



PostgreSQL под нагрузкой — от ноды до кластера

Модуль 1 - Введение в путь PostgreSQL-мастера

Урок 1 - Что такое масштабирование PostgreSQL

Шаг 01:

Миссия 1

Молодой разработчик Костя впервые развернул PostgreSQL у себя на ноутбуке. Он спрашивает у вас: что в первую очередь означает масштабирование базы данных?

Подсказка: Масштабирование связано с ростом нагрузки и способностью системы выдерживать этот рост.

Ответ:

- **Увеличение производительности системы при росте нагрузки**
- Добавление комментариев в код
- Увеличение размера таблицы логами
- Смена версии драйвера PostgreSQL

Шаг 02:

Миссия 2

Системный администратор Оля обсуждает с коллегами, когда стоит масштабировать PostgreSQL. В каком случае масштабирование становится необходимым?

Подсказка: Подумайте о ситуации, когда одна машина перестаёт справляться с запросами.

Ответ:

- **Когда один сервер перестаёт справляться с возросшим количеством запросов**
- Когда программисту скучно
- Когда в базе всего одна таблица
- Когда размер базы меньше 1 ГБ

🔮 Шаг 03:

Миссия 3

Фрилансер Лена задумалась: какой вариант масштабирования проще внедрить в PostgreSQL на старте проекта?

Подсказка: Начинать всегда проще с усиления железа, а не со сложных кластеров.

Ответ:

- **Вертикальное масштабирование (добавить CPU/RAM)**
- Горизонтальное масштабирование с десятками реплик
- Полное переписывание приложения
- Удаление индексов для ускорения

🔮 Шаг 04:

Миссия 4

Архитектор Саша объясняет стажёру разницу между вертикальным и горизонтальным масштабированием. Что верно?

Подсказка: Одно связано с мощностью одной машины, другое — с количеством машин.

Ответ:

- **Вертикальное** — увеличение ресурсов одной машины, горизонтальное — добавление новых серверов
- **Вертикальное** — это про индексы, горизонтальное — про запросы
- **Вертикальное** — значит работать стоя, горизонтальное — сидя
- Разницы между ними нет

🔮 Шаг 05:

Миссия 5

DevOps-инженер Артем думает, что даёт горизонтальное масштабирование PostgreSQL?

Подсказка: Вспомните про распределение нагрузки между машинами.

Ответ:

- **Возможность распределять нагрузку между несколькими серверами**
- Ускорение одиночного запроса на одном сервере
- Добавление GUI к PostgreSQL
- Бесплатный хостинг от разработчиков PostgreSQL

🔮 Шаг 06:

Миссия 6

Аналитик Катя заметила, что при росте нагрузки PostgreSQL можно масштабировать по-разному. Какой подход чаще используется для балансировки чтения?

Подсказка: Подумайте, что можно копировать без риска потерять данные.

Ответ:

- **Репликация на несколько серверов только для чтения**
- Удаление всех индексов
- Запрет на SELECT-запросы
- Переписывание SQL на Python

🔗 **Шаг 07:**

Миссия 7

Тимлид Игорь рассказывает: иногда PostgreSQL масштабируется не железом, а архитектурой. Как называется подход с разбиением таблиц на сегменты?

Подсказка: Это слово связано с делением на части.

Ответ:

- **Шардинг**
- Рефакторинг
- Кэширование
- Тюнинг индексов

🔗 **Шаг 08:**

Миссия 8

Инженер базы данных Петя спорит: зачем вообще нужна репликация при масштабировании?

Подсказка: Репликация — это не только про отказоустойчивость.

Ответ:

- **Чтобы распределять нагрузку на чтение и повышать отказоустойчивость**
- Чтобы уменьшить размер таблиц
- Чтобы PostgreSQL выглядел моднее
- Чтобы запретить все транзакции

🔗 **Шаг 09:**

Миссия 9

📖 Стажёр Дима изучает базовые термины масштабирования. Помогите ему сопоставить понятия и объяснения.

🔗 **Подсказка:**

Внимательно подумайте: некоторые слова связаны с ростом ресурсов, а некоторые — с архитектурой.

Ответ:

- **Вертикальное масштабирование** — Увеличение ресурсов одной машины (CPU, RAM, диск)
- **Горизонтальное масштабирование** — Добавление новых серверов в кластер
- **Репликация** — Создание копий данных для распределения нагрузки и отказоустойчивости
- **Шардинг** — Разделение данных на сегменты и хранение на разных серверах

🔗 **Шаг 10:**

Миссия 10

📖 Программист Миша перепутал термины. Сопоставьте методы масштабирования с их практическим применением.

💡 **Подсказка:**

Репликация — для чтения, шардинг — для больших таблиц, а вертикальное — для быстрой оптимизации.

Ответ:

- **Репликация** — Разгрузка чтения на копии базы
- **Шардинг** — Разделение данных пользователей по сегментам
- **Вертикальное масштабирование** — Добавление оперативной памяти или процессоров
- **Горизонтальное масштабирование** — Добавление новых серверов в систему

💡 **Шаг 11:**

Миссия 11

📖 Разработчица Настя тренируется отличать преимущества разных стратегий. Сопоставьте стратегию масштабирования с её плюсом.

💡 **Подсказка:**

Не все плюсы одинаковы: где-то проще реализовать, а где-то выше отказоустойчивость.

Ответ:

- **Вертикальное масштабирование** — Простота реализации — достаточно улучшить сервер
- **Горизонтальное масштабирование** — Позволяет выдерживать очень большие нагрузки
- **Репликация** — Обеспечивает высокую доступность при сбоях
- **Шардинг** — Снижает размер отдельных таблиц и ускоряет запросы

💡 **Шаг 12:**

Миссия 12

📖 DevOps-команда обсуждает, где уместны разные типы масштабирования. Сопоставьте сценарий и подход.

💡 **Подсказка:**

Представьте разные нагрузки: маленький проект, огромный магазин, аналитика.

Ответ:

- **Маленький стартап** — Вертикальное масштабирование
- **Интернет-магазин с тысячами покупателей** — Репликация
- **Социальная сеть с миллионами пользователей** — Шардинг
- **Система аналитики для BI** — Горизонтальное масштабирование

💡 **Шаг 13:**

Миссия 13

📖 Архитектор Даниил составил список терминов, а стажёры путаются. Сопоставьте их с верными пояснениями.

💡 **Подсказка:**

Обратите внимание на то, что связано с отказоустойчивостью и скоростью запросов.

Ответ:

- **Балансировщик нагрузки** — Распределяет запросы между серверами
- **Фейловер** — Автоматическое переключение на резервную систему

- **Кэширование** — Хранение часто используемых данных для ускорения
- **Партиционирование** — Разделение таблицы на логические части

🔮 Шаг 14:

Миссия 14

✳ Инженер Андрей планирует план масштабирования PostgreSQL. Расставьте шаги в правильном порядке.

🔮 Подсказка:

Сначала измерьте нагрузку, потом определите узкие места, далее выберите стратегию и только потом применяйте.

Ответ:

- Собрать метрики нагрузки
- Определить узкие места
- Выбрать стратегию масштабирования
- Реализовать выбранные изменения

🔮 Шаг 15:

Миссия 15

✳ Команда из стартапа решила оптимизировать PostgreSQL. Расположите действия по внедрению репликации в правильном порядке.

🔮 Подсказка:

Сначала подготовка мастер-сервера, потом настройка реплики, затем тестирование и только после — балансировка.

Ответ:

- Настроить главный сервер (master)
- Развернуть реплику (standby)
- Проверить работу репликации
- Настроить балансировку чтения

📖 Урок 2 - Архитектура PostgreSQL как всё устроено

🔮 Шаг 01:

Миссия 1

Разработчик Костя только начал изучать PostgreSQL и удивился: что именно является ядром системы?

Подсказка: Это главный процесс, который управляет остальными.

Ответ:

- **postmaster (основной процесс)**
- `psql` (клиент для работы с базой)
- `pg_dump` (утилита бэкапа)
- `pgbench` (тестирующий нагрузки)

🔮 Шаг 02:

Миссия 2

Сисадмин Оля объясняет стажёру: PostgreSQL состоит из нескольких уровней. Что отвечает за хранение данных на диске?

Подсказка: Подумайте о том, что работает с файлами напрямую.

Ответ:

- **Менеджер хранения (storage manager)**
- Парсер SQL-запросов
- Оптимизатор запросов
- Клиентская библиотека libpq

🔮 Шаг 03:

Миссия 3

Аналитик Лена спросила: какой компонент PostgreSQL отвечает за выбор наилучшего плана выполнения SQL-запроса?

Подсказка: Этот процесс похож на планирование маршрута для навигатора.

Ответ:

- **Оптимизатор запросов**
- Исполнитель запросов
- Процесс walwriter
- pg_stat_statements

🔮 Шаг 04:

Миссия 4

Тимлид Саша объясняет студенту: за что отвечает процесс bgwriter ?

Подсказка: Этот процесс помогает с буферами и диском.

Ответ:

- **Сбрасывает изменённые страницы из буфера на диск**
- Отвечает за подключение клиентов
- Оптимизирует SQL-запросы
- Создаёт новые таблицы

🔮 Шаг 05:

Миссия 5

DevOps-инженер Артём проверяет работу walwriter . За что отвечает этот процесс?

Подсказка: Вспомните про журнал транзакций WAL.

Ответ:

- **Записывает изменения в WAL (журнал транзакций)**
- Удаляет старые базы
- Создаёт индексы
- Запускает VACUUM

🔮 Шаг 06:

Миссия 6

Марина из команды поддержки удивилась: какой процесс отвечает за очистку и переиспользование пространства?

Подсказка: Подумайте о процессе, связанном с VACUUM.

Ответ:

- **autovacuum launcher**
- bgwriter
- archiver
- seqscan

💡 **Шаг 07:**

Миссия 7

Инженер базы Петя спрашивает: какой процесс в PostgreSQL отвечает за создание файлов архивов WAL?

Подсказка: Этот процесс работает, если включено архивирование.

Ответ:

- **archiver**
- walwriter
- autovacuum
- executor

💡 **Шаг 08:**

Миссия 8

Студент Ваня изучает архитектуру PostgreSQL. За что отвечает процесс checkpointer ?

Подсказка: Этот процесс связан с периодическим сохранением состояния.

Ответ:

- **Записывает все грязные страницы из буфера на диск в определённый момент**
- Устанавливает соединения клиентов
- Оптимизирует запросы
- Создаёт резервные копии

💡 **Шаг 09:**

Миссия 9

📖 Стажёр Дима путается в процессах PostgreSQL. Сопоставьте процесс и его назначение.

💡 **Подсказка:**

Вспомните: одни процессы работают с журналами, другие — с очисткой, третьи — с диском.

Ответ:

- **bgwriter** — Сбрасывает изменённые страницы из буфера
- **walwriter** — Записывает изменения в журнал WAL
- **checkpointer** — Выполняет контрольные точки
- **autovacuum** — Автоматическая очистка и переиспользование

💡 **Шаг 10:**

Миссия 10

📖 Программист Миша изучает уровни PostgreSQL. Сопоставьте уровень и его функцию.

💡 **Подсказка:**

Каждый уровень решает свою задачу: хранение, парсинг, планирование, выполнение.

Ответ:

- **Storage Manager** — Управление файлами данных на диске
- **Parser** — Разбор SQL-запроса
- **Planner/Optimizer** — Выбор оптимального плана выполнения
- **Executor** — Реальное выполнение запроса

💡 **Шаг 11:**

Миссия 11

📖 DBA Настя объясняет коллегам важность процессов. Сопоставьте процесс и его описание.

💡 **Подсказка:**

Не перепутайте фоновые процессы с клиентскими.

Ответ:

- **Postmaster** — Главный процесс, управляющий другими
- **Archiver** — Создание архивных файлов WAL
- **Client Backend** — Обрабатывает запросы конкретного клиента
- **Stats Collector** — Собирает статистику работы сервера

💡 **Шаг 12:**

Миссия 12

📖 DevOps-команда обсуждает архитектуру. Сопоставьте элемент и его роль.

💡 **Подсказка:**

Думайте о том, кто выполняет, кто управляет, а кто хранит.

Ответ:

- **Executor** — Выполняет план запроса
- **Planner** — Создаёт план выполнения
- **Parser** — Разбирает синтаксис SQL
- **Buffer Manager** — Работает с кэшированными страницами

💡 **Шаг 13:**

Миссия 13

📖 Архитектор Даниил составил список. Сопоставьте понятия с их описанием.

💡 **Подсказка:**

Фоновые процессы обеспечивают стабильность работы, а уровни отвечают за обработку запросов.

Ответ:

- **Checkpoint** — Сохраняет состояние базы в определённые моменты
- **Autovacuum** — Очищает и переиспользует место
- **Parser** — Разбирает SQL-запрос
- **Optimizer** — Выбирает лучший план

💡 **Шаг 14:**

Миссия 14

📖 Студенты готовятся к экзамену. Сопоставьте процесс PostgreSQL и его задачу.

💡 **Подсказка:**

Некоторые процессы работают с журналами, другие с метриками.

Ответ:

- **WALwriter** — Записывает изменения в WAL
- **Stats Collector** — Собирает статистику
- **Archiver** — Архивирует WAL-файлы
- **Bgwriter** — Сбрасывает страницы из буфера

🔧 **Шаг 15:**

Миссия 15

✂ Инженер Андрей решил проследить путь SQL-запроса в PostgreSQL. Расставьте этапы в правильном порядке.

🔧 **Подсказка:**

Сначала запрос нужно разобрать, потом спланировать, затем выполнить.

Ответ:

- Парсинг SQL-запроса
- Создание плана оптимизатором
- Выполнение плана исполнителем
- Возврат результата клиенту

🔧 **Шаг 16:**

Миссия 16

✂ DBA Марина описывает процессы сохранения данных. Расставьте шаги в правильной последовательности.

🔧 **Подсказка:**

Сначала изменения идут в буфер, потом в WAL, потом на диск.

Ответ:

- Запись изменений в буфер
- Фиксация в журнале WAL
- Выполнение контрольной точки
- Сброс страниц на диск

📖 **Урок 3 - Горизонтальное и вертикальное масштабирование**

🔧 **Шаг 01:**

Миссия 1

Разработчик Вика запустила приложение и заметила, что при росте пользователей база работает всё медленнее. Какой первый шаг для улучшения производительности можно отнести к вертикальному масштабированию PostgreSQL?

Подсказка: Вертикальное масштабирование — это усиление одного сервера, а не добавление новых.

Ответ:

- **Добавить серверу больше оперативной памяти и процессоров**
- Настроить репликацию между несколькими серверами
- Разбить данные на несколько шардов
- Подключить балансировщик нагрузки

Шаг 02:

Миссия 2

Андрей заметил, что его сервер PostgreSQL начал упираться в диск. Что можно сделать в рамках вертикального масштабирования?

Подсказка: Подумайте о замене или усилении оборудования.

Ответ:

- **Поставить более быстрые SSD-диски**
- Перенести часть таблиц в другой кластер
- Включить асинхронную репликацию
- Добавить больше логических реплик

Шаг 03:

Миссия 3

Программист Лена обсуждает с командой разницу между горизонтальным и вертикальным масштабированием. Что из перечисленного ближе к горизонтальному?

Подсказка: Горизонталь — это про распределение нагрузки по многим узлам.

Ответ:

- Увеличить оперативную память сервера
- Установить процессор нового поколения
- **Добавить дополнительные серверы с PostgreSQL**
- Заменить диск на NVMe

Шаг 04:

Миссия 4

База компании хранит миллионы заказов. При горизонтальном масштабировании программист Саша думает о шардировании. Что это означает?

Подсказка: Представьте, что вы делите огромный список клиентов на несколько частей по разным шкафам.

Ответ:

- **Разделение данных по определённому признаку между несколькими серверами**
- Увеличение размера кэша PostgreSQL
- Хранение всех данных только на SSD
- Оптимизация одного запроса индексацией

Шаг 05:

Миссия 5

Инженер Миша добавил несколько реплик для PostgreSQL. Что это даёт с точки зрения масштабирования?

Подсказка: Реплика чаще всего помогает распределить нагрузку, но не всегда ускоряет запись.

Ответ:

- **Возможность разгрузить мастер, выполняя чтение с реплик**
- Ускорение операций записи
- Снижение объёма данных в таблицах
- Удаление дубликатов строк

Шаг 06:

Миссия 6

Когда нагрузка выросла, программист Дима думает, что лучше: вертикальное или горизонтальное масштабирование. В каком случае горизонтальное решение предпочтительнее?

Подсказка: Один сервер не может бесконечно увеличиваться.

Ответ:

- **Когда один сервер уже достиг предела своих характеристик**
- Когда сервер простаивает и недогружен
- Когда нужно просто заменить диск на больший
- Когда достаточно добавить немного памяти

Шаг 07:

Миссия 7

Архитектор базы данных обсуждает плюсы и минусы. Какой главный минус горизонтального масштабирования?

Подсказка: Чем больше серверов, тем сложнее управлять всей системой.

Ответ:

- Ограничение по размеру памяти
- Нельзя использовать SSD-диски
- **Усложнение администрирования и синхронизации данных**
- Невозможность использовать индексы

Шаг 08:

Миссия 8

 Задача: сопоставьте тип масштабирования с его примером.

Подсказка: Вертикальное связано с усилением одного узла, горизонтальное — с добавлением новых узлов.

Ответ:

- **Вертикальное масштабирование** — Добавить серверу больше памяти и процессоров
- **Горизонтальное масштабирование** — Разделить данные на несколько серверов
- **Репликация** — Создать копии базы для распределения нагрузки
- **Шардирование** — Разделить таблицу заказов по регионам

Шаг 09:

Миссия 9

 Задача: сопоставьте ситуацию с правильным типом масштабирования.

Подсказка: Если усиливаем один сервер — это вертикаль, если добавляем новые — горизонталь.

Ответ:

- **Сервер перегружен по CPU** — Вертикальное масштабирование
- **Нужно обслуживать миллионы пользователей одновременно** — Горизонтальное масштабирование
- **Хотим распределить запросы на чтение** — Репликация
- **Необходимо делить данные по регионам** — Шардирование

Шаг 10:

Миссия 10

 Задача: сопоставьте технологию PostgreSQL с её назначением.

Подсказка: Подумайте, что ускоряет чтение, а что помогает хранить огромные таблицы.

Ответ:

- **Реплика** — Разгрузка мастера за счёт чтения
- **Шард** — Хранение части данных на отдельном сервере
- **Вертикальное масштабирование** — Увеличение ресурсов одного сервера
- **Балансировщик нагрузки** — Распределение запросов между узлами

Шаг 11:

Миссия 11

Расположите шаги внедрения горизонтального масштабирования в правильном порядке:

Подсказка: Сначала план, потом разделение данных, затем настройка инфраструктуры и только потом балансировка нагрузки.

Ответ:

- Планирование стратегии шардирования
- Разделение данных на шарды
- Настройка реплик и серверов
- Добавление балансировщика нагрузки

Шаг 12:

Миссия 12

Расположите действия при вертикальном масштабировании:

Подсказка: Сначала определяем узкое место, потом усиливаем сервер, затем оптимизируем конфигурацию.

Ответ:

- Анализ узких мест
- Добавление ресурсов (CPU, RAM, SSD)
- Тонкая настройка PostgreSQL
- Перезапуск и проверка производительности

Шаг 13:

Миссия 13

Команда решила внедрить репликацию. Расположите шаги в правильном порядке:

Подсказка: Сначала настраивается мастер, потом создаются реплики, и только затем распределяются запросы.

Ответ:

- Настройка мастера
- Создание реплик
- Синхронизация данных
- Распределение запросов между узлами

Шаг 14:

Миссия 14

Компания масштабируется. Расположите решения от простого к сложному:

Подсказка: Увеличить память проще, чем построить кластер с шардами.

Ответ:

- Добавить оперативную память
- Заменить диск на SSD
- Добавить реплики для чтения
- Реализовать шардирование

Урок 4 - Обзор стратегий HA, репликация, шардирование

Шаг 01:

Миссия 1

Команда стартапа запускает новый сервис на PostgreSQL. Им нужно понять, что значит High Availability (HA).

Подсказка: Подумайте, что гарантирует непрерывную работу базы данных даже при сбоях.

Ответ:

- **Система, обеспечивающая минимальный простой при сбоях**
- Просто быстрое железо с большим количеством ядер
- Установка PostgreSQL на один сервер без резервов
- Добавление индексов во все таблицы

Шаг 02:

Миссия 2

Разработчик Костя спорит с коллегами: что такое репликация в PostgreSQL?

Подсказка: Это связано с созданием копий данных.

Ответ:

- **Процесс копирования данных с одного сервера на другой**
- Сжатие базы данных для экономии места
- Удаление старых записей из таблиц
- Изменение структуры индексов

Шаг 03:

Миссия 3

В компании растёт нагрузка на БД. Что делает шардирование?

Подсказка: Представьте, что одна огромная таблица делится на части между серверами.

Ответ:

- **Разделяет данные между несколькими серверами**
- Создаёт копию базы для бэкапа
- Обновляет драйверы PostgreSQL
- Собирает статистику по запросам

Шаг 04:

Миссия 4

Программист Анна решила разобраться: в чём отличие master-slave репликации?

Подсказка: Один сервер главный, другие — подчинённые.

Ответ:

- **Master принимает запись, slave только читает**
- Все сервера одновременно могут писать и читать
- Slave управляет master-сервером
- Master используется только для чтения

Шаг 05:

Миссия 5

Алексей оптимизирует проект и думает: в каких случаях нужно HA?

Подсказка: Представьте критически важный сервис, который не должен останавливаться.

Ответ:

- **Онлайн-банк с миллионами транзакций**
- **Сервис доставки еды, работающий круглосуточно**
- Тестовая база для студенческих проектов
- Локальный прототип для одного разработчика

Шаг 06:

Миссия 6

 Сопоставьте стратегии с их назначением:

Подсказка: Вспомните, где применяют отказоустойчивость, а где — масштабирование.

Ответ:

- **High Availability (HA)** — Обеспечение непрерывной работы сервиса при сбоях
- **Репликация** — Создание копий данных на других серверах
- **Шардирование** — Разделение базы на части для распределения нагрузки
- **Backup** — Создание резервной копии базы данных

Шаг 07:

Миссия 7

 Сопоставьте тип репликации с описанием:

Подсказка: Одна бывает синхронной, другая — асинхронной.

Ответ:

- **Синхронная репликация** — Подтверждение транзакции только после записи на оба сервера
- **Асинхронная репликация** — Транзакция подтверждается сразу, а запись на slave позже
- **Логическая репликация** — Передаются изменения на уровне SQL-операторов
- **Физическая репликация** — Передаются блоки данных с диска

Шаг 08:

Миссия 8

 Сопоставьте понятия «горизонтальное» и «вертикальное» масштабирование:

Подсказка: Одно про добавление серверов, другое про усиление железа.

Ответ:

- **Горизонтальное масштабирование** — Добавление новых серверов в кластер
- **Вертикальное масштабирование** — Увеличение мощности одного сервера
- **HA** — Повышение доступности
- **Балансировщик нагрузки** — Распределяет запросы между серверами

Шаг 09:

Миссия 9

 Сопоставьте инструменты PostgreSQL с их назначением:

Подсказка: pgpool и Patroni решают разные задачи.

Ответ:

- **Patroni** — Управление кластером и автоматический failover
- **pgpool-II** — Балансировка нагрузки и пул подключений
- **pg_basebackup** — Создание резервной копии для репликации
- **pg_stat_replication** — Мониторинг состояния репликации

Шаг 10:

Миссия 10

Расположите шаги настройки потоковой репликации PostgreSQL в правильном порядке:

Подсказка: Начните с подготовки master.

Ответ:

- Настройка параметров wal_level и replication на master
- Создание бэкапа с помощью pg_basebackup
- Запуск standby-сервера
- Проверка синхронизации

Шаг 11:

Миссия 11

Расположите этапы организации HA-кластера:

Подсказка: Сначала настройка узлов, потом добавление мониторинга.

Ответ:

- Настройка нескольких PostgreSQL-узлов
- Установка и настройка Patroni
- Подключение etcd/consul для координации
- Настройка балансировщика

Шаг 12:

Миссия 12

Расставьте шаги шардирования базы данных:

Подсказка: Сначала проектирование, потом физическая реализация.

Ответ:

- Определение стратегии деления данных (по пользователям, регионам и т.д.)
- Настройка нескольких серверов

- Реализация маршрутизации запросов
- Тестирование распределённой системы

Шаг 13:

Миссия 13

Расположите действия администратора при падении master-сервера:

Подсказка: Failover — ключевой момент.

Ответ:

- Определение сбоя
- Назначение нового master
- Переключение клиентов на новый master
- Восстановление старого узла как standby

Шаг 14:

Миссия 14

Расположите этапы настройки логической репликации:

Подсказка: Вспомните про публикацию и подписку.

Ответ:

- Создание публикации на master
- Настройка подписки на slave
- Проверка синхронизации данных
- Тестирование записи новых транзакций

Шаг 15:

Миссия 15

Расположите шаги для построения отказоустойчивого кластера с репликацией и балансировкой:

Подсказка: Сначала база, потом слои отказоустойчивости.

Ответ:

- Настройка master и standby серверов
- Включение репликации
- Подключение Patroni для автоматического failover
- Настройка балансировщика запросов

Модуль 2 - Осваиваем репликацию — путь данных

Урок 1 - Что такое WAL и как он работает

Шаг 01:

Миссия 1

Андрей заметил, что при каждом изменении данных в PostgreSQL создаются специальные записи в WAL. Зачем они нужны?

Подсказка: Подумайте о том, что будет, если сервер внезапно выключится. Где взять данные для восстановления?

Ответ:

- Для восстановления изменений после сбоя

- Для ускорения индексации таблиц
- Для хранения резервных копий базы
- Для распределения нагрузки между серверами

🔮 Шаг 02:

Миссия 2

Марина читает логи PostgreSQL и видит, что WAL пишется раньше, чем обновление попадает в таблицу. Почему так?

Подсказка: Важен порядок: сначала фиксируем изменения, а потом применяем их в памяти и на диске.

Ответ:

- **Чтобы гарантировать целостность данных при сбое**
- Чтобы ускорить выполнение SELECT-запросов
- Чтобы уменьшить нагрузку на сеть
- Чтобы сразу отдать данные клиенту без изменений

🔮 Шаг 03:

Миссия 3

Игорь настраивает систему и думает: «Если WAL так важен, то куда его лучше писать?»

Подсказка: Подумайте про производительность ввода-вывода и надежность хранения.

Ответ:

- **На быстрый и надежный диск (SSD, отдельный том)**
- В оперативную память, чтобы было быстрее
- На тот же диск, где системные логи ОС
- В облако без локального кэша

🔮 Шаг 04:

Миссия 4

Сергей запускает нагрузочные тесты. Вдруг сервер перезагрузился. Какие файлы PostgreSQL помогут ему восстановить состояние базы?

Подсказка: Вспомните: данные применяются не напрямую, а сначала фиксируются в журнале.

Ответ:

- **Файлы WAL**
- Конфигурационные файлы postgresql.conf
- Файлы pg_hba.conf
- Журналы ошибок ОС

🔮 Шаг 05:

Миссия 5

Алексей удивился: почему размер базы сильно вырос при большом количестве транзакций? Что может быть причиной?

Подсказка: Вспомните про накопление журналов и их архивирование.

Ответ:

- **Слишком много неархивированных WAL-файлов**
- Увеличение размера индексов при SELECT

- Избыточное кеширование данных в памяти
- Слишком маленький `shared_buffers`

🔮 Шаг 06:

Миссия 6

Катя настраивает потоковую репликацию. Что именно передается реплике?

Подсказка: Реплика не получает готовые данные, а применяет журнал изменений.

Ответ:

- **Записи WAL**
- Снимки всей базы
- Только данные таблиц без индексов
- Результаты запросов SELECT

🔮 Шаг 07:

Миссия 7

Вася проверяет работу WAL и спрашивает: «А можно ли применить WAL только частично?»

Подсказка: Думайте о консистентности — база должна быть целой после восстановления.

Ответ:

- **Нет, применяются только целые записи транзакций**
- Да, можно откатить до произвольного байта WAL
- Да, можно пропустить ненужные записи
- Нет, WAL вообще не участвует в восстановлении

🔮 Шаг 08:

Миссия 8

Ольга хочет понять, почему WAL называется «журналом предзаписи». В чем суть?

Подсказка: Сначала в журнал, потом в данные.

Ответ:

- **Запись в WAL происходит до изменения таблиц**
- Запись в WAL идет только после коммита
- Запись в WAL полностью заменяет UPDATE
- Запись в WAL дублирует содержимое памяти

🔮 Шаг 09:

Миссия 9

📄 Сопоставьте ключевые элементы WAL с их назначением.

Подсказка: Представьте, что WAL — это «черный ящик» самолета. У каждой его части есть роль.

Ответ:

- **WAL сегмент** — Файл фиксированного размера, где хранятся записи журнала
- **WAL запись** — Конкретное изменение данных в транзакции
- **Архивация WAL** — Сохранение журналов для восстановления или репликации
- **Чекпоинт** — Момент, когда данные сбрасываются с журнала на диск

🔮 Шаг 10:

Миссия 10

📖 Сопоставьте понятия с их функцией в работе WAL.

Подсказка: Репликация, восстановление, консистентность — всё завязано на журнал.

Ответ:

- **fsync** — Гарантирует запись WAL на диск
- **pg_wal каталог** — Хранит активные журналы WAL
- **restore_command** — Команда для восстановления архивированных WAL
- **replication slot** — Позволяет удерживать WAL для реплик

🔮 Шаг 11:

Миссия 11

📖 Сопоставьте команды и их назначение в контексте WAL.

Подсказка: Эти команды часто используют при настройке резервного копирования и восстановления.

Ответ:

- **pg_basebackup** — Создание бэкапа с поддержкой WAL
- **archive_command** — Копирование WAL-файлов в архив
- **pg_resetwal** — Сброс журнала WAL в аварийных случаях
- **pg_receivewal** — Получение WAL для потоковой репликации

🔮 Шаг 12:

Миссия 12

📖 Сопоставьте термины WAL с их объяснением.

Подсказка: Это как словарь терминов WAL для новичка.

Ответ:

- **REDO** — Повторное применение изменений при восстановлении
- **LSN** — Логическая позиция в WAL
- **Hot standby** — Реплика, на которой можно выполнять SELECT
- **Point-in-time recovery** — Восстановление базы на конкретный момент

🔮 Шаг 13:

Миссия 13

📖 Сопоставьте проблемы с возможными причинами в работе WAL.

Подсказка: Если база растёт или падает — виноваты настройки или архивирование.

Ответ:

- **Быстрое переполнение диска** — WAL не архивируется
- **Долгий откат после сбоя** — Слишком редкие чекпоинты
- **Падение реплики** — Нет нужных WAL в мастере
- **Рост нагрузки на диск** — Слишком частые fsync

🔮 Шаг 14:

Миссия 14

Расположите шаги прохождения данных через WAL в правильном порядке:

Подсказка:

Сначала фиксируем изменения в журнале, затем сохраняем на диск, и только потом используем для репликации или восстановления.

Ответ:

- Запись изменений в WAL
- fsync для сохранения на диск
- Архивация WAL или отправка на реплики
- Применение изменений при восстановлении

Шаг 15:

Миссия 15

Расположите действия администратора при настройке архивации WAL в верном порядке:

Подсказка:

Нужно сначала настроить конфигурацию, потом проверить архивирование, а в конце протестировать восстановление.

Ответ:

- Указать archive_command в конфиге
- Проверить появление файлов в архиве
- Настроить restore_command для восстановления
- Провести тестовое восстановление

Шаг 16:

Миссия 16

Расположите шаги восстановления базы с помощью WAL после сбоя:

Подсказка:

Сначала нужен бэкап, потом применяем WAL, и только потом база запускается в рабочем режиме.

Ответ:

- Восстановить из резервной копии
- Применить WAL из архива
- Достроить изменения до последнего LSN
- Запустить базу в normal mode

Урок 2 - Физическая vs логическая репликация

Шаг 01:

Миссия 1

Андрей развернул PostgreSQL и впервые услышал о физической репликации. Какое главное отличие физической репликации от логической?

Подсказка: Подумайте о том, что именно передаётся: биты или SQL-запросы.

Ответ:

- **Физическая репликация копирует сегменты WAL на уровне байтов**
- Физическая репликация передаёт только SQL-запросы
- Физическая репликация копирует только структуру схемы без данных
- Физическая репликация работает только с JSON-данными

Шаг 02:

Миссия 2

Оля настраивает логическую репликацию, чтобы читать данные в аналитической системе. Что является основой передачи данных в логической репликации?

Подсказка: Вспомните, что именно логическая репликация передаёт подписчикам.

Ответ:

- **Изменения на уровне строк (INSERT/UPDATE/DELETE)**
- Только операции CREATE и DROP
- Сырые блоки памяти на диске
- Только операции SELECT

Шаг 03:

Миссия 3

Дима пытается понять, зачем вообще нужна репликация. Какое из преимуществ является общим и для физической, и для логической репликации?

Подсказка: Подумайте о базовом назначении репликации в любой системе.

Ответ:

- **Повышение отказоустойчивости и доступности системы**
- Автоматическая генерация отчётов в PDF
- Миграция данных между MySQL и PostgreSQL
- Сжатие WAL в архивы

Шаг 04:

Миссия 4

Катя включила репликацию и увидела, что физическая реплика требует одинаковых версий PostgreSQL. Почему?

Подсказка: Вспомните, на каком уровне физическая репликация работает и какие структуры она повторяет.

Ответ:

- **Из-за строгой зависимости от формата страниц базы данных**
- Потому что PostgreSQL запрещает разные версии для всех подключений
- Из-за несовместимости SQL-синтаксиса
- Потому что так удобнее администраторам

Шаг 05:

Миссия 5

Алексей планирует использовать логическую репликацию для переноса данных в облако. Что является ограничением логической репликации?

Подсказка: Вспомните, что логическая репликация работает на уровне таблиц.

Ответ:

- **Не реплицируются DDL-операции (CREATE/ALTER TABLE)**
- Не поддерживаются INSERT и UPDATE
- Нельзя использовать индексы на подписчике
- Нельзя подключить более одного подписчика

🔗 Шаг 06:

Миссия 6

Марина проверяет, что будет при сбое основного узла в кластере с физической репликацией. Какой сценарий обычно используется для восстановления?

Подсказка: Подумайте о механизме failover и роли standby.

Ответ:

- **Промоут реплики в новый мастер (failover)**
- Автоматическая конвертация в логическую репликацию
- Остановка всех подключений до ручного вмешательства
- Автоматическая миграция на Oracle

🔗 Шаг 07:

Миссия 7

Сергей настраивает публикацию и подписку для логической репликации. Какой объект в PostgreSQL отвечает за отправку изменений подписчику?

Подсказка: Подумайте о том, что создаётся на стороне издания (publisher).

Ответ:

- **Публикация (publication)**
- Триггер AFTER INSERT
- Функция generate_series()
- Слот расширения hstore

🔗 Шаг 08:

Миссия 8

📄 Задача: Сопоставьте элементы физической и логической репликации.

Подсказка: Вспомните, что физическая работает с байтами, а логическая — с данными на уровне строк.

Ответ:

- **Физическая репликация** — Передача WAL-блоков на уровне страниц
- **Логическая репликация** — Передача изменений строк
- **Publication** — Набор таблиц для логической репликации
- **Standby** — Узел-копия в физической репликации

🔗 Шаг 09:

Миссия 9

📄 Задача: Разные роли в репликации. Соотнесите их с правильными определениями.

Подсказка: Кто отправляет данные, а кто принимает?

Ответ:

- **Publisher** — Источник изменений в логической репликации
- **Subscriber** — Получатель данных в логической репликации
- **Primary** — Основной сервер в физической репликации
- **Replica** — Копия данных, синхронизированная с Primary

Шаг 10:

Миссия 10

 Задача: Сопоставьте тип данных WAL-записи с тем, что в ней содержится.

Подсказка: WAL фиксирует изменения на низком уровне, а логическая репликация их интерпретирует.

Ответ:

- **INSERT WAL** — Информация о вставке новой строки
- **UPDATE WAL** — Старое и новое значения строки
- **DELETE WAL** — Удалённая строка
- **DDL WAL** — Операции изменения структуры

Шаг 11:

Миссия 11

 Задача: Соотнесите инструмент с его применением в контексте репликации.

Подсказка: pg_basebackup, pglogical и т.п. имеют разные назначения.

Ответ:

- **pg_basebackup** — Создание копии для физической реплики
- **pglogical** — Расширение для логической репликации
- **pg_stat_replication** — Мониторинг состояния репликации
- **pg_wal** — Хранение журналов WAL

Шаг 12:

Миссия 12

Расположите шаги настройки физической реплики в правильном порядке.

Подсказка: Сначала создаётся база для реплики, затем подключение к мастеру, и только потом запуск.

Ответ:

- Создать резервную копию с помощью pg_basebackup
- Настроить recovery.conf или параметры standby
- Запустить сервер-реплику
- Проверить статус через pg_stat_replication

Шаг 13:

Миссия 13

Расположите шаги настройки логической репликации в правильном порядке.

Подсказка: Сначала создаётся публикация, затем подписка.

Ответ:

- Создать публикацию на стороне источника
- Создать подписку на стороне получателя
- Проверить состояние через pg_stat_subscription
- Выполнить тестовую транзакцию

Шаг 14:

Миссия 14

Расположите действия администратора при аварийном переключении (failover).

Подсказка: Нужно остановить мастер, промоутить реплику и перенастроить подключения.

Ответ:

- Остановить недоступный основной сервер
- Выполнить promote реплики
- Перенастроить клиентов на новый мастер
- Проверить синхронизацию

Шаг 15:

Миссия 15

У программистов возник спор: как развивается репликация от простой к сложной. Расставьте варианты по возрастанию сложности.

Подсказка: Начните с простого standby, затем логическая и дальше кластерные решения.

Ответ:

- Одиночный hot standby
- Логическая репликация с выборочными таблицами
- Синхронная физическая репликация
- Кластер с балансировкой нагрузки

Урок 3 - Настройка физической реплики с pg_basebackup

Шаг 01:

Миссия 1

Андрей впервые запускает pg_basebackup. Для чего в первую очередь используется эта команда?

Подсказка: Вспомните, что нужно реплике, чтобы стартовать с чистого состояния.

Ответ:

- **Создание начальной копии базы данных для физической реплики**
- Восстановление данных после удаления таблицы
- Оптимизация запросов SELECT
- Создание логической публикации

Шаг 02:

Миссия 2

Оля пытается понять: почему pg_basebackup требует подключения к основному серверу через репликационного пользователя?

Подсказка: Подумайте, какие права нужны для доступа к журналам WAL и данным.

Ответ:

- **Чтобы иметь права читать WAL и системные данные**
- Чтобы создавать новые базы данных на мастере
- Чтобы изменять pg_hba.conf
- Чтобы выполнять VACUUM на всех таблицах

Шаг 03:

Миссия 3

Дима запускает pg_basebackup с параметром -R. Что произойдёт?

Подсказка: Вспомните, что нужно, чтобы реплика автоматически знала, куда подключаться.

Ответ:

- **Будет автоматически создан конфигурационный файл standby.signal**
- Будет выполнен полный VACUUM
- Будет создана новая публикация для логической репликации
- Будет удалена папка data на мастере

🔮 Шаг 04:

Миссия 4

Катя запускает pg_basebackup с ключом -X stream. Зачем он нужен?

Подсказка: Подумайте, как репликация должна получать журналы WAL во время копирования.

Ответ:

- **Для потоковой передачи WAL во время резервного копирования**
- Для создания текстового отчёта о копировании
- Для параллельного копирования таблиц
- Для активации логической репликации

🔮 Шаг 05:

Миссия 5

Алексей заметил, что pg_basebackup можно запускать с параметром -P. Что он даёт?

Подсказка: Вспомните, как удобно следить за прогрессом копирования.

Ответ:

- **Отображение прогресса копирования в консоли**
- Параллельная передача WAL
- Резервное копирование только схемы без данных
- Создание дополнительной реплики

🔮 Шаг 06:

Миссия 6

Марина думает, почему иногда используют параметр -Fp в pg_basebackup. Что он означает?

Подсказка: Вспомните, как можно сохранить структуру каталогов при копировании.

Ответ:

- **Формат plain (каталоги и файлы в исходном виде)**
- Формат tar (архивирование)
- Формат gzip (сжатие)
- Формат sql (дамп)

🔮 Шаг 07:

Миссия 7

Сергей пробует использовать ключ -c fast в pg_basebackup. Что это изменяет?

Подсказка: Подумайте о типах чекпоинтов в PostgreSQL.

Ответ:

- **Принудительное создание быстрого чекпоинта**

- Сжатие файлов на лету
- Отключение передачи WAL
- Запуск логической публикации

🔧 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте параметры `pg_basebackup` с их назначением.

Подсказка: Вспомните, какие ключи отвечают за формат, прогресс и WAL.

Ответ:

- **-Fp** — Сохраняет файлы в исходной структуре
- **-Ft** — Создает архив tar
- **-P** — Показывает прогресс выполнения
- **-X stream** — Передаёт WAL во время копии

🔧 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте команду с её результатом.

Подсказка: Подумайте о том, какие файлы создает `pg_basebackup` с разными опциями.

Ответ:

- **`pg_basebackup -R`** — Создает `standby.signal`
- **`pg_basebackup -Ft`** — Создает архив tar
- **`pg_basebackup -Fp`** — Каталоги и файлы без архивации
- **`pg_basebackup -X fetch`** — Скачивает WAL после завершения копии

🔧 Шаг 10:

Миссия 10

📖 Задача: Соотнесите объекты с их ролью в процессе настройки физической реплики.

Подсказка: WAL, `pg_basebackup` и конфигурационные файлы играют разные роли.

Ответ:

- **`pg_basebackup`** — Создание копии базы для реплики
- **`standby.signal`** — Определяет сервер как реплику
- **`postgresql.conf`** — Задаёт параметры работы PostgreSQL
- **`pg_hba.conf`** — Управляет доступом клиентов

🔧 Шаг 11:

Миссия 11

📖 Задача: Сопоставьте действия администратора с их результатом при настройке реплики.

Подсказка: Вспомните последовательность: создать копию, подключить WAL, включить `standby`.

Ответ:

- **Запуск `pg_basebackup`** — Получение копии базы
- **Создание `standby.signal`** — Установка режима реплики
- **Включение параметра `hot_standby`** — Разрешение чтения на реплике
- **Подключение к мастеру** — Начало получения WAL

Шаг 12:

Миссия 12

 Задача: Сопоставьте файлы и каталоги PostgreSQL с их ролью при репликации.

Подсказка: `pg_wal` хранит журналы, а `data` — всё содержимое базы.

Ответ:

- **`pg_wal`** — Каталог с журналами WAL
- **`standby.signal`** — Файл запуска реплики
- **`pg_basebackup.log`** — Отчёт о процессе копирования
- **`data/`** — Основной каталог базы

Шаг 13:

Миссия 13

 Задача: Сопоставьте опцию запуска `pg_basebackup` с её эффектом.

Подсказка: Разные ключи отвечают за вывод, структуру и поведение копии.

Ответ:

- **`-D`** — Каталог для сохранения данных
- **`-v`** — Подробный вывод процесса
- **`-C`** — Создание репликационного слота
- **`-Z`** — Сжатие архива

Шаг 14:

Миссия 14

Расположите шаги настройки физической реплики в правильном порядке.

Подсказка: Сначала делается копия, потом конфигурация, потом запуск.

Ответ:

- Запуск `pg_basebackup` для создания копии
- Создание `standby.signal`
- Изменение `postgresql.conf` и `pg_hba.conf`
- Запуск PostgreSQL в режиме реплики

Шаг 15:

Миссия 15

Расположите шаги восстановления реплики после сбоя мастера.

Подсказка: Нужно сначала проверить мастера, потом продвинуть реплику.

Ответ:

- Остановить недоступный мастер
- Промоутить реплику в новый основной
- Перенастроить клиентов на новый сервер
- Проверить синхронизацию

Шаг 16:

Миссия 16

Расположите действия администратора при тестировании реплики.

Подсказка: Нужно выполнить запросы и проверить логи.

Ответ:

- Выполнить тестовый INSERT на мастере
- Проверить появление данных на реплике
- Проверить pg_stat_replication
- Сравнить данные на мастере и реплике

Урок 4 - Настройка restore_command и архивация WAL

Шаг 01:

Миссия 1

Алексей настроил архивирование WAL, но забыл указать restore_command на реплике. Что произойдет при попытке восстановления?

Подсказка: Реплика должна знать, где искать недостающие WAL-файлы.

Ответ:

- **Восстановление остановится при первом недостающем WAL**
- Реплика автоматически переключится в режим logical replication
- Postgres создаст пустые WAL-файлы
- Восстановление пройдет успешно, но данные будут повреждены

Шаг 02:

Миссия 2

Марина написала restore_command='cp /var/lib/pg_wal/%f %p'. Почему это ошибка?

Подсказка: Восстановление не может брать файлы из того же каталога, куда они записываются.

Ответ:

- **Источником должен быть внешний архив, а не pg_wal**
- Команда cp не поддерживается Postgres
- %f и %p нельзя использовать вместе
- restore_command вообще не нужен

Шаг 03:

Миссия 3

Иван хочет протестировать работу restore_command. Какой способ корректный?

Подсказка: Достаточно симулировать недостающий WAL и проверить реакцию.

Ответ:

- **Удалить один WAL из pg_wal и перезапустить реплику**
- Остановить сервер и удалить postgresql.conf
- Запустить pg_basebackup заново
- Сделать VACUUM FULL

Шаг 04:

Миссия 4

Дима написал restore_command='scp backup/%f %p'. Что нужно добавить, чтобы команда работала?

Подсказка: scp требует указания удаленного хоста.

Ответ:

- **Имя или IP удаленного сервера**
- Флаг `--wal-only`
- `sudo` перед командой
- Параметр `synchronous_commit`

🔗 Шаг 05:

Миссия 5

Ольга хочет ускорить восстановление и добавляет `restore_command='gzip -d < /archive/%f > %p'`. Почему это полезно?

Подсказка: Сжатие снижает нагрузку на диск и сеть.

Ответ:

- **Архив можно хранить в сжатом виде, экономя место**
- Postgres сам сжимает WAL перед записью
- `gzip` заменяет `pg_basebackup`
- Сжатие уменьшает количество транзакций

🔗 Шаг 06:

Миссия 6

Сергей забыл кавычки в `restore_command`. Что произойдет?

Подсказка: `restore_command` должен быть строкой.

Ответ:

- **Postgres не запустится и выдаст ошибку конфигурации**
- Файлы будут восстановлены, но медленнее
- Команда выполнится с `sudo` по умолчанию
- Postgres проигнорирует `restore_command`

🔗 Шаг 07:

Миссия 7

Сопоставьте параметры с их назначением при настройке WAL архивации.

Подсказка: Некоторые параметры управляют логикой копирования, другие — местоположением.

Ответ:

- **`archive_command`** — Команда для сохранения WAL в архив
- **`restore_command`** — Команда для восстановления WAL из архива
- **`archive_timeout`** — Интервал, после которого WAL отправляется в архив даже если не заполнен
- **`archive_mode`** — Флаг, включающий или отключающий архивацию

🔗 Шаг 08:

Миссия 8

Сопоставьте элементы `restore_command` с их значением.

Подсказка: Каждый placeholder подставляется Postgres автоматически.

Ответ:

- **`%f`** — Имя файла WAL, который нужно восстановить

- **%p** — Полный путь, куда скопировать WAL
- **scp** — Передача файлов по сети
- **gzip -d** — Расжатие файлов из архива

🔮 Шаг 09:

Миссия 9

Сопоставьте ситуации с вероятной причиной ошибки при восстановлении WAL.

Подсказка: Ошибки могут быть как в путях, так и в правах.

Ответ:

- **Файл найден, но не читается** — Нет прав доступа
- **restore_command выполняется, но файл пуст** — Ошибка при передаче данных
- **Postgres не находит WAL в архиве** — Неверно указан путь к архиву
- **Ошибка синтаксиса в restore_command** — Пропущены кавычки или аргументы

🔮 Шаг 10:

Миссия 10

Сопоставьте команды с их ролью в сценарии бэкапа и восстановления.

Подсказка: Подумайте, какая команда архивирует, а какая разворачивает данные.

Ответ:

- **pg_basebackup** — Создание базовой копии кластера
- **archive_command** — Сохраняет WAL-файлы в архив
- **restore_command** — Достает WAL из архива при восстановлении
- **pg_ctl start** — Запуск кластера

🔮 Шаг 11:

Миссия 11

Сопоставьте преимущества с технологиями.

Подсказка: Сжатие помогает хранению, а scp — передаче.

Ответ:

- **gzip** — Экономия места за счет сжатия WAL
- **scp** — Передача WAL по сети
- **rsync** — Инкрементальная передача файлов
- **локальное копирование** — Простая и быстрая операция в пределах одного сервера

🔮 Шаг 12:

Миссия 12

Сопоставьте типичные ошибки с их решением.

Подсказка: Иногда достаточно проверить владельца файла, иногда путь.

Ответ:

- **Нет доступа к архиву** — Проверить права пользователя postgres
- **restore_command не выполняется** — Проверить синтаксис команды
- **WAL не находится** — Проверить путь в конфигурации
- **Файл копируется, но поврежден** — Проверить целостность архива

🔧 Шаг 13:

Миссия 13

Расположите шаги настройки архивации WAL в правильном порядке:

Подсказка: Сначала включите режим, потом настройте команду и протестируйте.

Ответ:

- Включить `archive_mode`
- Указать `archive_command`
- Создать директорию архива
- Проверить архивирование с помощью `pg_switch_wal`

🔧 Шаг 14:

Миссия 14

Расположите шаги восстановления реплики с использованием `restore_command`:

Подсказка: Восстановление требует сначала конфигурации, потом запуска.

Ответ:

- Настроить `restore_command` в `postgresql.conf`
- Убедиться, что архив WAL доступен
- Запустить сервер
- Проверить, что реплика воспроизводит WAL

🔧 Шаг 15:

Миссия 15

Расположите шаги тестирования сжатого архива WAL:

Подсказка: Нужно сначала создать архив, потом проверить его чтение.

Ответ:

- Настроить `archive_command` с `gzip`
- Создать архив WAL
- Настроить `restore_command` с `gzip -d`
- Запустить восстановление и проверить

📖 Урок 5 - Промоутинг standby в primary

🔧 Шаг 01:

Миссия 1

Андрей ночью поддерживает PostgreSQL-кластер. Реплика внезапно стала основной, и он хочет убедиться, что команда для промоутинга корректна. Какой инструмент чаще всего используется?

Подсказка: Вспомните стандартную утилиту PostgreSQL для переключения standby в primary.

Ответ:

- **`pg_ctl promote`**
- `pg_upgrade`
- `pg_rewind`
- `pg_restore`

🔧 Шаг 02:

Миссия 2

Марина настраивает standby и хочет подготовить его к возможному промоутингу. Какой файл нужно удалить или игнорировать при превращении standby в primary?

Подсказка: Этот файл блокирует запуск в режиме primary, пока сервер работает как реплика.

Ответ:

- **standby.signal**
- postgresql.auto.conf
- postmaster.pid
- archive_status

🔧 Шаг 03:

Миссия 3

Сергей случайно запустил pg_ctl promote дважды подряд. Что произойдёт?

Подсказка: Подумайте, возможно ли повторное превращение standby в primary.

Ответ:

- База удалится
- **Ничего** — сервер уже primary
- Сервер станет replica снова
- Все подключения будут сброшены

🔧 Шаг 04:

Миссия 4

Компания перешла на режим асинхронной репликации. Что произойдёт при аварийном промоутинге standby?

Подсказка: Подумайте, что может потеряться при асинхронной репликации.

Ответ:

- **Могут потеряться последние неподтверждённые транзакции**
- Произойдёт мгновенное восстановление без потерь
- Сервер заблокируется навсегда
- **Ничего не произойдёт** — всё синхронизируется

🔧 Шаг 05:

Миссия 5

Игорь читал документацию и встретил команду SELECT pg_promote(); . Чем она отличается от pg_ctl promote ?

Подсказка: Вспомните, где именно выполняется SQL-команда и чем отличается от консольной утилиты.

Ответ:

- **Выполняется как SQL-команда внутри БД**
- Удаляет файлы реплики вручную
- Работает только в Windows
- Запускает pg_restore автоматически

🔦 Шаг 06:

Миссия 6

Лена разрабатывает скрипт автоматизации. Ей нужно дождаться завершения промоутинга. Какая опция функции `pg_promote()` позволяет задать таймаут ожидания?

Подсказка: Обратите внимание на аргументы этой функции.

Ответ:

- **wait_seconds**
- `max_connections`
- `standby_timeout`
- `archive_delay`

🔦 Шаг 07:

Миссия 7

📖 Задача: Сопоставьте инструмент и его назначение при промоутинге standby.

Подсказка: Одни команды работают на уровне ОС, другие — внутри PostgreSQL.

Ответ:

- **pg_ctl promote** — Команда ОС для смены standby на primary
- **pg_promote()** — SQL-функция для промоутинга изнутри БД
- **standby.signal** — Файл, наличие которого указывает на режим реплики
- **recovery.signal** — Файл, который используется для запуска в recovery-режиме

🔦 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте риск и его описание при аварийном промоутинге.

Подсказка: Вспомните про синхронный и асинхронный режимы.

Ответ:

- **Async-репликация** — Риск потери последних транзакций
- **Sync-репликация** — Гарантия согласованности между primary и standby
- **Network split** — Ситуация, когда узлы теряют связь
- **Форсированный promote** — Может привести к расхождению данных

🔦 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте понятие и правильное определение в контексте промоутинга.

Подсказка: Представьте себе типичный сценарий отказа кластера.

Ответ:

- **Failover** — Автоматическое переключение standby в primary
- **Switchover** — Плановое переключение ролей
- **Recovery.conf** — Устаревший файл для настройки standby до PostgreSQL 12
- **pg_wal** — Каталог, где хранятся WAL-файлы

🔦 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте команды и их ожидаемый результат.

Подсказка: Думайте о последствиях каждой команды.

Ответ:

- **pg_ctl promote** — Выполняет промоутинг через ОС
- **pg_is_in_recovery()** — Показывает, работает ли сервер как standby
- **pg_wal_replay_resume()** — Возобновляет применение WAL
- **pg_stat_wal_receiver** — Показывает статус приёма WAL

Шаг 11:

Миссия 11

 Задача: Сопоставьте ситуации и действия администратора.

Подсказка: Иногда лучше плановое переключение, иногда аварийное.

Ответ:

- **Плановые работы** — Использовать switchover
- **Сбой primary** — Выполнить promote standby
- **Нужно проверить реплику** — Оставить standby без промоутинга
- **Смена роли для теста** — Временный switchover

Шаг 12:

Миссия 12

 Задача: Сопоставьте термины кластера и их роль при промоутинге.

Подсказка: Термины связаны с режимами standby и primary.

Ответ:

- **Primary** — Основной сервер, принимающий записи
- **Standby** — Реплика, принимающая WAL
- **Promote** — Процесс перевода standby в primary
- **Failback** — Возврат роли к исходному primary

Шаг 13:

Миссия 13

Расставьте шаги промоутинга standby в правильном порядке.

Подсказка: Подумайте, с чего начинается процесс и чем заканчивается.

Ответ:

- Удаление standby.signal
- Выполнение pg_ctl promote или pg_promote()
- Проверка статуса через pg_is_in_recovery()
- Переключение приложений на новый primary

Шаг 14:

Миссия 14

Упорядочьте действия DBA при аварийном failover.

Подсказка: Сначала фиксируется проблема, затем идёт промоутинг, а уже потом настройка клиентов.

Ответ:

- Обнаружение недоступности primary

- Выполнение promote на standby
- Обновление конфигурации приложений
- Проверка репликации и логов

💡 Шаг 15:

Миссия 15

Расположите шаги при switchover (плановое переключение).

Подсказка: Тут оба узла координируются.

Ответ:

- Остановить записи на primary
- Выполнить promote standby
- Перенастроить старый primary в standby
- Возобновить работу приложений

📖 Урок 6 - Отслеживание репликации и задержки

💡 Шаг 01:

Миссия 1

Программист Олег заметил, что его standby-узел отстаёт от primary. Какой системный вью позволяет быстро оценить статус репликации?

Подсказка: В PostgreSQL есть специальный view, в котором можно увидеть все процессы репликации.

Ответ:

- **pg_stat_replication**
- pg_stat_activity
- pg_locks
- pg_stat_wal_receiver

💡 Шаг 02:

Миссия 2

Аня пишет дашборд для мониторинга задержки. Какое поле в pg_stat_replication покажет реальное отставание standby от primary?

Подсказка: Есть несколько lag-полей, но одно отвечает именно за задержку в доставке WAL на standby.

Ответ:

- **write_lag**
- state
- flush_location
- replay_lsn

💡 Шаг 03:

Миссия 3

Дмитрий тестирует задержку на standby. Какой SQL-запрос поможет оценить разницу между текущим LSN на primary и LSN на standby?

Подсказка: Сравните функции, которые возвращают текущий LSN для записи и воспроизведения.

Ответ:

- **SELECT pg_current_wal_lsn() - pg_last_wal_replay_lsn();**
- SELECT * FROM pg_stat_activity;
- SELECT txid_current();
- SELECT * FROM pg_stat_database;

🔗 **Шаг 04:**

Миссия 4

Ивану нужно понять, в каком состоянии находится поток репликации. Какое значение поля state в pg_stat_replication говорит о том, что standby воспроизводит WAL?

Подсказка: Подумайте, какое слово лучше всего описывает процесс применения WAL-записей.

Ответ:

- **streaming**
- startup
- catchup
- idle

🔗 **Шаг 05:**

Миссия 5

Маша заметила, что standby иногда сильно отстаёт. Какой параметр конфигурации влияет на то, как быстро standby получает WAL-записи?

Подсказка: Этот параметр управляет частотой сброса WAL на диск primary.

Ответ:

- **wal_writer_delay**
- shared_buffers
- work_mem
- max_connections

🔗 **Шаг 06:**

Миссия 6

Разработчик Стас хочет знать, какая функция на standby показывает последний применённый LSN.

Подсказка: Слово replay тут ключевое.

Ответ:

- **pg_last_wal_replay_lsn()**
- pg_current_wal_insert_lsn()
- pg_switch_wal()
- pg_is_in_recovery()

🔗 **Шаг 07:**

Миссия 7

Инженер Василий анализирует графики репликации. Какое поле в pg_stat_replication показывает задержку между flush и replay?

Подсказка: Это одно из lag-полей, связанное именно с применением WAL.

Ответ:

- **replay_lag**
- **write_lsn**
- **sent_lsn**
- **backend_xid**

🔮 Шаг 08:

Миссия 8

📖 Сопоставьте метрики репликации с их описанием.

Подсказка: Подумайте: одни поля отвечают за отправку WAL, другие за запись, третьи — за воспроизведение.

Ответ:

- **sent_lsn** — LSN, до которого WAL был отправлен standby.
- **write_lsn** — LSN, до которого WAL был записан на диск standby.
- **flush_lsn** — LSN, подтверждённый как сброшенный на диск standby.
- **replay_lsn** — LSN, до которого WAL был применён на standby.

🔮 Шаг 09:

Миссия 9

📖 Сопоставьте функции PostgreSQL с их назначением при мониторинге задержки.

Подсказка: Вспомните, какие функции работают на primary, а какие на standby.

Ответ:

- **pg_current_wal_lsn()** — Показывает текущую точку генерации WAL на primary.
- **pg_last_wal_receive_lsn()** — Возвращает последний LSN, полученный standby.
- **pg_last_wal_replay_lsn()** — Показывает LSN последнего применённого WAL на standby.
- **pg_is_in_recovery()** — Проверяет, находится ли узел в режиме standby.

🔮 Шаг 10:

Миссия 10

📖 Сопоставьте состояния репликации с их смыслом.

Подсказка: Каждое состояние отражает стадию работы standby.

Ответ:

- **startup** — Standby запускается и подключается к источнику WAL.
- **catchup** — Standby догоняет основной сервер по WAL.
- **streaming** — Standby получает WAL в режиме реального времени.
- **backup** — Используется при создании базовой копии для репликации.

🔮 Шаг 11:

Миссия 11

📖 Сопоставьте задержки с их объяснением.

Подсказка: Разные lag показывают разные этапы доставки и применения WAL.

Ответ:

- **write_lag** — Задержка между получением WAL и его записью на диск standby.
- **flush_lag** — Задержка между записью и сбросом WAL на диск standby.

- **replay_lag** — Задержка между сбросом WAL и его применением на standby.
- **sync_lag** — Искусственная задержка для синхронной репликации (standby ждёт подтверждения).

🔧 Шаг 12:

Миссия 12

📖 Сопоставьте инструменты мониторинга с их назначением.

Подсказка: Одни инструменты встроены в PostgreSQL, другие внешние.

Ответ:

- **pg_stat_replication** — Показывает активные соединения репликации и их статус.
- **pg_stat_wal_receiver** — Дает информацию о процессе получения WAL на standby.
- **pg_stat_activity** — Показывает общую активность соединений.
- **Prometheus + exporters** — Собирает метрики PostgreSQL и репликации для внешнего мониторинга.

🔧 Шаг 13:

Миссия 13

Расположите шаги получения WAL на standby в правильном порядке:

Подсказка: Сначала WAL отправляется, потом пишется, сбрасывается и воспроизводится.

Ответ:

- WAL отправляется с primary (sent_lsn)
- WAL записывается на диск standby (write_lsn)
- WAL сбрасывается на диск (flush_lsn)
- WAL применяется (replay_lsn)

🔧 Шаг 14:

Миссия 14

Расположите шаги анализа задержки между primary и standby:

Подсказка: Сначала проверяем генерацию WAL, затем получение, запись и воспроизведение.

Ответ:

- Проверка pg_current_wal_lsn()
- Сравнение с pg_last_wal_receive_lsn()
- Сравнение с pg_last_wal_replay_lsn()
- Оценка разницы (lag)

🔧 Шаг 15:

Миссия 15

Расположите этапы настройки внешнего мониторинга репликации:

Подсказка: Сначала настраиваем сбор данных, потом хранение, затем визуализацию.

Ответ:

- Установка Postgres exporter
- Сбор метрик о репликации
- Передача данных в Prometheus
- Отображение графиков в Grafana

Модуль 3 - Логическая репликация — контроль частиц

Урок 1 - Разница между логической и физической репликацией

Шаг 01:

Миссия 1

Анна, DBA, разрабатывает стратегию репликации для новой базы. Что из нижеперечисленного является ****ключевым отличием логической репликации**** по сравнению с физической?

Подсказка: Логическая репликация оперирует на уровне реляционных объектов.

Ответ:

- **Реплицируются отдельные таблицы и данные на уровне SQL**
- Копируется весь файловый образ WAL-файлов
- Работает только на уровне блоков файловой системы
- Нельзя селектить данные на подписчике

Шаг 02:

Миссия 2

Максим подключает новый подписчик к логической репликации. Какой объект обязательно должен существовать на публикации?

Подсказка: Публикация объявляет, что именно будет реплицироваться.

Ответ:

- **Публикация (publication)**
- WAL-поток
- Точка восстановления (restore point)
- Слепок файловой системы

Шаг 03:

Миссия 3

Во внутренней встрече команда обсуждает: можно ли использовать логическую репликацию для схем, которые ещё не существуют на подписчике? Что верно?

Подсказка: Подписчик должен знать таблицу до получения данных.

Ответ:

- **На подписчике схема и таблица должны быть созданы заранее**
- Подписчик сам создаёт схему и таблицу
- Подписчик игнорирует структуры
- Подписчик создаёт только индексы

Шаг 04:

Миссия 4

Во время демонстрации Ирина говорит, что физическая репликация обеспечивает ****полную копию кластера****. Почему это преимущество одновременно может быть недостатком?

Подсказка: Полная копия – это и большой объём данных.

Ответ:

- **Избыточно реплицировать неиспользуемые данные**
- Нельзя читать на подписчике

- Нельзя делать failover
- Работает только с таблицами

🔗 Шаг 05:

Миссия 5

Олег хочет реплицировать только одну таблицу с логами. Какой способ более естествен в PostgreSQL под нагрузкой?

Подсказка: Логическая репликация позволяет фильтрацию.

Ответ:

- **Использовать логическую репликацию с публикацией конкретных таблиц**
- Использовать физическую репликацию и игнорировать остальные таблицы
- Ручной экспорт/импорт
- Снимок файловой системы

🔗 Шаг 06:

Миссия 6

Команда сомневается: можно ли применять логическую репликацию между различными версиями PostgreSQL (например, с 12 на 15)? Что верно?

Подсказка: Логическая репликация использует SQL-уровень, не низкоуровневые форматы.

Ответ:

- **Да, можно, если версии поддерживают совместимый логический протокол**
- Нет, версии должны совпадать
- Можно только если версии отличаются на одну мажорную
- Логическое обновится само

🔗 Шаг 07:

Миссия 7

Сопоставьте понятие и характеристику репликаций (логической и физической):

Подсказка: Думайте, на каком уровне выполняется копирование.

Понятия:

Ответ:

- Физическая репликация
- Логическая репликация
- Передача WAL-журналов
- Репликация конкретных таблиц

Определения:

1. Копируется бинарный образ из файлов PostgreSQL
2. Работает через трансляцию изменений DML/DDDL
3. Нельзя фильтровать таблицы или схемы
4. Можно выборочно реплицировать лишь часть данных

- **Физическая репликация** — Копируется бинарный образ из файлов PostgreSQL
- **Логическая репликация** — Работает через трансляцию изменений DML/DDDL
- **Передача WAL-журналов** — Нельзя фильтровать таблицы или схемы

- **Репликация конкретных таблиц** — Можно выборочно реплицировать лишь часть данных

Шаг 08:

Миссия 8

Сопоставьте ситуацию из жизни со свойством логической/физической репликации:

Подсказка: Вспомните, что делал герой в реальной истории.

Сценарии:

Ответ:

- Подписчик читает данные, не влияя на мастер
- Подписчик точно идентичен мастеру в блоках
- Нельзя опускать столбцы при репликации
- Допустимо добавлять новые колонки на подписчике

Свойства:

1. Характерно для логической репликации
 2. Характерно для физической репликации
 3. Ограничение физической репликации
 4. Допустимо в логической репликации
- **Подписчик читает данные, не влияя на мастер** — Характерно для логической репликации
 - **Подписчик точно идентичен мастеру в блоках** — Характерно для физической репликации
 - **Нельзя опускать столбцы при репликации** — Ограничение физической репликации
 - **Допустимо добавлять новые колонки на подписчике** — Допустимо в логической репликации

Шаг 09:

Миссия 9

Рассортируйте этапы настройки логической репликации от мастера к подписчику:

1. Создать подписку на подписчике
2. Настроить публикацию на мастере и выбрать таблицы
3. Убедиться, что схемы и таблицы присутствуют на подписчике
4. Запустить подписку и начать синхронизацию

Подсказка: Подумайте, что надо подготовить до подключения.

Ответ:

- Создать подписку на подписчике
- Настроить публикацию на мастере и выбрать таблицы
- Убедиться, что схемы и таблицы присутствуют на подписчике
- Запустить подписку и начать синхронизацию

Шаг 10:

Миссия 10

У Веры на проекте несколько подписчиков. Расположите шаги failover от логического мастера к новому мастеру:

1. Отключить подписки на старом мастере
2. Перенастроить публикацию/подписку, чтобы новый мастер был источником
3. Обновить подписчики, чтобы читали с нового мастера
4. Синхронизировать изменения между старым мастером и новым

Подсказка: Подумайте, как переключить потоки с минимальным простоем.

Ответ:

- Отключить подписки на старом мастере
- Перенастроить публикацию/подписку, чтобы новый мастер был источником
- Обновить подписчики, чтобы читали с нового мастера
- Синхронизировать изменения между старым мастером и новым

Шаг 11:

Миссия 11

Сергей готовит подробную инструкцию. Упорядочите шаги сравнения физической и логической репликации по показателю «гибкость»:

1. Узнать, можно ли фильтровать таблицы или схемы
2. Узнать, можно ли изменять структуру подписчика
3. Узнать, можно ли читать с подписчика
4. Оценить полную копию или выборочную копию данных

Подсказка: Гибкость — способность изменять, выбирать, фильтровать.

Ответ:

- Узнать, можно ли фильтровать таблицы или схемы
- Узнать, можно ли изменять структуру подписчика
- Узнать, можно ли читать с подписчика
- Оценить полную копию или выборочную копию данных

Шаг 12:

Миссия 12

Юля сравнивает шаги запуска физической и логической репликации. Расположите действия в порядке, общий для логической:

1. Настроить публикацию на мастере
2. Настроить приемник подписчика (subscription)
3. Синхронизировать существующие данные (copy) или snapshot
4. Проследить поток изменений в режиме реального времени

Подсказка: Логическая репликация подразумевает snapshot + поток DML.

Ответ:

- Настроить публикацию на мастере
- Настроить приемник подписчика (subscription)
- Синхронизировать существующие данные (copy) или snapshot
- Проследить поток изменений в режиме реального времени

Шаг 13:

Миссия 13

Во время аудита Павел хочет сравнить порядок действий failover для физической репликации. Упорядочите шаги:

1. Переключить слейв в мастер-режим
2. Остановить старый мастер
3. Обновить точки восстановления (recovery.conf или настройку)
4. Перенастроить слейв как новый мастер

Подсказка: Физический failover — низкоуровневый.

Ответ:

- Переключить слейв в мастер-режим
- Остановить старый мастер
- Обновить точки восстановления (recovery.conf или настройку)
- Перенастроить слейв как новый мастер

Шаг 14:

Миссия 14

Итоговый квест: расположите этапы принятия решения, какую репликацию выбрать (логическую или физическую) под нагрузкой:

1. Оценить необходимость фильтрации таблиц и схем
2. Оценить нагрузку на сеть и объём данных
3. Оценить гибкость в изменении схемы подписчика
4. Сравнить требования к идентичности данных и доступности

Подсказка: Решение — не просто техническое, а баланс требований.

Ответ:

- Оценить необходимость фильтрации таблиц и схем
- Оценить нагрузку на сеть и объём данных
- Оценить гибкость в изменении схемы подписчика
- Сравнить требования к идентичности данных и доступности

Урок 2 - Подготовка БД к логической репликации

Шаг 01:

Миссия 1

Вы — инженер DBA, и ваш коллега попросил вас включить логическую репликацию на новом сервере PostgreSQL. Что нужно сделать в первую очередь?

Подсказка: без соответствующих параметров в конфигурации сам репликационный механизм не запустится.

Ответ:

- **Добавить настройки wal_level и max_replication_slots**
- Запустить pg_dump всех баз
- Сразу создать подписки на реплики
- Отключить все соединения к базе

🔧 Шаг 02:

Миссия 2

Вы уже изменили конфигурацию PostgreSQL (postgresql.conf). Что далее необходимо, чтобы изменения вступили в силу?

Подсказка: сервер должен прочитать обновлённый файл.

Ответ:

- **Перезапустить сервер PostgreSQL**
- Выполнить VACUUM FULL
- Создать репликационные слоты
- Запустить подписку

🔧 Шаг 03:

Миссия 3

При попытке создать публикацию вы получаете ошибку: «could not find replication slot». Что, вероятно, забыли сделать?

Подсказка: публикация пользуется слотом для логической репликации.

Ответ:

- **Не создан репликационный слот (CREATE REPLICATION SLOT)**
- Не настроен pg_hba.conf
- Не запущен сервер
- Не установлен pglogical

🔧 Шаг 04:

Миссия 4

Перед включением логической репликации вы хотите удостовериться, что база готова. Какой шаг будет полезным?

Подсказка: подумайте об ожидании устойчивости схем и данных.

Ответ:

- **Выполнить pg_dump и сравнить контрольные суммы**
- Отключить все активные транзакции
- Удалить все фоновые задачи
- Включить synchronous_commit = on

🔧 Шаг 05:

Миссия 5

Сопоставьте параметр конфигурации PostgreSQL со значением, часто используемым при логической репликации:

Параметры:

Ответ:

- wri_level
- max_replication_slots
- max_wal_senders
- wal_level

Рекомендуемые значения / описания:

1. **logical** — чтобы включить логическую репликацию
2. не менее 1 (то есть достаточное число слотов)
3. количество одновременных отправителей WAL
4. общий уровень журналирования WAL (например, minimal, replica, logical)

Подсказка: подумайте: wal_level управляет типом репликации, max_wal_senders — число клиентов, max_replication_slots — слоты.

- **wal_level** — logical — чтобы включить логическую репликацию
- **max_replication_slots** — не менее 1 (то есть достаточное число слотов)
- **max_wal_senders** — количество одновременных отправителей WAL
- **wri_level** — общий уровень журналирования WAL (например, minimal, replica, logical)

Шаг 06:

Миссия 6

Сопоставьте файл / таблицу с его ролью при подготовке к логической репликации:

Объекты:

Ответ:

- postgresql.conf
- pg_hba.conf
- pg_ident.conf
- pg_replication_slot (каталог)

Роли / описания:

1. Здесь разрешаются подключения от хостов / пользователей
2. Конфигурация основных параметров WAL и репликации
3. Переадресация идентификаторов (непосредственно не связана с репликацией)
4. Хранилище слотов логической репликации

Подсказка: вспомните, где настраивается доступ, а где — операционные параметры.

- **postgresql.conf** — Конфигурация основных параметров WAL и репликации
- **pg_hba.conf** — Здесь разрешаются подключения от хостов / пользователей
- **pg_ident.conf** — Переадресация идентификаторов (непосредственно не связана с репликацией)
- **pg_replication_slot (каталог)** — Хранилище слотов логической репликации

Шаг 07:

Миссия 7

Сопоставьте состояние базы (до репликации) с задачей, которую нужно выполнить:

Состояния:

Ответ:

- Активные DDL-операции выполняются часто
- База с интенсивными обновлениями строк
- Низкий объем WAL
- Многие мелкие транзакции

Необходимые подготовки:

1. Убедиться, что репликация выбранных таблиц поддерживает DDL
2. Настроить достаточный общий размер WAL буферов и параметров чекпойнта
3. Установить параметры `wal_segment_size` и `checkpoint_segments` (или их аналоги)
4. Оценить нагрузку на I/O и убедиться, что подписчики успевают догонять

Подсказка: подумайте, какие настройки влияют на скорость записи WAL и нагрузку на диск.

- **Активные DDL-операции выполняются часто** — Убедиться, что репликация выбранных таблиц поддерживает DDL
- **База с интенсивными обновлениями строк** — Настроить достаточный общий размер WAL буферов и параметров чекпойнта
- **Низкий объем WAL** — Установить параметры `wal_segment_size` и `checkpoint_segments` (или их аналоги)
- **Многие мелкие транзакции** — Оценить нагрузку на I/O и убедиться, что подписчики успевают догонять

🔗 Шаг 08:

Миссия 8

Сопоставьте шаг подготовки базы к логической репликации с его описанием:

Шаги:

Ответ:

- Отключить репликацию системных каталогов
- Убедиться, что публикации не конфликтуют
- Сделать резервную копию схемы
- Проверить права пользователя репликации

Описания:

1. Пользователь должен иметь REPLICATION-привилегии и подходящий доступ через `pg_hba.conf`
2. Необходимо исключить нежелательные таблицы из публикации
3. Публикации не должны пересекаться по объектам
4. Сохранить состояние схемы до репликации для проверки

Подсказка: подумайте, какая репликация включает системные каталоги, и зачем копия схемы.

- **Проверить права пользователя репликации** — Пользователь должен иметь REPLICATION-привилегии и подходящий доступ через `pg_hba.conf`
- **Отключить репликацию системных каталогов** — Необходимо исключить нежелательные таблицы из публикации
- **Убедиться, что публикации не конфликтуют** — Публикации не должны пересекаться по объектам
- **Сделать резервную копию схемы** — Сохранить состояние схемы до репликации для проверки

🔗 Шаг 09:

Миссия 9

Сопоставьте термин с его ролью в логической репликации:

Термины:

Ответ:

- Публикация
- Подписка
- Логический декодер
- Слот репликации

Роли / описания:

1. Определяет, какие изменения (таблицы) отправлять
2. Подключается к публикации и принимает изменения
3. Интерфейс чтения WAL в виде логических изменений
4. Хранит состояние точки чтения WAL на стороне сервера-источника

Подсказка: подумайте, кто что делает: публикация — отправитель, подписка — получатель.

- **Публикация** — Определяет, какие изменения (таблицы) отправлять
- **Подписка** — Подключается к публикации и принимает изменения
- **Логический декодер** — Интерфейс чтения WAL в виде логических изменений
- **Слот репликации** — Хранит состояние точки чтения WAL на стороне сервера-источника

 Шаг 10:

Миссия 10

Сопоставьте потенциальную проблему подготовки к репликации с рекомендацией:

Проблемы:

Ответ:

- Недостаточное количество слотов
- Неправильные права доступа
- Высокий уровень DDL изменений
- Конфликт публикаций

Рекомендации:

1. Добавить или расширить max_replication_slots
2. Настроить роли и записи в pg_hba.conf
3. Включать только безопасные изменения или отдельные таблицы
4. Пересмотреть публикации, чтобы не пересекались объекты

Подсказка: каждая проблема напрямую связана с исключением или правами или пересечением объектов.

- **Недостаточное количество слотов** — Добавить или расширить max_replication_slots
- **Неправильные права доступа** — Настроить роли и записи в pg_hba.conf
- **Высокий уровень DDL изменений** — Включать только безопасные изменения или отдельные таблицы
- **Конфликт публикаций** — Пересмотреть публикации, чтобы не пересекались объекты

 Шаг 11:

Миссия 11

Расположите шаги подготовки к логической репликации в правильном порядке:

1. Настроить pg_hba.conf и права доступа
2. Изменить postgresql.conf (wal_level, слоты и пр.)

3. Создать публикацию на мастере
4. Проверить готовность схем / выполнить резервную копию
5. Создать подписку на реплику

Подсказка: сначала конфигурация доступа, потом WAL и слоты, затем публикация и подписка.

Ответ:

- Настроить `pg_hba.conf` и права доступа
- Изменить `postgresql.conf` (`wal_level`, слоты и пр.)
- Проверить готовность схем / выполнить резервную копию
- Создать публикацию на мастере
- Создать подписку на реплику

Шаг 12:

Миссия 12

Расположите этапы проверки перед запуском логической репликации:

1. Убедиться, что таблицы не находятся под heavy lock
2. Проверить несоответствие схем между узлами
3. Проверить, что пользователь имеет репликационные права
4. Закрыть все активные транзакции на момент старта
5. Сделать тестовый дамп / контрольную сумму

Подсказка: сначала права и схемы, потом состояние транзакций и контрольные проверки.

Ответ:

- Убедиться, что таблицы не находятся под heavy lock
- Проверить несоответствие схем между узлами
- Проверить, что пользователь имеет репликационные права
- Закрыть все активные транзакции на момент старта
- Сделать тестовый дамп / контрольную сумму

Шаг 13:

Миссия 13

Расположите шаги после создания публикации, но до подписки:

1. Проверить свободные слоты и ресурсы
2. Проверить, что слоты созданы и в списке репликации
3. Обновить `pg_hba.conf` на стороне подписчика
4. Проверить сетевое соединение между узлами
5. Протестировать подключение с правами репликации

Подсказка: сначала проверка слотов, затем права доступа, потом подключение и сеть.

Ответ:

- Проверить свободные слоты и ресурсы
- Проверить, что слоты созданы и в списке репликации
- Обновить `pg_hba.conf` на стороне подписчика
- Проверить сетевое соединение между узлами
- Протестировать подключение с правами репликации

Шаг 14:

Миссия 14

Расположите шаги, ведущие к активации подписки:

1. Запрос данных из публикации (синхронизация initial copy)
2. Убедиться, что база подписчика пуста или готова
3. Запустить подписку
4. Отключить вспомогательные блокировки на подписчике
5. Включить репликацию потоков изменений (apply)

Подсказка: сначала база должна быть готова, потом запуск подписки, затем данные и поток изменений.

Ответ:

- Убедиться, что база подписчика пуста или готова
- Запустить подписку
- Запрос данных из публикации (синхронизация initial copy)
- Отключить вспомогательные блокировки на подписчике
- Включить репликацию потоков изменений (apply)

Шаг 15:

Миссия 15

Расположите контрольные шаги после запуска подписки:

1. Проверить задержку подписки (lag)
2. Сравнить контрольные суммы между мастер-базой и подписчиком
3. Проверить логи на ошибки репликации
4. Проверить, что публикация продолжает работать корректно
5. Проанализировать ресурсы (I/O, CPU) на подписчике

сначала проверка ошибок и статус, затем сравнение данных и ресурсы.

Ответ:

- Проверить логи на ошибки репликации
- Проверить задержку подписки (lag)
- Сравнить контрольные суммы между мастер-базой и подписчиком
- Проверить, что публикация продолжает работать корректно
- Проанализировать ресурсы (I/O, CPU) на подписчике

Шаг 16:

Миссия 16

Расположите полную последовательность подготовки к логической репликации:

1. Настроить доступы (pg_hba.conf, привилегии)
2. Настроить postgresql.conf (wal_level, слоты, wal_senders и др.)
3. Проверить схемы и права, сделать резервную копию
4. Создать публикацию на мастере
5. Проверить слоты, сети, подключение
6. Создать подписку на реплику
7. Запустить синхронизацию и применить поток изменений
8. Проверить логи, задержку, целостность и ресурсы

Подсказка: объёмная задача — объедините все предыдущие шаги в один сценарий.

Ответ:

- Настроить доступы (pg_hba.conf, привилегии)
- Настроить postgresql.conf (wal_level, слоты, wal_senders и др.)
- Проверить схемы и права, сделать резервную копию
- Создать публикацию на мастере
- Проверить слоты, сети, подключение
- Создать подписку на реплику
- Запустить синхронизацию и применить поток изменений
- Проверить логи, задержку, целостность и ресурсы

Урок 3 - Создание публикаций и подписок

Шаг 01:

Миссия 1

Аня, DBA, готовится создать первую публикацию. Что должно существовать на главном узле перед созданием публикации?

Подсказка: без репликационного слота публикация не сможет читать WAL.

Ответ:

- **Репликационный слот, настроенный сервер и WAL уровень**
- Подписка на другом узле
- Полная резервная копия на другом сервере
- Синхронизация с физическим слейвом

Шаг 02:

Миссия 2

Когда вы выполняете `CREATE PUBLICATION my_pub FOR TABLE users, orders;`, и получаете ошибку «permission denied», что, скорее всего, забыли настроить?

Подсказка: публикации создаются от имени пользователя; он должен иметь нужные права.

Ответ:

- **Не дал права SELECT на таблицы пользователю публикации**
- Не создал слот
- Не включил логическую репликацию в конфигурации
- Не запустил подписчика

Шаг 03:

Миссия 3

Иван настроил публикацию с `FOR ALL TABLES`, но некоторые таблицы не реплицируются. Что могло быть причиной?

Подсказка: публикация FOR ALL TABLES не затрагивает таблицы без права доступа или в схемах без включения.

Ответ:

- **У пользователя публикации нет доступа к этим таблицам**
- Таблицы созданы после публикации и не добавлены автоматически
- Сервера не синхронизированы физически
- Подписка была создана до публикации

🔗 Шаг 04:

Миссия 4

Пётр хочет реплицировать не все таблицы, а только некоторые колонки (subset). Можно ли это сделать через стандартную публикацию?

Подсказка: логическая репликация PostgreSQL не поддерживает фильтрацию колонок (на уровне ядра).

Ответ:

- **Нет, нужно публиковать всю строку и фильтровать на уровне подписчика**
- **Да** — через параметр COLUMNS

🔗 Шаг 05:

Миссия 5

Сопоставьте опцию публикации с её поведением:

Опции публикации:

Ответ:

- FOR ALL TABLES
- FOR TABLE table1, table2
- FOR ALL TABLES WITH (publish = 'insert, update')
- FOR TABLE ... WITH (publish = 'delete')

Поведение:

1. Реплицировать вставки, обновления и удаления для всех таблиц
2. Реплицировать только указанные таблицы со всеми действиями
3. Реплицировать только insert и update во всех таблицах
4. Реплицировать только удаление для указанных таблиц

Подсказка: WITH (publish = ...) ограничивает операции, FOR ALL означает все таблицы.

- **FOR ALL TABLES** — Реплицировать вставки, обновления и удаления для всех таблиц
- **FOR TABLE table1, table2** — Реплицировать только указанные таблицы со всеми действиями
- **FOR ALL TABLES WITH (publish = 'insert, update')** — Реплицировать только insert и update во всех таблицах
- **FOR TABLE ... WITH (publish = 'delete')** — Реплицировать только удаление для указанных таблиц

🔗 Шаг 06:

Миссия 6

Сопоставьте параметр подписки с его назначением:

Параметры подписки:

Ответ:

- copy_data = true
- copy_data = false
- slot_name = '...'
- connect = '...' (коннекشن строка)

Назначения:

1. Не копировать начальные данные при создании подписки
2. Имя слота репликации на сервере источника
3. Копировать начальные данные из публикации
4. Строка подключения к главному серверу

Подсказка: `copy_data` управляет `initial data`, `slot_name` — слот, `connect` — связь.

- **`copy_data = true`** — Копировать начальные данные из публикации
- **`copy_data = false`** — Не копировать начальные данные при создании подписки
- **`slot_name = '...'`** — Имя слота репликации на сервере источника
- **`connect = '...' (коннекшн строка)`** — Строка подключения к главному серверу

Шаг 07:

Миссия 7

Сопоставьте состояние мониторинга после создания подписки с тем, что нужно проверить:

Состояния / симптомы:

Ответ:

- Подписка находится в состоянии “`сору`”
- Подписка активна, но нет данных
- Подписка завершила `initial сору` и идет `apply`
- Подписка выдает ошибку соединения

Что проверить:

1. Проверьте логи и `network, credentials`
2. Проверьте наличие данных и фильтры публикации/подписки
3. Наличие таблиц/схем на подписчике
4. Состояние `copy_data` и слот в `use`

Подсказка: разные стадии подписки требуют своих проверок.

- **Подписка находится в состоянии “`сору`”** — Состояние `copy_data` и слот в `use`
- **Подписка активна, но нет данных** — Проверьте наличие данных и фильтры публикации/подписки
- **Подписка завершила `initial сору` и идет `apply`** — Наличие таблиц/схем на подписчике
- **Подписка выдает ошибку соединения** — Проверьте логи и `network, credentials`

Шаг 08:

Миссия 8

Сопоставьте проблему при публикации / подписке с возможным решением:

Проблемы:

Ответ:

- Подписка застряла в стадии `сору` навсегда
- Пользователь репликации не может подключиться
- Удаления не реплицируются
- Подписчик получает несоответствие схемы

Решения:

1. Проверить правило `connect / pg_hba / сеть`

2. Убедиться, что publication включает DELETE (publish = 'delete')
3. Сравнить структуру схем и добавить missing columns
4. Сделать copy_data = true или проверить initial snapshot

Подсказка: подбирайте решение под характер проблемы.

- **Подписка застряла в стадии copy навсегда** — Сделать copy_data = true или проверить initial snapshot
- **Пользователь репликации не может подключиться** — Проверить правило connect / pg_hba / сеть
- **Удаления не реплицируются** — Убедиться, что publication включает DELETE (publish = 'delete')
- **Подписчик получает несоответствие схемы** — Сравнить структуру схем и добавить missing columns

Шаг 09:

Миссия 9

Сопоставьте команду SQL с эффектом при работе с подпиской:

Команды:

Ответ:

- ALTER SUBSCRIPTION my_sub ENABLE
- ALTER SUBSCRIPTION my_sub DISABLE
- ALTER SUBSCRIPTION my_sub REFRESH PUBLICATION
- DROP SUBSCRIPTION my_sub

Эффекты:

1. Подписка перестаёт принимать изменения (пауза apply)
2. Перестроить подписку в соответствии с текущими публикациями
3. Удалить подписку и остановить поток
4. Включить приостановленную подписку

Подсказка: ENABLE / DISABLE — активация / деактивация, REFRESH — пересинхронизация.

- **ALTER SUBSCRIPTION my_sub ENABLE** — Включить приостановленную подписку
- **ALTER SUBSCRIPTION my_sub DISABLE** — Подписка перестаёт принимать изменения (пауза apply)
- **ALTER SUBSCRIPTION my_sub REFRESH PUBLICATION** — Перестроить подписку в соответствии с текущими публикациями
- **DROP SUBSCRIPTION my_sub** — Удалить подписку и остановить поток

Шаг 10:

Миссия 10

Расположите шаги создания публикации и подписки:

1. CREATE PUBLICATION на мастере
2. Настроить pg_hba.conf / права
3. CREATE SUBSCRIPTION на реплике
4. Проверить соединение и доступы
5. Убедиться, что слот существует

Подсказка: сначала доступ, потом публикация, потом подписка с проверкой.

Ответ:

- Настроить pg_hba.conf / права
- CREATE PUBLICATION на мастере
- Убедиться, что слот существует
- Проверить соединение и доступы
- CREATE SUBSCRIPTION на реплике

 Шаг 11:**Миссия 11**

Расположите этапы после создания подписки:

1. Начальный копий данных (initial copy)
2. Проверка логов подписчика на ошибки
3. Проверка задержки (lag) подписки
4. Сопоставление данных между мастер/реплика
5. Переход к применению изменений (apply)

Подсказка: сначала copy, затем проверка, затем apply и контроль.

Ответ:

- Начальный копий данных (initial copy)
- Проверка логов подписчика на ошибки
- Проверка задержки (lag) подписки
- Сопоставление данных между мастер/реплика
- Переход к применению изменений (apply)

 Шаг 12:**Миссия 12**

Расположите шаги при ошибке несоответствия схемы:

1. Остановить подписку (DISABLE)
2. Добавить отсутствующие столбцы / индекс на подписчике
3. Выполнить ALTER TABLE / DDL на подписчике
4. REFRESH PUBLICATION / подписки
5. ENABLE подписку

Подсказка: при ошибке схемы надо внести правки, затем пересинхронизировать.

Ответ:

- Остановить подписку (DISABLE)
- Добавить отсутствующие столбцы / индекс на подписчике
- Выполнить ALTER TABLE / DDL на подписчике
- REFRESH PUBLICATION / подписки
- ENABLE подписку

 Шаг 13:**Миссия 13**

Расположите шаги, если подписка работает, но данные не приходят:

1. Проверить наличие фильтров публикации / подписки
2. Проверить права доступа / pg_hba

3. Проверить логи и соединение
4. REFRESH PUBLICATION / подписки
5. ENABLE подписку, если была отключена

Подсказка: диагностика: фильтры, доступ, сеть, пересинхронизация.

Ответ:

- Проверить наличие фильтров публикации / подписки
- Проверить права доступа / pg_hba
- Проверить логи и соединение
- REFRESH PUBLICATION / подписки
- ENABLE подписку, если была отключена

 Шаг 14:

Миссия 14

Расположите шаги при изменении публикации (добавление таблицы) после работы подписки:

1. ALTER PUBLICATION ADD TABLE new_table
2. REFRESH PUBLICATION на подписчике
3. ALTER SUBSCRIPTION my_sub REFRESH PUBLICATION
4. Проверить, что новая таблица появилась на подписчике
5. Проверить данные / целостность по новой таблице

Подсказка: добавление таблицы требует обновления подписки и проверок.

Ответ:

- ALTER PUBLICATION ADD TABLE new_table
- REFRESH PUBLICATION на подписчике
- ALTER SUBSCRIPTION my_sub REFRESH PUBLICATION
- Проверить, что новая таблица появилась на подписчике
- Проверить данные / целостность по новой таблице

 Шаг 15:

Миссия 15

Расположите шаги при удалении таблицы из публикации:

1. ALTER PUBLICATION DROP TABLE old_table
2. ALTER SUBSCRIPTION my_sub REFRESH PUBLICATION
3. Проверить, что подписчик перестал получать изменения по этой таблице
4. Удалить таблицу вручную на подписчике (если нужно)
5. Проверить, что подписка работает без ошибок

Подсказка: удаление таблицы требует обновления и проверки, как и добавление.

Ответ:

- ALTER PUBLICATION DROP TABLE old_table
- ALTER SUBSCRIPTION my_sub REFRESH PUBLICATION
- Проверить, что подписчик перестал получать изменения по этой таблице
- Удалить таблицу вручную на подписчике (если нужно)
- Проверить, что подписка работает без ошибок

Шаг 16:

Миссия 16

Расположите полную последовательность команд и проверок для создания и проверки публикации/подписки:

1. Настроить доступы / pg_hba / права
2. Создать репликационный слот
3. CREATE PUBLICATION с нужными таблицами
4. CREATE SUBSCRIPTION с правильной строкой подключения
5. Проверить стадию сору и логи
6. **Когда сору завершится** — перейти к apply
7. Проверить данные / целостность / задержку
8. По необходимости обновлять публикацию / подписку

Подсказка: это финальный сценарий с диагностикой и шагами модификации.

Ответ:

- Настроить доступы / pg_hba / права
- Создать репликационный слот
- CREATE PUBLICATION с нужными таблицами
- CREATE SUBSCRIPTION с правильной строкой подключения
- Проверить стадию сору и логи
- **Когда сору завершится** — перейти к apply
- Проверить данные / целостность / задержку
- По необходимости обновлять публикацию / подписку

Урок 4 - Репликация отдельных таблиц

Шаг 01:

Миссия 1

Катя хочет реплицировать только таблицы `orders` и `customers`, а не всю базу. Какой синтаксис публикации ей нужен?

Подсказка: публикация может ограничивать таблицы явно.

Ответ:

- **CREATE PUBLICATION mypub FOR TABLE orders, customers**
- CREATE PUBLICATION mypub FOR ALL TABLES
- CREATE PUBLICATION mypub FOR SCHEMA public

Шаг 02:

Миссия 2

Лиза хочет, чтобы подписчик при подписке не копировал начальные данные из таблиц, только принимал будущие изменения. Какой параметр использовать?

Подсказка: copy_data управляет начальным копированием.

Ответ:

- **CREATE SUBSCRIPTION mysub CONNECTION '...' PUBLICATION mypub WITH (copy_data = false)**
- CREATE SUBSCRIPTION mysub CONNECTION '...' PUBLICATION mypub WITH (copy_data = true)

🔗 Шаг 03:

Миссия 3

Игорь настроил публикацию `FOR TABLE customers WITH (publish = 'insert, update')`. Он удивлён, что удаление не реплицируется. Почему?

Подсказка: publish ограничивает операции.

Ответ:

- Удаления не включены в **publish = 'insert, update'**
- Удаления запрещены на подписчике

🔗 Шаг 04:

Миссия 4

После создания подписки, Саша замечает, что данные для одной из таблиц не пришли. Что стоит проверить первым делом?

Подсказка: проблема часто в согласованности схем на мастер/подписчике.

Ответ:

- **Сравнить структуру таблицы на мастере и подписчике**
- Проверить сеть
- Перезапустить подписку

🔗 Шаг 05:

Миссия 5

Сопоставьте вариант публикации с её свойством:

Варианты публикации:

Ответ:

- FOR TABLE t1, t2
- FOR ALL TABLES
- FOR TABLE t1 WITH (publish = 'insert, update')
- FOR TABLE t2 WITH (publish = 'delete')

Свойства:

1. Реплицирует только t1 и t2 со всеми операциями
2. Реплицирует вставки и обновления из t1
3. Реплицирует удаления из t2
4. Реплицирует все таблицы полностью

Подсказка: FOR TABLE ограничивает набор, publish = ... ограничивает операции.

- **FOR TABLE t1, t2** — Реплицирует только t1 и t2 со всеми операциями
- **FOR ALL TABLES** — Реплицирует все таблицы полностью
- **FOR TABLE t1 WITH (publish = 'insert, update')** — Реплицирует вставки и обновления из t1
- **FOR TABLE t2 WITH (publish = 'delete')** — Реплицирует удаления из t2

🔗 Шаг 06:

Миссия 6

Сопоставьте параметр подписки с его смыслом при репликации отдельных таблиц:

Параметры:

Ответ:

- publication
- copy_data
- slot_name
- connection

Смыслы:

1. Имя публикации, которую подписка будет читать
2. Копировать ли начальные данные таблиц при подписке
3. Имя слота репликации на мастере для этой подписки
4. Строка подключения к мастеру-серверу

Подсказка: эти параметры — стандартные части CREATE SUBSCRIPTION.

- **publication** — Имя публикации, которую подписка будет читать
- **copy_data** — Копировать ли начальные данные таблиц при подписке
- **slot_name** — Имя слота репликации на мастере для этой подписки
- **connection** — Строка подключения к мастеру-серверу

Шаг 07:

Миссия 7

Сопоставьте ситуацию с таблицей и что нужно сделать:

Ситуации:

Ответ:

- таблица была добавлена в публикацию после создания подписки
- таблица была удалена из публикации
- определён publish = 'update' и удаления не видны
- на подписке структура не совпадает с мастером

Действия:

1. ALTER PUBLICATION ADD TABLE + REFRESH SUBSCRIPTION
2. ALTER PUBLICATION DROP TABLE + REFRESH SUBSCRIPTION
3. ALTER PUBLICATION ... WITH publish = 'insert, update, delete' + REFRESH SUBSCRIPTION
4. Исправить схему, выполнить REFRESH SUBSCRIPTION

Подсказка: изменения публикации часто требуют REFRESH, а схемы надо синхронизировать.

- **таблица была добавлена в публикацию после создания подписки** — ALTER PUBLICATION ADD TABLE + REFRESH SUBSCRIPTION
- **таблица была удалена из публикации** — ALTER PUBLICATION DROP TABLE + REFRESH SUBSCRIPTION
- **определён publish = 'update' и удаления не видны** — ALTER PUBLICATION ... WITH publish = 'insert, update, delete' + REFRESH SUBSCRIPTION
- **на подписке структура не совпадает с мастером** — Исправить схему, выполнить REFRESH SUBSCRIPTION

Шаг 08:

Миссия 8

Сопоставьте этап подписки отдельных таблиц с проверкой:

Этапы / симптомы:

Ответ:

- initial copy
- apply изменений
- ошибка связи
- отсутствие данных после copy

Что проверить:

1. проверить логи и связь
2. проверить, что таблицы существуют и структура корректна
3. проверить задержку и статус подписки
4. провести повторный REFRESH или новый initial copy

Подсказка: разные стадии требуют разных проверок.

- **ошибка связи** — проверить логи и связь
- **absence данных после copy** — провести повторный REFRESH или новый initial copy
- **initial copy** — проверить, что таблицы существуют и структура корректна
- **apply изменений** — проверить задержку и статус подписки

Шаг 09:

Миссия 9

Сопоставьте SQL-команду с её действием при работе с подпиской на отдельные таблицы:

Команды:

Ответ:

- ALTER SUBSCRIPTION ... REFRESH PUBLICATION
- ALTER SUBSCRIPTION ... ENABLE
- ALTER SUBSCRIPTION ... DISABLE
- DROP SUBSCRIPTION ...

Действия:

1. перезапустить получение изменений после приостановки
2. пересинхронизировать структуру публикации и подписки
3. приостановить применение изменений (pause)
4. удалить подписку и остановить поток репликации

Подсказка: ENABLE и DISABLE управляют состоянием, REFRESH — синхронизация.

- **ALTER SUBSCRIPTION ... REFRESH PUBLICATION** — пересинхронизировать структуру публикации и подписки
- **ALTER SUBSCRIPTION ... ENABLE** — перезапустить получение изменений после приостановки
- **ALTER SUBSCRIPTION ... DISABLE** — приостановить применение изменений (pause)
- **DROP SUBSCRIPTION ...** — удалить подписку и остановить поток репликации

Шаг 10:

Миссия 10

Расположите шаги создания подписки на отдельные таблицы:

1. CREATE PUBLICATION FOR specific tables

2. Настроить pg_hba.conf и права доступа
3. CREATE SUBSCRIPTION с указанием этой публикации
4. Проверить соединение и доступы
5. Убедиться, что структуры таблиц совпадают

Подсказка: доступы и структура важны прежде, чем подписка начнёт работать.

Ответ:

- Настроить pg_hba.conf и права доступа
- CREATE PUBLICATION FOR specific tables
- Убедиться, что структуры таблиц совпадают
- Проверить соединение и доступы
- CREATE SUBSCRIPTION с указанием этой публикации

 Шаг 11:

Миссия 11

Расположите этапы подписки после её создания:

1. initial copy данных по таблицам
2. проверка логов на ошибки
3. переход к apply изменений
4. мониторинг задержки (lag)
5. сравнение данных с мастером

Подсказка: copy, проверка, apply, мониторинг, верификация.

Ответ:

- initial copy данных по таблицам
- проверка логов на ошибки
- переход к apply изменений
- мониторинг задержки (lag)
- сравнение данных с мастером

 Шаг 12:

Миссия 12

Расположите шаги при изменении набора таблиц в публикации:

1. ALTER PUBLICATION ADD / DROP TABLE
2. REFRESH PUBLICATION на подписчике
3. Создать/обновить подписку, если требуется
4. Проверить, что новые таблицы / изменения отражены
5. Проверить корректность данных

Подсказка: изменение публикации требует обновления подписки и проверки.

Ответ:

- ALTER PUBLICATION ADD / DROP TABLE
- REFRESH PUBLICATION на подписчике
- Создать/обновить подписку, если требуется
- Проверить, что новые таблицы / изменения отражены
- Проверить корректность данных

Шаг 13:

Миссия 13

Расположите шаги при ошибке несоответствия схемы:

1. DISABLE подписку
2. Внести изменения схемы на подписчике (ALTER TABLE и др.)
3. REFRESH PUBLICATION или подписку
4. ENABLE подписку
5. Проверить, что данные пошли корректно

Подсказка: остановка, правки, пересинхронизация, проверка.

Ответ:

- DISABLE подписку
- Внести изменения схемы на подписчике (ALTER TABLE и др.)
- REFRESH PUBLICATION или подписку
- ENABLE подписку
- Проверить, что данные пошли корректно

Шаг 14:

Миссия 14

Расположите диагностические шаги, если данных не приходит:

1. Проверить лог файлы подписчика и мастера
2. Проверить имя публикации и подписки на совпадение
3. Проверить права доступа и pg_hba записей
4. REFRESH PUBLICATION / подписку
5. Сравнить структуру таблиц и схемы

Подсказка: лог → публикация → доступ → пересинхронизация → схема.

Ответ:

- Проверить лог файлы подписчика и мастера
- Проверить имя публикации и подписки на совпадение
- Проверить права доступа и pg_hba записей
- REFRESH PUBLICATION / подписку
- Сравнить структуру таблиц и схемы

Шаг 15:

Миссия 15

Расположите шаги для безопасного добавления новой таблицы к подписке:

1. ALTER PUBLICATION ADD TABLE new_t
2. REFRESH PUBLICATION на подписчике
3. Проверить, что подписчик получил таблицу и данные
4. Проверить задержку и целостность новых данных
5. **В случае ошибок** — откат / правки схемы

Подсказка: добавление требует пересинхронизации и проверки.

Ответ:

- ALTER PUBLICATION ADD TABLE new_t

- REFRESH PUBLICATION на подписчике
- Проверить, что подписчик получил таблицу и данные
- Проверить задержку и целостность новых данных
- **В случае ошибок** — откат / правки схемы

🔮 Шаг 16:

Миссия 16

Расположите полную последовательность создания репликации отдельных таблиц и проверки:

1. Настроить права доступа и pg_hba
2. CREATE PUBLICATION FOR TABLE t1, t2
3. CREATE SUBSCRIPTION WITH (copy_data = true/false)
4. Проверить, что подписка перешла в copy / apply стадии
5. Проверить логи и ошибки
6. Проанализировать задержку и статус
7. Сравнить данные t1, t2 между мастер и подписчиком
8. При изменении публикации выполнить REFRESH / ALTER и проверку

Подсказка: объедините шаги конфигурации, подписки, мониторинга и модификации.

Ответ:

- Настроить права доступа и pg_hba
- CREATE PUBLICATION FOR TABLE t1, t2
- CREATE SUBSCRIPTION WITH (copy_data = true/false)
- Проверить, что подписка перешла в copy / apply стадии
- Проверить логи и ошибки
- Проанализировать задержку и статус
- Сравнить данные t1, t2 между мастер и подписчиком
- При изменении публикации выполнить REFRESH / ALTER и проверку

📖 Урок 5 - Ограничения логической репликации

🔮 Шаг 01:

Миссия 1

Марина хочет использовать логическую репликацию для системных каталогов (pg_catalog, pg_toast). Что скажет PostgreSQL?

Подсказка: логическая репликация не распространяется на системные каталоги.

Ответ:

- **Нельзя** — системные каталоги не могут быть опубликованы
- Можно, если указать FOR ALL TABLES
- Можно, если изменить wal_level на full

🔮 Шаг 02:

Миссия 2

Иван пытается реплицировать последовательное значение через `serial` (auto increment). Что важно помнить о таких столбцах?

Подсказка: логическая репликация передаёт изменения, а не смысл последовательности.

Ответ:

- **Подписчик получит конкретные значения, но последовательность может разойтись**
- Подписчик будет использовать ту же последовательность автоматически
- Нельзя реплицировать таблицу с serial

🔮 Шаг 03:

Миссия 3

Алексей хочет реплицировать large object-ы (LOB). Поддерживается ли это в логической репликации?

Подсказка: логическая репликация поддерживает обычные таблицы, но есть ограничения.

Ответ:

- **Нет, large objects (LOB) не поддерживаются логической репликацией**
- Да, но только в PostgreSQL 14+

🔮 Шаг 04:

Миссия 4

Ольга планирует реплицировать таблицу с `oid` типом. Почему это может быть проблемой?

Подсказка: oid и внутренние идентификаторы могут не повторяться точно.

Ответ:

- **Значения oid могут не согласоваться между узлами**
- oid запрещены для репликации

🔮 Шаг 05:

Миссия 5

Василий не хочет реплицировать DELETE-операции. Можно ли это настроить?

Подсказка: публикация позволяет ограничивать операции, но не все.

Ответ:

- **Да** — можно указать publish = 'insert, update' исключая DELETE
- Нет, нельзя исключить отдельные операции

🔮 Шаг 06:

Миссия 6

Сопоставьте ограничения логической репликации с их последствиями:

Ограничения:

Ответ:

- Нет системных каталогов
- Не поддерживаются LOB / large object
- DELETE / TRUNCATE можно исключить
- Синхронизация последовательностей не обеспечивается

Последствия:

1. Невозможно реплицировать содержимое pg_catalog таблиц
2. Объекты large object не реплицируются

3. Можно не реплицировать удаления, если указано publish
4. Счётчики последовательностей могут расходиться между узлами

Подсказка: каждое ограничение вызывает типичное поведение.

- **Нет системных каталогов** — Невозможно реплицировать содержимое pg_catalog таблиц
- **Не поддерживаются LOB / large object** — Объекты large object не реплицируются
- **DELETE / TRUNCATE можно исключить** — Можно не реплицировать удаления, если указано publish
- **Синхронизация последовательностей не обеспечивается** — Счётчики последовательностей могут расходиться между узлами

Шаг 07:

Миссия 7

Сопоставьте тип данных / сценарий с ограничением репликации:

Тип / сценарий:

Ответ:

- Тип bytea / large binary
- Использование sequence.nextval в приложении
- Триггеры, изменяющие другие таблицы
- UNLOGGED таблицы

Ограничения / результат:

1. LOB / large object не реплицируются нормально
2. sequence значения не синхронизируются автоматически
3. изменения через триггеры, вызывающие побочные эффекты, могут не попасть в публикацию
4. UNLOGGED таблицы не записывают WAL и не реплицируются

Подсказка: подумаем, что WAL нужен для репликации.

- **Тип bytea / large binary** — LOB / large object не реплицируются нормально
- **Использование sequence.nextval в приложении** — sequence значения не синхронизируются автоматически
- **Триггеры, изменяющие другие таблицы** — изменения через триггеры, вызывающие побочные эффекты, могут не попасть в публикацию
- **UNLOGGED таблицы** — UNLOGGED таблицы не записывают WAL и не реплицируются

Шаг 08:

Миссия 8

Сопоставьте сценарий с подходящим ограничением или предостережением:

Сценарии:

Ответ:

- Публикация очень многих таблиц
- Большое количество мелких транзакций
- DDL (ALTER TABLE) часто применяются
- Физические реплики параллельно

Ограничение / предостережение:

1. увеличенная нагрузка на decoding / логическую нагрузку

2. задержка подписки может возрасть значительно
3. DDL могут не попасть, если не включены или поддержаны
4. конфликт между физической и логической репликацией возможен

Подсказка: логическая нагрузка и согласованность — важны.

- **Публикация очень многих таблиц** — увеличенная нагрузка на decoding / логическую нагрузку
- **Большое количество мелких транзакций** — задержка подписки может возрасть значительно
- **DDL (ALTER TABLE) часто применяются** — DDL могут не попасть, если не включены или поддержаны
- **Физические реплики параллельно** — конфликт между физической и логической репликацией возможен

Шаг 09:

Миссия 9

Сопоставьте тип ограничения с примером, когда это проявляется:

Ограничения:

Ответ:

- Отсутствие репликации TRUNCATE
- Не поддерживается публикация через FOREIGN DATA WRAPPER
- Не поддерживаются расширения DDL-only
- Нет репликации для unlogged таблиц

Примеры:

1. TRUNCATE my_table не передаётся подписчику
2. Запрос к внешнему серверу через FDW не реплицируется как часть публикации
3. CREATE EXTENSION myext не реплицируется логически
4. Таблица, помеченная UNLOGGED, игнорируется логической репликацией

Подсказка: вспомните ограничения документации.

- **Отсутствие репликации TRUNCATE** — TRUNCATE my_table не передаётся подписчику
- **Не поддерживается публикация через FOREIGN DATA WRAPPER** — Запрос к внешнему серверу через FDW не реплицируется как часть публикации
- **Не поддерживаются расширения DDL-only** — CREATE EXTENSION myext не реплицируется логически
- **Нет репликации для unlogged таблиц** — Таблица, помеченная UNLOGGED, игнорируется логической репликацией

Шаг 10:

Миссия 10

Сопоставьте команду / операция с тем, будет ли она реплицирована логически:

Операции:

Ответ:

- VACUUM FULL
- UPDATE обычной таблицы
- ALTER TABLE ADD COLUMN
- TRUNCATE таблицы

Будет / не будет:

1. не будет реплицировано (очистка физическая)
2. будет, если таблица входит в публикацию
3. будет, если публикация поддерживает DDL
4. не будет реплицировано

Подсказка: логическая репликация передаёт DML + поддерживаемые DDL, но не физические операции.

- **VACUUM FULL** — не будет реплицировано (очистка физическая)
- **UPDATE обычной таблицы** — будет, если таблица входит в публикацию
- **ALTER TABLE ADD COLUMN** — будет, если публикация поддерживает DDL
- **TRUNCATE таблицы** — не будет реплицировано

Шаг 11:

Миссия 11

Расположите ограничения логической репликации по возрастанию их сложности / влияния:

1. Не реплицируются sys-каталоги
2. Не поддерживаются LOB / large object
3. Отсутствие синхронизации sequence
4. Не реплицируется TRUNCATE
5. DDL не всегда поддерживается / требует включения

Подсказка: сначала самые простые ограничения, потом более тонкие.

Ответ:

- Не реплицируются sys-каталоги
- Не поддерживаются LOB / large object
- Отсутствие синхронизации sequence
- Не реплицируется TRUNCATE
- DDL не всегда поддерживается / требует включения

Шаг 12:

Миссия 12

Расположите этапы проверки, чтобы обнаружить ограничение:

1. Проверить логи на ошибки DDL / операции
2. Сравнить схему мастер / подписчик
3. Проверить, что TRUNCATE-операции не приходят
4. Проверить поведение sequence / автоинкремента
5. Проверить репликацию бинарных полей / LOB

Подсказка: диагностика идёт от общих ошибок к специфическим.

Ответ:

- Проверить логи на ошибки DDL / операции
- Сравнить схему мастер / подписчик
- Проверить, что TRUNCATE-операции не приходят
- Проверить поведение sequence / автоинкремента
- Проверить репликацию бинарных полей / LOB

Шаг 13:

Миссия 13

Расположите сценарии, при которых ограничения проявляются наиболее остро:

1. Таблицы с large object
2. Использование sequence в приложении
3. Частые DDL изменения
4. TRUNCATE вручную
5. Очень большие публикации с многими таблицами

Подсказка: подумайте, что нагружает систему и что не поддерживается.

Ответ:

- Таблицы с large object
- Использование sequence в приложении
- Частые DDL изменения
- TRUNCATE вручную
- Очень большие публикации с многими таблицами

Шаг 14:

Миссия 14

Расположите шаги, если обнаружен LOB в вашей схеме и вы хотите понимать его обработку:

1. Выявить таблицы / столбцы, содержащие LOB / large objects
2. Проверить, как они используются (с потоками, внешними системами)
3. Рассмотреть альтернативы (например bytea или внешнее хранение)
4. Оценить, можно ли обойтись без репликации LOB
5. При необходимости исключить такие таблицы из публикации

Подсказка: просто диагностика и план действий.

Ответ:

- Выявить таблицы / столбцы, содержащие LOB / large objects
- Проверить, как они используются (с потоками, внешними системами)
- Рассмотреть альтернативы (например bytea или внешнее хранение)
- Оценить, можно ли обойтись без репликации LOB
- При необходимости исключить такие таблицы из публикации

Шаг 15:

Миссия 15

Расположите шаги, если часто происходит ALTER TABLE в продакшн, и нужно минимизировать сбои репликации:

1. Планировать DDL изменения в окна обслуживания
2. Отключать подписку (DISABLE) на время DDL
3. Вносить изменения на мастере и подписчике вручную, если нужно
4. REFRESH PUBLICATION / подписку после изменений
5. Включать подписку (ENABLE) и проверять синхронизацию

Подсказка: безопасные DDL — это паузы и пересинхронизация.

Ответ:

- Планировать DDL изменения в окна обслуживания
- Отключать подписку (DISABLE) на время DDL
- Вносить изменения на мастере и подписчике вручную, если нужно
- REFRESH PUBLICATION / подписку после изменений
- Включать подписку (ENABLE) и проверять синхронизацию

 Шаг 16:

Миссия 16

Расположите полную последовательность проверки и обхода ограничений при внедрении логической репликации:

1. Проанализировать схему и типы данных на наличие LOB / unlogged / oid
2. Оценить использование sequence и стратегии синхронизации
3. Исключить неподдерживаемые объекты из публикации
4. Настроить публикацию с нужными опциями publish / таблицы
5. Создать подписку и провести начальное копирование (copy)
6. Мониторинг ошибок, задержек, несоответствий
7. При изменениях DDL использовать DISABLE / REFRESH / ENABLE
8. Регулярно проверять sequence и целостность данных вручную

Подсказка: это сценарий, объединяющий все ограничения и способы их обхода.

Ответ:

- Проанализировать схему и типы данных на наличие LOB / unlogged / oid
- Оценить использование sequence и стратегии синхронизации
- Исключить неподдерживаемые объекты из публикации
- Настроить публикацию с нужными опциями publish / таблицы
- Создать подписку и провести начальное копирование (copy)
- Мониторинг ошибок, задержек, несоответствий
- При изменениях DDL использовать DISABLE / REFRESH / ENABLE
- Регулярно проверять sequence и целостность данных вручную

 Урок 6 - Примеры миграций между версиями

 Шаг 01:

Миссия 1

Андрей запускает логическую репликацию между PostgreSQL 13 и 14 и хочет обновить слейв до версии 15. Что он должен сделать первым?

Подсказка: сначала подготовка слейва важнее, чем переключение слотов.

Ответ:

- **Остановить поток репликации и выключить слейв**
- Непосредственно запустить pg_upgrade на мастер-сервере
- Удалить слот публикации на мастере
- Остановить мастер и сделать offline дамп

 Шаг 02:

Миссия 2

После остановки репликации Андрей хочет запустить `pg\_upgrade` на слейве. Какой режим лучше выбрать?

Подсказка: минимальное время простоя важнее, чем экономия дискового пространства.

Ответ:

- **--link (in-place) режим**
- --copy режим
- Запуск pg_upgrade без опций
- Использовать pg_dump/pg_restore вместо pg_upgrade

Шаг 03:

Миссия 3

При миграции Андрей обнаруживает, что схема публикации в версии 14 не поддерживает новую фичу версии 15. Что делать?

Подсказка: часто помогает пересоздание публикации после обновления.

Ответ:

- **Пересоздать публикацию на мастере после обновления**
- Откатить мастера на версию 14
- Оставить старую публикацию и ждать патчей
- Использовать логическую репликацию только для части таблиц

Шаг 04:

Миссия 4

Во время миграции на продакшене у Андрея возникли конфликты из-за удаления столбца на мастере. Как правильно поступить?

Подсказка: изменение схемы когда есть активная репликация — риск.

Ответ:

- **Сначала внести изменения на слейве, дождаться синхронизации, затем переключить**
- Удалить колонку сразу на мастере
- Отключить публикацию только для этой таблицы
- Использовать триггеры для поддержания старых данных

Шаг 05:

Миссия 5

После успешного обновления Андрей переключил репликацию. Как он проверит, что слейв в версии 15 работает корректно?

Подсказка: проверьте журнал реплики и сравните LSN с мастером.

Ответ:

- **Сравнить текущий LSN мастера и слейва**
- Проверить только версию PostgreSQL
- Запустить vacuum на мастере
- Проверить размер таблиц слейва

Шаг 06:

Миссия 6

Сопоставьте шаг с его назначением при миграции версии через логическую репликацию:

Шаги:

Ответ:

- Остановка реплики
- `pg_upgrade` слейва
- Пересоздание публикации
- Запуск репликации

Назначения:

1. Обеспечивает минимальное время простоя
2. Подготовить схему к новому функционалу
3. Отключение слежения перед изменениями
4. Возобновить синхронизацию после всех изменений

 **Подсказка:**

Подумайте, какой шаг идет первым и какой последний.

- **Остановка реплики** — Отключение слежения перед изменениями
- **`pg_upgrade` слейва** — Обеспечивает минимальное время простоя
- **Пересоздание публикации** — Подготовить схему к новому функционалу
- **Запуск репликации** — Возобновить синхронизацию после всех изменений

 **Шаг 07:**

Миссия 7

Сопоставьте проблему миграции с её типичным решением:

Проблема:

Ответ:

- Несоответствие типов данных
- Ограничения, отсутствующие на слейве
- Зависимости функций между версиями
- Большие таблицы без downtime

Решение:

1. Добавление CHECK с условием CAST
2. Создание ограничений после репликации
3. Ручное портирование функций
4. Использование логической репликации с ограниченным тиражом

 **Подсказка:**

Проанализируйте, какую проблему решает каждое действие.

- **Несоответствие типов данных** — Добавление CHECK с условием CAST
- **Ограничения, отсутствующие на слейве** — Создание ограничений после репликации
- **Зависимости функций между версиями** — Ручное портирование функций
- **Большие таблицы без downtime** — Использование логической репликации с ограниченным тиражом

 **Шаг 08:**

Миссия 8

Расположите этапы миграции через логическую репликацию в правильном порядке:

1. Пересоздание подписки на слейве
2. Остановка репликации
3. pg_upgrade слейва до новой версии
4. Установка публикации на мастере в новой версии
5. Запуск репликации

🔗 **Подсказка:**

Что нужно сделать перед апгрейдом, и что после него?

Ответ:

- Остановка репликации
- pg_upgrade слейва до новой версии
- Установка публикации на мастере в новой версии
- Пересоздание подписки на слейве
- Запуск репликации

🔗 **Шаг 09:**

Миссия 9

Расположите действия по верификации после миграции в логическом порядке:

1. Сравнение LSN мастера и слейва
2. Проверка логов реплики на ошибки
3. Запуск тестового приложения к слейву
4. Сравнение выборок из таблиц

🔗 **Подсказка: Сначала проверить техническую синхронизацию, затем логи, затем бизнес-данные.**

Ответ:

- Сравнение LSN мастера и слейва
- Проверка логов реплики на ошибки
- Запуск тестового приложения к слейву
- Сравнение выборок из таблиц

🔗 **Шаг 10:**

Миссия 10

Расположите фазы MITIGATION при миграции:

1. Минимизация блокировок схемы
2. Плавное переключение клиентов
3. Мониторинг задержек репликации
4. Откат в случае ошибок

🔗 **Подсказка: Какие шаги нужно предпринять заранее, а какие при возможности отката?**

Ответ:

- Минимизация блокировок схемы
- Мониторинг задержек репликации
- Плавное переключение клиентов
- Откат в случае ошибок

🔮 Шаг 11:

Миссия 11

После миграции разработчик хочет добавить новую колонку с дефолтом. Как безопасно это сделать?

Подсказка: избегайте блокировки всей таблицы.

Ответ:

- **Добавить колонку без дефолта, затем обновить батчами**
- Добавить колонку с дефолтом сразу
- Удалить таблицу и создать заново
- Сразу применить ALTER ... USING формулу

🔮 Шаг 12:

Миссия 12

Сопоставьте советы с их пользой при миграции логической репликации:

Совет:

Ответ:

- Отключать триггеры во время catch-up
- Периодически перезапускать подписку
- Поддерживать zone-align WAL сегменты
- Проверять задержку в ms через pg_stat_subscription

Польза:

1. Уменьшение нагрузки на репликацию
2. Избежание накопления логов
3. Повышение стабильности замера времени задержки
4. Снижение накладных операций триггеров

🔮 **Подсказка: Каждый совет ориентирован на конкретную проблему.**

- **Отключать триггеры во время catch-up** — Снижение накладных операций триггеров
- **Периодически перезапускать подписку** — Избежание накопления логов
- **Поддерживать zone-align WAL сегменты** — Уменьшение нагрузки на репликацию
- **Проверять задержку в ms через pg_stat_subscription** — Повышение стабильности замера времени задержки

🔮 Шаг 13:

Миссия 13

Расположите шаги миграции с минимальным downtime:

1. Настроить публикацию с фильтром
2. Начать подписку на новом слейве
3. Синхронизовать базу до catch-up
4. Переключить трафик на новый слейв
5. Отключить старый репликующий мастер

🔮 **Подсказка: Сначала готовится инфраструктура, затем синхронизация, затем переключение.**

Ответ:

- Настроить публикацию с фильтром
- Начать подписку на новом слейве
- Синхронизовать базу до catch-up
- Переключить трафик на новый слейв
- Отключить старый репликующий мастер

 **Шаг 14:**

Миссия 14

Во время миграции нагрузка на мастер выросла. Как смягчить ситуацию?

Подсказка: подумайте о приоритете и batch-обновлениях.

Ответ:

- **Уменьшить приоритет репликационных процессов**
- Отключить логическую репликацию
- Запустить полную синхронную репликацию
- Полностью отказаться от репликации

 **Шаг 15:**

Миссия 15

Миграция окончена, Андрей хочет отлавливать регрессии. Как поступить?

Подсказка: подумайте о канареечных тестах и сравнении метрик.

Ответ:

- **Запустить A/B запросы к мастер/слейву и сравнить результаты**
- Удалить старую версию слейва мгновенно
- Отключить логи после миграции
- Включить полное логирование на мастер

Модуль 4 - Шардирование — путь распределения

Урок 1 - Что такое шардинг и зачем он нужен

 **Шаг 01:**

Миссия 1

Аня — разработчик из стартапа, который резко вырос. Она слышала слово «шардинг», но не до конца понимает, зачем он нужен. Что из перечисленного лучше всего описывает цель шардинга?

Подсказка: Шардинг помогает распределять нагрузку и данные между разными серверами.

Ответ:

- **Разделение больших данных на части для повышения производительности**
- Создание резервных копий базы данных
- Увеличение размера одного сервера
- Оптимизация запросов на одном сервере

 **Шаг 02:**

Миссия 2

Максим разрабатывает приложение с большим количеством пользователей. Он заметил, что база данных стала тормозить. Что из перечисленного является признаком, что пора подумать о шардинге?

Подсказка: Подумайте, когда нагрузка на один сервер становится слишком высокой.

Ответ:

- **Высокое время отклика и частые блокировки базы данных**
- Малое количество запросов к базе
- Отсутствие данных в базе
- Большое количество пользователей, но сервер работает быстро

Шаг 03:

Миссия 3

Ирина хочет объяснить коллегам, почему шардирование — это не просто копирование данных. Какое из утверждений лучше всего отражает отличие шардинга от репликации?

Подсказка: Репликация копирует данные целиком, шардинг — делит их.

Ответ:

- **Шардинг разделяет данные по частям между серверами, а репликация создает полные копии.**
- Шардинг копирует данные полностью, а репликация делит их.
- **Шардинг и репликация — одно и то же.**
- Репликация нужна для разделения данных, шардинг для резервного копирования.

Шаг 04:

Миссия 4

Павел ведет обсуждение в команде. Он утверждает, что шардинг — это только про горизонтальное масштабирование. Верно ли это?

Подсказка: Вспомните, какие есть виды масштабирования баз данных.

Ответ:

- **Да, шардинг** — это способ горизонтального масштабирования, а не вертикального.
- **Нет, шардинг** — это вертикальное масштабирование.
- Шардинг не относится к масштабированию.

Шаг 05:

Миссия 5

Алексей готовит презентацию о шардировании. Помогите ему сопоставить ключевые понятия с их описаниями.

Понятия:

Ответ:

- Шард
- Кластер
- Горизонтальное масштабирование
- Репликация

Определения:

1. Отдельная часть базы данных, хранящая часть данных

2. Множество серверов, работающих вместе для обеспечения высокой доступности и масштабирования
3. Добавление новых серверов для увеличения производительности
4. Создание копий данных для отказоустойчивости

🔗 Подсказка:

Подумайте, что именно делает каждый термин в архитектуре распределённых баз данных.

- **Шард** — Отдельная часть базы данных, хранящая часть данных
- **Кластер** — Множество серверов, работающих вместе для обеспечения высокой доступности и масштабирования
- **Горизонтальное масштабирование** — Добавление новых серверов для увеличения производительности
- **Репликация** — Создание копий данных для отказоустойчивости

🔗 Шаг 06:

Миссия 6

Наташа изучает способы распределения данных. Помогите ей сопоставить типы шардинга с их характеристиками.

Типы шардинга:

Ответ:

- Диапазонный шардинг
- Хэш-шардинг
- Шардинг по ключу

Характеристики:

1. Данные разбиваются по непрерывным диапазонам значений
2. Используется функция хэширования для равномерного распределения
3. Шардинг по определённому полю в записи, например, `user_id`

🔗 Подсказка:

Подумайте, как данные могут быть разделены, чтобы нагрузка распределялась равномерно или логично.

- **Диапазонный шардинг** — Данные разбиваются по непрерывным диапазонам значений
- **Хэш-шардинг** — Используется функция хэширования для равномерного распределения
- **Шардинг по ключу** — Шардинг по определённому полю в записи, например, `user_id`

🔗 Шаг 07:

Миссия 7

Дима планирует внедрить шардинг, но хочет понять риски. Сопоставьте потенциальные проблемы с их описаниями.

Проблемы:

Ответ:

- Балансировка нагрузки
- Сложность запросов
- Обеспечение целостности данных
- Администрирование

Описание:

1. Необходимо равномерно распределять запросы между шардами, иначе одни будут перегружены
2. Запросы могут требовать обращения к нескольким шардам, усложняя логику
3. Гарантировать согласованность данных становится сложнее при распределении
4. Поддержка множества серверов требует дополнительных усилий

 **Подсказка:**

Вспомните, какие трудности появляются при работе с распределёнными системами.

- **Балансировка нагрузки** — Необходимо равномерно распределять запросы между шардами, иначе одни будут перегружены
- **Сложность запросов** — Запросы могут требовать обращения к нескольким шардам, усложняя логику
- **Обеспечение целостности данных** — Гарантировать согласованность данных становится сложнее при распределении
- **Администрирование** — Поддержка множества серверов требует дополнительных усилий

 **Шаг 08:**

Миссия 8

Костя хочет понять, как шардирование влияет на резервное копирование. Помогите сопоставить понятия и их описание в контексте резервного копирования.

Понятия:

Ответ:

- Резервное копирование на уровне шардов
- Общий бэкап кластера
- Восстановление данных

Описание:

1. Создание копии данных каждого шарда отдельно
2. Создание полной копии всех данных из всех шардов вместе
3. Процесс восстановления данных из резервных копий в случае сбоя

 **Подсказка:**

Вспомните, почему распределённые данные требуют особого подхода к бэкапам.

- **Резервное копирование на уровне шардов** — Создание копии данных каждого шарда отдельно
- **Общий бэкап кластера** — Создание полной копии всех данных из всех шардов вместе
- **Восстановление данных** — Процесс восстановления данных из резервных копий в случае сбоя

 **Шаг 09:**

Миссия 9

Светлана хочет систематизировать знания о шардировании. Помогите ей сопоставить термины с их практическими примерами.

Термины:

Ответ:

- Диапазонный шардинг

- Хэш-шардинг
- Шардинг по ключу

Примеры:

1. **Пользователи с user_id от 1 до 10000 на одном шарде, от 10001 до 20000 — на другом**
2. Каждому user_id присваивается хэш, который определяет шард
3. Данные делятся по значению уникального ключа, например email

🔮 Подсказка:

Обратите внимание, как данные группируются — по диапазону, по вычисленному хэшу или по конкретному полю.

- **Диапазонный шардинг** — Пользователи с user_id от 1 до 10000 на одном шарде, от 10001 до 20000 — на другом
- **Хэш-шардинг** — Каждому user_id присваивается хэш, который определяет шард
- **Шардинг по ключу** — Данные делятся по значению уникального ключа, например email

🔮 Шаг 10:

Миссия 10

Игорь разрабатывает систему с шардингом и хочет настроить её правильно. Расположите этапы настройки шардинга в правильном порядке:

1. Определение ключа для шардинга
2. Настройка маршрутизации запросов на соответствующий шард
3. Добавление новых серверов для хранения шардов
4. Перенос данных на соответствующие шарды
5. Тестирование корректности распределения данных и запросов

🔮 Подсказка:

Сначала нужно понять, как будут разделены данные, потом настроить инфраструктуру и проверить работу.

Ответ:

- Определение ключа для шардинга
- Добавление новых серверов для хранения шардов
- Перенос данных на соответствующие шарды
- Настройка маршрутизации запросов на соответствующий шард
- Тестирование корректности распределения данных и запросов

🔮 Шаг 11:

Миссия 11

Анна объясняет команде процесс масштабирования с помощью шардинга. Помогите ей расположить правильный порядок действий:

1. Оценка нагрузки на текущую базу данных
2. Выбор подходящей стратегии шардинга
3. Реализация распределения данных между шард-серверами
4. Обновление приложений для работы с шардами
5. Мониторинг и оптимизация производительности

🔮 Подсказка:

Начинайте с оценки, затем планируйте, внедряйте и поддерживайте систему.

Ответ:

- Оценка нагрузки на текущую базу данных
- Выбор подходящей стратегии шардинга
- Реализация распределения данных между шард-серверами
- Обновление приложений для работы с шардами
- Мониторинг и оптимизация производительности

Шаг 12:

Миссия 12

Даша работает с большим проектом, где данные распределены по разным базам. Что является основной целью шардинга?

Подсказка: Подумайте, зачем данные разбивают на части.

Ответ:

- **Улучшить масштабируемость и производительность базы данных**
- Сократить объем резервных копий
- Объединить все данные в одной таблице
- Сделать базу данных менее доступной

Шаг 13:

Миссия 13

Иван заметил, что при шардинге нагрузка на сервер распределяется. Что из перечисленного помогает достигнуть этого эффекта?

Подсказка: Представьте, что данные разделены по ключам и обрабатываются параллельно.

Ответ:

- **Разделение данных на шарды по ключу**
- **Параллельная обработка запросов к разным шардам**
- Использование одного сервера для всех данных
- Сохранение всех данных в одном месте

Шаг 14:

Миссия 14

Максим хочет разобраться в типах шардинга. Помогите сопоставить тип шардинга и его описание.

Типы шардинга:

Ответ:

- Диапазонный шардинг
- Хэш-шардинг
- Список (List) шардинг

Описание:

1. Данные распределяются по диапазонам значений ключа
2. Данные распределяются по результату хэш-функции от ключа
3. Данные распределяются по заранее определённым спискам значений

 **Подсказка:**

Обратите внимание на способы группировки данных и распределения нагрузки.

- **Диапазонный шардинг** — Данные распределяются по диапазонам значений ключа
- **Хэш-шардинг** — Данные распределяются по результату хэш-функции от ключа
- **Список (List) шардинг** — Данные распределяются по заранее определённым спискам значений

Шаг 15:

Миссия 15

Аня сталкивается с проблемами при шардинге. Помогите ей сопоставить типичные трудности и их описание.

Проблемы:

Ответ:

- Балансировка нагрузки
- Распределённые транзакции
- Обновление схемы

Описание:

1. Сложность равномерного распределения запросов между шардами
2. Трудности поддержания атомарности операций между разными шардами
3. Проблемы с синхронизацией изменений структуры таблиц в разных шардах

Подсказка:

Вспомните, как распределённая система усложняет обычные операции.

- **Балансировка нагрузки** — Сложность равномерного распределения запросов между шардами
- **Распределённые транзакции** — Трудности поддержания атомарности операций между разными шардами
- **Обновление схемы** — Проблемы с синхронизацией изменений структуры таблиц в разных шардах

Шаг 16:

Миссия 16

Павел хочет внедрить шардинг. Расположите этапы внедрения шардинга в правильном порядке:

1. Определение ключа для шардинга
2. Настройка инфраструктуры и серверов
3. Разделение данных по выбранному ключу
4. Перенос и синхронизация данных
5. Тестирование работы системы

Подсказка:

Сначала выбирается логика разделения, потом подготавливаются ресурсы и данные.

Ответ:

- Определение ключа для шардинга
- Настройка инфраструктуры и серверов
- Разделение данных по выбранному ключу
- Перенос и синхронизация данных

- Тестирование работы системы

Шаг 17:

Миссия 17

Елена планирует масштабировать систему с шардингом. Расположите действия в правильном порядке, чтобы эффективно добавить новый шард:

1. Подготовка нового сервера
2. Определение границ нового шарда
3. Перенос части данных на новый шард
4. Обновление маршрутизации запросов
5. Проверка корректности работы системы

Подсказка:

Добавление шардов — это планирование, перенос, настройка и проверка.

Ответ:

- Подготовка нового сервера
- Определение границ нового шарда
- Перенос части данных на новый шард
- Обновление маршрутизации запросов
- Проверка корректности работы системы

Шаг 18:

Миссия 18

Алексей хочет понять, как происходит обработка запросов в шардинговой системе. Расположите шаги обработки запроса в правильном порядке:

1. Получение запроса от клиента
2. Определение, к какому шару нужно обратиться
3. Отправка запроса на соответствующий шард
4. Обработка запроса на шарде
5. Возврат результата клиенту

Подсказка:

Запрос должен пройти маршрут от клиента до правильного хранилища и обратно.

Ответ:

- Получение запроса от клиента
- Определение, к какому шару нужно обратиться
- Отправка запроса на соответствующий шард
- Обработка запроса на шарде
- Возврат результата клиенту

Урок 2 - Ручной шардинг с partitioning и routing

Шаг 01:

Миссия 1

Вы работаете над монолитным приложением и заметили, что таблица “orders” постоянно растёт, а запросы начинают тормозить. Что вы выберете как первый шаг к ручному шардингу?

Подсказка: Сначала нужно решить, по какому критерию вы будете распараллеливать данные.

Ответ:

- **Выбрать ключ шардинга и разбить таблицу по диапазонам**
- Перенести всю таблицу на отдельную машину
- Создать копию таблицы и писать туда

🔮 **Шаг 02:**

Миссия 2

Вы решили шардировать по customer_id. Какой подход будет наиболее надёжным с самого начала?

Подсказка: Лучше начать с простого механизма маршрутизации.

Ответ:

- **Маршрутизатор, который по customer_id выбирает нужную партицию**
- При вставке сразу вставлять в все шарды
- Оставить всё в одном шарде и ретроспективно делить

🔮 **Шаг 03:**

Миссия 3

При добавлении маршрутизатора вы обнаружили, что некоторые запросы SELECT всё ещё направляются к монолиту. Что делать?

Подсказка: Нужно обновить логику маршрутизации SELECT-запросов.

Ответ:

- **Добавить логику маршрутизации SELECT по ключу шарда**
- Оставлять SELECT на монолите всегда
- Игнорировать SELECT и ориентироваться только на INSERT

🔮 **Шаг 04:**

Миссия 4

Теперь вы хотите настроить partitioning в PostgreSQL, чтобы shard-разбиение стало встроенным. Какой тип partition лучше всего подойдёт для диапазонного шардинга?

Подсказка: Диапазонный шардинг — это расщепление по диапазонам ключа.

Ответ:

- **RANGE partitioning**
- LIST partitioning
- HASH partitioning

🔮 **Шаг 05:**

Миссия 5

Вы хотите, чтобы новые записи с customer_id от 1000 до 1999 уходили в новый сегмент. Какую команду вы примените?

Подсказка: Используйте конструкцию CREATE TABLE ... PARTITION OF ... FOR VALUES ...

Ответ:

- **CREATE TABLE orders_p1000 PARTITION OF orders FOR VALUES FROM (1000) TO (2000)**
- CREATE PARTITION orders_p1000 VALUES 1000 TO 2000
- ALTER TABLE orders ADD PARTITION FOR VALUES 1000..2000

🔗 Шаг 06:

Миссия 6

После создания пары партиций вы замечаете, что вставки всё ещё идут в главную таблицу и падают. Что требуется добавить?

Подсказка: Проверьте, что определён маршрутизатор и триггер вставки.

Ответ:

- **Создать INSERT-триггер, который направит вставку в нужный partition**
- Отключить главную таблицу
- Разрешить вставки только вручную

🔗 Шаг 07:

Миссия 7

Сопоставьте термин с соответствующим описанием в контексте шардинга:

Ответ:

- Parent table
 - Child partition
 - Trigger routing
 - Default partition
1. Таблица-контейнер без собственных данных, унаследованная дочерними
 2. Локальная шардинговая таблица, содержащая реальные строки
 3. Триггер, направляющий INSERT в нужную часть
 4. Партиция, в которую попадают записи, не удовлетворяющие остальным правилам

Подсказка: Подумайте, как PostgreSQL распределяет по partitioning и что происходит, если значение не попадает ни в один диапазон.

- **Parent table** — Таблица-контейнер без собственных данных, унаследованная дочерними
- **Child partition** — Локальная шардинговая таблица, содержащая реальные строки
- **Trigger routing** — Триггер, направляющий INSERT в нужную часть
- **Default partition** — Партиция, в которую попадают записи, не удовлетворяющие остальным правилам

🔗 Шаг 08:

Миссия 8

Сопоставьте SQL-конструкцию с её ролью при ручном шардинге:

Ответ:

- CREATE TABLE ... PARTITION OF ... FOR VALUES ...
 - ALTER TABLE ... ATTACH PARTITION
 - CREATE TRIGGER ... BEFORE INSERT
 - DEFAULT PARTITION clause
1. Связывает новую child-таблицу с parent-partition
 2. Определяет правила попадания данных в долю partition
 3. Обеспечивает обработку вставок вне заданных диапазонов
 4. Создаёт механизм маршрутизации вставки в child-таблицу

Подсказка: Вспомните синтаксис partitioning и как PostgreSQL управляет прикреплением.

- **ALTER TABLE ... ATTACH PARTITION** — Связывает новую child-таблицу с parent-partition
- **CREATE TABLE ... PARTITION OF ... FOR VALUES ...** — Определяет правила попадания данных в долю partition
- **DEFAULT PARTITION clause** — Обеспечивает обработку вставок вне заданных диапазонов
- **CREATE TRIGGER ... BEFORE INSERT** — Создаёт механизм маршрутизации вставки в child-таблицу

🔮 Шаг 09:

Миссия 9

Сопоставьте проблему с её возможным следствием в системе с ручным шардингом:

Ответ:

- Отсутствие default partition
 - Слабая логика маршрутизатора
 - Перекрывающиеся диапазоны partition
 - Неоднородная нагрузка по shard
1. Записи «вне диапазонов» будут отброшены или упадут с ошибкой
 2. Запросы направляются не в тот shard, появятся ошибки данных
 3. Неопределённость, куда помещать данные, конфликт диапазонов
 4. Один shard перегружается, остальные простаивают

Подсказка: Думайте о граничных случаях и распределении нагрузки.

- **Отсутствие default partition** — Записи «вне диапазонов» будут отброшены или упадут с ошибкой
- **Слабая логика маршрутизатора** — Запросы направляются не в тот shard, появятся ошибки данных
- **Перекрывающиеся диапазоны partition** — Неопределённость, куда помещать данные, конфликт диапазонов
- **Неоднородная нагрузка по shard** — Один shard перегружается, остальные простаивают

🔮 Шаг 10:

Миссия 10

Сопоставьте способ масштабирования с ограничением в реализации ручного шардинга:

Ответ:

- Добавление нового partition
 - Перебалансировка данных вручную
 - Ребалансировка через скрипты
 - Автоматическое перераспределение данных
1. Требуется downtime или сложной логики остановки вставок
 2. Сложно управлять вручную при большом объёме
 3. Можно автоматизировать, но надо писать миграции
 4. Не поддерживается нативно в PostgreSQL для partitioning

Подсказка: Подумайте, какая часть приходится на руки инженера, какая — на СУБД.

- **Добавление нового partition** — Требуется downtime или сложной логики остановки вставок
- **Перебалансировка данных вручную** — Сложно управлять вручную при большом объёме

- **Ребалансировка через скрипты** — Можно автоматизировать, но надо писать миграции
- **Автоматическое перераспределение данных** — Не поддерживается нативно в PostgreSQL для partitioning

🔮 Шаг 11:

Миссия 11

Упорядочите шаги ручного шардинга через partitioning:

1. Создать parent таблицу без данных
2. Добавить trigger-функцию RoutingInsert
3. Определить ключ шардинга и шаблоны диапазонов
4. Создать child partition для каждого диапазона
5. Настроить default partition на крайние случаи
6. Проверить маршрутизацию и корректность INSERT/SELECT

Подсказка: Сначала план, потом реализация, затем проверка.

Ответ:

- Создать parent таблицу без данных
- Определить ключ шардинга и шаблоны диапазонов
- Создать child partition для каждого диапазона
- Добавить trigger-функцию RoutingInsert
- Настроить default partition на крайние случаи
- Проверить маршрутизацию и корректность INSERT/SELECT

🔮 Шаг 12:

Миссия 12

Упорядочите типичные этапы отказа shard:

1. Shard становится недоступен (сбой)
2. Рута INSERT/SELECT направляет на недоступный shard
3. Система переключается на fallback или default shard
4. Восстанавливается недоступный shard и синхронизируются данные

Подсказка: Подумайте, как хамелеон-маршрутизатор будет реагировать на отказ.

Ответ:

- Shard становится недоступен (сбой)
- Рута INSERT/SELECT направляет на недоступный shard
- Система переключается на fallback или default shard
- Восстанавливается недоступный shard и синхронизируются данные

🔮 Шаг 13:

Миссия 13

Упорядочите действия при добавлении нового диапазона shard без простоя:

1. Создать новую child-partition
2. Обновить логику маршрутизатора (INSERT/SELECT)
3. Перенести часть данных из старого partition в новый
4. Обновить default partition, если нужно
5. Проверить целостность данных и корректность маршрутов

Подсказка: Перемещение данных можно делать постепенно, а не одним махом.

Ответ:

- Создать новую child-partition
- Обновить логику маршрутизатора (INSERT/SELECT)
- Перенести часть данных из старого partition в новый
- Обновить default partition, если нужно
- Проверить целостность данных и корректность маршрутов

Шаг 14:

Миссия 14

Упорядочите шаги отладки маршрутизатора:

1. Включить логирование маршрута
2. Запустить INSERT с разными значениями ключа
3. Сравнить куда направлены вставки vs. ожидание
4. Исправить логику routing-функции
5. Повторить тесты и убедиться в корректности

Подсказка: Отладка — цикл: наблюдение → изменение → проверка.

Ответ:

- Включить логирование маршрута
- Запустить INSERT с разными значениями ключа
- Сравнить куда направлены вставки vs. ожидание
- Исправить логику routing-функции
- Повторить тесты и убедиться в корректности

Шаг 15:

Миссия 15

Упорядочите шаги миграции при изменении критерия шардинга:

1. Оставить старые shard доступными для чтения
2. Писать в оба шарда по новому ключу (dual write)
3. Перенести исторические данные под новый ключ
4. Обновить маршрутизатор на новый ключ
5. Отключить старую логику после полного перехода

Подсказка: Минимизируйте окна, когда данные недоступны.

Ответ:

- Оставить старые shard доступными для чтения
- Писать в оба шарда по новому ключу (dual write)
- Перенести исторические данные под новый ключ
- Обновить маршрутизатор на новый ключ
- Отключить старую логику после полного перехода

Шаг 16:

Миссия 16

Упорядочите шаги восстановления shard после сбоя:

1. Поднять сервер shard

2. Синхронизировать недостающие данные (реверс репликация или дамп)
3. Переключить маршрутизатор вновь на shard
4. Проверить консистентность данных
5. Убрать fallback-режим маршрутизации

Подсказка: Восстановление делается аккуратно, чтобы не потерять данные.

Ответ:

- Поднять сервер shard
- Синхронизировать недостающие данные (реверс репликация или дамп)
- Переключить маршрутизатор вновь на shard
- Проверить консистентность данных
- Убрать fallback-режим маршрутизации

Шаг 17:

Миссия 17

Упорядочите полный сценарий запуска ручного шардинга в проекте:

1. Проанализировать данные и нагрузку
2. Выбрать ключ шардинга и диапазоны
3. Спроектировать partitioning и структуру child-таблиц
4. Реализовать routing-триггеры и логику SELECT
5. Провести миграцию исторических данных
6. Запустить на продакшн и контролировать метрики

Подсказка: Начинайте с анализа, заканчивайте мониторингом.

Ответ:

- Проанализировать данные и нагрузку
- Выбрать ключ шардинга и диапазоны
- Спроектировать partitioning и структуру child-таблиц
- Реализовать routing-триггеры и логику SELECT
- Провести миграцию исторических данных
- Запустить на продакшн и контролировать метрики

Урок 3 - Введение в Citus и его архитектура

Шаг 01:

Миссия 1

Вам выдали задачу: протестировать кластерный режим Citus. С чего логичнее начать?

Подсказка: Прежде чем развернуть кластер — убедитесь, что базовая установка работает.

Ответ:

- Сразу шардировать данные
- **Установить отдельный узел и попробовать распределённые запросы**
- Нарисовать архитектуру в блок-схеме

Шаг 02:

Миссия 2

Вы хотите узнать — как Citus распределяет данные внутри шарда. Что спросить у коллег?

Подсказка: Интересует физическая распределённость, не логика запросов.

Ответ:

- Какие индексы на таблице
- **На каком уровне Citus разбивает таблицу** — сегменты или шарды
- Какой автокоммит используется

Шаг 03:

Миссия 3

Сопоставьте компонент Citus и его роль.

Компоненты:

Ответ:

- Coordinator
- Worker
- Shard
- Distribution column

Роли:

1. Сервер, управляющий планированием запросов
2. Сервер, хранящий данные части шардов
3. Подмножество таблицы, хранящее часть строк
4. Колонка, по которой происходит распределение строк

Подсказка: Подумайте, как данные “рассеиваются” по узлам.

- **Coordinator** — Сервер, управляющий планированием запросов
- **Worker** — Сервер, хранящий данные части шардов
- **Shard** — Подмножество таблицы, хранящее часть строк
- **Distribution column** — Колонка, по которой происходит распределение строк

Шаг 04:

Миссия 4

Сопоставьте понятие и пример в контексте Citus.

Понятия:

Ответ:

- Reference table
- Distributed table
- Colocated joins
- Replication factor

Примеры:

1. Таблица, копируемая на каждый worker
2. Таблица, разбитая по distribution column
3. Связь таблиц, когда join не требует сетевого обмена
4. Количество реплик шардов

Подсказка: Подумайте, какие таблицы легче реплицировать, а какие — шардировать.

- **Reference table** — Таблица, копируемая на каждый worker

- **Distributed table** — Таблица, разбитая по distribution column
- **Colocated joins** — Связь таблиц, когда join не требует сетевого обмена
- **Replication factor** — Количество реплик шардов

🔮 Шаг 05:

Миссия 5

Сопоставьте тип запросов и как Citus их обрабатывает.

Типы запросов:

Ответ:

- Simple SELECT по distributed table с фильтром по distribution column
- Большой агрегат по всей таблице
- JOIN между двумя distributed таблицами с разным distribution column
- INSERT, задающий значение distribution column

Как обрабатывается:

1. Данные собираются с всех shard-узлов
2. Запрос направляется только к узлу, где находится нужный shard
3. Раздается перекрестный обмен данных между узлами
4. Вставка идет прямо на правильный shard без пересылки

Подсказка: Фильтры по ключам ускоряют маршрутизацию.

- **Simple SELECT по distributed table с фильтром по distribution column** — Запрос направляется только к узлу, где находится нужный shard
- **Большой агрегат по всей таблице** — Данные собираются с всех shard-узлов
- **JOIN между двумя distributed таблицами с разным distribution column** — Раздается перекрестный обмен данных между узлами
- **INSERT, задающий значение distribution column** — Вставка идет прямо на правильный shard без пересылки

🔮 Шаг 06:

Миссия 6

Сопоставьте термин и проблему при использовании Citus.

Термины:

Ответ:

- Skew (скаучивание)
- Hot shard
- Replication lag
- Cross-shard transaction

Проблемы:

1. Когда один shard получает слишком много запросов
2. Когда данные между репликами синхронизируются с задержкой
3. Когда распределение неравномерно (одни шарды перегружены)
4. Транзакция затрагивает несколько shard-узлов

Подсказка: Контроль за нагрузкой и задержками важен в кластере.

- **Skew (скаучивание)** — Когда распределение неравномерно (одни шарды перегружены)

- **Hot shard** — Когда один shard получает слишком много запросов
- **Replication lag** — Когда данные между репликами синхронизируются с задержкой
- **Cross-shard transaction** — Транзакция затрагивает несколько shard-узлов

Шаг 07:

Миссия 7

Расположите этапы запуска кластера Citus (coordinator + workers) в правильном порядке:

1. Привязать узлы workers к coordinator
2. Запустить PostgreSQL на каждом узле
3. Настроить конфигурацию сети между узлами
4. Создать distributed table на coordinator
5. Добавить worker-узлы в кластер

Подсказка: Сначала инфраструктура, потом конфигурация, потом логика распределения.

Ответ:

- Запустить PostgreSQL на каждом узле
- Настроить конфигурацию сети между узлами
- Привязать узлы workers к coordinator
- Добавить worker-узлы в кластер
- Создать distributed table на coordinator

Шаг 08:

Миссия 8

Расположите шаги при выполнении запроса SELECT с фильтром на distribution column:

1. Coordinator получает запрос от клиента
2. Coordinator направляет подзапрос к конкретному worker
3. Worker выполняет локальный SELECT на нужном шарде
4. Worker возвращает результат
5. Coordinator агрегирует и возвращает финальный ответ

Подсказка: Запрос обрабатывается как маршрут + исполнение.

Ответ:

- Coordinator получает запрос от клиента
- Coordinator направляет подзапрос к конкретному worker
- Worker выполняет локальный SELECT на нужном шарде
- Worker возвращает результат
- Coordinator агрегирует и возвращает финальный ответ

Шаг 09:

Миссия 9

Расположите шаги при выполнении глобального агрегата (например, COUNT) на distributed table:

1. Coordinator получает запрос от клиента
2. Coordinator преобразует запрос в подзапросы для всех шардов
3. Worker узлы выполняют локальные агрегаты
4. Workers возвращают промежуточные значения
5. Coordinator агрегирует промежуточные результаты и возвращает клиенту

Подсказка: Глобальный агрегат требует участия всех шардов.

Ответ:

- Coordinator получает запрос от клиента
- Coordinator преобразует запрос в подзапросы для всех шардов
- Worker узлы выполняют локальные агрегаты
- Workers возвращают промежуточные значения
- Coordinator агрегирует промежуточные результаты и возвращает клиенту

🔮 Шаг 10:

Миссия 10

Вы заметили, что один shard получает значительно больше нагрузки. Как вы поступите?

Подсказка: Может быть дисбаланс распределения (skew).

Ответ:

- **Перепределить distribution column или использовать hash partitioning**
- Добавить ещё реплик на этот shard
- **Игнорировать** — нагрузка сама сгладится
- Убрать этот shard из кластера

🔮 Шаг 11:

Миссия 11

Вы хотите уменьшить сетевой обмен при JOIN разных distributed таблиц. Что может помочь?

Подсказка: Подумайте о совпадении distribution column и стратегии colocated joins.

Ответ:

- **Выбрать одинаковый distribution column для обеих таблиц**
- Поставить индексы на обе таблицы
- Реплицировать обе таблицы

🔮 Шаг 12:

Миссия 12

Сопоставьте стратегию шардирования и её применение.

Стратегии:

Ответ:

- Hash-based sharding
- Range-based sharding
- List-based sharding
- Custom shard placement

Применение:

1. Данные распределяются по диапазонам значений, например, по времени
2. Задано множество ключей, каждая строка попадает в один список
3. Используется хэш функции от ключа
4. Определён вручную, на каких узлах будут шарды

Подсказка: Обычно Citus использует hash, но может быть range в некоторых случаях.

- **Hash-based sharding** — Используется хэш функции от ключа

- **Range-based sharding** — Данные распределяются по диапазонам значений, например, по времени
- **List-based sharding** — Задано множество ключей, каждая строка попадает в один список
- **Custom shard placement** — Определён вручную, на каких узлах будут шарды

🔮 Шаг 13:

Миссия 13

Сопоставьте проблему и способ её решения в Citus.

Проблемы:

Ответ:

- Skew
- High replication lag
- Cross-shard transactions медленные
- Worker недоступен

Решения:

1. Использовать распределение по более сбалансированному ключу
2. Увеличить частоту синхронизации реплик или настроить потоки
3. Свести транзакции к одному shard или перепроектировать логику
4. Переназначить shard на здоровый узел и восстановить репликацию

Подсказка: Представьте реальные сбои и как вы бы реагировали.

- **Skew** — Использовать распределение по более сбалансированному ключу
- **High replication lag** — Увеличить частоту синхронизации реплик или настроить потоки
- **Cross-shard transactions медленные** — Свести транзакции к одному shard или перепроектировать логику
- **Worker недоступен** — Переназначить shard на здоровый узел и восстановить репликацию

🔮 Шаг 14:

Миссия 14

Расположите этапы миграции существующей PostgreSQL-базы в кластер Citus:

1. Создать distributed table на Citus coordinator
2. Экспортировать данные из обычной базы по фрагментам
3. Импортировать данные в shard-таблицы на worker-узлах
4. Перенести ссылки на приложение на coordinator
5. Проверить консистентность данных и целостность

Подсказка: Данные мигрируются последовательно, с проверкой на каждом этапе.

Ответ:

- Создать distributed table на Citus coordinator
- Экспортировать данные из обычной базы по фрагментам
- Импортировать данные в shard-таблицы на worker-узлах
- Перенести ссылки на приложение на coordinator
- Проверить консистентность данных и целостность

Шаг 15:

Миссия 15

Расположите шаги восстановления при отказе одного worker-узла:

1. Выключить кластер или пометить узел как нерабочий
2. Переназначить shard-реплики на другой узел
3. Запустить восстановление данных из реплик
4. Проверить целостность и синхронизацию реплик
5. Вернуть узел в кластер как реплика или мастер

Подсказка: Восстановление — это переадресация, восстановление и проверка.

Ответ:

- Выключить кластер или пометить узел как нерабочий
- Переназначить shard-реплики на другой узел
- Запустить восстановление данных из реплик
- Проверить целостность и синхронизацию реплик
- Вернуть узел в кластер как реплика или мастер

Шаг 16:

Миссия 16

Расположите ключевые этапы оптимизации производительности Citus на кластере:

1. Мониторинг метрик (CPU, I/O, latency)
2. Обнаружение hotspot или skew
3. Корректировка распределения или repartitioning
4. Повышение реплик или конфигурация потоковой репликации
5. Тестирование после изменений

Подсказка: Оптимизация — это итеративный цикл.

Ответ:

- Мониторинг метрик (CPU, I/O, latency)
- Обнаружение hotspot или skew
- Корректировка распределения или repartitioning
- Повышение реплик или конфигурация потоковой репликации
- Тестирование после изменений

Урок 4 - Шардирование таблиц с помощью Citus

Шаг 01:

Миссия 1

Аня — разработчик, запускает первую шарду с помощью Citus. Какой компонент отвечает за распределение записей по шардам?

Подсказка: Это компонент, который решает, в какую ноду кластер распределить запись.

Ответ:

- **Distributor (Citus coordinator)**
- Worker node
- PostgreSQL master instance

Шаг 02:

Миссия 2

Вася замечает, что некоторые запросы идут только на координатор. Что следует сделать, чтобы запросы читались с воркеров?

Подсказка: У Citus есть настройка, позволяющая читать на рабочих узлах.

Ответ:

- **Включить reference tables или репликацию read-only на воркерах**
- Отключить распределение
- Перенести все данные на координатор

Шаг 03:

Миссия 3

Света хочет изменить ключ шардинга таблицы, но таблица уже содержит данные. Какой подход безопаснее?

Подсказка: Подумайте о миграции: копирование + переименование.

Ответ:

- **Создать новую распределённую таблицу с новым ключом и постепенно перелить данные**
- Просто изменить ключ в DDL
- Удалить старую таблицу и сразу переименовать новую

Шаг 04:

Миссия 4

Игорь думает, можно ли сделать подзадачи (shard co-location), чтобы две таблицы хранились на одних и тех же шардах. Верно ли это?

Подсказка: Citus поддерживает co-location, если таблицы имеют одинаковый distribution key.

Ответ:

- **Да, если у таблиц одинаковый ключ распределения**
- Нет, co-location не поддерживается

Шаг 05:

Миссия 5

 Сопоставьте термин Citus с его описанием.

Термины:

Ответ:

- Coordinator
- Worker
- Reference table
- Distributed table

Описание:

1. Узел, принимающий клиентские запросы и распределяющий их
2. Узел, хранящий шардированные данные
3. Таблица, которая реплицируется на все узлы, не шардуется
4. Таблица, разбитая на шарды и распределённая по воркерам

Подсказка: Координатор — точка входа, reference — часто маленькие справочные таблицы.

- **Coordinator** — Узел, принимающий клиентские запросы и распределяющий их
- **Worker** — Узел, хранящий шардированные данные
- **Reference table** — Таблица, которая реплицируется на все узлы, не шардуется
- **Distributed table** — Таблица, разбитая на шарды и распределённая по воркерам

🔧 Шаг 06:

Миссия 6

📖 Сопоставьте понятие и пример из практики шардинга:

Понятия:

Ответ:

- Co-location
- Rebalancing
- Hash distribution
- Range distribution

Примеры:

1. Разгрузка данных при изменении числа шардов
2. Две таблицы с одинаковым ключом хранятся на тех же шардах
3. Записи распределяются по хешу ключа
4. Записи распределяются по диапазону значений ключа

Подсказка: Rebalancing — это перераспределение, co-location — совместное хранение.

- **Co-location** — Две таблицы с одинаковым ключом хранятся на тех же шардах
- **Rebalancing** — Разгрузка данных при изменении числа шардов
- **Hash distribution** — Записи распределяются по хешу ключа
- **Range distribution** — Записи распределяются по диапазону значений ключа

🔧 Шаг 07:

Миссия 7

📖 Сопоставьте ситуацию с рекомендацией при шардинге:

Ситуации:

Ответ:

- Одновременные горячие точки на одном ключе
- Большие блоки данных между узлами
- Небольшие справочные таблицы
- Сложные join'ы по разным ключам

Рекомендации:

1. Использовать reference table или копирование
2. Перераспределить шард или увеличить число шардов (rebalancing)
3. Выбрать равномерный ключ распределения
4. Настроить co-location или применять colocated joins

Подсказка: Join'ы проще, когда таблицы находятся на тех же шардах.

- **Одновременные горячие точки на одном ключе** — Выбрать равномерный ключ распределения

- **Большие блоки данных между узлами** — Перераспределить шард или увеличить число шардов (rebalancing)
- **Небольшие справочные таблицы** — Использовать reference table или копирование
- **Сложные join'ы по разным ключам** — Настроить co-location или применять colocated joins

🔧 Шаг 08:

Миссия 8

📖 Сопоставьте вид нагрузки и соответствующее действие с Citus:

Нагрузка:

Ответ:

- Запросы, читающие массово данные по диапазону
- Поток вставок по случайным ключам
- Агрегации по ключу
- Сложные join'ы между таблицами с разными ключами

Действия:

1. Использовать range distribution
2. Выбрать hash distribution
3. Использовать colocated tables или reference
4. Добавить агрегированные таблицы на воркерах (local aggregates)

Подсказка: Разные распределения соответствуют разным сценариям чтения/записи.

- **Запросы, читающие массово данные по диапазону** — Использовать range distribution
- **Поток вставок по случайным ключам** — Выбрать hash distribution
- **Агрегации по ключу** — Добавить агрегированные таблицы на воркерах (local aggregates)
- **Сложные join'ы между таблицами с разными ключами** — Использовать colocated tables или reference

🔧 Шаг 09:

Миссия 9

Расположите шаги создания распределённой таблицы в правильном порядке:

1. Добавить worker-ноды к кластеру
2. Создать таблицу как distributed (sharded)
3. Настроить DDL распределения (distribution key)
4. Заполнить таблицу данными

Подсказка: Сначала надо подготовить инфраструктуру, потом определиться с распределением, а затем уже наполнять таблицу.

Ответ:

- Добавить worker-ноды к кластеру
- Создать таблицу как distributed (sharded)
- Настроить DDL распределения (distribution key)
- Заполнить таблицу данными

🔧 Шаг 10:

Миссия 10

Расположите шаги безопасной миграции распределения ключа:

1. Создать новую распределённую таблицу с новым ключом
2. Копировать данные пакетами из старой таблицы
3. Переименовать старую таблицу как backup
4. Переименовать новую таблицу в имя старой

Подсказка: Это стандартный паттерн zero-downtime миграции: создать новую, скопировать, поменять имена.

Ответ:

- Создать новую распределённую таблицу с новым ключом
- Копировать данные пакетами из старой таблицы
- Переименовать старую таблицу как backup
- Переименовать новую таблицу в имя старой

Шаг 11:

Миссия 11

Расположите этапы rebalancing при добавлении нового worker:

1. Обновление распределения метаданных
2. Перенос шардов на новый worker
3. Отключение новых записей на время миграции (в некоторых сценариях)
4. Проверка целостности и завершение миграции

Подсказка: Перенос шардов — важный этап; обновление метаданных должно быть синхронизировано.

Ответ:

- Обновление распределения метаданных
- Перенос шардов на новый worker
- Отключение новых записей на время миграции (в некоторых сценариях)
- Проверка целостности и завершение миграции

Шаг 12:

Миссия 12

Расположите действия при планировании join'ов между таблицами:

1. Проверка, имеют ли таблицы одинаковый distribution key
2. **Если нет** — рассмотреть reference одну из таблиц
3. **Если совместимы** — выполнить colocated join
4. **Если очень разнесены** — возможно сделать промежуточную агрегацию

Подсказка: Join'ы проще, когда ключи совпадают; иначе нужны обходные пути.

Ответ:

- Проверка, имеют ли таблицы одинаковый distribution key
- **Если нет** — рассмотреть reference одну из таблиц
- **Если совместимы** — выполнить colocated join
- **Если очень разнесены** — возможно сделать промежуточную агрегацию

Шаг 13:

Миссия 13

Расположите шаги диагностики горячего ключа:

1. Проанализировать распределение записей по шард-ключам
2. Выявить ключи с непропорционально большим числом записей
3. Изменить ключ распределения на более равномерный
4. Перераспределить данные или создать дополнительную шардную группу

Подсказка: Найти «горячий ключ» → исправить распределение → перераспределить.

Ответ:

- Проанализировать распределение записей по шард-ключам
- Выявить ключи с непропорционально большим числом записей
- Изменить ключ распределения на более равномерный
- Перераспределить данные или создать дополнительную шардную группу

Шаг 14:

Миссия 14

Расположите этапы отказоустойчивости кластера с Citus:

1. Настроить репликацию слотов и потоковую репликацию между воркерами
2. Разместить копии шардов на разных узлах
Проверить переключение пода на резервный узел при сбое
3. Обновлять координатор резервными координаторами

Подсказка: Высокая доступность требует реплик и тестов переключения.

Ответ:

- Настроить репликацию слотов и потоковую репликацию между воркерами
- Разместить копии шардов на разных узлах
- Проверить переключение пода на резервный узел при сбое
- Обновлять координатор резервными координаторами

Шаг 15:

Миссия 15

Расположите шаги мониторинга нагрузки шардинга:

1. Собирать метрики распределения записи и чтения per shard
2. Отслеживать дисбаланс между воркерами
3. Запуск rebalancing при отклонении порога
4. Оповестить инженеров и провести анализ корня проблемы

Подсказка: Мониторинг → выявление дисбаланса → реакция → анализ.

Ответ:

- Собирать метрики распределения записи и чтения per shard
- Отслеживать дисбаланс между воркерами
- Запуск rebalancing при отклонении порога
- Оповестить инженеров и провести анализ корня проблемы

🔧 Шаг 16:

Миссия 16

Расположите стратегические шаги масштабирования кластера:

1. Проанализировать текущую нагрузку по узлам
2. Решить, добавить воркеры или пересчитать шардовку
3. Выполнить rebalancing / перераспределение
4. Тестировать отказоустойчивость и производительность после изменений

Подсказка: Масштабирование — это не просто добавление узлов, а также балансировка и верификация.

Ответ:

- Проанализировать текущую нагрузку по узлам
- Решить, добавить воркеры или пересчитать шардовку
- Выполнить rebalancing / перераспределение
- Тестировать отказоустойчивость и производительность после изменений

📖 Урок 5 - Распределённые запросы и ограничения

🔧 Шаг 01:

Миссия 1

Вы — инженер, работающий с распределённой базой данных. Какой из вариантов будет самым простым способом направить запрос к нужному шардированному узлу?

Подсказка: Подумайте, что должен знать маршрутизатор запросов.

Ответ:

- **Маршрутизатор на основе ключа шардирования знает целевой узел**
- Каждый запрос отправляем на все узлы, выбирая первый ответ
- Клиент сам угадывает адрес узла вручную

🔧 Шаг 02:

Миссия 2

При шардировании по пользователю `user_id`, какой вариант неверен?

Подсказка: Подумайте, какие варианты могут приводить к «горячим» шардам.

Ответ:

- **Все пользователи с id 1-10000 на одном шарде**
- Распределённый хеш `user_id mod N`
- Диапазонные шарды по `user_id`
- Гибрид хеш+диапазон по `region` и `user_id`

🔧 Шаг 03:

Миссия 3

Вы обнаружили, что один шард перегружен. Что можно предпринять первым шагом?

Подсказка: Подумайте о перераспределении или репликации.

Ответ:

- **Рассмотреть правило миграции части диапазона на другой шард**
- Немедленно остановить все записи на этом шарде

- Дублировать все данные вручную

🔮 Шаг 04:

Миссия 4

Какой подход можно использовать, чтобы распределённый запрос к двум разным шардам не нарушал целостность данных?

Подсказка: Подумайте о распределённых транзакциях.

Ответ:

- **Двухфазная фиксация (2PC) между шардами**
- Просто обновить независимые шарды без координации
- Игнорировать целостность
- Использовать одну централизованную таблицу на одном узле

🔮 Шаг 05:

Миссия 5

Когда приложение посылает распределённый SELECT на все шарды, что нужно делать на стороне сборщика результатов?

Подсказка: Подумайте о дедупликации и сортировке.

Ответ:

- **Собрать, отсортировать и убрать дубликаты**
- Просто взять первые N от каждого шарда
- Игнорировать порядок

🔮 Шаг 06:

Миссия 6

Сопоставьте тип распределённого запроса и его характер:

Типы запросов:

Ответ:

- Горизонтальный SELECT
- Горизонтальный INSERT/UPDATE/DELETE
- Кросс-шардовый JOIN
- Агрегатный запрос по всем шардам

Характер запроса:

1. Объединение строк из разных таблиц на разных шардах
2. Выборка из таблицы, распределённой по диапазону
3. Операция записи, где ключ определяет целевой шард
4. Подсчёт, например, SUM или COUNT по всем узлам

Подсказка: Подумайте, что делает каждая операция и как распределение влияет на неё.

- **Горизонтальный SELECT** — Выборка из таблицы, распределённой по диапазону
- **Горизонтальный INSERT/UPDATE/DELETE** — Операция записи, где ключ определяет целевой шард
- **Кросс-шардовый JOIN** — Объединение строк из разных таблиц на разных шардах
- **Агрегатный запрос по всем шардам** — Подсчёт, например, SUM или COUNT по всем узлам

🔗 Шаг 07:

Миссия 7

Сопоставьте проблему распределённых ограничений и её решение:

Проблемы:

Ответ:

- Нарушение внешнего ключа между разными шардированными таблицами
- Уникальность значения по всем шартам
- Транзакции охватывают разные шарды
- Формирование агрегатов в распределённой среде

Решения:

1. Борьба с «утечкой» ограничений через 2PC или согласование
2. Глобальные sequence или централизованный каталог значений
3. Разделение данных так, чтобы каскад ограничений был локальным
4. Сбор агрегатов на координаторе с частичными вычислениями на шардах

Подсказка: Каждой проблеме нужен свой подход, не пытайтесь применять универсальное решение сразу.

- **Нарушение внешнего ключа между разными шардированными таблицами** — Разделение данных так, чтобы каскад ограничений был локальным
- **Уникальность значения по всем шартам** — Глобальные sequence или централизованный каталог значений
- **Транзакции охватывают разные шарды** — Борьба с «утечкой» ограничений через 2PC или согласование
- **Формирование агрегатов в распределённой среде** — Сбор агрегатов на координаторе с частичными вычислениями на шардах

🔗 Шаг 08:

Миссия 8

Сопоставьте поведение запроса и проблему, которая может возникнуть:

Запрос:

Ответ:

- SELECT с ORDER BY по не-шардированному полю
- UPDATE, меняющий ключ шардирования
- DELETE из всех шардов без указания ключа
- INSERT с случайным ключом вне распределения

Возможная проблема:

1. **Нельзя пересчитать локальный шард** — требуется полное сканирование всех узлов
2. Запись может попасть на неправильный узел, потребуется миграция строки между шардами
3. Невозможно быстро определить, куда удалить без знания ключа
4. Запись попадёт в неправильный диапазон или вызовет конфликт распределения

Подсказка: Подумайте, что именно нарушается, если запрос не учитывает шардирование.

- **SELECT с ORDER BY по не-шардированному полю** — Нельзя пересчитать локальный шард — требуется полное сканирование всех узлов

- **UPDATE, меняющий ключ шардирования** — Запись может попасть на неправильный узел, потребуется миграция строки между шардами
- **DELETE из всех шардов без указания ключа** — Невозможно быстро определить, куда удалить без знания ключа
- **INSERT с случайным ключом вне распределения** — Запись попадёт в неправильный диапазон или вызовет конфликт распределения

🔮 Шаг 09:

Миссия 9

Сопоставьте стратегию с её преимуществом:

Стратегии:

Ответ:

- Репликация+шардирование
- Таблица глобальных уникальных ID
- Композиционные ключи (region + id)
- Локальные внешние ключи в пределах шарда

Преимущества:

1. Снижение конфликтов уникальности при вставках
2. Уменьшение межшардовых зависимостей
3. Высокое чтение на репликах плюс разделение нагрузки
4. **Часть ключа определяет шард, другая** — уникальность внутри него

Подсказка: Представьте, что делает каждая стратегия и какой вызов она решает.

- **Репликация+шардирование** — Высокое чтение на репликах плюс разделение нагрузки
- **Таблица глобальных уникальных ID** — Снижение конфликтов уникальности при вставках
- **Композиционные ключи (region + id)** — Часть ключа определяет шард, другая — уникальность внутри него
- **Локальные внешние ключи в пределах шарда** — Уменьшение межшардовых зависимостей

🔮 Шаг 10:

Миссия 10

Расположите этапы выполнения распределённого SELECT по шардам:

1. Отправка запроса ко всем шардам или выбранным
2. Сбор ответов с узлов
3. Фильтрация/дедупликация/сортировка на координаторе
4. Возврат результата клиенту

🔮 **Подсказка:**

Подумайте: сначала отправка, затем сбор, потом постобработка и выдача клиенту.

Ответ:

- Отправка запроса ко всем шардам или выбранным
- Сбор ответов с узлов
- Фильтрация/дедупликация/сортировка на координаторе
- Возврат результата клиенту

Шаг 11:

Миссия 11

Расположите стадии распределённой транзакции (2PC):

1. Фаза подготовки (prepare) на каждом участнике
2. Фаза подтверждения (commit) или отката (rollback)
3. Запрос координатором о готовности участников
4. Сбор ответов от участников (yes/no)

Подсказка:

Координатор спрашивает, затем собирает ответы, затем решает и сообщает.

Ответ:

- Фаза подготовки (prepare) на каждом участнике
- Фаза подтверждения (commit) или отката (rollback)
- Запрос координатором о готовности участников
- Сбор ответов от участников (yes/no)

Шаг 12:

Миссия 12

Расположите шаги обработки агрегатного запроса по шардам:

1. На каждом шарде выполняется частичная агрегация (например, SUM)
2. Отправка частичных результатов на координатор
3. Координатор объединяет результаты и вычисляет финальный агрегат
4. Клиент получает итог

Подсказка: Частичные вычисления помогают уменьшить передаваемые данные.

Ответ:

- На каждом шарде выполняется частичная агрегация (например, SUM)
- Отправка частичных результатов на координатор
- Координатор объединяет результаты и вычисляет финальный агрегат
- Клиент получает итог

Шаг 13:

Миссия 13

Расположите уровни ограничения целостности в распределённой БД:

1. Локальные ограничения на шард-таблицах (CHECK, UNIQUE внутри шарда)
2. Координация глобальных ограничений (например, через централизованный сервис)
3. Проверка при распределённых транзакциях (2PC)
4. Проверка на уровне приложения (до отправки запросов)

Подсказка: Контроль может быть на разных уровнях: приложение, шард, координация.

Ответ:

- Локальные ограничения на шард-таблицах (CHECK, UNIQUE внутри шарда)
- Координация глобальных ограничений (например, через централизованный сервис)
- Проверка при распределённых транзакциях (2PC)
- Проверка на уровне приложения (до отправки запросов)

Шаг 14:

Миссия 14

Расположите шаги реагирования на отказ шарда:

1. Определить, что шард недоступен
2. Переключить маршрутизацию на резервную реплику или другой шард
3. Восстановить или пересоздать недоступный шард
4. Синхронизировать данные и вернуть шард в работу

 **Подсказка: Жизненный цикл отказа — обнаружить, переключиться, восстановить, вернуться к норме.**

Ответ:

- Определить, что шард недоступен
- Переключить маршрутизацию на резервную реплику или другой шард
- Восстановить или пересоздать недоступный шард
- Синхронизировать данные и вернуть шард в работу

Шаг 15:

Миссия 15

Расположите полный цикл работы распределённого запроса с ограничениями:

1. Клиент посылает запрос через координатор
2. Координатор направляет запрос к соответствующим шардам с учётом ограничений
3. Шард проверяет локальные ограничения, участвует в 2PC при необходимости
4. Координатор агрегирует ответы и проверяет глобальные ограничения
5. Клиент получает окончательный результат или ошибку

 **Подсказка: Думайте о порядке: от клиента к узлам и обратно, с проверками на каждом этапе.**

Ответ:

- Клиент посылает запрос через координатор
- Координатор направляет запрос к соответствующим шардам с учётом ограничений
- Шард проверяет локальные ограничения, участвует в 2PC при необходимости
- Координатор агрегирует ответы и проверяет глобальные ограничения
- Клиент получает окончательный результат или ошибку

Модуль 5 - Высокая доступность — путь отказа

Урок 1 - Архитектура ActivePassive и ActiveActive

Шаг 01:

Миссия 1

Сопоставьте режим репликации и его преимущество:

Режимы репликации:

Ответ:

- Синхронная репликация
- Асинхронная репликация
- Полусинхронная репликация (с ожиданием подтверждения)

- Логическая репликация

Преимущества:

1. Минимальная потеря данных при сбое
2. Меньшая задержка для записи
3. Гибкость в схемах, возможность репликации отдельных таблиц
4. Баланс между задержкой и безопасностью

Подсказка: Подумайте, какому сценарию соответствует каждый режим.

- **Синхронная репликация** — Минимальная потеря данных при сбое
- **Асинхронная репликация** — Меньшая задержка для записи
- **Полусинхронная репликация (с ожиданием подтверждения)** — Баланс между задержкой и безопасностью
- **Логическая репликация** — Гибкость в схемах, возможность репликации отдельных таблиц

Шаг 02:

Миссия 2

Сопоставьте состояние HA-системы и характер поведения:

Состояние:

Ответ:

- Мастер в фокусе
- Failover происходит
- Пассивный узел стал мастером
- Восстановление старого мастера

Характер поведения:

1. Запросы на мастер перенаправляются на нового мастера
2. Система переключается после отказа мастера
3. Пассивный узел начинает принимать записи
4. Старая нода синхронизируется как реплика

Подсказка: Задумайтесь, как проходит жизненный цикл переключения.

- **Мастер в фокусе** — Запросы на мастер перенаправляются на нового мастера
- **Failover происходит** — Система переключается после отказа мастера
- **Пассивный узел стал мастером** — Пассивный узел начинает принимать записи
- **Восстановление старого мастера** — Старая нода синхронизируется как реплика

Шаг 03:

Миссия 3

Сопоставьте компонент HA-системы и его функцию:

Компоненты:

Ответ:

- Watcher / мониторинг
- Служба failover (arbiter или менеджер)
- Virtual IP / floating IP
- Репликация между узлами

Функции:

1. Следит за доступностью главного узла
2. Меняет роли узлов при сбое
3. Перенаправляет трафик к активному узлу
4. Синхронизирует данные между узлами

Подсказка: Какую роль играет каждый компонент HA?

- **Watcher / мониторинг** — Следит за доступностью главного узла
- **Служба failover (arbiter или менеджер)** — Меняет роли узлов при сбое
- **Virtual IP / floating IP** — Перенаправляет трафик к активному узлу
- **Репликация между узлами** — Синхронизирует данные между узлами

Шаг 04:

Миссия 4

Сопоставьте проблему HA и способ её смягчения:

Проблема:

Ответ:

- Split-brain
- Задержки репликации
- Неопределённость состояния при переключении
- Утеря данных при асинхронной репликации

Способ смягчения:

1. Кворум или голосование для избежания split-brain
2. Использование синхронной репликации или ожидание подтверждения
3. Согласованный failover через менеджер
4. Использование журналов и подтверждение записи перед фиксацией

Подсказка: Соотнесите каждую проблему с подходящим решением.

- **Split-brain** — Кворум или голосование для избежания split-brain
- **Задержки репликации** — Использование синхронной репликации или ожидание подтверждения
- **Неопределённость состояния при переключении** — Согласованный failover через менеджер
- **Утеря данных при асинхронной репликации** — Использование журналов и подтверждение записи перед фиксацией

Шаг 05:

Миссия 5

Сопоставьте тип конфликта Active/Active и метод его разрешения:

Тип конфликта:

Ответ:

- Конфликт по версии (версионное противоречие)
- Последовательность уникальных ID в обоих узлах
- Синхронизация настроек (конфигурации)
- Зависимости между записями на разных узлах

Метод разрешения:

1. Применение последнего write или разрешение вручную
2. Использование глобальных sequence или сегментации ID
3. Централизованная конфигурационная служба
4. Выбор доминирующего узла или разделение по области ответственности

Подсказка: Обдумайте, каким образом можно согласовывать конфликты.

- **Конфликт по версии (версионное противоречие)** — Применение последнего write или разрешение вручную
- **Последовательность уникальных ID в обоих узлах** — Использование глобальных sequence или сегментации ID
- **Синхронизация настроек (конфигурации)** — Централизованная конфигурационная служба
- **Зависимости между записями на разных узлах** — Выбор доминирующего узла или разделение по области ответственности

Шаг 06:

Миссия 6

Расположите шаги failover в Active/Passive системе:

1. Мониторинг обнаруживает отказ мастера
2. Система переключает VIP или DNS на пассивный узел
3. Пассивный узел становится мастером и начинает принимать записи
4. Старая нода восстанавливается как реплика

Подсказка:

Подумайте, в каком порядке происходят события при failover.

Ответ:

- Мониторинг обнаруживает отказ мастера
- Система переключает VIP или DNS на пассивный узел
- Пассивный узел становится мастером и начинает принимать записи
- Старая нода восстанавливается как реплика

Шаг 07:

Миссия 7

Расположите этапы восстановления после сбоя:

1. Оценка повреждённых данных и логов
2. Восстановление базы данных или повторная репликация
3. Переключение на восстановленную ноду, если она готова
4. Проверка целостности и согласованности

Подсказка:

Подумайте о порядке: сначала диагностика, затем восстановление, затем проверка и возвращение в кластер.

Ответ:

- Оценка повреждённых данных и логов
- Восстановление базы данных или повторная репликация
- Переключение на восстановленную ноду, если она готова

- Проверка целостности и согласованности

Шаг 08:

Миссия 8

Расположите этапы согласованного активного режима (Active/Active) при конфликте записи:

1. Обе ноды получают запись
2. Обнаружен конфликт версии
3. Система запускает алгоритм разрешения конфликта
4. Согласованный результат отправляется обеим нодам
5. Клиент получает окончательный ответ или ошибку

Подсказка:

Как разрешаются противоречия и как синхронизируются данные?

Ответ:

- Обе ноды получают запись
- Обнаружен конфликт версии
- Система запускает алгоритм разрешения конфликта
- Согласованный результат отправляется обеим нодам
- Клиент получает окончательный ответ или ошибку

Шаг 09:

Миссия 9

Расположите уровни контроля при Active/Active архитектуре:

1. Конфликт в приложении (client-side checks)
2. Решение на уровне БД (триггеры, правила)
3. Координация через механизм синхронизации
4. Разрешение через административный скрипт

Подсказка:

Контроль может быть на разных уровнях — представьте, где логика проверок наиболее уместна.

Ответ:

- Конфликт в приложении (client-side checks)
- Решение на уровне БД (триггеры, правила)
- Координация через механизм синхронизации
- Разрешение через административный скрипт

Шаг 10:

Миссия 10

Расположите шаги перехода с Active/Passive на Active/Active режим:

1. Оценка требований к согласованности и конфликтам
2. Настройка двунаправленной репликации между узлами
3. Определение схемы разрешения конфликтов
4. Тестирование сценариев конфликтов
5. Переключение приложения на оба узла для записи

Подсказка:

Переход требует планирования и тестов на конфликтные сценарии.

Ответ:

- Оценка требований к согласованности и конфликтам
- Настройка двунаправленной репликации между узлами
- Определение схемы разрешения конфликтов
- Тестирование сценариев конфликтов
- Переключение приложения на оба узла для записи

 Шаг 11:

Миссия 11

Расположите шаги инцидент-реакции при разделении сети (network partition):

1. Обнаружить отсутствие связи между узлами
2. Оценить, какую ветвь считать действительной (quorum)
3. Отключить одну из ветвей или заморозить записи
4. **При восстановлении связи** — разрешить конфликты и синхронизировать

Подсказка:

При разрыве сети нужно решить, какая часть продолжает работать, а затем потом синхронизировать.

Ответ:

- Обнаружить отсутствие связи между узлами
- Оценить, какую ветвь считать действительной (quorum)
- Отключить одну из ветвей или заморозить записи
- **При восстановлении связи** — разрешить конфликты и синхронизировать

 Шаг 12:

Миссия 12

Расположите полный жизненный цикл отказа и восстановления в Active/Active конфигурации:

1. Один из узлов выходит из строя или теряет связь
2. Система обнаруживает сбой и останавливает записи на убитой ветви
3. Оставшийся узел продолжает обслуживание
4. Восстановленный узел синхронизируется и применяет разрешение конфликтов
5. Записи возвращаются к обоим узлам с учётом согласования

Подсказка:

Процесс отказа и восстановления должен учитывать остановку записей, синхронизацию, разрешения конфликтов и восстановление записи на обоих узлах.

Ответ:

- Один из узлов выходит из строя или теряет связь
- Система обнаруживает сбой и останавливает записи на убитой ветви
- Оставшийся узел продолжает обслуживание
- Восстановленный узел синхронизируется и применяет разрешение конфликтов
- Записи возвращаются к обоим узлам с учётом согласования

Урок 2 - Обзор Patroni, Repmgr и auto_failover

Шаг 01:

Миссия 1

Вы — DBA, хотите внедрить Patroni для кластера PostgreSQL. Что из следующего наиболее фундаментально описывает роль Patroni?

Подсказка: Patroni — это не сам БД-кластер, а координатор HA.

Ответ:

- **Служба координации управления ролью мастера и автоматического failover**
- СУБД-движок, выполняющий запросы
- Монитор визуализатор нагрузки
- Бэкап-менеджер кластерных копий

Шаг 02:

Миссия 2

В сценарии с Repmgr вы хотите, чтобы failover происходил автоматически. Какая ключевая функция Repmgr обеспечивает это?

Подсказка: Процесс failover — не просто переключение, нужен мониторинг и управление.

Ответ:

- **Daemon-процесс repmgrd, который следит за состоянием и инициирует failover**
- Библиотека SQL-функций для репликации
- Интерфейс командной строки без демона
- GUI-панель управления кластером

Шаг 03:

Миссия 3

Сергей развернул кластер с Patroni. Какой компонент обычно используется совместно с Patroni для хранения конфигурации и кворума?

Подсказка: Подумайте о распределенном хранилище метаданных.

Ответ:

- **etcd или Consul или ZooKeeper (координационный сервис)**
- Redis
- MySQL
- Нативная таблица PostgreSQL без внешнего кворума

Шаг 04:

Миссия 4

Команда решила использовать auto_failover через Patroni. Что приоритетнее выбрать: синхронную репликацию или асинхронную, и почему?

Подсказка: Подумайте о максимально возможной потере данных.

Ответ:

- **Синхронную** — чтобы минимизировать потерю данных при failover
- **Асинхронную** — чтобы повысить производительность
- Выключить репликацию
- Использовать журналы WAL-архивации вместо репликации

🔮 Шаг 05:

Миссия 5

Ольга настраивает автоматический failover в Repmgr. Какой параметр в конфигурации особенно важен?

Подсказка: Настройка порога, при котором происходит переключение.

Ответ:

- **failover_on_failures и параметры мониторинга (election, health_check)**
- Чтение узла роли standby_only
- max_connections
- autovacuum настройки

🔮 Шаг 06:

Миссия 6

Сопоставьте компонент кластера с его основной ролью в Patroni/Repmgr архитектуре:

Компоненты:

Ответ:

- Клиентское приложение
- Patroni / Repmgr Daemon
- etcd / Consul / ZooKeeper
- PostgreSQL узел (мастер/реплика)

Роли:

1. Хранить состояние кластера и кворум
2. Принимать SQL-запросы и хранить данные
3. Слушать отказ и инициировать failover
4. Посылать запросы к активному узлу

Подсказка: Подумайте, какой компонент отвечает за что в системе HA.

- **Клиентское приложение** — Посылать запросы к активному узлу
- **Patroni / Repmgr Daemon** — Слушать отказ и инициировать failover
- **etcd / Consul / ZooKeeper** — Хранить состояние кластера и кворум
- **PostgreSQL узел (мастер/реплика)** — Принимать SQL-запросы и хранить данные

🔮 Шаг 07:

Миссия 7

Сопоставьте сценарий отказа и реакцию патрона (Patroni) или Repmgr:

Сценарии:

Ответ:

- Мастер перестал отвечать
- Реплика отстала по WAL
- Узел разделился от кворума (network partition)
- Новая нода стартовала и пытается подключиться как мастер

Реакции:

1. Иницируется failover к другому узлу
2. Отключить репликацию или отстранить узел от кворума

3. Отклонить роль мастера и стать репликой
4. Проверить состояние кластера, разрешить роль или запретить

Подсказка: Подумайте, как система реагирует в различных случаях отказов.

- **Мастер перестал отвечать** — Иницируется failover к другому узлу
- **Реплика отстала по WAL** — Отключить репликацию или отстранить узел от кворума
- **Узел разделился от кворума (network partition)** — Отклонить роль мастера и стать репликой
- **Новая нода стартовала и пытается подключиться как мастер** — Проверить состояние кластера, разрешить роль или запретить

Шаг 08:

Миссия 8

Сопоставьте параметр конфигурации Patroni/Repmgr и его назначение:

Параметры:

Ответ:

- `tli / loop_wait`
- `maximum_lag_on_failover`
- `retry_timeout`
- `slots / replication_slots`

Назначения:

1. Лимит задержки реплики для допуска к failover
2. Интервал ожидания между циклами мониторинга
3. Время, после которого пытается повторный опрос или попытка
4. Слоты для логической/физической репликации

Подсказка: У каждого параметра есть собственная роль, не путайте время и слоты.

- **`tli / loop_wait`** — Интервал ожидания между циклами мониторинга
- **`maximum_lag_on_failover`** — Лимит задержки реплики для допуска к failover
- **`retry_timeout`** — Время, после которого пытается повторный опрос или попытка
- **`slots / replication_slots`** — Слоты для логической/физической репликации

Шаг 09:

Миссия 9

Сопоставьте вызов CLI и его эффект в контексте Patroni/Repmgr:

Вызовы:

Ответ:

- `patronictl failover`
- `patronictl pause`
- `repmgr standby promote`
- `repmgr cluster show`

Эффекты:

1. Остановить автоматические переключения
2. Промоутировать реплику в мастер
3. Показать текущее состояние кластера

4. Запустить ручной failover

Подсказка: Некоторые команды запускают действия, другие — приостанавливают или информируют.

- **patronictl failover** — Запустить ручной failover
- **patronictl pause** — Остановить автоматические переключения
- **repmgr standby promote** — Промоутировать реплику в мастер
- **repmgr cluster show** — Показать текущее состояние кластера

Шаг 10:

Миссия 10

Расположите этапы автоматического failover через Patroni:

1. Patroni обнаруживает, что master не отвечает
2. Patroni пытается выбрать нового лидера через кворум etcd/Consul
3. Patroni переключает роли мастер/реплика
4. Переадресация клиента на новый мастер

Подсказка: Порядок: детекшн, кворум, переключение, клиентская переадресация.

Ответ:

- Patroni обнаруживает, что master не отвечает
- Patroni пытается выбрать нового лидера через кворум etcd/Consul
- Patroni переключает роли мастер/реплика
- Переадресация клиента на новый мастер

Шаг 11:

Миссия 11

Расположите шаги failover через Repmgr:

1. repmgrd обнаруживает недоступность primary
2. repmgrd выбирает нового ведущего узла согласно правилам голосования
3. repmgrd запускает promote на выбранной реплике
4. Клиенты переключаются к новому primary

Подсказка: Логика Repmgr похожа: мониторинг, выбор, повышение, переключение клиента.

Ответ:

- repmgrd обнаруживает недоступность primary
- repmgrd выбирает нового ведущего узла согласно правилам голосования
- repmgrd запускает promote на выбранной реплике
- Клиенты переключаются к новому primary

Шаг 12:

Миссия 12

Расположите стадии восстановления после автоматического failover:

1. Старый мастер становится репликой или отключается
2. Синхронизация данных между новым мастером и бывшим мастером
3. Обновление конфигурации HA (Patroni/Repmgr) о новых ролях

Возобновление обычной работы кластера

Подсказка: После failover надо синхронизировать, конфигурировать и восстановить нормальный статус.

Ответ:

- Старый мастер становится репликой или отключается
- Синхронизация данных между новым мастером и бывшим мастером
- Обновление конфигурации HA (Patroni/Repmgr) о новых ролях
- Возобновление обычной работы кластера

Шаг 13:

Миссия 13

Расположите шаги добавления новой реплики в кластер с Patroni/Repmgr:

1. Инициализация узла как standby / replica
2. Настройка конфигурации HA для нового узла
3. Запуск службы Patroni / репликация Repmgr
4. Подтверждение узла как робастной реплики в кластере

Подсказка: Новый узел должен быть правильно сконфигурирован, подключён и признан кластером.

Ответ:

- Инициализация узла как standby / replica
- Настройка конфигурации HA для нового узла
- Запуск службы Patroni / репликация Repmgr
- Подтверждение узла как робастной реплики в кластере

Шаг 14:

Миссия 14

Расположите шаги реагирования при split-brain в Patroni/Repmgr:

1. Обнаружить разделение кластеров / отсутствие кворума
2. Прекратить запись на ветвях без кворума
3. Применить процедуру разрешения конфликта (quorum, override)
4. Синхронизировать ветви и восстановить единство кластера

Подсказка: Split-brain требует остановки записи и согласования перед восстановлением.

Ответ:

- Обнаружить разделение кластеров / отсутствие кворума
- Прекратить запись на ветвях без кворума
- Применить процедуру разрешения конфликта (quorum, override)
- Синхронизировать ветви и восстановить единство кластера

Шаг 15:

Миссия 15

Расположите шаги тестирования автоматического failover на тестовом стенде:

1. Искусственно убить мастер-процесс или остановить узел
2. Наблюдать, как Patroni/Repmgr переключается на реплику

3. Проверить, что новая роль мастера принята и работает записи
4. Вернуть прежний узел и синхронизировать репликацию

Подсказка: Тестирование требует выключения, мониторинга, проверки и восстановления.

Ответ:

- Искусственно убить мастер-процесс или остановить узел
- Наблюдать, как Patroni/Repmgr переключается на реплику
- Проверить, что новая роль мастера принята и работает записи
- Вернуть прежний узел и синхронизировать репликацию

Шаг 16:

Миссия 16

Расположите полный цикл отказа и восстановления в Patroni/Repmgr с auto_failover:

1. Мастер теряет доступ или выходит из строя
2. Patroni/Repmgr обнаруживает отказ через демона и мониторинг
3. Инициализируется выбор нового master с помощью кворума и голосования
4. Новая реплика становится мастером, клиенты переключаются
5. Старый узел синхронизируется и возвращается как реплика
6. Кластер возвращается к нормальной работе

Подсказка: Это объединение всех фаз: отказ, переключение, восстановление, возврат к норме.

Ответ:

- Мастер теряет доступ или выходит из строя
- Patroni/Repmgr обнаруживает отказ через демона и мониторинг
- Инициализируется выбор нового master с помощью кворума и голосования
- Новая реплика становится мастером, клиенты переключаются
- Старый узел синхронизируется и возвращается как реплика
- Кластер возвращается к нормальной работе

Урок 3 - Настройка балансировки с Pgpool-II

Шаг 01:

Миссия 1

Вы настроили Pgpool-II перед основным рабочим днем, и один из слотов показывает статус «down». Что вы делаете первым?

Подсказка: важно диагностировать причину до перезапуска.

Ответ:

- **Посмотреть логи Pgpool и проверить соединение с бэкендами**
- Сразу перезапустить весь кластер Pgpool
- Отключить мониторинг слота и игнорировать его
- Удалить конфигурацию Pgpool и начать настройку заново

Шаг 02:

Миссия 2

При конфигурации Pgpool-II вам необходимо указать режим балансировки. Какой режим выбираете для минимального времени ответа клиентам?

Подсказка: разные режимы влияют, как распределяются SELECT-запросы.

Ответ:

- **least_connections (наименьшее число соединений)**
- round robin
- random
- load balance by weight

Шаг 03:

Миссия 3

Вы хотите, чтобы Pgpool-II автоматически переключал клиентов на реплику при падении мастера. Какой параметр включаете?

Подсказка: попробуйте подумать об автоматическом failover.

Ответ:

- **enable_pool_hba и failover_command**
- master_slave_mode = off
- disable load balancing
- health_check_period = 0

Шаг 04:

Миссия 4

Для режима health check вы хотите снизить нагрузку. Какой параметр стоит уменьшить?

Подсказка: здоровье проверяется через периодические запросы.

Ответ:

- **health_check_period**
- num_init_children
- connection_life_time
- client_idle_limit_in_recovery

Шаг 05:

Миссия 5

Один из слейвов сильно отстает по репликации, и вы не хотите, чтобы к нему шли SELECT. Что сделать?

Подсказка: Pgpool может исключать слейв из пула.

Ответ:

- **set backend flag по health check, чтобы не направлять запросы**
- удалить слейв из конфигурации навсегда
- увеличить load_balance_mode
- уменьшить timeout клиентского соединения

Шаг 06:

Миссия 6

Вы хотите логировать, какие запросы отправляются к репликам. Какую опцию включить?

Подсказка: это может помочь в дебаге распределения запросов.

Ответ:

- **log_statement_stats**
- log_min_error_statement
- statement_timeout
- log_connections

🔮 **Шаг 07:**

Миссия 7

Сопоставьте параметр Pgpool-II с его значением или смыслом.

Подсказка: представьте, что вы конфигурируете его шаг за шагом.

Параметры:

Ответ:

- listen_addresses
- port
- backend_hostname0
- health_check_timeout

Значения / смысл:

1. IP или интерфейс, на котором слушает Pgpool
2. имя или адрес первого бэкенд-сервера
3. максимальное время ожидания health check ответа
4. порт, на котором Pgpool принимает подключения

- **listen_addresses** — IP или интерфейс, на котором слушает Pgpool
- **port** — порт, на котором Pgpool принимает подключения
- **backend_hostname0** — имя или адрес первого бэкенд-сервера
- **health_check_timeout** — максимальное время ожидания health check ответа

🔮 **Шаг 08:**

Миссия 8

Сопоставьте механизм отказоустойчивости с его задачей в Pgpool-II.

Подсказка: подумайте, что делает failover и failback.

Механизмы:

Ответ:

- failover
- failback
- watchdog
- health check

Задачи:

1. возврат бывшего мастера в кластер, когда он снова доступен
2. мониторинг состояния серверов и реагирование на сбои
3. переключение клиентов на новый мастер при сбое текущего мастера
4. проверка доступности backend-серверов

- **failover** — переключение клиентов на новый мастер при сбое текущего мастера
- **failback** — возврат бывшего мастера в кластер, когда он снова доступен

- **watchdog** — мониторинг состояния серверов и реагирование на сбои
- **health check** — проверка доступности backend-серверов

🔧 Шаг 09:

Миссия 9

Сопоставьте логические термины с контекстом Pgpool-II.

Подсказка: подумайте, что означает «frontend» и «backend».

Термины:

Ответ:

- frontend
- backend
- pool
- node

Контексты:

1. клиентское соединение к Pgpool
 2. сервер PostgreSQL за Pgpool
 3. набор подключений к бэкендам
 4. отдельный экземпляр сервера в кластере
- **frontend** — клиентское соединение к Pgpool
 - **backend** — сервер PostgreSQL за Pgpool
 - **pool** — набор подключений к бэкендам
 - **node** — отдельный экземпляр сервера в кластере

🔧 Шаг 10:

Миссия 10

Сопоставьте команды Pgpool-II с тем, что они делают.

Подсказка: вспомните основные утилиты.

Команды:

Ответ:

- pcp_attach_node
- pcp_detach_node
- pcp_pool_status
- pcp_recovery_node

Действия:

1. проверка статуса пула бэкендов
 2. отключение ноды от пула
 3. присоединение ноды к пулу
 4. восстановление ноды после сбоя
- **pcp_attach_node** — присоединение ноды к пулу
 - **pcp_detach_node** — отключение ноды от пула
 - **pcp_pool_status** — проверка статуса пула бэкендов
 - **pcp_recovery_node** — восстановление ноды после сбоя

Шаг 11:

Миссия 11

Сопоставьте термины Pgpool-II с их параметрами конфигурации.

Подсказка: вспомните, что отвечает за таймауты и подключения.

Термины:

Ответ:

- `client_idle_limit`
- `connection_life_time`
- `num_init_children`
- `max_pool`

Назначения:

1. количество дочерних процессов при старте
2. максимальное число клиентских соединений в пуле
3. время жизни подключения от клиента к Pgpool
4. максимальное время, которое клиент может быть бездействующим

- **`client_idle_limit`** — максимальное время, которое клиент может быть бездействующим
- **`connection_life_time`** — время жизни подключения от клиента к Pgpool
- **`num_init_children`** — количество дочерних процессов при старте
- **`max_pool`** — максимальное число клиентских соединений в пуле

Шаг 12:

Миссия 12

Сопоставьте состояние backend-нод в Pgpool-II с их статусом.

Подсказка: обратите внимание, когда нода исключена или недоступна.

Состояния:

Ответ:

- `up`
- `down`
- `reject`
- `disabled`

Значения:

1. нода активно используется для запросов
2. нода недоступна или не отвечает
3. нода не берёт новых соединений
4. нода временно выключена из пула

- **`up`** — нода активно используется для запросов
- **`down`** — нода недоступна или не отвечает
- **`reject`** — нода не берёт новых соединений
- **`disabled`** — нода временно выключена из пула

Шаг 13:

Миссия 13

Расположите шаги failover через Pgpool-II в правильном порядке:

1. Запуск команды failover или автоматическое обнаружение
2. Обновление конфигурации Pgpool, перевод слейв в мастер
3. Перенаправление клиентов на новый мастер
4. Отключение старого мастера из пула
5. Проверка состояния нового мастера

Подсказка: подумайте, что должно произойти именно при сбое мастера.

Ответ:

- Запуск команды failover или автоматическое обнаружение
- Обновление конфигурации Pgpool, перевод слейв в мастер
- Перенаправление клиентов на новый мастер
- Отключение старого мастера из пула
- Проверка состояния нового мастера

Шаг 14:

Миссия 14

Расположите действия по настройке статистики и логирования Pgpool-II:

1. Включить log_statement_stats или подобную опцию
2. Настроить уровень логирования и файл логов
3. Перезапустить службу Pgpool-II
4. Убедиться, что статистика логируется и правильно показывает балансировку

Подсказка: выполняйте настройку постепенно, проверяя каждый шаг.

Ответ:

- Включить log_statement_stats или подобную опцию
- Настроить уровень логирования и файл логов
- Перезапустить службу Pgpool-II
- Убедиться, что статистика логируется и правильно показывает балансировку

Шаг 15:

Миссия 15

Расположите этапы реагирования на медленного слейва:

1. Обнаружение отставания репликации с помощью health check
2. Отключение слейва из пула
3. Устранение причины репликации (сеть, нагрузка)
4. Когда слейв догонит, вернуть его в пул
5. Проверка распределения запросов после возврата

Подсказка: сначала реакция, потом восстановление.

Ответ:

- Обнаружение отставания репликации с помощью health check
- Отключение слейва из пула
- Устранение причины репликации (сеть, нагрузка)
- Когда слейв догонит, вернуть его в пул
- Проверка распределения запросов после возврата

Шаг 16:

Миссия 16

Расположите шаги при ручной миграции мастера с помощью Pgpool:

1. Поднять нового мастера из слейва
2. Обновить конфигурацию Pgpool на всех нодах
3. Отключить старый мастер из пула
4. Выполнить switchover или failover команду
5. Перезапустить Pgpool на клиентах или приложениям уведомить о новом мастере

Подсказка: важен порядок переключения и уведомления.

Ответ:

- Поднять нового мастера из слейва
- Обновить конфигурацию Pgpool на всех нодах
- Отключить старый мастер из пула
- Выполнить switchover или failover команду
- Перезапустить Pgpool на клиентах или приложениям уведомить о новом мастере

Шаг 17:

Миссия 17

Расположите этапы полного цикла восстановления после отказа мастера:

1. Обнаружить сбой мастера
2. Failover через Pgpool или вручную
3. Убедиться, что новый мастер работает корректно
4. Failback: вернуть старый мастер, если возможен
5. Проверить и оптимизировать конфигурацию кластеров и Pgpool

Подсказка: цикл включает как аварийное переключение, так и возвращение к норме.

Ответ:

- Обнаружить сбой мастера
- Failover через Pgpool или вручную
- Убедиться, что новый мастер работает корректно
- Failback: вернуть старый мастер, если возможен
- Проверить и оптимизировать конфигурацию кластеров и Pgpool

Урок 4 - Использование HAProxy для фейловеров

Шаг 01:

Миссия 1

Ваша команда запускает HAProxy для обеспечения отказоустойчивости кластера PostgreSQL. Что является главной задачей HAProxy в этом сценарии?

Подсказка: Подумайте, какую роль может играть балансировщик нагрузки при отказе основного сервера.

Ответ:

- **Перенаправлять запросы на работающий сервер при сбое основного**
- Хранить данные пользователей
- Выполнять резервное копирование базы данных

- Оптимизировать SQL-запросы

Шаг 02:

Миссия 2

При настройке HAProxy вы обнаружили, что серверы PostgreSQL отвечают с задержкой. Какой параметр в конфигурации HAProxy поможет быстро определить неработающий сервер?

Подсказка: Вспомните, какой параметр отвечает за проверку доступности серверов.

Ответ:

- **health-check timeout**
- maxconn
- balance roundrobin
- timeout client

Шаг 03:

Миссия 3

Команда заметила, что при переключении на резервный сервер запросы становятся медленнее. Что из следующего могло стать причиной?

Подсказка: Обратите внимание на влияние кеша и состояния сессий.

Ответ:

- **Отсутствие актуального кеша и сессий на резервном сервере**
- Плохое качество сети у основного сервера
- Неправильная версия PostgreSQL
- Перегрузка HAProxy

Шаг 04:

Миссия 4

Команда заметила, что после перезапуска HAProxy все соединения с базой данных обрываются. Что следует добавить в конфигурацию, чтобы уменьшить влияние таких перезапусков?

Подсказка: Подумайте про управление соединениями с клиентами.

Ответ:

- **option httpclose**
- timeout connect
- maxconn
- balance source

Шаг 05:

Миссия 5

 Сопоставьте параметры HAProxy с их назначением в контексте работы с PostgreSQL:

Подсказка: Обратите внимание, какие параметры отвечают за балансировку, время ожидания и проверку состояния.

Ответ:

- balance
- timeout connect
- option tcp-check
- maxconn

- server

Определения:

1. Максимальное количество одновременных соединений с сервером
 2. Метод распределения нагрузки между серверами
 3. Настройка таймаута при подключении к серверу
 4. Опция для проверки здоровья TCP-соединения
 5. Определение адреса и состояния бэкенд-сервера
- **balance** — Метод распределения нагрузки между серверами
 - **timeout connect** — Настройка таймаута при подключении к серверу
 - **option tcp-check** — Опция для проверки здоровья TCP-соединения
 - **maxconn** — Максимальное количество одновременных соединений с сервером
 - **server** — Определение адреса и состояния бэкенд-сервера

Шаг 06:

Миссия 6

 Сопоставьте возможные проблемы с использованием HAProxy и их причины:

Подсказка: Проанализируйте, как конфигурация влияет на поведение системы при сбоях.

Ответ:

- Задержка при переключении на резервный сервер
- Частые разрывы соединений
- Неправильное распределение нагрузки
- Неопределенность состояния серверов

Причины:

1. Отсутствие health-check или слишком большой интервал проверок
 2. Неправильные timeout параметры
 3. Использование алгоритма roundrobin для серверов с разной производительностью
 4. Отсутствие или неправильная настройка параметров server
- **Задержка при переключении на резервный сервер** — Отсутствие health-check или слишком большой интервал проверок
 - **Частые разрывы соединений** — Неправильные timeout параметры
 - **Неправильное распределение нагрузки** — Использование алгоритма roundrobin для серверов с разной производительностью
 - **Неопределенность состояния серверов** — Отсутствие или неправильная настройка параметров server

Шаг 07:

Миссия 7

 Сопоставьте типы health-check в HAProxy с их описанием:

Подсказка: Разные типы проверок подходят под разные сценарии нагрузки.

Ответ:

- tcp-check
- http-check
- ssl-hello-check
- pgsql-check

Описание:

1. Проверка TCP-соединения без передачи данных
 2. Проверка HTTPS-соединения через отправку HTTP-запроса
 3. Проверка SSL-соединения с инициацией рукопожатия
 4. Проверка состояния PostgreSQL с помощью протокола базы данных
- **tcp-check** — Проверка TCP-соединения без передачи данных
 - **http-check** — Проверка HTTPS-соединения через отправку HTTP-запроса
 - **ssl-hello-check** — Проверка SSL-соединения с инициацией рукопожатия
 - **pgsql-check** — Проверка состояния PostgreSQL с помощью протокола базы данных

Шаг 08:

Миссия 8

 Сопоставьте команды в конфигурации HAProxy и их действия при работе с PostgreSQL:

Подсказка: Вспомните, как задаются backend-сервера и их параметры.

Ответ:

- `server db1 192.168.1.10:5432 check`
- `option redispatch`
- `balance leastconn`
- `timeout server 5s`
- `maxconn 100`

Назначения:

1. Перенаправление запроса на другой сервер при сбое
 2. Настройка максимального числа соединений к серверу
 3. Определение сервера с IP и проверкой здоровья
 4. Балансировка нагрузки по наименьшему количеству соединений
 5. Таймаут ожидания ответа от сервера
- **server db1 192.168.1.10:5432 check** — Определение сервера с IP и проверкой здоровья
 - **option redispatch** — Перенаправление запроса на другой сервер при сбое
 - **balance leastconn** — Балансировка нагрузки по наименьшему количеству соединений
 - **timeout server 5s** — Таймаут ожидания ответа от сервера
 - **maxconn 100** — Настройка максимального числа соединений к серверу

Шаг 09:

Миссия 9

Расположите в правильном порядке шаги настройки HAProxy для обеспечения фейловера PostgreSQL:

1. Настройка health-check для серверов
2. Определение backend-серверов PostgreSQL
3. Настройка frontend для приема клиентских подключений
4. Выбор алгоритма балансировки нагрузки
5. Перезапуск HAProxy для применения конфигурации

Подсказка: Начните с определения серверов, затем настройте их проверку и распределение, потом frontend и примените изменения.

Ответ:

- Определение backend-серверов PostgreSQL
- Настройка health-check для серверов
- Выбор алгоритма балансировки нагрузки
- Настройка frontend для приема клиентских подключений
- Перезапуск HAProxy для применения конфигурации

 Шаг 10:**Миссия 10**

Расположите этапы диагностики проблем с HAProxy при отказе основного сервера PostgreSQL:

1. Проверка логов HAProxy
2. Проверка состояния health-check
3. Анализ сетевых задержек между HAProxy и PostgreSQL
4. Проверка конфигурации HAProxy на ошибки
5. Тестирование переключения на резервный сервер

Подсказка: Сначала посмотрите логи, потом убедитесь, что проверки работают, проверьте сеть и конфигурацию, затем проведите тест переключения.

Ответ:

- Проверка логов HAProxy
- Проверка состояния health-check
- Анализ сетевых задержек между HAProxy и PostgreSQL
- Проверка конфигурации HAProxy на ошибки
- Тестирование переключения на резервный сервер

 Шаг 11:**Миссия 11**

Расположите в правильном порядке действия для реализации плавного фейловера с использованием HAProxy и PostgreSQL:

1. Настройка синхронной репликации между серверами
2. Настройка health-check и мониторинга HAProxy
3. Настройка конфигурации HAProxy для переключения на резервный сервер
4. Тестирование автоматического переключения при отказе
5. Обновление приложения для поддержки повторного подключения

Подсказка: Начинайте с репликации, настройте мониторинг, потом конфигурацию HAProxy, затем протестируйте и подготовьте приложение.

Ответ:

- Настройка синхронной репликации между серверами
- Настройка health-check и мониторинга HAProxy
- Настройка конфигурации HAProxy для переключения на резервный сервер
- Тестирование автоматического переключения при отказе
- Обновление приложения для поддержки повторного подключения

 Шаг 12:**Миссия 12**

Аня, девопс-инженер, настраивает HAProxy для PostgreSQL. Она хочет, чтобы HAProxy автоматически переключался на резервный сервер при недоступности основного. Какой параметр отвечает за автоматический фейловер?

Подсказка: Ищите опцию, которая помогает перенаправлять трафик при сбоях.

Ответ:

- **option redispatch**
- balance roundrobin
- timeout connect
- maxconn

Шаг 13:

Миссия 13

Игорь, системный администратор, хочет понять, как разные параметры HAProxy влияют на поведение PostgreSQL под нагрузкой. Сопоставьте параметры HAProxy с их назначением.

Подсказка: Подумайте, как каждый параметр влияет на производительность и устойчивость.

Параметры:

Ответ:

- timeout connect
- maxconn
- option httpclose
- balance leastconn

Назначения:

1. Максимальное число одновременных подключений
2. Время ожидания подключения к серверу
3. Балансировка нагрузки на основе наименьшего количества соединений
4. Закрытие HTTP-сессий после каждого запроса

- **timeout connect** — Время ожидания подключения к серверу
- **maxconn** — Максимальное число одновременных подключений
- **option httpclose** — Закрытие HTTP-сессий после каждого запроса
- **balance leastconn** — Балансировка нагрузки на основе наименьшего количества соединений

Шаг 14:

Миссия 14

Вадим настраивает фейловер для PostgreSQL через HAProxy. Расположите действия в правильном порядке для корректного переключения на резервный сервер при сбое основного.

Подсказка: Начинайте с обнаружения проблемы, заканчивайте контролем результата.

1. HAProxy обнаруживает сбой через health-check
2. Переключение трафика на резервный сервер
3. Конфигурация backend с резервным сервером
4. Мониторинг состояния после переключения

Ответ:

- Конфигурация backend с резервным сервером
- HAProxy обнаруживает сбой через health-check

- Переключение трафика на резервный сервер
- Мониторинг состояния после переключения

🔗 Шаг 15:

Миссия 15

Светлана замечает, что при работе HAProxy в кластере PostgreSQL некоторые запросы часто теряются при переключении на резервный сервер. Какой параметр поможет уменьшить потерю запросов?

Подсказка: Обратите внимание на настройки повторной отправки запросов.

Ответ:

- **option redispatch**
- timeout server
- balance roundrobin
- maxconn

🔗 Шаг 16:

Миссия 16

Дмитрий изучает настройки HAProxy и их влияние на балансировку и отказоустойчивость. Сопоставьте понятия с их определениями.

Подсказка: Внимательно прочитайте, что означает каждый термин.

Понятия:

Ответ:

- Health-check
- Balance roundrobin
- Maxconn
- Redispatch

Определения:

1. Проверка доступности сервера
2. Последовательное распределение запросов по серверам
3. Максимальное число одновременных соединений
4. Перенаправление запросов на другой сервер при недоступности

- **Health-check** — Проверка доступности сервера
- **Balance roundrobin** — Последовательное распределение запросов по серверам
- **Maxconn** — Максимальное число одновременных соединений
- **Redispatch** — Перенаправление запросов на другой сервер при недоступности

🔗 Шаг 17:

Миссия 17

Алексей хочет настроить плавное обновление HAProxy без прерывания соединений в PostgreSQL. Расположите этапы процесса в правильном порядке.

Подсказка: Помните, что сначала настраивается новое ПО, потом переключаются соединения.

1. Запуск нового экземпляра HAProxy параллельно с текущим
2. Перенос сессий и плавное переключение трафика
3. Остановка старого экземпляра после завершения сессий
4. Проверка корректности работы нового экземпляра

Ответ:

- Запуск нового экземпляра HAProxy параллельно с текущим
- Перенос сессий и плавное переключение трафика
- Остановка старого экземпляра после завершения сессий
- Проверка корректности работы нового экземпляра

Урок 5 - Репликация + HA лучшее сочетание

Шаг 01:

Миссия 1

При настройке асинхронной репликации: что произойдёт, если первичный сервер записал транзакцию, но слейв ещё не успел её получить?

Подсказка: Подумайте, что такое «асинхронный» — нет гарантии мгновенной доставки.

Ответ:

- **Транзакция считается завершённой на первичном, но может быть потеряна при сбое**
- Транзакция блокируется до подтверждения со слейва
- Слейв сразу применяет транзакцию синхронно

Шаг 02:

Миссия 2

Мы решили использовать синхронную репликацию. Что будет, если слейв недоступен?

Подсказка: Синхронная репликация ждёт ответа. Что если никто не отвечает?

Ответ:

- **Первичный будет ожидать ответа и задерживать коммиты**
- Репликация перейдёт в асинхронный режим автоматически
- **Ничего не изменится** — запись пройдет как обычно

Шаг 03:

Миссия 3

Вы читаете лог WAL на слейве: зачем нужен параметр `hot_standby_feedback``?

Подсказка: Он связан с тем, чтобы первичный не удалял старые WAL.

Ответ:

- **Чтобы слейв сообщал первичному, какие WAL ещё нужны**
- Чтобы ускорить репликацию в асинхронном режиме
- Чтобы слейв мог писать клиентские запросы на чтение

Шаг 04:

Миссия 4

У вас настроен кластер с одним первичным и двумя синхронными слейвами (`quorum = 2`). Какой риск остаётся при потере одного слейва?

Подсказка: Подумайте, сколько реплик всё ещё участвует в синхронной схеме.

Ответ:

- **Риск потери коммитов увеличится, если потеряется вторую реплику**
- Кластер сразу станет недоступен

- **Никакого риска** — один слейв достаточно

🔮 Шаг 05:

Миссия 5

Вы заметили лаг в репликации. Что первым делом стоит проверить?

Подсказка: Начните с сетевого и дискового здоровья.

Ответ:

- **Задержку сети между первичным и слейвом**
- Версию PostgreSQL на слейве (обычно не влияет)
- Запретить все записи на первичном
- Удалить все WAL-файлы вручную

🔮 Шаг 06:

Миссия 6

Сопоставьте ****тип отказа / события**** с ****последствием в репликации****:

Типы:

Ответ:

- Потеря слейва
- Задержка сети
- Удаление WAL на первичном
- Сбой первичного

Последствия:

1. Репликация перестаёт работать
2. Слейв получает устаревшие данные
3. Слейв не сможет догнать первичный
4. Нужен failover / promotion

Подсказка: Подумайте: каждое событие вызывает характерную проблему.

- **Потеря слейва** — Репликация перестаёт работать
- **Задержка сети** — Слейв получает устаревшие данные
- **Удаление WAL на первичном** — Слейв не сможет догнать первичный
- **Сбой первичного** — Нужен failover / promotion

🔮 Шаг 07:

Миссия 7

В контексте HA-кластера: сопоставьте ****компонент**** с его ролью:

Компоненты:

Ответ:

- PgBouncer
- Patroni
- etcd / Consul
- WAL-шардинг / archive_segments

Роли:

1. Хранит конфигурацию кворума / лидерства

2. Балансирует подключения к базе
3. Контролирует автоматическое переключение первичного
4. Обеспечивает распределение WAL между узлами

Подсказка: Подумайте: что отвечает за управление, что — за транспорт, что — за маршрутизацию.

- **PgBouncer** — Балансирует подключения к базе
- **Patroni** — Контролирует автоматическое переключение первичного
- **etcd / Consul** — Хранит конфигурацию кворума / лидерства
- **WAL-шардинг / archive_segments** — Обеспечивает распределение WAL между узлами

Шаг 08:

Миссия 8

Сопоставьте ****состояние кластера**** с ****нужным действием администратора****:

Состояния:

Ответ:

- Первичный упал
- Слейв догнал первичный после недоступности
- Другое событие split-brain
- Все слейвы отстают сильно

Действия:

1. Запустить ручной failover после проверки данных
2. Подождать репликацию, затем перевести в режим чтения
3. Разрешить конфликт лидерства / проверить quorum
4. Перераспределить нагрузку, проверить диск I/O

Подсказка: Подумайте, какое действие безопасно в каждом характерном сценарии.

- **Первичный упал** — Запустить ручной failover после проверки данных
- **Слейв догнал первичный после недоступности** — Подождать репликацию, затем перевести в режим чтения
- **Другое событие split-brain** — Разрешить конфликт лидерства / проверить quorum
- **Все слейвы отстают сильно** — Перераспределить нагрузку, проверить диск I/O

Шаг 09:

Миссия 9

Сопоставь ****метрику**** с ****индикатором проблемы в репликации****:

Метрики:

Ответ:

- replication_lag
- write_lag
- flush_lag
- replay_lag

Индикаторы:

1. Как долго слейв ждет WAL перед записью
2. Разница времени между записями на первичном и слейве

3. Время задержки перед сбросом WAL на диск
4. Время задержки применения WAL на слейве

Подсказка: Помните: write -> запись, flush -> сброс, replay -> применение.

- **replication_lag** — Разница времени между записями на первичном и слейве
- **write_lag** — Как долго слейв ждёт WAL перед записью
- **flush_lag** — Время задержки перед сбросом WAL на диск
- **replay_lag** — Время задержки применения WAL на слейве

🔮 Шаг 10:

Миссия 10

Отсортируйте шаги failover-процедуры при падении первичного:

1. Назначение нового первичного
2. Отключение старого первичного (если он оживёт)
3. Перенаправление клиентов на новый первичный
4. Проверка согласованности данных
5. Поднятие слейва как новый мастер

Подсказка: Сначала нужно убедиться, что данные в порядке, затем сменить роли и трафик.

Ответ:

- Проверка согласованности данных
- Поднятие слейва как новый мастер
- Назначение нового первичного
- Отключение старого первичного (если он оживёт)
- Перенаправление клиентов на новый первичный

🔮 Шаг 11:

Миссия 11

Отсортируйте уровни read_replica (чтения) по возрастанию риска устаревших данных:

1. Hot standby
2. Streaming replica (асинхронная)
3. Snapshot-based replica
4. Logical replication subscriber

Подсказка: Чем дальше от синхронного, тем выше риск устаревания.

Ответ:

- Hot standby
- Streaming replica (асинхронная)
- Snapshot-based replica
- Logical replication subscriber

🔮 Шаг 12:

Миссия 12

Отсортируйте элементы HA-архитектуры по порядку запуска при инициализации кластера:

1. Запуск etcd / Consul
2. Запуск Patroni / координатора

3. Инициализация первичного и слейвов
4. Подключение клиентов через балансировщик

Подсказка: Сначала нужно обеспечить quorum-сервис, потом координатор, потом базы, потом клиентский доступ.

Ответ:

- Запуск etcd / Consul
- Запуск Patroni / координатора
- Инициализация первичного и слейвов
- Подключение клиентов через балансировщик

Шаг 13:

Миссия 13

Отсортируйте этапы переключения реплик в топологии HA при деградации:

1. Обнаружение сбоя
2. Выбор кандидата на роль мастера
3. Promotion кандидата
4. Обновление маршрутов клиента
5. Синхронизация журналов, если возможно

Подсказка: Оценивайте сначала событие, потом выбираете нового, затем переключаете.

Ответ:

- Обнаружение сбоя
- Выбор кандидата на роль мастера
- Promotion кандидата
- Синхронизация журналов, если возможно
- Обновление маршрутов клиента

Шаг 14:

Миссия 14

После failover кластер остаётся в split-brain: два узла считают себя мастер-нодами. Что делать?

Подсказка: Нужно решить конфликт лидерства, а не просто исправить одну ноду.

Ответ:

- **Остановить оба узла, выбрать один в качестве мастера и восстановить второй как слейв**
- Оставить текущий мастер, второй отключить навсегда
- Оставить оба в работе, клиенты пусть решают

Шаг 15:

Миссия 15

Вы заметили, что один из слейвов часто отстаёт на большое число WAL-сегментов. Как оптимальный шаг?

Подсказка: Иногда проще пересоздать реплику, чем догонять.

Ответ:

- **Пересоздать реплику из свежего бэкапа**
- Увеличить max_wal_senders на первичном

- Выключить синхронность для этого слейва

Шаг 16:

Миссия 16

Отсортируйте шаги восстановления после полной потери кворума (когда первичный и большинство слейвов недоступны):

1. Поднять резервную ноду или временный мастер
2. Восстановить данные из архива WAL / резервных копий
3. Проверить контрольные суммы, целостность данных
4. Подключить остальных слейвов как реплики к новому мастер-узлу
5. Обновить конфигурацию HA (Patroni, etcd) и маршруты клиентов

Подсказка: Восстановление — это сначала данные, затем роли, затем конфигурация и трафик.

Ответ:

- Восстановить данные из архива WAL / резервных копий
- Проверить контрольные суммы, целостность данных
- Поднять резервную ноду или временный мастер
- Подключить остальных слейвов как реплики к новому мастер-узлу
- Обновить конфигурацию HA (Patroni, etcd) и маршруты клиентов

Модуль 6 - Наблюдение и проверка — путь контроля

Урок 1 - Обзор внутренних метрик PostgreSQL

Шаг 01:

Миссия 1

Вы админ Сашка, и на сервере PostgreSQL вы видите резкий рост числа подключений. Что из перечисленного вы проверите в первую очередь?

Подсказка: Начните с основных метрик, которые отражают текущее состояние сервера.

Ответ:

- **Количество активных соединений (numbackends)**
- Количество запросов в очереди на диск
- Размер WAL-архива
- Число временных файлов

Шаг 02:

Миссия 2

Вы видите, что задержка на запись WAL растёт. Какую внутреннюю метрику вы посмотрите?

Подсказка: Подумайте, какая статистика отражает, сколько WAL записано.

Ответ:

- **pg_stat_wal**
- pg_stat_activity
- pg_stat_bgwriter
- pg_stat_user_tables

🔮 Шаг 03:

Миссия 3

В PostgreSQL наблюдается рост «dead tuples» в таблице. Что побудит вас сначала?

Подсказка: Подумайте, какая команда может помочь очистить старые записи.

Ответ:

- Запустить **VACUUM**
- Сделать REINDEX
- Перезапустить кластер
- Увеличить shared_buffers

🔮 Шаг 04:

Миссия 4

Саша заметил, что нагрузка на диск сильно выросла. Какая метрика даст информацию об I/O деятельности?

Подсказка: Она находится в pg_stat_io инструменте расширения или в расширении (pg_stat_stat_io).

Ответ:

- **pg_stat_io**
- pg_stat_sql
- pg_stat_net
- pg_stat_scheduler

🔮 Шаг 05:

Миссия 5

При повторяющихся репликах вы замечаете расхождение WAL задержек. Что вы проверите?

Подсказка: В кластере важно следить за задержками при приёме и применении WAL.

Ответ:

- **pg_stat_replication (write_lag, flush_lag)**
- pg_stat_activity
- pg_stat_statements
- pg_stat_bgwriter

🔮 Шаг 06:

Миссия 6

Саша собрал данные метрик и хочет понять, что означает каждая из них. Сопоставьте метрику и её смысл.

Метрики:

Ответ:

- numbackends
- dead_tuples
- buffers_alloc
- write_lag (replica)

Значения:

1. Количество активных соединений

2. Количество «мёртвых» строк, подлежащих VACUUM
3. Число выделенных буферов для записи
4. Задержка реплики при записи WAL

Подсказка: numbackends — это про подключения, dead_tuples — что-то про удалённые записи.

- **numbackends** — Количество активных соединений
- **dead_tuples** — Количество «мёртвых» строк, подлежащих VACUUM
- **buffers_alloc** — Число выделенных буферов для записи
- **write_lag (replica)** — Задержка реплики при записи WAL

🔮 Шаг 07:

Миссия 7

Сопоставьте метрику и её типичный диапазон нормы (примерно):

Метрики:

Ответ:

- buffers_checkpoint
- buffers_clean
- buffers_backend
- buffers_alloc

Нормальный диапазон:

1. **Низкий** — десятки / сотни в секунду
2. **Средний** — сотни / тысячи в секунду
3. **Высокий** — тысячи / десятки тысяч в секунду
4. **Очень высокий** — десятки тысяч и выше

Подсказка: При высокой нагрузке alloc может быть высоким, checkpoint – умеренным.

- **buffers_checkpoint** — Средний — сотни / тысячи в секунду
- **buffers_clean** — Высокий — тысячи / десятки тысяч в секунду
- **buffers_backend** — Низкий — десятки / сотни в секунду
- **buffers_alloc** — Очень высокий — десятки тысяч и выше

🔮 Шаг 08:

Миссия 8

Сопоставьте статистику из pg_stat_replication и её смысл:

Метрики:

Ответ:

- flush_lag
- replay_lag
- write_lag

Смыслы:

1. Время, которое потребовалось реплике записать WAL
2. Время, которое потребовалось реплике применить WAL
3. Время, которое потребовалось основной ноде передать WAL реплике

Подсказка: write_lag — передача, flush_lag — запись на диск, replay_lag — применение.

- **write_lag** — Время, которое потребовалось основной ноде передать WAL реплике
- **flush_lag** — Время, которое потребовалось реплике записать WAL
- **replay_lag** — Время, которое потребовалось реплике применить WAL

🧭 Шаг 09:

Миссия 9

Сопоставьте расширение/сборщик и метрики, которые оно добавляет к PostgreSQL:

Расширения:

Ответ:

- pg_stat_statements
- pg_stat_io
- pg_buffercache
- auto_explain

Метрики/данные:

1. Статистика I/O операций по таблицам
2. **Кэш буферов** — какие страницы в shared_buffers используются
3. Детализированная статистика запросов (время, частота)
4. План выполнения медленных запросов

Подсказка: pg_stat_statements — про запросы, auto_explain — про планы, buffercache — про кеш.

- **pg_stat_statements** — Детализированная статистика запросов (время, частота)
- **pg_stat_io** — Статистика I/O операций по таблицам
- **pg_buffercache** — Кэш буферов — какие страницы в shared_buffers используются
- **auto_explain** — План выполнения медленных запросов

🧭 Шаг 10:

Миссия 10

Сопоставьте метрику и возможную проблема, если она чрезмерно растёт:

Метрики:

Ответ:

- buffers_backend
- dead_tuples
- replay_lag
- buffers_clean

Проблемы:

1. Неоптимальные запросы без индексов
2. Отсутствие регулярного VACUUM
3. **Реплика отстаёт** — возможно медленное применение WAL
4. Слишком частые очистки приводят к перегрузке I/O

Подсказка: dead_tuples — про вакуум, replay_lag — реплика.

- **buffers_backend** — Неоптимальные запросы без индексов
- **dead_tuples** — Отсутствие регулярного VACUUM
- **replay_lag** — Реплика отстаёт — возможно медленное применение WAL

- **buffers_clean** — Слишком частые очистки приводят к перегрузке I/O

🔧 Шаг 11:

Миссия 11

Последовательность действий при росте latency на записи WAL:

1. Смотреть `pg_stat_bgwriter`
2. Проверить конфигурацию `checkpoint_timeout` и `wal_writer_delay`
3. Увеличить частоту checkpoint или изменить параллельность
4. Мониторить изменение задержек по `pg_stat_wal` и `pg_stat_replication`

🔧 Подсказка:

Сначала смотрят статистику, потом меняют конфигурацию, затем применяют и проверяют.

Ответ:

- Смотреть `pg_stat_bgwriter`
- Проверить конфигурацию `checkpoint_timeout` и `wal_writer_delay`
- Увеличить частоту checkpoint или изменить параллельность
- Мониторить изменение задержек по `pg_stat_wal` и `pg_stat_replication`

🔧 Шаг 12:

Миссия 12

Аня замечает, что реплика сильно отстает. Расположите шаги диагностики:

1. Посмотреть `write_lag`, `flush_lag`, `replay_lag` в `pg_stat_replication`
2. Проверить нагрузку на диск реплики
3. Проверить настройки `wal_receiver_timeout` и `hot_standby_feedback`
4. Оценить необходимость увеличения ресурсов или оптимизации WAL

🔧 Подсказка:

Начните с метрик, затем — анализ среды и параметров, потом уже — выводы и действия.

Ответ:

- Посмотреть `write_lag`, `flush_lag`, `replay_lag` в `pg_stat_replication`
- Проверить нагрузку на диск реплики
- Проверить настройки `wal_receiver_timeout` и `hot_standby_feedback`
- Оценить необходимость увеличения ресурсов или оптимизации WAL

🔧 Шаг 13:

Миссия 13

Олег хочет уменьшить размер базы, не потеряв данных. Расположите его шаги:

1. Оценить размер таблиц и индексов (`pg_total_relation_size`)
2. Запустить `VACUUM FULL` на нужных объектах
3. Рестарт сервиса для сброса кеша, если необходимо
4. Проверить `pg_stat_user_tables` для подтверждения очистки

🔧 Подсказка:

Сначала измерения, потом действия, потом валидация результата.

Ответ:

- Оценить размер таблиц и индексов (`pg_total_relation_size`)

- Запустить VACUUM FULL на нужных объектах
- Рестарт сервиса для сброса кеша, если необходимо
- Проверить pg_stat_user_tables для подтверждения очистки

🔮 Шаг 14:

Миссия 14

Света решила внедрить мониторинг PostgreSQL. Упорядочите шаги внедрения:

1. Определить ключевые метрики (connections, WAL, bgwriter, locks)
2. Настроить сбор через pg_stat* и сторонние агенты
3. Визуализировать данные в Grafana / Zabbix
4. Настроить алерты на критические значения

🔮 Подсказка:

Сначала надо понять, что именно мониторить, затем — как, и только потом — визуализация и алерты.

Ответ:

- Определить ключевые метрики (connections, WAL, bgwriter, locks)
- Настроить сбор через pg_stat* и сторонние агенты
- Визуализировать данные в Grafana / Zabbix
- Настроить алерты на критические значения

🔮 Шаг 15:

Миссия 15

Паша готовит PostgreSQL к нагрузочному тесту. Расположите его действия:

1. Сбросить статистику pg_stat*
2. Запустить нагрузочный тест (например, pgbench)
3. Собрать метрики с pg_stat_database и pg_stat_statements
4. Сравнить с предыдущими тестами и оценить результат

🔮 Подсказка:

Сначала обнуляем метрики, потом тест, потом снимаем и анализируем.

Ответ:

- Сбросить статистику pg_stat*
- Запустить нагрузочный тест (например, pgbench)
- Собрать метрики с pg_stat_database и pg_stat_statements
- Сравнить с предыдущими тестами и оценить результат

🔮 Шаг 16:

Миссия 16

Ира расследует рост времени выполнения запросов. Упорядочите шаги:

1. Сравнить метрики pg_stat_statements до и после
2. Проверить изменения плана (EXPLAIN ANALYZE)
3. Проверить наличие вакуума и статистики (ANALYZE)
4. Оптимизировать запрос или добавить индекс

🔮 Подсказка:

Сначала анализ, потом копание в плане, затем уже действия.

Ответ:

- Сравнить метрики `pg_stat_statements` до и после
- Проверить изменения плана (`EXPLAIN ANALYZE`)
- Проверить наличие вакуума и статистики (`ANALYZE`)
- Оптимизировать запрос или добавить индекс

Урок 2 - Использование `pg_stat_statements` и `EXPLAIN`

Шаг 01:

Миссия 1

Аня запускает нагрузочный тест на PostgreSQL и хочет увидеть статистику выполнения запросов. Какой модуль нужно включить?

Подсказка: Это расширение записывает информацию о выполнениях запросов.

Ответ:

- `auto_explain`
- **`pg_stat_statements`**
- `pg_stat_activity`
- `pg_stat_io`

Шаг 02:

Миссия 2

Борис видит, что некоторые запросы долго выполняются. Какой параметр позволит ограничить включённые запросы по времени?

Подсказка: В `pg_stat_statements` есть «мин. время» порог.

Ответ:

- **`pg_stat_statements.min_time`**
- `pg_stat_statements.max_time`
- `pg_stat_statements.query_limit`
- `pg_stat_statements.track_activity`

Шаг 03:

Миссия 3

Когда включено `pg_stat_statements`, куда сохраняются данные по умолчанию?

Подсказка: Это системная схема PostgreSQL.

Ответ:

- `public.pg_stat_statements`
- **`pg_catalog.pg_stat_statements`**
- `information_schema.pg_stat_statements`
- `pg_stats.pg_stat_statements`

Шаг 04:

Миссия 4

Катя хочет сбросить статистику запросов в `pg_stat_statements`. Какой SQL вызов?

Подсказка: Используется функция из расширения.

Ответ:

- RESET pg_stat_statements
- CLEAR pg_stat_statements()
- **SELECT pg_stat_statements_reset()**
- TRUNCATE pg_stat_statements

🔮 Шаг 05:

Миссия 5

Иван хочет сравнить план запроса с фактическим исполнением. Какой команда подходит?

Подсказка: Нужно сочетать EXPLAIN с ключевым словом.

Ответ:

- EXPLAIN VERBOSE SELECT ...
- **EXPLAIN ANALYZE SELECT ...**
- EXPLAIN BUFFERS SELECT ...
- EXPLAIN SUMMARY SELECT ...

🔮 Шаг 06:

Миссия 6

Сопоставьте название колонки из pg_stat_statements с её значением.

Колонки:

Ответ:

- total_time
- calls
- rows
- mean_time

Значения:

1. Общее время, затраченное на все выполнения этого запроса
2. Общее число выполнений
3. Среднее время выполнения одного вызова
4. Сумма строк, возвращённых всеми вызовами

Подсказка: $\text{mean_time} = \text{total_time} / \text{calls}$.

- **total_time** — Общее время, затраченное на все выполнения этого запроса
- **calls** — Общее число выполнений
- **rows** — Сумма строк, возвращённых всеми вызовами
- **mean_time** — Среднее время выполнения одного вызова

🔮 Шаг 07:

Миссия 7

Сопоставьте фразу с её смыслом в контексте EXPLAIN ANALYZE.

Фразы:

Ответ:

- “Actual Loops”
- “Planning Time”

- “Buffers”
- “Shared Hit Blocks”

Смысл:

1. Количество повторов данного узла плана
2. Время, потраченное на планирование запроса
3. Число обращений к общей буферной памяти, не требующих загрузки с диска
4. Детали о работе с буфером (например, чтение и запись блоков)

Подсказка: Смотрите раздел "Buffers" в выводе EXPLAIN.

- **Actual Loops** — Количество повторов данного узла плана
- **Planning Time** — Время, потраченное на планирование запроса
- **Shared Hit Blocks** — Число обращений к общей буферной памяти, не требующих загрузки с диска
- **Buffers** — Детали о работе с буфером (например, чтение и запись блоков)

Шаг 08:

Миссия 8

Иван пытается понять отношение между полями EXPLAIN: сопоставьте статистику и её объяснение.

Поля:

Ответ:

- Rows Removed by Filter
- Rows
- Startup Cost
- Total Cost

Объяснения:

1. Оценка затрат на запуск узла до выдачи первой строки
2. Количество строк, отфильтрованных условием
3. Оценка затрат после обработки всех строк
4. Число строк, которые прошли через узел

Подсказка: Смотрите раздел «cost=» и «rows». Стартовые и итоговые оценки важны.

- **Startup Cost** — Оценка затрат на запуск узла до выдачи первой строки
- **Total Cost** — Оценка затрат после обработки всех строк
- **Rows** — Число строк, которые прошли через узел
- **Rows Removed by Filter** — Количество строк, отфильтрованных условием

Шаг 09:

Миссия 9

Сопоставьте SQL-команду и её эффект по pg_stat_statements или EXPLAIN.

Команды:

Ответ:

- SELECT pg_stat_statements_reset()
- EXPLAIN ANALYZE SELECT ...
- SELECT * FROM pg_stat_statements

- `CREATE EXTENSION pg_stat_statements`

Эффекты:

1. Сброс накопленной статистики
2. Получение плана с фактическими временами
3. Подключение расширения
4. Просмотр агрегированных статистик запросов

Подсказка: Думайте, какая команда что делает: `reset`, `explain`, `create`, `select`.

- **`SELECT pg_stat_statements_reset()`** — Сброс накопленной статистики
- **`EXPLAIN ANALYZE SELECT ...`** — Получение плана с фактическими временами
- **`SELECT * FROM pg_stat_statements`** — Просмотр агрегированных статистик запросов
- **`CREATE EXTENSION pg_stat_statements`** — Подключение расширения

Шаг 10:

Миссия 10

Расположите шаги анализа медленного запроса с помощью `EXPLAIN` + статистики:

1. Посмотреть aggregated-статистику в `pg_stat_statements`
2. Запустить `EXPLAIN ANALYZE` для проблемного запроса
3. Определить узел, где тратится больше всего времени
4. Проверить результаты операторов буферов и чтения диска
5. Сравнить средние времена с историей

Подсказка:

Сначала собираются статистики, затем детальный план, после — анализ узлов и I/O, и только потом сравнение с историческим контекстом.

Ответ:

- Посмотреть aggregated-статистику в `pg_stat_statements`
- Запустить `EXPLAIN ANALYZE` для проблемного запроса
- Определить узел, где тратится больше всего времени
- Проверить результаты операторов буферов и чтения диска
- Сравнить средние времена с историей

Шаг 11:

Миссия 11

Расположите типы метрик в порядке возрастания детализации:

1. Обобщённая статистика из `pg_stat_statements`
2. Среднее время на вызов
3. Данные узла в `EXPLAIN ANALYZE`
4. Информация о буферах и I/O в плане

 **Подсказка: От общего к конкретному: сначала агрегаты, потом детализация узлов и операций I/O.**

Ответ:

- Обобщённая статистика из `pg_stat_statements`
- Среднее время на вызов
- Данные узла в `EXPLAIN ANALYZE`

- Информация о буферах и I/O в плане

🔧 Шаг 12:

Миссия 12

Расположите действия для интеграции pg_stat_statements в продакшен:

1. Добавить extension в postgresql.conf / shared_preload_libraries
2. Перезапустить PostgreSQL
3. Создать расширение в БД (CREATE EXTENSION)
4. Настроить track и min_time параметры
5. Проверить, что статистика накапливается (SELECT из pg_stat_statements)

🔧 **Подсказка: Нужна загрузка модуля на старте, потом создание, конфигурирование, и проверка.**

Ответ:

- Добавить extension в postgresql.conf / shared_preload_libraries
- Перезапустить PostgreSQL
- Создать расширение в БД (CREATE EXTENSION)
- Настроить track и min_time параметры
- Проверить, что статистика накапливается (SELECT из pg_stat_statements)

🔧 Шаг 13:

Миссия 13

Расположите шаги при диагностике неожиданного роста времени запроса:

1. Сравнить с историческими данными pg_stat_statements
2. Запустить EXPLAIN ANALYZE новой версии запроса
3. Сбросить статистику, если она слишком «шла»
4. Анализ узлов плана и буферов
5. Корректировать индекс или запрос

🔧 **Подсказка: Сначала выяснить, что изменилось, затем план, сброс (при необходимости), анализ и оптимизация.**

Ответ:

- Сравнить с историческими данными pg_stat_statements
- Запустить EXPLAIN ANALYZE новой версии запроса
- Сбросить статистику, если она слишком «шла»
- Анализ узлов плана и буферов
- Корректировать индекс или запрос

🔧 Шаг 14:

Миссия 14

Расположите категории проблем, которые выявляет pg_stat_statements + EXPLAIN, по убыванию важности:

1. И/О-узкие места (чтение диска)
2. Слишком частые вызовы (высокая нагрузка)
3. Неэффективные планы (неверные эвристики)
4. Слабое использование индексов

🔮 **Подсказка: Важно: сначала I/O, потом нагрузка, затем логика плана и индекс.**

Ответ:

- I/O-узкие места (чтение диска)
- Слишком частые вызовы (высокая нагрузка)
- Неэффективные планы (неверные эвристики)
- Слабое использование индексов

🔮 **Шаг 15:**

Миссия 15

Расположите шаги оптимизации по итогам анализа плана:

1. Добавить или пересмотреть индекс
2. Переписать запрос или распределить агрегации
3. Принудительно задать JOIN/задать разные типы соединения
4. Выполнить анализ (ANALYZE) и сбросить статистику
5. Проверить эффект с EXPLAIN ANALYZE и собрать новую статистику

🔮 **Подсказка: После изменений всегда анализируем и проверяем снова.**

Ответ:

- Добавить или пересмотреть индекс
- Переписать запрос или распределить агрегации
- Принудительно задать JOIN/задать разные типы соединения
- Выполнить анализ (ANALYZE) и сбросить статистику
- Проверить эффект с EXPLAIN ANALYZE и собрать новую статистику

🔮 **Шаг 16:**

Миссия 16

Расположите этапы при использовании истории запросов для предсказания узких мест:

1. Собрать статистику по периодам (утро/вечер)
2. Сравнить средние времена с трендами
3. Выделить группы запросов с ростом времени
4. Запустить EXPLAIN ANALYZE для «проблемной» группы

🔮 **Подсказка: От анализа истории — к группам — к плану.**

Ответ:

- Собрать статистику по периодам (утро/вечер)
- Сравнить средние времена с трендами
- Выделить группы запросов с ростом времени
- Запустить EXPLAIN ANALYZE для «проблемной» группа

🔮 **Шаг 17:**

Миссия 17

Расположите полные этапы исследования производительности от ноды до кластера:

1. Собирать статистику на отдельной ноды через pg_stat_statements
2. Диагностировать медленные запросы через EXPLAIN ANALYZE
3. Оптимизировать запросы и индексы

4. Реплицировать конфигурацию и тестировать нагрузку на кластере
5. Сравнивать поведение запросов на разных нодах кластера

💡 **Подсказка: От одного узла к кластеру: сначала диагностика, потом оптимизация, затем масштабирование и сравнение.**

Ответ:

- Собирать статистику на отдельной ноде через `pg_stat_statements`
- Диагностировать медленные запросы через `EXPLAIN ANALYZE`
- Оптимизировать запросы и индексы
- Реплицировать конфигурацию и тестировать нагрузку на кластере
- Сравнивать поведение запросов на разных нодах кластера

📖 Урок 3 - Профилирование с `auto_explain` и `pgbench`

💡 Шаг 01:

Миссия 1

Вы запускаете `pgbench` на тестовой базе, и заметили, что некоторые команды медленные. Что логично включить для первичной диагностики в PostgreSQL?

Подсказка: Сначала включают общий механизм логирования медленных запросов.

Ответ:

- `pg_stat_statements`
- **`log_min_duration_statement`**
- `pg_repack`
- `auto_explain.log_min_duration`

💡 Шаг 02:

Миссия 2

Вы включили `auto_explain` для медленных запросов. Какой параметр отвечает за порог (в миллисекундах), после которого запрос будет логироваться?

Подсказка: Это аналог `log_min_duration_statement`, но для `auto_explain`.

Ответ:

- **`auto_explain.log_min_duration`**
- `auto_explain.log_threshold`
- `auto_explain.duration_ms`
- `auto_explain.min_duration`

💡 Шаг 03:

Миссия 3

Ваша коллега предлагает включить `auto_explain.log_analyze = on`. Что это даст?

Подсказка: Это влияет на включение вывода плана с дополнительной статистикой.

Ответ:

- **Добавится статистика исполнения (например, время на узлы) в лог плана**
- Будет логироваться только анализ без плана
- Выключит лог плана, только текст запроса
- Заставит логировать все запросы независимо от времени

Шаг 04:

Миссия 4

Вы получили лог `auto_explain` — там есть план с узлами и времена. Какой тип плана наиболее информативен?

Подсказка: Полные планы с подробностями дают максимум информации.

Ответ:

- **EXPLAIN ANALYZE PLAN (с информацией о времени на узлах)**
- EXPLAIN (без анализа)
- EXPLAIN BUFFERS
- EXPLAIN VERBOSE

Шаг 05:

Миссия 5

Вы запустили `pgbench` с большим числом клиентов, и запросы стали более медленными. Что вы предполагаете как возможную проблему?

Подсказка: При росте нагрузки часто появляются блокировки или конкуренция за ресурсы.

Ответ:

- **Контенция по буферам/синхронизация**
- **Блокировки на таблицах/строках**
- Неоптимальный алгоритм сортировки в клиенте
- Неправильный порт PostgreSQL

Шаг 06:

Миссия 6

 Задача: сопоставить ключевые параметры `auto_explain` и их назначение.

Параметры:

Ответ:

- `auto_explain.log_buffers`
- `auto_explain.log_timing`
- `auto_explain.log_triggers`
- `auto_explain.log_nested_statements`

Назначения:

1. Логировать триггеры, выполняемые в запросе
2. Логировать вложенные запросы (например, внутри функций)
3. Добавлять информацию о буферах (использованные блоки)
4. Логировать временные детали шагов плана

Подсказка: Название параметра часто указывает на то, что он логирует.

- **`auto_explain.log_buffers`** — Добавлять информацию о буферах (использованные блоки)
- **`auto_explain.log_timing`** — Логировать временные детали шагов плана
- **`auto_explain.log_triggers`** — Логировать триггеры, выполняемые в запросе
- **`auto_explain.log_nested_statements`** — Логировать вложенные запросы (например, внутри функций)

🔦 Шаг 07:

Миссия 7

📖 Сопоставьте виды блокировок с ситуациями, которые они могут вызывать при нагрузке.

Типы блокировок:

Ответ:

- ShareLock (SHARE)
- ExclusiveLock (EXCLUSIVE)
- RowExclusiveLock
- ShareRowExclusiveLock

Ситуации:

1. Множественные транзакции читают одну строку одновременно
2. Одна транзакция модифицирует строку, другие ждут
3. Таблица блокируется эксклюзивно для DDL
4. Комбинация чтения и записи внутри одной транзакции вызывает ожидание

Подсказка: Share vs Exclusive — разница в доступе к модификации.

- **ShareLock (SHARE)** — Множественные транзакции читают одну строку одновременно
- **ExclusiveLock (EXCLUSIVE)** — Таблица блокируется эксклюзивно для DDL
- **RowExclusiveLock** — Одна транзакция модифицирует строку, другие ждут
- **ShareRowExclusiveLock** — Комбинация чтения и записи внутри одной транзакции вызывает ожидание

🔦 Шаг 08:

Миссия 8

📖 Сопоставьте шаги профилирования с задачами, которые они помогают решить.

Шаги:

Ответ:

- Запуск pgbench с метриками latency/tps
- Анализ логов auto_explain
- Подсчёт медленных запросов из log_min_duration_statement
- Сравнение планов при разных нагрузках

Задачи:

1. Выявить точки перегрузки по времени отклика
2. Найти узкие места в плане запросов
3. Оценить рост числа медленных запросов при нагрузке
4. Увидеть, как план меняется под нагрузкой

Подсказка: Каждый шаг связан с конкретной задачей анализа.

- **Запуск pgbench с метриками latency/tps** — Выявить точки перегрузки по времени отклика
- **Анализ логов auto_explain** — Найти узкие места в плане запросов
- **Подсчёт медленных запросов из log_min_duration_statement** — Оценить рост числа медленных запросов при нагрузке
- **Сравнение планов при разных нагрузках** — Увидеть, как план меняется под нагрузкой

💡 Шаг 09:

Миссия 9

📖 Сопоставьте термины и определения, относящиеся к нагрузочному тестированию:

Термины:

Ответ:

- Latency
- TPS (transactions per second)
- Concurrency
- Throughput

Определения:

1. Количество одновременно работающих клиентов/потоков
2. Время отклика одной транзакции
3. Общее количество операций, выполненных за единицу времени
4. Количество транзакций, обработанных в секунду

Подсказка: Concurrency — это число игроков, throughput часто ближе к суммарной нагрузке.

- **Latency** — Время отклика одной транзакции
- **TPS (transactions per second)** — Количество транзакций, обработанных в секунду
- **Concurrency** — Количество одновременно работающих клиентов/потоков
- **Throughput** — Общее количество операций, выполненных за единицу времени

💡 Шаг 10:

Миссия 10

📖 Сопоставьте метрики rpgbench и их значимость:

Метрики:

Ответ:

- latency average
- latency stddev
- tps (including connections)
- sets per transaction

Значения:

1. Среднее время отклика
2. Стабильность (отклонение)
3. Общая пропускная способность
4. Количество операций INSERT/UPDATE в транзакции

Подсказка: Stddev показывает разброс, sets указывает на модификации.

- **latency average** — Среднее время отклика
- **latency stddev** — Стабильность (отклонение)
- **tps (including connections)** — Общая пропускная способность
- **sets per transaction** — Количество операций INSERT/UPDATE в транзакции

💡 Шаг 11:

Миссия 11

Расположите шаги включения auto_explain последовательно:

1. Установить extension auto_explain (если нужно)
2. Включить параметр shared_preload_libraries
3. Настроить параметры auto_explain (логирование порога, буферов и т.д.)
4. Перезапустить PostgreSQL
5. Запустить нагрузочный тест и проанализировать логи

Подсказка: Некоторые шаги обязательны до загрузки библиотеки.

Ответ:

- Установить extension auto_explain (если нужно)
- Включить параметр shared_preload_libraries
- Настроить параметры auto_explain (логирование порога, буферов и т.д.)
- Перезапустить PostgreSQL
- Запустить нагрузочный тест и проанализировать логи

Шаг 12:

Миссия 12

Расположите шаги профилирования нагрузки:

1. Запись логов slow query / auto_explain
2. Запуск pgbench с контролем параметров
3. Сбор статистик (pg_stat views, logs)
4. Анализ узких мест и планов запросов
5. Повторный прогон при корректировках

Подсказка: Процесс повторяется итеративно.

Ответ:

- Запуск pgbench с контролем параметров
- Запись логов slow query / auto_explain
- Сбор статистик (pg_stat views, logs)
- Анализ узких мест и планов запросов
- Повторный прогон при корректировках

Шаг 13:

Миссия 13

Расположите по возрастанию уровни детализации логов auto_explain:

1. minimal (только запрос)
2. buffers
3. timing
4. verbose (все детали)

Подсказка: Детализация растёт от базового к полному.

Ответ:

- minimal (только запрос)
- buffers
- timing
- verbose (все детали)

Шаг 14:

Миссия 14

Расположите этапы подбора порога для `auto_explain.log_min_duration`:

1. Установить начальный порог (например, 100 мс)
2. Запустить тест с нагрузкой и собрать логи
3. Проанализировать сколько запросов попало в лог
4. Снизить порог, если слишком мало запросов
5. Повторить до компромиссного значения

Подсказка: Идёт постепенное уточнение порога.

Ответ:

- Установить начальный порог (например, 100 мс)
- Запустить тест с нагрузкой и собрать логи
- Проанализировать сколько запросов попало в лог
- Снизить порог, если слишком мало запросов
- Повторить до компромиссного значения

Шаг 15:

Миссия 15

Расположите этапы выявления горячих запросов:

1. Настроить `log_min_duration_statement` / `auto_explain`
2. Собрать логи за часть нагрузки
3. Сгруппировать запросы по тексту / хэшу
4. Отсортировать по суммарному времени или частоте
5. Углубиться в план самого «тяжёлого» запроса

Подсказка: Ищем запросы, которые съедают ресурс.

Ответ:

- Настроить `log_min_duration_statement` / `auto_explain`
- Собрать логи за часть нагрузки
- Сгруппировать запросы по тексту / хэшу
- Отсортировать по суммарному времени или частоте
- Углубиться в план самого «тяжёлого» запроса

Шаг 16:

Миссия 16

Расположите шаги оптимизации после анализа плана:

1. Добавить / изменить индексы
2. Проверить статистику (ANALYZE)
3. Переписать запрос или использовать CTE / подзапросы по-другому
4. Перезапустить нагрузочный тест
5. Оценить изменение `latency` / `tps`

Подсказка: После изменений нужно проверить эффект.

Ответ:

- Добавить / изменить индексы

- Проверить статистику (ANALYZE)
- Переписать запрос или использовать CTE / подзапросы по-другому
- Перезапустить нагрузочный тест
- Оценить изменение latency / tps

🔮 Шаг 17:

Миссия 17

Расположите полный цикл профилирования и оптимизации под нагрузкой:

1. Запуск базового теста pgbench
2. Включение log_min_duration_statement / auto_explain
3. Сбор логов и статистик
4. Выявление горячих запросов и анализ планов
5. Внесение изменений (индексы, переписать запросы)
6. Повторный тест и сравнение метрик
7. Документирование результатов и рассуждений

Подсказка: Это итеративный процесс, включающий тест → анализ → оптимизацию → проверку.

Ответ:

- Запуск базового теста pgbench
- Включение log_min_duration_statement / auto_explain
- Сбор логов и статистик
- Выявление горячих запросов и анализ планов
- Внесение изменений (индексы, переписать запросы)
- Повторный тест и сравнение метрик
- Документирование результатов и рассуждений

📖 Урок 4 - Отслеживание задержек репликации

🔮 Шаг 01:

Миссия 1

Вы как инженер реплики получаете сообщения от мониторинга: задержка репликации выросла до 500 мс. Что в первую очередь стоит проверить?

Подсказка: Проверьте, сколько WAL-сегментов уже отправлено и не применено.

Ответ:

- **pg_stat_replication** — поля sent_lsn и replay_lsn
- pg_stat_activity на главном узле
- Сетевую маршрутизацию между узлами
- Конфигурацию autovacuum

🔮 Шаг 02:

Миссия 2

Вы настраиваете слейв и думаете, какая опция в recovery.conf может помочь снизить задержку репликации при упоре на производительность?

Подсказка: Рассмотрите агрессивную стратегию применения WAL.

Ответ:

- **recovery_target_timeline = 'latest' / synchronous_commit = off**

- hot_standby = off
- archive_mode = off
- max_standby_archive_delay = 0

🔧 Шаг 03:

Миссия 3

Ваша команда предлагает использовать синхронную репликацию для уменьшения задержек. Какой риск это несёт?

Подсказка: Подумайте о влиянии на производительность мастера.

Ответ:

- **Увеличение задержки записи на мастер до подтверждения реплики**
- Невозможность читать с мастера
- Резкое снижение пропускной способности сети
- Отключение автоматического вакуума

🔧 Шаг 04:

Миссия 4

Монитор показывает, что replay_lag остаётся нулём, но write_lag растёт. Что это говорит?

Подсказка: Значит реплика успевает применять, но не успевает получить WAL.

Ответ:

- **Узел реплики отстаёт на приёме (network / I/O)**
- Replay заблокирован блокировкой
- Мастер не генерирует WAL
- Синхронная репликация не работает

🔧 Шаг 05:

Миссия 5

При длительных задержках репликации вы решили включить слежение через pg_stat_wal_receiver. Какое поле особенно важно смотреть?

Подсказка: Нас интересует, как далеко реплика продвинулась в приёме WAL.

Ответ:

- **received_lsn**
- state_change
- last_msg_send_time
- slot_name

🔧 Шаг 06:

Миссия 6

📄 Сопоставьте термин из репликации со значением.

Термины:

Ответ:

- write_lag
- flush_lag
- replay_lag
- sync_state

Значения:

1. Задержка применения WAL на реплике
2. Состояние синхронной репликации (sync, async и т.д.)
3. Время записи WAL в буфер на реплике после получения
4. Время записи WAL на диск на реплике после flush

Подсказка: В названиях — write, flush, replay подсказывают стадию.

- **write_lag** — Время записи WAL в буфер на реплике после получения
- **flush_lag** — Время записи WAL на диск на реплике после flush
- **replay_lag** — Задержка применения WAL на реплике
- **sync_state** — Состояние синхронной репликации (sync, async и т.д.)

Шаг 07:

Миссия 7

 Сопоставьте представление или вид статистики с тем, что оно показывает в контексте репликации:

Источники:

Ответ:

- pg_stat_replication
- pg_stat_wal_receiver
- pg_stat_wal_sender
- pg_stat_activity

Что показывает:

1. Состояние слейва и его полученные LSN
2. Активные соединения WAL-отправки на мастер
3. Активные сессии запросов на мастере/реплике
4. Объединённая информация о репликах (send, replay и др.)

Подсказка: Названия дают подсказки, где что находится.

- **pg_stat_replication** — Объединённая информация о репликах (send, replay и др.)
- **pg_stat_wal_receiver** — Состояние слейва и его полученные LSN
- **pg_stat_wal_sender** — Активные соединения WAL-отправки на мастер
- **pg_stat_activity** — Активные сессии запросов на мастере/реплике

Шаг 08:

Миссия 8

 Сопоставьте факторы с их влиянием на задержку репликации:

Факторы:

Ответ:

- Высокая нагрузка I/O на дисках реплики
- Медленная сеть между мастер/реплика
- Большие транзакции на мастере
- Синхронная репликация и подтверждения

Влияния:

1. Увеличивает время передачи WAL сегментов

2. Увеличивает время записи / применения WAL
3. Увеличивает задержку подтверждения записи на мастер
4. Увеличивает размер пакета WAL, и может вызвать накладные расходы

Подсказка: Каждое влияние соответствует стадии передачи / записи / подтверждения.

- **Высокая нагрузка I/O на дисках реплики** — Увеличивает время записи / применения WAL
- **Медленная сеть между мастер/реплика** — Увеличивает время передачи WAL сегментов
- **Большие транзакции на мастере** — Увеличивает размер пакета WAL, и может вызвать накладные расходы
- **Синхронная репликация и подтверждения** — Увеличивает задержку подтверждения записи на мастер

Шаг 09:

Миссия 9

 Сопоставьте команду SQL и информацию, которую она возвращает (в контексте репликации):

Команды:

Ответ:

- `SELECT pg_last_wal_receive_lsn()`
- `SELECT pg_last_wal_replay_lsn()`
- `SELECT pg_xlog_location_diff(...) / pg_wal_lsn_diff(...)`
- `SELECT * FROM pg_stat_replication;`

Информация:

1. **Разница между двух LSN** — сколько WAL ещё нужно применить
2. LSN, до которого реплика применяет WAL
3. LSN, до которого реплика получила WAL
4. Статусы всех подключённых реплик (lag, state и т.д.)

Подсказка: Две из них возвращают LSN, две — вычисляют или показывают статус.

- **`SELECT pg_last_wal_receive_lsn()`** — LSN, до которого реплика получила WAL
- **`SELECT pg_last_wal_replay_lsn()`** — LSN, до которого реплика применяет WAL
- **`SELECT pg_xlog_location_diff(...) / pg_wal_lsn_diff(...)`** — Разница между двух LSN — сколько WAL ещё нужно применить
- **`SELECT * FROM pg_stat_replication;`** — Статусы всех подключённых реплик (lag, state и т.д.)

Шаг 10:

Миссия 10

 Сопоставьте режим репликации и характер задержек:

Режимы:

Ответ:

- Asynchronous
- Semi-synchronous
- Synchronous
- Logical replication

Характер задержек:

1. Минимальные задержки, подтверждение не ждётся
2. Мастер ждёт одну реплику перед завершением транзакции
3. Все реплики должны подтвердить запись перед завершением транзакции
4. Применение WAL с задержкой зависит от логической обработки (слегка выше)

Подсказка: Помните об обязательствах мастера при разных режимах.

- **Asynchronous** — Минимальные задержки, подтверждение не ждётся
- **Semi-synchronous** — Мастер ждёт одну реплику перед завершением транзакции
- **Synchronous** — Все реплики должны подтвердить запись перед завершением транзакции
- **Logical replication** — Применение WAL с задержкой зависит от логической обработки (слегка выше)

Шаг 11:

Миссия 11

Расположите шаги устранения задержки репликации, когда `write_lag` растёт:

1. Проверить нагрузку I/O на дисках реплики
2. Проверить сетевую задержку / packet loss
3. Оптимизировать конфигурацию `wal_receiver` / `wal_sender`
4. Внедрить сжатие передачи, если поддерживается (compression)
5. Провести нагрузочный тест с похожим паттерном транзакций

Подсказка: Сначала диагностируйте физические узкие места, затем переходите к конфигурации и тестам.

Ответ:

- Проверить нагрузку I/O на дисках реплики
- Проверить сетевую задержку / packet loss
- Оптимизировать конфигурацию `wal_receiver` / `wal_sender`
- Внедрить сжатие передачи, если поддерживается (compression)
- Провести нагрузочный тест с похожим паттерном транзакций

Шаг 12:

Миссия 12

Расположите шаги диагностики при `replay_lag` $\gg$ 0:

1. Проверить lock / блокировки на реплике
2. Просмотреть `plan heavy queries`, выполняющиеся на реплике
3. Проверить загрузку CPU на реплике
4. Проверить влияние `autovacuum` / `maintenance` задач на реплике
5. Измерить, как часто реплика применяет WAL (apply rate)

Подсказка: Применение может тормозить из-за конкуренции и ресурсов.

Ответ:

- Проверить lock / блокировки на реплике
- Просмотреть `plan heavy queries`, выполняющиеся на реплике
- Проверить загрузку CPU на реплике
- Проверить влияние `autovacuum` / `maintenance` задач на реплике
- Измерить, как часто реплика применяет WAL (apply rate)

Шаг 13:

Миссия 13

Расположите шаги мониторинга задержек в кластере:

1. Собирать `write_lag`, `flush_lag`, `replay_lag` на всех репликах
2. Собирать `sync_state` у реплик
3. Вычислять разницу LSN между мастер и репликами
4. Визуализировать тренды задержек во времени
5. Установить предупреждения (alerting) при превышении порога

Подсказка: Мониторинг — это сбор → анализ → оповещение.

Ответ:

- Собирать `write_lag`, `flush_lag`, `replay_lag` на всех репликах
- Собирать `sync_state` у реплик
- Вычислять разницу LSN между мастер и репликами
- Визуализировать тренды задержек во времени
- Установить предупреждения (alerting) при превышении порога

Шаг 14:

Миссия 14

Расположите этапы улучшения задержки репликации путем конфигурации:

1. Увеличить `wal_receiver_buffers` на реплике
2. Увеличить `wal_sender_timeout` / `keepalive` параметры
3. Установить `wal_compression = on` (если CPU позволяет)
4. Настроить `synchronous_commit = local / remote_write` при необходимости
5. Перезапустить службы, протестировать задержки

Подсказка: Параметры должны быть скорректированы до тестов.

Ответ:

- Увеличить `wal_receiver_buffers` на реплике
- Увеличить `wal_sender_timeout` / `keepalive` параметры
- Установить `wal_compression = on` (если CPU позволяет)
- Настроить `synchronous_commit = local / remote_write` при необходимости
- Перезапустить службы, протестировать задержки

Шаг 15:

Миссия 15

Расположите шаги реакции на сигналы мониторинга (когда задержка > threshold):

1. Получить оповещение / алерт от системы мониторинга
2. Автоматически переключиться на другую реплику (failover/смена мастер)
3. Провести диагностику задержки (см. миссии выше)
4. Снизить нагрузку на мастер или реплику
5. Вернуть систему в стабильное состояние и закрыть алерт

Подсказка: Важен баланс между автоматикой и диагностикой.

Ответ:

- Получить оповещение / алерт от системы мониторинга

- Автоматически переключиться на другую реплику (failover/смена мастер)
- Провести диагностику задержки (см. миссии выше)
- Снизить нагрузку на мастер или реплику
- Вернуть систему в стабильное состояние и закрыть алерт

Шаг 16:

Миссия 16

Расположите полный цикл работы с проблемой задержки репликации:

1. Мониторинг задержек
2. Анализ отклонений и диагностика причин
3. Внедрение конфигурационных или аппаратных изменений
4. Тестирование после изменений
5. Обратная проверка задержки, подтверждение улучшения
6. Документация и принятие новых порогов

Подсказка: Это итеративный workflow.

Ответ:

- Мониторинг задержек
- Анализ отклонений и диагностика причин
- Внедрение конфигурационных или аппаратных изменений
- Тестирование после изменений
- Обратная проверка задержки, подтверждение улучшения
- Документация и принятие новых порогов

Шаг 17:

Миссия 17

Расположите этапы масштабирования кластера с минимальными задержками репликации:

1. Добавить реплики ближе по сети к мастеру
2. Использовать cascading репликацию
3. Использовать логическую репликацию для части данных
4. Настроить параллельное применение WAL на репликах (parallel apply)
5. Мониторинг и адаптация конфигурации каждой реплики

Подсказка: Масштабирование — это сочетание топологии, применения и мониторинга.

Ответ:

- Добавить реплики ближе по сети к мастеру
- Использовать cascading репликацию
- Использовать логическую репликацию для части данных
- Настроить параллельное применение WAL на репликах (parallel apply)
- Мониторинг и адаптация конфигурации каждой реплики

Урок 5 - Тестирование отказов и фейловеров

Шаг 01:

Миссия 1

Саша запускает PostgreSQL в контейнере, и хочет проверить, как система отреагирует на выключение основного процесса. Что он должен сделать первым?

Подсказка: подумайте, что обычно делается до имитации сбоя.

Ответ:

- **Сделать резервную копию данных и проверить состояние кластера**
- Немедленно остановить основной процесс
- Начать нагрузочное тестирование
- Выключить весь узел виртуальной машины

Шаг 02:

Миссия 2

Анна хочет проверить поведение PgBouncer при падении основной БД. Что она должна сделать?

Подсказка: вспомните роли PgBouncer как пула соединений.

Ответ:

- **Остановить основной сервер и посмотреть, перенаправляет ли PgBouncer на реплики**
- Остановить PgBouncer
- Остановить все реплики
- Перезапустить PgBouncer

Шаг 03:

Миссия 3

Игорь готов тестировать автоматический failover. Что он должен удостовериться до начала теста?

Подсказка: условия для failover обычно проверяются заранее.

Ответ:

- **Что порог задержки или пропажи мастера в конфигурации настроен**
- Что все пользователи отключены
- Что репликации нет
- Что нет критичных запросов

Шаг 04:

Миссия 4

Во время тестирования отказа мастер-узла в кластере Patroni, что наиболее ожидаемое поведение?

Подсказка: Patroni управляет лидерством.

Ответ:

- **Одна из реплик должна стать новым мастером**
- Кластер полностью останавливается
- Все запросы пишутся в старый мастер
- Нельзя подключиться ни к какому узлу

Шаг 05:

Миссия 5

Кирилл хочет убедиться, что WAL-архив корректно работает при отказе архивационного узла. Что он проверит первым?

Подсказка: архивирование WAL — ключ к восстановлению.

Ответ:

- **Логи архивации и ошибки записи WAL**
- Нагрузку CPU
- Объём памяти
- Максимальный размер БД

Шаг 06:

Миссия 6

 **Задача:** сопоставьте сценарий отказа с ожидаемым поведением системы PostgreSQL с репликацией

Сценарии отказа:

Ответ:

- Отказ мастера
- Отказ реплики
- Падение сети между узлами
- Задержка репликации

Ожидаемые реакции:

1. Одна из реплик становится мастером
2. Реплика исключается из синхронных записей
3. Временная деградация доступности
4. Конфигурация переключения на горячего резервного узла

Подсказка: подумайте, как ведёт себя система при потере главного узла или реплики.

- **Отказ мастера** — Одна из реплик становится мастером
- **Отказ реплики** — Реплика исключается из синхронных записей
- **Падение сети между узлами** — Временная деградация доступности
- **Задержка репликации** — Конфигурация переключения на горячего резервного узла

Шаг 07:

Миссия 7

 **Задача:** сопоставьте инструментарий с его назначением при тестировании отказов

Инструменты / механизмы:

Ответ:

- pg_isready
- patronictl
- pgbench
- repmgr

Назначения:

1. Запрос статуса кластера Patroni
2. Проверка доступности PostgreSQL
3. Управление репликацией и failover
4. Нагрузочное тестирование

Подсказка: подумайте, какая утилита отвечает за мониторинг, какая — за нагрузку.

- **pg_isready** — Проверка доступности PostgreSQL

- **patronictl** — Запрос статуса кластера Patroni
- **pgbench** — Нагрузочное тестирование
- **repmgr** — Управление репликацией и failover

🔧 Шаг 08:

Миссия 8

📖 Задача: сопоставьте тип отказа с краеугольным риском при его испытании

Типы отказов:

Ответ:

- Отключение питания узла
- Сбой диска
- Отключение сети
- Сбой репликации

Риски / последствия:

1. Потеря данных
2. Недоступность сервиса
3. Нарушение целостности данных
4. Разрыв связи между узлами

Подсказка: подумайте, что может случиться при физическом повреждении или сетевой изоляции.

- **Отключение питания узла** — Недоступность сервиса
- **Сбой диска** — Потеря данных
- **Отключение сети** — Разрыв связи между узлами
- **Сбой репликации** — Нарушение целостности данных

🔧 Шаг 09:

Миссия 9

📖 Задача: сопоставьте термин при отказе кластера с его описанием

Термины:

Ответ:

- Split-brain
- Automatic failover
- Consistent failover
- Failback

Описания:

1. Возврат старого мастера в кластер после восстановления
2. Сценарий, когда два узла считают себя мастером
3. Переключение на резервный узел автоматически
4. Гарантия консистентного состояния после переключения

Подсказка: обратите внимание на порядок: переключение, консистентность, возврат.

- **Split-brain** — Сценарий, когда два узла считают себя мастером
- **Automatic failover** — Переключение на резервный узел автоматически
- **Consistent failover** — Гарантия консистентного состояния после переключения

- **Failback** — Возврат старого мастера в кластер после восстановления

Шаг 10:

Миссия 10

Расположите шаги проведения теста failover от начала до конца:

1. Запустить нагрузку
2. Имитировать отказ узла
3. Наблюдать переключение
4. Проверить целостность данных
5. Вернуть систему в нормальное состояние

Подсказка: поймите, что должно быть сначала: тест, сбой, проверка, восстановление.

Ответ:

- Запустить нагрузку
- Имитировать отказ узла
- Наблюдать переключение
- Проверить целостность данных
- Вернуть систему в нормальное состояние

Шаг 11:

Миссия 11

Расположите фазы при восстановлении после отказа:

1. Выбор нового мастера
2. Синхронизация данных
3. Оповещение клиентов
4. Отключение старого узла
5. Возобновление операций записи

Подсказка: вспомните: сначала выбор, потом синхронизация, затем операции.

Ответ:

- Выбор нового мастера
- Синхронизация данных
- Оповещение клиентов
- Отключение старого узла
- Возобновление операций записи

Шаг 12:

Миссия 12

Расположите команды Patroni / etcd-реплики, используемые при отказе, по порядку:

1. patronictl failover
2. patronictl show-config
3. patronictl list
4. patronictl switchover
5. patronictl restart

Подсказка: сначала статус, потом конфигурация, затем действия по переключению и восстановлению.

Ответ:

- patronictl list
- patronictl show-config
- patronictl failover
- patronictl switchover
- patronictl restart

 Шаг 13:

Миссия 13

Расположите события во время теста отказа с логами:

1. "promote request sent"
2. "master down"
3. "switchover complete"
4. "clients reconnect"
5. "new master is ready"

Подсказка: по логам видно: сначала сбой, потом промоушен, затем переключение и запросы.

Ответ:

- master down
- promote request sent
- new master is ready
- switchover complete
- clients reconnect

 Шаг 14:

Миссия 14

Расположите шаги для проверки отката (failback) после восстановления мастера:

1. Остановить старый мастер как реплику
2. Восстановить его данные с WAL
3. Включить его как реплику синхронно
4. Переключить роли через switchover
5. Отключить прежнего вспомогательного мастера

Подсказка: подумайте, как безопасно вернуть первоначальный мастер в кластер.

Ответ:

- Остановить старый мастер как реплику
- Восстановить его данные с WAL
- Включить его как реплику синхронно
- Переключить роли через switchover
- Отключить прежнего вспомогательного мастера

Модуль 7 - Финальный квест — построение кластера

Урок 1 - Проектирование архитектуры кластера

Шаг 01:

Миссия 1

Наташа — начинающий DBA, и ей нужно выбрать тип репликации для кластера PostgreSQL, который обеспечит простую настройку и минимальную задержку. Что лучше выбрать?

Подсказка: Подумайте, какой тип репликации проще и обеспечивает почти мгновенную передачу данных.

Ответ:

- **Синхронная репликация**
- Асинхронная репликация
- Логическая репликация
- Физическая репликация

Шаг 02:

Миссия 2

Алексей хочет обеспечить отказоустойчивость кластера. Какой элемент архитектуры PostgreSQL поможет автоматически переключать нагрузку на резервный сервер в случае сбоя главного?

Подсказка: Подумайте, что отвечает за автоматическое переключение между серверами.

Ответ:

- **Промежуточный менеджер (failover manager)**
- Только горячий резервный сервер
- Ручное переключение
- Физическая репликация без менеджера

Шаг 03:

Миссия 3

Игорь планирует использовать кластер для аналитики, где важна высокая нагрузка на чтение. Какой тип узлов лучше всего использовать в архитектуре?

Подсказка: Для распределения нагрузки на чтение подходят серверы, которые не принимают запись.

Ответ:

- **Репликационные слейвы (read replicas)**
- Главный мастер-сервер
- Локальный кэш
- Резервный сервер без репликации

Шаг 04:

Миссия 4

Ольга хочет минимизировать время отклика при записи данных в распределённом кластере. Какой метод репликации ей стоит выбрать?

Подсказка: Подумайте, какой способ репликации уменьшит задержки при записи, но может снизить уровень защиты от потери данных.

Ответ:

- **Асинхронная репликация**
- Синхронная репликация
- Периодическая репликация
- Мастер-мастер репликация

Шаг 05:

Миссия 5

 Задача: Сопоставьте компоненты кластера PostgreSQL с их функциями.

Компоненты:

Ответ:

- Master (главный сервер)
- Slave (реплика)
- Failover manager
- Load balancer

Функции:

1. Распределяет запросы между узлами для повышения производительности
2. Автоматически переключает работу на резервный сервер при сбое
3. Обрабатывает все операции записи в базу данных
4. Копирует данные с главного сервера для чтения

Подсказка: Подумайте, кто принимает записи, кто копирует данные, и кто отвечает за отказоустойчивость и балансировку.

- **Master (главный сервер)** — Обрабатывает все операции записи в базу данных
- **Slave (реплика)** — Копирует данные с главного сервера для чтения
- **Failover manager** — Автоматически переключает работу на резервный сервер при сбое
- **Load balancer** — Распределяет запросы между узлами для повышения производительности

Шаг 06:

Миссия 6

 Задача: Сопоставьте типы репликации с их особенностями.

Типы репликации:

Ответ:

- Физическая репликация
- Логическая репликация
- Синхронная репликация
- Асинхронная репликация

Особенности:

1. Копирует данные на уровне файлов WAL
2. Позволяет реплицировать отдельные таблицы или данные
3. Ждёт подтверждения записи на слейве перед продолжением
4. Не ждёт подтверждения записи на слейве, повышая производительность

Подсказка: Подумайте, что именно копируется и как происходит подтверждение записи.

- **Физическая репликация** — Копирует данные на уровне файлов WAL
- **Логическая репликация** — Позволяет реплицировать отдельные таблицы или данные
- **Синхронная репликация** — Ждёт подтверждения записи на слейве перед продолжением
- **Асинхронная репликация** — Не ждёт подтверждения записи на слейве, повышая производительность

🔮 Шаг 07:

Миссия 7

📖 Задача: Сопоставьте термины из архитектуры кластера с их назначением.

Термины:

Ответ:

- Hot standby
- Warm standby
- Cold standby
- Patroni

Назначения:

1. Автоматическое управление отказоустойчивостью PostgreSQL
2. Резервный сервер, готовый к переключению, но без активного чтения
3. Резервный сервер, доступный только после запуска
4. Резервный сервер, который может обслуживать запросы на чтение

Подсказка: Обратите внимание на различия в готовности и доступности резервных серверов.

- **Hot standby** — Резервный сервер, который может обслуживать запросы на чтение
- **Warm standby** — Резервный сервер, готовый к переключению, но без активного чтения
- **Cold standby** — Резервный сервер, доступный только после запуска
- **Patroni** — Автоматическое управление отказоустойчивостью PostgreSQL

🔮 Шаг 08:

Миссия 8

Расположите шаги создания отказоустойчивого кластера PostgreSQL в правильном порядке:

1. Настройка репликации между главным и резервным сервером
2. Тестирование автоматического переключения (failover)
3. Установка и конфигурация PostgreSQL на серверах
4. Настройка менеджера отказов (например, Patroni)

Подсказка: Помните, что сначала нужно подготовить серверы, затем соединить их, настроить автоматику и проверить её.

Ответ:

- Установка и конфигурация PostgreSQL на серверах
- Настройка репликации между главным и резервным сервером
- Настройка менеджера отказов (например, Patroni)
- Тестирование автоматического переключения (failover)

Шаг 09:

Миссия 9

Расположите этапы масштабирования PostgreSQL кластера для увеличения производительности:

1. Добавление реплик для распределения чтения
2. Оптимизация запросов и индексов
3. Внедрение балансировщика нагрузки
4. Мониторинг производительности и узких мест

Подсказка: Масштабирование — это и технические улучшения, и мониторинг результата.

Ответ:

- Оптимизация запросов и индексов
- Добавление реплик для распределения чтения
- Внедрение балансировщика нагрузки
- Мониторинг производительности и узких мест

Шаг 10:

Миссия 10

Расположите этапы настройки синхронной репликации PostgreSQL:

1. Конфигурация параметров репликации на главном сервере
2. Настройка параметров приёма реплики на слейве
3. Запуск потоковой репликации
4. Тестирование задержки и подтверждения записи

Подсказка: Важно правильно настроить и главную, и второстепенную сторону, а затем проверить.

Ответ:

- Конфигурация параметров репликации на главном сервере
- Настройка параметров приёма реплики на слейве
- Запуск потоковой репликации
- Тестирование задержки и подтверждения записи

Шаг 11:

Миссия 11

Дмитрий столкнулся с проблемой — после переключения на резервный сервер некоторые транзакции пропали. Что скорее всего неправильно настроено?

Подсказка: Обратите внимание на тип репликации и синхронность подтверждений.

Ответ:

- **Используется асинхронная репликация без подтверждения**
- Переключение не выполнялось
- Резервный сервер был не готов
- Проблемы с сетевым соединением

Шаг 12:

Миссия 12

Анна хочет внедрить мастер-мастер репликацию для повышения доступности. Какой инструмент PostgreSQL может помочь в этом?

Подсказка: Подумайте, что позволяет синхронизировать данные в обе стороны.

Ответ:

- **Bucardo**
- Patroni
- Pgpool-II
- Slony

Шаг 13:

Миссия 13

Максим проектирует кластер PostgreSQL и хочет выбрать инструмент для автоматического failover. Какой из инструментов лучше всего подойдет?

Подсказка: Ищите решение, которое следит за состоянием мастера и быстро переключает роли.

Ответ:

- **Patroni**
- Pgpool-II
- Bucardo
- Slony

Шаг 14:

Миссия 14

Сопоставьте типы репликации PostgreSQL с их характеристиками:

Ответ:

- Синхронная репликация
- Асинхронная репликация
- Логическая репликация
- Физическая репликация

1. Данные передаются в реальном времени, подтверждение от слейва требуется перед коммитом
2. Данные передаются с задержкой, мастер не ждет подтверждения слейва
3. Репликация на уровне WAL файлов, копирует байтовый поток данных
4. Репликация таблиц и схем с возможностью фильтрации и трансформации

Подсказка: Подумайте, какие задачи решает каждая репликация и на каком уровне работают.

- **Синхронная репликация** — Данные передаются в реальном времени, подтверждение от слейва требуется перед коммитом
- **Асинхронная репликация** — Данные передаются с задержкой, мастер не ждет подтверждения слейва
- **Физическая репликация** — Репликация на уровне WAL файлов, копирует байтовый поток данных
- **Логическая репликация** — Репликация таблиц и схем с возможностью фильтрации и трансформации

Шаг 15:

Миссия 15

Расположите шаги настройки кластера PostgreSQL для отказоустойчивости в правильном порядке:

1. Настройка репликации между мастер-сервером и слейвами
2. Настройка мониторинга состояния серверов
3. Настройка автоматического failover
4. Проверка корректной синхронизации данных

Подсказка: Важно последовательно выстроить процесс, от базовой настройки к автоматизации и контролю.

Ответ:

- Настройка репликации между мастер-сервером и слейвами
- Настройка мониторинга состояния серверов
- Настройка автоматического failover
- Проверка корректной синхронизации данных

Урок 2 - Развёртывание master и реплик

Шаг 01:

Миссия 1

Алексей разворачивает мастер-сервер PostgreSQL для будущего кластера. Какой параметр в postgresql.conf указывает на то, что сервер будет принимать подключения от реплик?

Подсказка: Вспомните про параметр, связанный с записью журнала транзакций (WAL).

Ответ:

- **wal_level = replica**
- max_connections = 200
- shared_buffers = 2GB
- listen_addresses = '*'

Шаг 02:

Миссия 2

Ирина готовит сервер к роли мастера. Какой параметр в pg_hba.conf нужно настроить, чтобы реплики могли подключаться по протоколу репликации?

Подсказка: Этот файл управляет правилами доступа клиентов к серверу.

Ответ:

- **host replication реплика 192.168.1.0/24 md5**
- local all all trust
- host all all 127.0.0.1/32 trust
- hostssl all all 0.0.0.0/0 cert

Шаг 03:

Миссия 3

Сергей хочет проверить, что мастер готов к работе с репликами. Какой SQL-запрос показывает текущее значение wal_level?

Подсказка: Вспомните команду для проверки параметров конфигурации.

Ответ:

- **SHOW wal_level;**

- `SELECT wal_level FROM pg_settings;`
- `SELECT * FROM pg_stat_replication;`
- `SHOW replication;`

🔧 Шаг 04:

Миссия 4

Маша настраивает реплику. Какой способ инициализации данных является корректным?

Подсказка: Думайте про копирование содержимого data-директории с мастера.

Ответ:

- **`pg_basebackup -h master -D /var/lib/postgresql/replica -U реплика -P`**
- `cp -r /var/lib/postgresql/master/* /var/lib/postgresql/replica/`
- `initdb -D /var/lib/postgresql/replica`
- `pg_dumpall | psql -h replica`

🔧 Шаг 05:

Миссия 5

Иван хочет запустить реплику. Какой файл необходимо создать в data-директории реплики для указания настроек подключения к мастеру?

Подсказка: Этот файл был введён начиная с PostgreSQL 12.

Ответ:

- **`standby.signal`**
- `replica.conf`
- `postgresql.auto.conf`
- `pg_hba.conf`

🔧 Шаг 06:

Миссия 6

Марина проверяет подключение реплики к мастеру. Какая системная вьюшка на мастере показывает состояние подключенных реплик?

Подсказка: Эта вьюшка содержит информацию о слотах репликации и передаче WAL.

Ответ:

- **`pg_stat_replication`**
- `pg_stat_activity`
- `pg_stat_wal_receiver`
- `pg_stat_database`

🔧 Шаг 07:

Миссия 7

Дмитрий планирует синхронную репликацию. Какой параметр в `postgresql.conf` указывается для выбора списка синхронных реплик?

Подсказка: Этот параметр содержит имена подключений реплик.

Ответ:

- **`synchronous_standby_names`**
- `replica_hosts`
- `standby_list`

- wal_receivers

🔮 Шаг 08:

Миссия 8

Егор хочет проверить работу асинхронной реплики. Какое преимущество у асинхронного режима?

Подсказка: Вспомните про задержки записи и скорость ответа мастера.

Ответ:

- **Мастер отвечает клиенту без ожидания реплики**
- Запросы всегда выполняются быстрее на реплике
- Реплика может принимать запись
- Мастер никогда не теряет данные

🔮 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте термины и их объяснения в настройке репликации.

Подсказка: Смотрите на то, кто отправляет WAL и кто принимает.

Ответ:

- **WAL Sender** — Процесс на мастере, который отправляет WAL-данные.
- **WAL Receiver** — Процесс на реплике, который принимает WAL-данные.
- **Replication Slot** — Механизм на мастере, гарантирующий хранение WAL до подтверждения реплики.
- **Streaming Replication** — Механизм передачи WAL в реальном времени по сети.

🔮 Шаг 10:

Миссия 10

📖 Сопоставьте конфигурационные файлы и их назначение в процессе репликации.

Подсказка: Подумайте, где задаются параметры доступа, где — системные настройки, а где — сигнал о standby.

Ответ:

- **postgresql.conf** — Основной файл настроек сервера PostgreSQL.
- **pg_hba.conf** — Файл правил доступа клиентов к серверу.
- **standby.signal** — Файл, указывающий, что сервер работает как реплика.
- **postgresql.auto.conf** — Файл, куда автоматически сохраняются изменения через ALTER SYSTEM.

🔮 Шаг 11:

Миссия 11

📖 Сопоставьте режимы репликации с их особенностями.

Подсказка: Один режим быстрее, другой надёжнее.

Ответ:

- **Асинхронная репликация** — Мастер отвечает клиенту сразу после записи в WAL.
- **Синхронная репликация** — Мастер ждёт подтверждения от реплики перед ответом клиенту.

- **Логическая репликация** — Передаются отдельные изменения таблиц, а не бинарный WAL.
- **Физическая репликация** — Передаётся поток WAL-блоков побайтно.

💡 Шаг 12:

Миссия 12

📖 Сопоставьте команды с их результатом при работе с кластером.

Подсказка: Обратите внимание, какие команды смотрят настройки, а какие — состояние процессов.

Ответ:

- **SHOW synchronous_commit;** — Отображает текущий режим подтверждения транзакций.
- **pg_basebackup** — Создаёт резервную копию базы для инициализации реплики.
- **SELECT * FROM pg_stat_replication;** — Показывает подключённых реплик и их статус.
- **SELECT * FROM pg_stat_wal_receiver;** — Показывает состояние приёма WAL на реплике.

💡 Шаг 13:

Миссия 13

📖 Сопоставьте возможные проблемы репликации и их причины.

Подсказка: Думайте про сеть, доступ и версии.

Ответ:

- **Реплика не подключается** — Ошибка в pg_hba.conf или учётных данных.
- **Задержка реплики** — Слабая сеть или высокая нагрузка на мастере.
- **Несовместимость версий** — Мастер и реплика используют разные версии PostgreSQL.
- **Нет записи WAL** — Параметр wal_level не установлен в replica.

💡 Шаг 14:

Миссия 14

Упорядочите шаги инициализации реплики PostgreSQL.

Подсказка: Сначала готовим пользователя и доступ, потом копируем данные, и только после этого запускаем.

Ответ:

- Создать пользователя репликации на мастере
- Выполнить pg_basebackup для копирования данных
- Создать файл standby.signal на реплике
- Запустить сервер PostgreSQL на реплике

💡 Шаг 15:

Миссия 15

Расположите шаги настройки синхронной репликации в правильном порядке.

Подсказка: Подумайте: сначала меняем конфиг мастера, потом конфиг реплики, потом проверяем.

Ответ:

- Установить synchronous_standby_names в postgresql.conf на мастере
- Создать standby.signal и настроить подключение на реплике
- Перезапустить мастер и реплику

- Проверить `pg_stat_replication` для подтверждения

Урок 3 - Шардирование части данных через Citus

Шаг 01:

Миссия 1

Алиса готовит кластер на PostgreSQL с расширением Citus. Она думает: с чего начать?

Подсказка: Первым делом нужно определить таблицы, которые будут распределяться.

Ответ:

- **Выбор таблиц для распределения (distributed tables)**
- Сразу добавить больше реплик
- Настроить `pgpool` перед выбором таблиц
- Удалить все индексы для скорости

Шаг 02:

Миссия 2

Боб заметил, что таблица заказов растёт быстрее всех. Что нужно сделать для её шардирования через Citus?

Подсказка: Без распределяющего ключа Citus не сможет понять, как делить данные.

Ответ:

- **Выбрать распределяющий ключ (distribution column)**
- Создать внешние таблицы
- Сделать `VACUUM FULL`
- Настроить `pg_hba.conf`

Шаг 03:

Миссия 3

Команда решает, какой тип ключа выбрать для таблицы. Какой вариант чаще всего подходит для заказов?

Подсказка: Обычно выбирают идентификатор клиента или заказчика, чтобы минимизировать межнодовые соединения.

Ответ:

- **customer_id**
- дата создания
- сумма заказа
- случайный UUID на каждый INSERT

Шаг 04:

Миссия 4

Катя подключает новые воркер-ноды. Что нужно сделать, чтобы Citus начал использовать их?

Подсказка: В Citus воркеры нужно явно регистрировать на координаторе.

Ответ:

- **Добавить их с помощью `SELECT master_add_node(...)`**
- Создать `dblink` на каждую ноду
- Прописать IP в `pg_hba.conf` и этого достаточно

- Запустить ANALYZE

🔗 Шаг 05:

Миссия 5

Денис замечает: join между заказами и пользователями идёт очень медленно. Что может помочь?

Подсказка: Подумайте, как уменьшить межнодовые операции.

Ответ:

- **Использовать один и тот же ключ шардирования для связанных таблиц**
- Добавить индекс только на колонку email
- Сделать таблицу пользователей reference
- Разбить join на подзапросы

🔗 Шаг 06:

Миссия 6

Вася сделал таблицу reference для справочника валют. Почему это хорошая идея?

Подсказка: Reference таблицы доступны на каждой ноде полностью.

Ответ:

- **Это снижает межнодовые join при обращении к справочнику**
- Это ускоряет INSERT в разы
- Это автоматически создаёт индексы
- Это уменьшает размер кластера

🔗 Шаг 07:

Миссия 7

📄 Сопоставьте команды Citus с их назначением.

Подсказка: Вспомните функции master_add_node, create_distributed_table и др.

Ответ:

- **master_add_node** — Добавление воркер-ноды в кластер
- **create_distributed_table** — Распределение таблицы по шардам
- **create_reference_table** — Создание reference-таблицы
- **master_remove_node** — Удаление воркер-ноды из кластера

🔗 Шаг 08:

Миссия 8

Сопоставьте тип таблицы с описанием.

Подсказка: Есть distributed, reference и локальные таблицы.

Ответ:

- **Distributed table** — Таблица разбивается на шарды и хранится на воркерах
- **Reference table** — Таблица копируется целиком на все ноды
- **Local table** — Таблица существует только на координаторе
- **Colocated tables** — Таблицы, распределённые по одному ключу для оптимизации join

🔗 Шаг 09:

Миссия 9

Сопоставьте задачу разработчиков с оптимальным решением в Citus.

Подсказка: Обратите внимание на reference-таблицы и колокацию.

Ответ:

- **Нужно быстро соединять таблицы заказов и клиентов** — Колокация по customer_id
- **Нужно иметь справочник стран во всех запросах** — Создать reference-таблицу
- **Нужно распределить огромный лог транзакций** — Distributed table по transaction_id
- **Нужно хранить временные данные только на координаторе** — Local table

🔗 Шаг 10:

Миссия 10

Сопоставьте ситуацию с правильной стратегией выбора ключа шардирования.

Подсказка: Старайтесь минимизировать межнодовые join.

Ответ:

- **Большая таблица заказов и таблица клиентов** — Выбрать ключ customer_id для обеих
- **Отчёты по дате** — Использовать дату как ключ не рекомендуется
- **Справочник валют** — Сделать reference-таблицу
- **Логи транзакций с уникальными UUID** — UUID подойдёт, если равномерность важнее

🔗 Шаг 11:

Миссия 11

Свяжите задачу оптимизации с методом в Citus.

Подсказка: Иногда reference быстрее, иногда распределение.

Ответ:

- **Ускорить join справочника с заказами** — Сделать справочник reference
- **Распределить нагрузку по логам** — Distributed table
- **Хранить таблицу статусов без шардинга** — Local table
- **Снизить нагрузку на координатор при массовых join** — Колокация по ключу

🔗 Шаг 12:

Миссия 12

🔗 Сопоставьте компонент кластера с его ролью.

Подсказка: Вспомните разницу между координатором и воркером.

Ответ:

- **Координатор** — Принимает запросы и распределяет их
- **Воркер-нода** — Хранит шарды и выполняет запросы
- **Reference table** — Одинакова на всех нодах
- **Distributed table** — Делится на части и распределяется по нодам

🔗 Шаг 13:

Миссия 13

Расставьте шаги создания распределённой таблицы через Citus в правильном порядке.

Подсказка: Сначала создаётся таблица, затем регистрируется как distributed.

Ответ:

- Создать таблицу обычным CREATE TABLE
- Выбрать колонку для шардирования
- Вызвать create_distributed_table
- Проверить, что шарды распределились по нодам

 Шаг 14:

Миссия 14

Поставьте шаги подключения новой воркер-ноды в правильной последовательности.

Подсказка: Нода должна быть готова и доступна перед добавлением.

Ответ:

- Настроить PostgreSQL на воркер-ноде
- Открыть доступ в pg_hba.conf
- Вызвать master_add_node
- Проверить распределение шардов

 Шаг 15:

Миссия 15

Отсортируйте шаги оптимизации join с помощью колокации.

Подсказка: Ключи должны совпадать.

Ответ:

- Определить связанные таблицы
- Выбрать общий ключ шардирования
- Создать таблицы с этим ключом
- Выполнить join и проверить план

 Шаг 16:

Миссия 16

Расположите шаги деплоя reference-таблицы на все ноды.

Подсказка: Reference нужно объявить и синхронизировать.

Ответ:

- Создать таблицу обычным CREATE TABLE
- Вызвать create_reference_table
- Заполнить таблицу справочными данными
- Проверить доступность на воркерах

 Шаг 17:

Миссия 17

Выстройте шаги тестирования кластера после шардирования.

Подсказка: Важно убедиться, что распределение и запросы работают как задумано.

Ответ:

- Выполнить INSERT и проверить распределение
- Сделать SELECT с join

- Проверить нагрузку на воркеры
- Сравнить время с прежними результатами

Урок 4 - Тестирование отказов и балансировки

Шаг 01:

Миссия 1

Команда стартапа запускает PostgreSQL-кластер. Разработчик Петя предлагает протестировать его отказоустойчивость. С чего логичнее начать?

Подсказка: Сначала важно проверить поведение при отключении самой простой ноды.

Ответ:

- **Отключить одну реплику и посмотреть реакцию кластера**
- Сразу отключить мастер-ноду
- Сымитировать сетевую катастрофу во всём дата-центре
- Изменить конфиги балансировщика без тестирования отказов

Шаг 02:

Миссия 2

Аня заметила, что при падении мастера нужно переключиться на другую ноду. Как называется этот процесс?

Подсказка: Этот термин часто встречается в кластерах и балансировщиках.

Ответ:

- **Failover**
- Failback
- Load balancing
- Sharding

Шаг 03:

Миссия 3

Программист Ваня включил балансировщик для распределения запросов. Что он в первую очередь должен проверить?

Подсказка: Балансировщик важен, но главное — убедиться, что запросы реально распределяются.

Ответ:

- **Равномерность распределения запросов между репликами**
- Версию PostgreSQL у каждой ноды
- Размер кэша планировщика
- Количество открытых соединений к балансировщику

Шаг 04:

Миссия 4

Лена решила протестировать поведение кластера при падении сети между узлами. Какой тип сбоя она моделирует?

Подсказка: Это связано не с железом, а с коммуникацией.

Ответ:

- **Сетевой раздел (network partition)**
- Аппаратный отказ диска
- Ошибки SQL-запроса
- Неправильная настройка `pg_hba.conf`

🔮 **Шаг 05:**

Миссия 5

Сергей заметил, что нагрузка на кластер выросла. Что можно сделать, чтобы снизить нагрузку на мастер-ноду?

Подсказка: Реплики могут выполнять не только роль резервов.

Ответ:

- **Перенести часть запросов на реплики для чтения**
- Удалить часть данных из таблиц
- Увеличить таймауты соединений
- Отключить журнал транзакций (WAL)

🔮 **Шаг 06:**

Миссия 6

Инженер Катя заметила, что кластер иногда выбирает новый мастер слишком долго. Какой инструмент чаще всего отвечает за этот процесс?

Подсказка: Подумайте о менеджерах кластера, которые управляют ролями узлов.

Ответ:

- **Система оркестрации (например, Patroni)**
- `VACUUM ANALYZE`
- `pg_dump`
- `pgAdmin`

🔮 **Шаг 07:**

Миссия 7

📄 Задача: Сопоставьте сбой с его описанием.

Подсказка: Вспомните: сбои бывают сетевые, аппаратные и логические.

Ответ:

- **Сетевой раздел** — Ноды теряют связь друг с другом, хотя сами работают.
- **Аппаратный отказ диска** — Физическое повреждение или сбой накопителя.
- **SQL-ошибка** — Некорректный запрос от разработчика.
- **Отказ памяти** — Сбой оперативной памяти сервера.

🔮 **Шаг 08:**

Миссия 8

📄 Задача: Сопоставьте понятие балансировки с его характеристикой.

Подсказка: Балансировка может быть разной: по соединениям, по запросам, по ресурсам.

Ответ:

- **Балансировка по соединениям** — Запросы одного клиента идут всегда на одну ноду.

- **Балансировка по запросам** — Каждый новый запрос распределяется независимо.
- **Балансировка по нагрузке CPU** — Учитывается загрузка процессора узлов.
- **Балансировка по памяти** — Учитывается использование RAM узлов.

🔮 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте инструмент с его назначением при работе кластера.

Подсказка: Подумайте, что делает каждый инструмент.

Ответ:

- **Patroni** — Управление ролями узлов и автоматический failover.
- **pgpool-II** — Балансировка запросов и соединений.
- **etcd** — Хранение конфигурации и координация.
- **repmgr** — Управление репликацией и мониторинг состояния нод.

🔮 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте метрику мониторинга с её значением.

Подсказка: Метрики показывают, где узкое место: CPU, память, сеть или диски.

Ответ:

- **CPU usage** — Загрузка процессора сервера.
- **Memory usage** — Использование оперативной памяти.
- **Disk I/O** — Количество операций чтения/записи на диск.
- **Network latency** — Время отклика между узлами.

🔮 Шаг 11:

Миссия 11

📖 Задача: Сопоставьте действия администратора с их результатом.

Подсказка: Вспомните: перенастройка — это про параметры, а тесты — про наблюдение.

Ответ:

- **Изменить synchronous_commit** — Влияет на скорость подтверждения транзакций.
- **Запустить pgbench** — Создать нагрузку для тестирования.
- **Перезапустить Patroni** — Запустить повторное избрание мастера.
- **Отключить реплику** — Проверить реакцию кластера на потерю узла.

🔮 Шаг 12:

Миссия 12

📖 Задача: Сопоставьте тип теста отказоустойчивости с его примером.

Подсказка: Failover — про потерю мастера, recovery — про восстановление.

Ответ:

- **Failover test** — Отключение мастера для проверки переключения.
- **Recovery test** — Возврат узла в кластер после сбоя.
- **Load test** — Создание искусственной нагрузки pgbench.
- **Network partition test** — Искусственное разделение кластера по сети.

Шаг 13:

Миссия 13

Расставьте шаги тестирования failover в правильном порядке.

Подсказка: Сначала создаём нагрузку, затем выключаем мастер, ждем переключения и проверяем результаты.

Ответ:

- Создать нагрузку с помощью rgbench
- Выключить мастер-ноду
- Дождаться переключения на новую ноду
- Проверить корректность выполнения запросов

Шаг 14:

Миссия 14

Расположите шаги настройки балансировки в правильном порядке.

Подсказка: Сначала настройка соединений, потом правила распределения, затем тестирование.

Ответ:

- Настроить параметры подключения к репликам
- Определить стратегию балансировки
- Запустить нагрузочный тест
- Проанализировать результаты

Шаг 15:

Миссия 15

Расположите действия администратора при восстановлении реплики.

Подсказка: Реплику сначала чинят, потом синхронизируют, потом возвращают в строй.

Ответ:

- Исправить физическую проблему или сбой
- Перезапустить процесс репликации
- Синхронизировать данные с мастером
- Добавить реплику обратно в балансировщик

Шаг 16:

Миссия 16

Расположите шаги подготовки плана тестирования отказоустойчивости.

Подсказка: Сначала определяем цели, потом сценарии, затем запускаем тесты и анализируем.

Ответ:

- Определить цели тестирования
- Составить сценарии отказов
- Провести тесты на кластере
- Проанализировать результаты

Модуль 8 - Бонус. Тюнинг — путь настройки под нагрузку

Урок 1 - Настройка памяти, WAL и autovacuum

Шаг 01:

Миссия 1

Программист Саша настраивает сервер PostgreSQL и заметил, что база стала медленно отдавать результаты при сложных запросах. С чего стоит начать настройку памяти?

Подсказка: Подумайте о параметрах, влияющих на кэширование часто используемых данных.

Ответ:

- Настроить `shared_buffers`
- Увеличить `work_mem` до максимума
- Изменить `random_page_cost`
- Отключить `autovacuum`

Шаг 02:

Миссия 2

Марина проводит нагрузочное тестирование и замечает, что дисковая активность выросла из-за журнала транзакций. Какой параметр помогает контролировать размер WAL?

Подсказка: Это параметр ограничивает общий объем логов перед их архивированием.

Ответ:

- `max_wal_size`
- `checkpoint_timeout`
- `wal_level`
- `synchronous_commit`

Шаг 03:

Миссия 3

Андрей столкнулся с замедлением при сортировках больших выборок. Какой параметр памяти стоит проверить в первую очередь?

Подсказка: Этот параметр влияет на использование памяти при сортировках и хэш-операциях.

Ответ:

- `work_mem`
- `shared_buffers`
- `effective_cache_size`
- `maintenance_work_mem`

Шаг 04:

Миссия 4

Инженер Коля настраивает систему архивирования WAL. Какой уровень `wal_level` нужно выбрать для репликации?

Подсказка: Этот уровень фиксирует все изменения, необходимые для логической или физической репликации.

Ответ:

- `replica`

- minimal
- archive
- default

🔧 Шаг 05:

Миссия 5

Лена обнаружила, что база быстро разрастается из-за мёртвых строк. Какой сервис должен автоматически чистить их?

Подсказка: Этот процесс работает в фоновом режиме и поддерживает таблицы в актуальном состоянии.

Ответ:

- **autovacuum**
- checkpointer
- bgwriter
- analyze

🔧 Шаг 06:

Миссия 6

📖 Задача: Сопоставьте параметры памяти PostgreSQL с их назначением.

Подсказка: Подумайте, какие параметры отвечают за кэш, а какие — за временные операции.

Ответ:

- **shared_buffers** — Используется для хранения страниц таблиц в памяти
- **work_mem** — Используется для сортировок и хэш-таблиц
- **maintenance_work_mem** — Применяется при VACUUM и CREATE INDEX
- **effective_cache_size** — Подсказывает планировщику размер доступного кэша ОС

🔧 Шаг 07:

Миссия 7

Сопоставьте параметры WAL с их описанием.

Подсказка: Подумайте, какой параметр ограничивает размер, а какой задаёт частоту контрольных точек.

Ответ:

- **max_wal_size** — Ограничивает максимальный размер WAL
- **min_wal_size** — Минимальный размер WAL до очистки
- **checkpoint_timeout** — Максимальное время между контрольными точками
- **wal_compression** — Сжимает записи WAL для экономии места

🔧 Шаг 08:

Миссия 8

Сопоставьте параметры autovacuum с их функцией.

Подсказка: Один параметр регулирует интервал, другой — порог активности.

Ответ:

- **autovacuum_naptime** — Интервал запуска процесса autovacuum
- **autovacuum_vacuum_threshold** — Минимальное количество обновлений для запуска VACUUM

- **autovacuum_analyze_threshold** — Минимальное количество изменений для запуска ANALYZE
- **autovacuum_freeze_max_age** — Максимальный возраст транзакций до принудительного VACUUM

🧭 Шаг 09:

Миссия 9

Сопоставьте типичные проблемы с подходящими параметрами для их решения.

Подсказка: Вспомните, что помогает при сортировках, а что при росте таблиц.

Ответ:

- **Медленные сортировки** — Увеличить `work_mem`
- **Частые обращения к диску** — Настроить `shared_buffers`
- **Переполнение WAL** — Отрегулировать `max_wal_size`
- **Разрастание таблиц** — Включить `autovacuum`

🧭 Шаг 10:

Миссия 10

Сопоставьте роли фоновых процессов PostgreSQL.

Подсказка: Один отвечает за очистку, другой за запись буферов.

Ответ:

- **Autovacuum launcher** — Запускает процессы очистки таблиц
- **Checkpoint** — Создает контрольные точки
- **WAL writer** — Записывает журналы транзакций
- **Background writer** — Сбрасывает изменённые страницы в диск

🧭 Шаг 11:

Миссия 11

Расположите шаги настройки памяти PostgreSQL в правильном порядке.

Подсказка: Сначала учитывается кэш, затем настройки временных операций.

Ответ:

- Оценить `shared_buffers`
- Настроить `work_mem`
- Задать `maintenance_work_mem`
- Учесть `effective_cache_size`

🧭 Шаг 12:

Миссия 12

Расставьте шаги настройки WAL.

Подсказка: Сначала определите размер, затем настройте контрольные точки.

Ответ:

- Установить `max_wal_size`
- Задать `min_wal_size`
- Настроить `checkpoint_timeout`
- Включить `wal_compression`

Шаг 13:

Миссия 13

Расположите шаги оптимизации autovacuum.

Подсказка: Начинаем с частоты запуска, затем переходим к порогам.

Ответ:

- Установить autovacuum_naptime
- Задать autovacuum_vacuum_threshold
- Установить autovacuum_analyze_threshold
- Настроить autovacuum_freeze_max_age

Шаг 14:

Миссия 14

Расставьте шаги действий администратора при росте базы данных.

Подсказка: Сначала анализ, затем корректировки параметров.

Ответ:

- Проверить статистику autovacuum
- Скорректировать work_mem
- Изменить shared_buffers
- Проверить max_wal_size

Шаг 15:

Миссия 15

Определите порядок оптимизации под высокую нагрузку.

Подсказка: Сначала память, потом WAL, потом autovacuum.

Ответ:

- Оптимизировать shared_buffers
- Настроить max_wal_size
- Включить autovacuum
- Сбалансировать work_mem

Шаг 16:

Миссия 16

Расположите шаги при настройке тестового стенда.

Подсказка: Сначала память, затем проверка WAL, потом autovacuum.

Ответ:

- Настроить shared_buffers
- Проверить work_mem
- Отрегулировать max_wal_size
- Проверить autovacuum_naptime

Шаг 17:

Миссия 17

Определите правильный порядок настройки production-сервера.

Подсказка: Логика: память → WAL → autovacuum → тесты.

Ответ:

- Настроить shared_buffers и work_mem
- Задать max_wal_size и checkpoint_timeout
- Оптимизировать параметры autovacuum
- Провести нагрузочные тесты

Урок 2 - Индексация выбор и оптимизация

Шаг 01:

Миссия 1

Программист Саша заметил, что запросы на поиск по тексту выполняются медленно. Какой индекс чаще всего помогает ускорить поиск по строковым полям с шаблонами?

Подсказка: Подумайте о типе индекса, который хорошо работает с шаблонами и LIKE.

Ответ:

- **GIN или GiST индекс**
- Hash индекс
- B-Tree индекс
- BRIN индекс

Шаг 02:

Миссия 2

Аня пишет аналитический сервис. Ей нужно быстро искать данные по диапазону дат. Какой индекс лучше выбрать?

Подсказка: Этот индекс хорошо работает с диапазонами и упорядоченными значениями.

Ответ:

- **B-Tree индекс**
- Hash индекс
- GIN индекс
- BRIN индекс

Шаг 03:

Миссия 3

Влад делает систему логирования с миллионами записей. Какой индекс лучше подходит для очень больших таблиц с данными, упорядоченными по времени?

Подсказка: Этот индекс хранит только минимальные и максимальные значения блоков.

Ответ:

- **BRIN индекс**
- Hash индекс
- GIN индекс
- B-Tree индекс

Шаг 04:

Миссия 4

Разработчик Петр жалуется, что Hash индекс не ускоряет его запросы. Почему?

Подсказка: Подумайте, какие ограничения у Hash индекса в PostgreSQL.

Ответ:

- **Hash индекс поддерживает только операции равенства**
- Hash индекс работает быстрее всех
- Hash индекс подходит для диапазонов
- Hash индекс лучше всего для полнотекстового поиска

🔮 Шаг 05:

Миссия 5

Программист Маша думает, стоит ли создавать индекс на колонку с очень низкой избирательностью (например, пол 'М/Ж'). Это хорошая идея?

Подсказка: Индексы эффективны, когда в столбце много разных значений.

Ответ:

- **Нет, индекс будет бесполезным**
- Да, индекс ускорит все запросы

🔮 Шаг 06:

Миссия 6

Сопоставьте тип индекса с его основной особенностью:

Подсказка: Вспомните, какой индекс оптимален для полнотекста, какой — для диапазонов, а какой — для больших массивов данных.

Ответ:

- **B-Tree** — Хорошо работает с диапазонами и упорядоченными значениями
- **GIN** — Эффективен для полнотекстового поиска
- **BRIN** — Оптимален для очень больших таблиц с упорядоченными данными
- **Hash** — Поддерживает только операции равенства

🔮 Шаг 07:

Миссия 7

Саша хочет выбрать правильный индекс для разных задач. Сопоставьте задачу и индекс:

Подсказка: Даты → диапазон, текст → полнотекст, уникальные значения → равенство.

Ответ:

- **Поиск по диапазону дат** — B-Tree
- **Поиск по тексту с полнотекстовым поиском** — GIN
- **Хранение логов в таблице 1 млрд строк** — BRIN
- **Проверка равенства значений** — Hash

🔮 Шаг 08:

Миссия 8

Определите преимущества индексов:

Подсказка: Не все индексы универсальны, у каждого есть свой сценарий использования.

Ответ:

- **B-Tree** — Подходит для сортировок и поиска по диапазону
- **GIN** — Поддерживает массивы и JSONB

- **BRIN** — Занимает мало места
- **Hash** — Простая и быстрая проверка равенства

🔮 Шаг 09:

Миссия 9

Сопоставьте проблему и индекс, который может её решить:

Подсказка: Вспомните: большие таблицы → BRIN, сложные тексты → GIN.

Ответ:

- **Медленный поиск по длинным статьям** — GIN
- **Большие данные в логах** — BRIN
- **Поиск по дате и времени** — B-Tree
- **Проверка равенства идентификаторов** — Hash

🔮 Шаг 10:

Миссия 10

Какой индекс выбрать для каждой ситуации?

Подсказка: У каждой задачи свой инструмент.

Ответ:

- **Фильтрация по числовым диапазонам** — B-Tree
- **Фильтрация по JSONB полям** — GIN
- **Обработка огромных логов по дате** — BRIN
- **Быстрая проверка равенства** — Hash

🔮 Шаг 11:

Миссия 11

Расположите шаги создания индекса в PostgreSQL в правильном порядке:

Подсказка: Сначала нужно понять проблему, затем выбрать тип индекса, потом его создать и протестировать.

Ответ:

- Определение медленного запроса
- Выбор подходящего типа индекса
- Создание индекса
- Тестирование производительности

🔮 Шаг 12:

Миссия 12

Расположите шаги анализа индекса:

Подсказка: Сначала смотрим план, потом оцениваем эффективность, затем оптимизируем.

Ответ:

- Запуск EXPLAIN
- Оценка эффективности индекса
- Оптимизация при необходимости
- Подтверждение улучшений

Шаг 13:

Миссия 13

Как программист применяет индексы:

Подсказка: Вспомните: сначала запрос, потом индекс, тесты, поддержка.

Ответ:

- Анализ запроса
- Создание индекса
- Проверка ускорения
- Регулярная поддержка

Шаг 14:

Миссия 14

Порядок оптимизации запросов с индексами:

Подсказка: Важно — анализировать, создавать, проверять и поддерживать.

Ответ:

- Анализ медленных запросов
- Выбор типа индекса
- Создание индекса
- Тестирование ускорения

Шаг 15:

Миссия 15

Шаги работы с полнотекстовым поиском:

Подсказка: Нужно проанализировать, создать индекс, проверить и оптимизировать.

Ответ:

- Определение задачи поиска
- Создание GIN индекса
- Тестирование результатов
- Оптимизация при необходимости

Шаг 16:

Миссия 16

Оптимизация индексов в проекте:

Подсказка: Последовательность действий — от анализа до поддержки.

Ответ:

- Анализ структуры данных
- Выбор индекса
- Создание
- Поддержка

Шаг 17:

Миссия 17

Финальный вызов: как программист оптимизирует работу с индексами?

Подсказка: Думайте как DBA: анализируйте, действуйте и проверяйте.

Ответ:

- Анализ производительности
- Выбор оптимального индекса
- Создание и тестирование
- Мониторинг и поддержка

Урок 3 - Планировщик запросов и EXPLAIN ANALYZE

Шаг 01:

Миссия 1

Алексей пишет отчётный запрос для бухгалтерии, но он работает слишком медленно. Какой инструмент в PostgreSQL он должен использовать первым, чтобы понять, почему так происходит?

Подсказка: Этот инструмент показывает план выполнения запроса и время на каждом этапе.

Ответ:

- **EXPLAIN ANALYZE**
- VACUUM FULL
- pg_stat_activity
- ANALYZE TABLE

Шаг 02:

Миссия 2

Марина анализирует запрос с несколькими JOIN. В плане выполнения она увидела Nested Loop . Что это значит?

Подсказка: Это один из способов соединять таблицы, не всегда самый быстрый при больших объёмах.

Ответ:

- **Таблицы соединяются построчно, вложенными циклами**
- Таблицы сортируются и соединяются
- Используется хэш-таблица для соединения
- Таблицы объединяются в параллельных потоках

Шаг 03:

Миссия 3

Сергей заметил, что в плане выполнения появился Seq Scan по большой таблице. Что это означает?

Подсказка: Это стратегия, которая не использует индекс.

Ответ:

- **Таблица читается полностью построчно**
- Используется индекс по первичному ключу
- Данные сортируются перед выборкой
- Запрос выполняется параллельно

Шаг 04:

Миссия 4

Ольга исследует план запроса и видит Index Scan . Почему это может быть быстрее, чем Seq Scan ?

Подсказка: Индексы помогают быстрее находить нужные строки.

Ответ:

- **Потому что чтение идёт сразу по нужным индексным записям**
- Потому что таблица читается целиком
- Потому что данные сортируются автоматически
- Потому что используется параллельный режим

Шаг 05:

Миссия 5

Иванка выполняет EXPLAIN ANALYZE и замечает, что стоимость cost=0.00..120.00 сильно отличается от реального времени. Что это значит?

Подсказка: Планировщик строит прогнозы, которые могут расходиться с реальностью.

Ответ:

- **Оценка планировщика неточна из-за статистики**
- Запрос всегда оптимизирован идеально
- Планировщик не учитывает индексы
- Ошибка в синтаксисе запроса

Шаг 06:

Миссия 6

Дмитрий видит в плане Hash Join . Когда этот алгоритм соединения работает особенно хорошо?

Подсказка: Представьте, что данные заранее кладутся в хэш-таблицу для быстрого поиска.

Ответ:

- **Когда одна из таблиц помещается целиком в память**
- Когда обе таблицы отсортированы
- Когда таблицы соединяются построчно
- Когда таблицы очень маленькие

Шаг 07:

Миссия 7

Алина заметила в плане Bitmap Index Scan . Чем он отличается от обычного Index Scan?

Подсказка: Он полезен, когда нужно выбрать много строк через индекс.

Ответ:

- **Он сначала формирует битовую карту, а потом читает строки пачками**
- Он всегда быстрее обычного индекса
- Он работает только для первичных ключей
- Он сортирует данные перед выборкой

Шаг 08:

Миссия 8

Виктор анализирует план и видит Parallel Seq Scan . Что это означает?

Подсказка: Представьте, что несколько работников читают книгу одновременно по разным страницам.

Ответ:

- **Таблица читается одновременно несколькими процессами**
- Таблица читается только через индекс
- Таблица сортируется перед выборкой
- Запрос выполняется без планировщика

🧙 Шаг 09:

Миссия 9

Сопоставьте типы соединений в плане запроса с их особенностями.

Подсказка: Вспомните: один соединяет построчно, другой строит хэш, третий требует сортировки.

Ответ:

- **Nested Loop** — Построчное соединение с вложенными циклами
- **Hash Join** — Создание хэш-таблицы для соединения
- **Merge Join** — Соединение отсортированных таблиц

🧙 Шаг 10:

Миссия 10

Сопоставьте элементы EXPLAIN ANALYZE с их значением.

Подсказка: Одно показывает планируемую стоимость, другое — реальное время, третье — число строк.

Ответ:

- **cost** — Прогнозируемая стоимость выполнения
- **actual time** — Реальное время выполнения
- **rows** — Количество обработанных строк

🧙 Шаг 11:

Миссия 11

Сопоставьте ситуацию с инструментом для анализа.

Подсказка: Если нужно посмотреть активные запросы — это одно, если понять план — другое.

Ответ:

- **Понять текущие активные запросы** — pg_stat_activity
- **Посмотреть план выполнения** — EXPLAIN
- **Измерить фактическое время выполнения** — EXPLAIN ANALYZE

🧙 Шаг 12:

Миссия 12

Сопоставьте элементы плана с тем, на что они указывают.

Подсказка: Rows — это про количество, Buffers — про использование памяти.

Ответ:

- **rows** — Количество строк
- **loops** — Сколько раз выполнялась операция
- **buffers** — Информация об обращениях к памяти

Шаг 13:

Миссия 13

Сопоставьте вид сканирования с его характеристикой.

Подсказка: Seq Scan идёт подряд, Index Scan использует дерево, Bitmap — читает пачками.

Ответ:

- **Seq Scan** — Последовательное чтение всей таблицы
- **Index Scan** — Поиск по дереву индекса
- **Bitmap Index Scan** — Формирование битовой карты и чтение пачками

Шаг 14:

Миссия 14

Расставьте шаги анализа запроса в правильном порядке.

Подсказка: Сначала запускается план, потом проверяется статистика, потом интерпретируются цифры.

Ответ:

- Запустить EXPLAIN ANALYZE
- Посмотреть статистику строк и времени
- Сравнить прогнозы и фактические результаты
- Сделать выводы и оптимизировать

Шаг 15:

Миссия 15

Расставьте шаги оптимизации запроса.

Подсказка: Сначала анализируем, потом добавляем индексы, потом проверяем снова.

Ответ:

- Анализ плана с помощью EXPLAIN ANALYZE
- Определение узких мест (Seq Scan, Nested Loop)
- Добавление или корректировка индексов
- Проверка результата и сравнение с предыдущим планом

Шаг 16:

Миссия 16

Упорядочите действия администратора при подозрении на неэффективный план.

Подсказка: Сначала проверяется статистика, потом обновляются данные, затем повторный анализ.

Ответ:

- Проверить статистику таблиц
- Выполнить ANALYZE
- Запустить EXPLAIN ANALYZE
- Оптимизировать запрос при необходимости

Урок 4 - Параллельные запросы и их настройка

Шаг 01:

Миссия 1

Аня запускает отчёт по продажам и замечает, что запрос выполняется слишком долго. Она думает о включении параллельных запросов в PostgreSQL. Какой параметр нужно проверить в первую очередь?

Подсказка: Без этого параметра даже самые оптимальные настройки параллельности не заработают.

Ответ:

- **max_parallel_workers_per_gather**
- shared_buffers
- work_mem
- wal_level

Шаг 02:

Миссия 2

Игорь настраивает тестовый сервер. Он включил параллельные запросы, но они всё ещё не выполняются. Что ещё нужно проверить?

Подсказка: Иногда проблема не в параметрах конкретного запроса, а в общей настройке числа доступных рабочих процессов.

Ответ:

- work_mem
- **max_parallel_workers**
- random_page_cost
- effective_cache_size

Шаг 03:

Миссия 3

Марина запускает параллельный запрос на большом кластере. Однако запрос идёт только в один поток. В чём может быть причина?

Подсказка: Параллельность зависит от оценки объёма данных.

Ответ:

- Недостаток памяти
- **Планировщик считает, что данных мало для распараллеливания**
- Сломанный индекс
- Неправильный join

Шаг 04:

Миссия 4

Алексей хочет ускорить запросы с использованием агрегатов SUM и COUNT. Включение параллельных запросов даст эффект, если...

Подсказка: Не все функции умеют работать в многопоточном режиме.

Ответ:

- **Агрегат поддерживает parallel safe**

- Запрос короткий
- Таблица маленькая
- Используется индекс

🔮 Шаг 05:

Миссия 5

Дмитрий проверяет нагрузку на CPU. Один параллельный запрос занял все ядра и начал мешать другим задачам. Что стоит сделать?

Подсказка: Иногда нужно ограничивать, а не расширять.

Ответ:

- **Уменьшить `max_parallel_workers_per_gather`**
- Увеличить `shared_buffers`
- Поменять `wal_level`
- Снизить `random_page_cost`

🔮 Шаг 06:

Миссия 6

Олег изучает параметры для настройки параллельных запросов. Сопоставьте параметр с его назначением.

Подсказка: Один отвечает за количество на один запрос, другой — за общее количество, третий — за распределение по узлам.

Ответ:

- **`max_parallel_workers_per_gather`** — Ограничивает число воркеров для одного запроса
- **`max_parallel_workers`** — Максимальное количество воркеров в системе
- **`parallel_setup_cost`** — Стоимость инициализации параллельного плана
- **`parallel_tuple_cost`** — Стоимость передачи одной строки между воркерами

🔮 Шаг 07:

Миссия 7

Вика настраивает кластер для параллельных запросов. Соотнесите тип операции с её влиянием на параллельность.

Подсказка: Вспомните, что не все join равны, а seq scan часто параллелизируется.

Ответ:

- **Sequential Scan** — Часто может выполняться параллельно
- **Nested Loop Join** — Редко параллелизируется
- **Hash Join** — Может быть параллельным
- **Index Scan** — Обычно выполняется в одном потоке

🔮 Шаг 08:

Миссия 8

Степан анализирует EXPLAIN ANALYZE. Сопоставьте элементы плана с их значением.

Подсказка: Смотрите на ключевые слова Gather и Parallel.

Ответ:

- **Gather** — Собирает результаты от параллельных воркеров
- **Parallel Seq Scan** — Последовательное чтение таблицы в несколько потоков

- **Worker** — Один из процессов, выполняющий часть работы
- **Cost** — Оценка затрат выполнения узла

🔮 Шаг 09:

Миссия 9

Катя сравнивает разные планы выполнения. Сопоставьте ситуацию с объяснением.

Подсказка: Параллельный план выбирается только при достаточной выгоде.

Ответ:

- **Планировщик выбрал последовательный план** — Данных слишком мало для параллельного исполнения
- **Планировщик выбрал параллельный план** — Большой объём данных делает распараллеливание выгодным
- **Выполняется Gather** — Результаты собираются в один поток
- **Worker запускается** — Один из процессов выполняет подзадачу

🔮 Шаг 10:

Миссия 10

Никита настраивает параметры стоимости параллельных запросов. Соотнесите параметр с его ролью.

Подсказка: Один влияет на старт, другой — на каждую строку.

Ответ:

- **parallel_setup_cost** — Влияет на выбор параллельного плана при инициализации
- **parallel_tuple_cost** — Стоимость передачи каждой строки
- **max_worker_processes** — Общее число процессов воркеров в системе
- **effective_io_concurrency** — Количество одновременных операций ввода-вывода

🔮 Шаг 11:

Миссия 11

Саша готовит нагрузочные тесты. Соотнесите инструмент с его назначением в контексте параллельных запросов.

Подсказка: Один инструмент для измерения, другой для анализа плана.

Ответ:

- **pgbench** — Используется для нагрузочного тестирования
- **EXPLAIN ANALYZE** — Показывает фактическое выполнение плана
- **auto_explain** — Автоматически логирует планы выполнения
- **pg_stat_activity** — Показывает активные запросы

🔮 Шаг 12:

Миссия 12

Расставьте шаги настройки параллельных запросов в правильном порядке.

Подсказка: Сначала активируем общую возможность, потом задаём лимиты, затем анализируем планы.

Ответ:

- Включить `parallel_setup_cost` и `parallel_tuple_cost`
- Настроить `max_parallel_workers`

- Настроить `max_parallel_workers_per_gather`
- Проверить EXPLAIN ANALYZE для запросов

Шаг 13:

Миссия 13

Сергей готовит отчёт для руководства. Расставьте шаги работы с EXPLAIN ANALYZE для проверки параллельности.

Подсказка: Сначала запускаем команду, потом ищем ключевые слова.

Ответ:

- Выполнить EXPLAIN ANALYZE
- Найти Gather
- Найти Parallel Seq Scan
- Сравнить время выполнения с обычным планом

Шаг 14:

Миссия 14

Ирина оптимизирует отчёты. Расставьте действия для контроля использования ресурсов параллельными запросами.

Подсказка: Нужно сначала включить параллельность, потом ограничить и проверить нагрузку.

Ответ:

- Проверить настройки `max_parallel_workers`
- Установить `max_parallel_workers_per_gather`
- Выполнить запрос под нагрузкой
- Анализировать CPU и I/O

Шаг 15:

Миссия 15

Артём отлаживает сложный отчёт. Расположите шаги диагностики параллельного запроса.

Подсказка: Диагностика начинается с анализа плана, затем проверки параметров, и в конце — повторный запуск.

Ответ:

- Выполнить EXPLAIN
- Проверить `max_parallel_workers_per_gather`
- Запустить запрос
- Сравнить результаты времени

Урок 5 - Управление чекпоинтами и кешем

Шаг 01:

Миссия 1

Сопоставьте параметры PostgreSQL с их назначением (чекпоинты и кеш).

Подсказка: Подумайте: один параметр задаёт частоту, другой — объём памяти, третий — сглаживание нагрузки.

Ответ:

- **`checkpoint_timeout`** — Максимальное время между чекпоинтами

- **shared_buffers** — Объем памяти для кеша страниц
- **checkpoint_completion_target** — Доля времени на растягивание записи во время чекпоинта
- **wal_buffers** — Буфер изменений перед записью в WAL

🔧 Шаг 02:

Миссия 2

Вы видите, что чекпоинты происходят слишком часто. Сопоставьте действия администратора с правильным эффектом.

Подсказка: Вспомните: увеличение параметра не всегда сразу улучшает всё, важно балансировать.

Ответ:

- **Увеличение checkpoint_timeout** — Реже чекпоинты, но риск длинного восстановления
- **Снижение checkpoint_timeout** — Чаше чекпоинты, меньше WAL при сбое
- **Увеличение shared_buffers** — Меньше обращений к диску, но выше нагрузка при чекпоинтах
- **Снижение shared_buffers** — Чаше обращения к диску

🔧 Шаг 03:

Миссия 3

Стажёр путается в терминах. Помогите ему: сопоставьте параметр PostgreSQL с примером значения.

Подсказка: Таймаут измеряется в секундах, кеш — в мегабайтах, target — доля, а wal_buffers — блоки.

Ответ:

- **checkpoint_timeout** — 300s
- **shared_buffers** — 4GB
- **checkpoint_completion_target** — 0.7
- **wal_buffers** — 16MB

🔧 Шаг 04:

Миссия 4

Представьте: DBA настраивает новый сервер. Расположите шаги оптимизации чекпоинтов в правильном порядке.

Подсказка: Сначала анализ логов, потом настройка параметров, затем мониторинг.

Ответ:

- Анализ частоты чекпоинтов в логах
- Изменение checkpoint_timeout
- Настройка checkpoint_completion_target
- Мониторинг результата

🔧 Шаг 05:

Миссия 5

DBA оптимизирует кеш. Расставьте действия в правильном порядке.

Подсказка: Сначала нужно замерить, потом изменить, потом проверить.

Ответ:

- Анализ hit ratio кеша
- Изменение shared_buffers
- Перезапуск сервера
- Сравнение метрик после изменений

 Шаг 06:**Миссия 6**

В проекте неожиданно выросла нагрузка. DBA принимает решения. Расставьте шаги управления чекпоинтами по логике.

Подсказка: Сначала стабилизируем систему, потом анализируем, потом делаем тонкую настройку.

Ответ:

- Временное увеличение checkpoint_timeout
- Анализ статистики pg_stat_bgwriter
- Настройка checkpoint_completion_target
- Регулярный мониторинг

 Шаг 07:**Миссия 7**

Junior DBA решает оптимизировать WAL. Расставьте действия в правильной последовательности.

Подсказка: Сначала убедиться в нагрузке, потом корректировать wal_buffers, затем проверка.

Ответ:

- Проверка скорости записи WAL
- Изменение wal_buffers
- Тестирование на нагрузочном стенде
- Мониторинг стабильности

 Шаг 08:**Миссия 8**

Представьте, что проект попал в продакшен с неправильными настройками кеша. Восстановите шаги исправления ситуации.

Подсказка: Сначала заметить проблему, потом локализовать, затем применить настройку и перепроверить.

Ответ:

- Обнаружение низкого cache hit ratio
- Определение текущего значения shared_buffers
- Корректировка shared_buffers
- Проверка после изменения

 Шаг 09:**Миссия 9**

Финальный квест: команда должна согласованно настроить PostgreSQL под нагрузку. Расставьте шаги в правильной логике.

Подсказка: Думайте как о полном цикле — диагностика → настройка → тест → контроль.

Ответ:

- Диагностика логов чекпоинтов и кеша
- Настройка `shared_buffers` и `checkpoint_timeout`
- Тестирование под нагрузкой
- Непрерывный мониторинг

Урок 6 - Снижение bloating и рост производительности

Шаг 01:

Миссия 1

Антон заметил, что таблица с логами растёт слишком быстро. Что из ниже перечисленного поможет уменьшить bloating?

Подсказка: Обратите внимание на методы удаления старых данных и оптимизацию структуры таблиц.

Ответ:

- **Регулярный VACUUM и индексация**
- Добавление новых столбцов без индексов
- Запись всех данных в одну строку
- Полная перезапись таблицы каждый час

Шаг 02:

Миссия 2

Мария хочет уменьшить нагрузку на диск при работе с большими таблицами. Какой метод лучше использовать?

Подсказка: Подумайте о том, как PostgreSQL управляет удалёнными строками и индексами.

Ответ:

- **Автоматический autovacuum и настройка threshold**
- Увеличение размера page без изменения vacuum
- Использование только временных таблиц
- Удаление всех индексов

Шаг 03:

Миссия 3

Иван заметил, что частые UPDATE увеличивают размер таблиц. Что стоит сделать в первую очередь?

Подсказка: Подумайте, как PostgreSQL хранит старые версии строк.

Ответ:

- **Настроить регулярный VACUUM и analyze**
- Удалять таблицу и создавать заново каждый день
- Перевести все столбцы в тип TEXT
- Выключить WAL

Шаг 04:

Миссия 4

 Задача: Сопоставление терминов с определениями

Подсказка: Подумайте о том, как PostgreSQL управляет удалёнными и обновлёнными строками.

Ответ:

- **VACUUM** — Удаляет мёртвые строки и освобождает место для новых данных
- **ANALYZE** — Собирает статистику для оптимизатора запросов
- **autovacuum** — Автоматически запускает VACUUM и ANALYZE
- **bloating** — Раздувание таблицы из-за устаревших строк

Шаг 05:

Миссия 5

 Сопоставьте действия с эффектом на производительность

Подсказка: Подумайте, какие действия снижают или увеличивают bloating.

Ответ:

- **Регулярный VACUUM** — Снижает bloating и ускоряет SELECT
- **Частые UPDATE без vacuum** — Увеличивает bloating и замедляет операции
- **Создание индексов** — Ускоряет поиск данных
- **Удаление старых логов** — Освобождает место и улучшает производительность

Шаг 06:

Миссия 6

 Сопоставьте типы индексов с их особенностями

Подсказка: Подумайте о том, какой индекс лучше подходит для часто обновляемых таблиц.

Ответ:

- **B-tree** — Подходит для поиска по точным значениям и диапазонам
- **GIN** — Эффективен для полнотекстового поиска
- **BRIN** — Экономит место на больших таблицах
- **Hash** — Хорош для равенства, но менее универсален

Шаг 07:

Миссия 7

 Сопоставьте настройки autovacuum с их назначением

Подсказка: Подумайте о частоте запуска и размере порции строк для очистки.

Ответ:

- **autovacuum_vacuum_threshold** — Минимальное количество изменений перед запуском VACUUM
- **autovacuum_vacuum_scale_factor** — Процент от таблицы для запуска VACUUM
- **autovacuum_analyze_threshold** — Минимальное количество изменений перед ANALYZE
- **autovacuum_max_workers** — Количество одновременных процессов autovacuum

Шаг 08:

Миссия 8

Расположите действия для снижения bloating в правильном порядке:

Подсказка: Начните с выявления проблем, потом выполняйте очистку и настройку, затем проверку эффективности.

Ответ:

- Проанализировать таблицы и выявить bloating
- Запустить VACUUM и ANALYZE
- Настроить autovacuum для регулярного обслуживания
- Проверить производительность запросов

 Шаг 09:

Миссия 9

Расположите шаги оптимизации UPDATE для снижения bloating:

Подсказка: Сначала минимизируйте ненужные обновления, потом очищайте старые версии строк.

Ответ:

- Проанализировать какие поля реально изменяются
- Использовать partial UPDATE
- Регулярно запускать VACUUM
- Проверить размер таблицы и производительность

 Шаг 10:

Миссия 10

Расположите действия DBA для контроля bloating:

Подсказка: Сначала мониторинг, потом настройка, потом автоматизация процессов.

Ответ:

- Настроить мониторинг размера таблиц и индексов
- Проверить частоту autovacuum и thresholds
- Внести коррективы в конфигурацию PostgreSQL
- Автоматизировать регулярные проверки и отчёты

 Шаг 11:

Миссия 11

Сергей хочет ускорить SELECT на большом массиве данных с frequent UPDATE. Какой подход эффективнее?

Подсказка: Подумайте о типах индексов и их влиянии на часто обновляемые таблицы.

Ответ:

- **Использовать BRIN индекс для больших таблиц**
- Создать B-tree индекс на все колонки
- Полностью отключить индексы
- Дублировать таблицу в памяти

 Шаг 12:

Миссия 12

Ольга заметила, что после миграции данных таблица раздулась. Какой инструмент поможет быстро оценить размер bloating?

Подсказка: PostgreSQL предоставляет специальные функции для анализа размера таблиц и индексов.

Ответ:

- **pg_stat_all_tables** и **pg_total_relation_size**
- EXPLAIN SELECT
- VACUUM FULL без анализа
- DROP TABLE и CREATE TABLE

Шаг 13:

Миссия 13

 Сопоставьте виды VACUUM с их особенностями

Подсказка: Есть разница между обычной очисткой и полной.

Ответ:

- **VACUUM** — Очищает мёртвые строки, не блокируя таблицу
- **VACUUM FULL** — Полностью переписывает таблицу, освобождая пространство
- **autovacuum** — Автоматически запускает VACUUM по заданным порогам
- **ANALYZE** — Собирает статистику для оптимизатора

Шаг 14:

Миссия 14

 Сопоставьте причины медленной работы SELECT с действиями по оптимизации

Подсказка: Обратите внимание, что bloating и индексы напрямую влияют на скорость.

Ответ:

- **Большой bloating** — Запуск VACUUM и настройка autovacuum
- **Отсутствие индекса** — Создание подходящего индекса
- **Слишком частые UPDATE** — Использование BRIN или частичных индексов
- **Старые статистики** — ANALYZE таблицы

Шаг 15:

Миссия 15

Расположите шаги работы DBA для регулярного контроля bloating:

Подсказка: Мониторинг, анализ, очистка, оптимизация.

Ответ:

- Мониторинг таблиц и индексов
- ANALYZE и сбор статистики
- Регулярный VACUUM или VACUUM FULL
- Корректировка настроек autovacuum

Шаг 16:

Миссия 16

Расположите действия разработчика для минимизации bloating при UPDATE:

Подсказка: Сначала подумайте как уменьшить лишние обновления, потом настройте очистку.

Ответ:

- Проверка, какие поля реально обновляются
- Использовать partial UPDATE или UPSERT
- Проверка размера таблицы после изменений

- Настройка autovacuum и регулярный VACUUM

Модуль 9 - Бонус. Конкуренция и блокировки — путь битвы

Урок 1 - MVCC и уровни изоляции

Шаг 01:

Миссия 1

Антон открыл две сессии в PostgreSQL и заметил, что одна транзакция не видит изменения другой до commit. Что объясняет такое поведение?

Подсказка: Подумайте о том, как MVCC управляет видимостью строк.

Ответ:

- **MVCC обеспечивает консистентный снимок данных для каждой транзакции**
- Транзакции блокируют все таблицы по умолчанию
- PostgreSQL не поддерживает параллельные транзакции
- Транзакции видят все изменения мгновенно

Шаг 02:

Миссия 2

Мария хочет предотвратить «грязное чтение» при одновременном изменении одной строки. Какой уровень изоляции следует выбрать?

Подсказка: Уровни изоляции контролируют, какие изменения видит транзакция.

Ответ:

- **Read Committed**
- Read Uncommitted
- Serializable без блокировок
- No Isolation

Шаг 03:

Миссия 3

Иван заметил, что при одновременном обновлении одной строки другая транзакция получает ошибку «could not serialize access». Что это значит?

Подсказка: Подумайте о конфликте транзакций на уровне Serializable.

Ответ:

- **Возник конфликт сериализуемости, транзакцию нужно повторить**
- Ошибка синтаксиса SQL
- Таблица заблокирована полностью
- MVCC не работает с UPDATE

Шаг 04:

Миссия 4

 Сопоставьте уровни изоляции с их характеристиками

Подсказка: Подумайте, какие аномалии предотвращает каждый уровень.

Ответ:

- **Read Uncommitted** — Позволяет видеть незакоммиченные изменения, могут быть грязные чтения
- **Read Committed** — Видит только зафиксированные изменения, предотвращает грязные чтения
- **Repeatable Read** — Гарантирует, что все SELECT внутри транзакции видят один и тот же снимок
- **Serializable** — Полностью предотвращает фантомные чтения и конфликты сериализуемости

Шаг 05:

Миссия 5

 Сопоставьте типы конфликтов с их описанием

Подсказка: Подумайте, какие ситуации вызывают блокировки и ошибки сериализации.

Ответ:

- **Dirty Read** — Транзакция читает незакоммиченные изменения другой транзакции
- **Non-Repeatable Read** — Повторный SELECT видит разные значения из-за commit другой транзакции
- **Phantom Read** — Новые строки появляются между SELECT в одной транзакции
- **Serialization Failure** — Конфликт при уровне Serializable, транзакцию нужно повторить

Шаг 06:

Миссия 6

 Сопоставьте MVCC-понятия с объяснением

Подсказка: Подумайте о том, как PostgreSQL хранит версии строк.

Ответ:

- **Tuple Version** — Каждая строка хранит свои версии для разных транзакций
- **xmin/xmax** — Идентификаторы транзакций, управляющие видимостью строк
- **Snapshot** — Консистентный снимок базы данных для транзакции
- **VACUUM** — Удаляет мёртвые строки, освобождая место

Шаг 07:

Миссия 7

 Сопоставьте поведение транзакций с типами чтения

Подсказка: Подумайте, что транзакция видит при разных уровнях изоляции.

Ответ:

- **Read Committed** — Каждый SELECT видит только зафиксированные изменения
- **Repeatable Read** — Все SELECT видят один и тот же снимок данных
- **Serializable** — Полное предотвращение фантомов и конфликтов
- **Read Uncommitted** — Может видеть незакоммиченные данные

Шаг 08:

Миссия 8

Расположите шаги решения конфликта сериализации в правильном порядке:

Подсказка: Сначала поймите, какие транзакции конфликтуют, потом откат и повтор.

Ответ:

- Выявить транзакции, вызывающие конфликт
- Откат одной из конфликтующих транзакций
- Повторить откатанную транзакцию
- Проверить успешность повторной транзакции

 Шаг 09:

Миссия 9

Расположите действия для минимизации фантомных чтений:

Подсказка: Начните с выбора подходящего уровня изоляции, далее используйте блокировки и индексацию.

Ответ:

- Выбрать уровень Repeatable Read или Serializable
- Использовать явные блокировки SELECT FOR UPDATE
- Создать индексы для ускорения поиска
- Проверить результаты SELECT

 Шаг 10:

Миссия 10

Расположите шаги работы DBA для контроля MVCC и уровней изоляции:

Подсказка: Мониторинг, анализ блокировок, настройка уровней изоляции.

Ответ:

- Мониторинг активных транзакций и блокировок
- Анализ частоты конфликтов сериализации
- Настройка подходящего уровня изоляции для нагрузки
- Документирование и автоматизация процедур

 Шаг 11:

Миссия 11

Ольга хочет убедиться, что повторные SELECT в транзакции видят одинаковые данные. Какой уровень изоляции выбрать?

Подсказка: Подумайте, какой уровень гарантирует одинаковый снимок данных для всех SELECT.

Ответ:

- **Repeatable Read**
- Read Committed
- Read Uncommitted
- Serializable без блокировок

 Шаг 12:

Миссия 12

Сергей хочет минимизировать количество откатов транзакций из-за конфликтов сериализации. Что поможет?

Подсказка: Подумайте о правильной настройке транзакций и индексах.

Ответ:

- **Использовать индексы и оптимизировать порядок операций**
- Отключить MVCC
- Уменьшить размер базы данных
- Использовать Read Uncommitted

Шаг 13:

Миссия 13

 Сопоставьте PostgreSQL-конфликты с реальными примерами

Подсказка: Подумайте, что происходит, если две транзакции одновременно изменяют одни и те же строки.

Ответ:

- **Dirty Read** — Транзакция читает изменения другой, ещё не зафиксированной транзакции
- **Non-Repeatable Read** — SELECT возвращает разные значения из-за commit другой транзакции
- **Phantom Read** — Новые строки появляются между SELECT
- **Serialization Failure** — Конфликт на уровне Serializable, требуется откат

Шаг 14:

Миссия 14

 Сопоставьте тип блокировки с её назначением

Подсказка: Подумайте, какая блокировка защищает строки, а какая таблицу целиком.

Ответ:

- **Row-level Lock** — Блокирует отдельные строки, позволяя параллельные изменения других
- **Table-level Lock** — Блокирует всю таблицу для операций записи
- **Share Lock** — Позволяет читать, но запрещает запись другим транзакциям
- **Exclusive Lock** — Полностью запрещает доступ другим транзакциям

Шаг 15:

Миссия 15

Расположите шаги работы разработчика для безопасного параллельного обновления:

Подсказка: Сначала планируйте операции, потом используйте блокировки, потом commit.

Ответ:

- Определить строки для обновления
- Использовать SELECT FOR UPDATE
- Выполнить изменения
- Закоммитить транзакцию

Шаг 16:

Миссия 16

Расположите действия DBA для минимизации ошибок сериализации при высокой нагрузке:

Подсказка: Мониторинг, анализ, оптимизация и автоматизация.

Ответ:

- Мониторинг активных транзакций и блокировок
- Анализ конфликтов Serializable
- Оптимизация порядка операций и индексов
- Автоматизация повторного запуска откатанных транзакций

Урок 2 - Конкуренция, блокировки и deadlocks

Шаг 01:

Миссия 1

Алексей заметил, что при одновременном обновлении двух строк одна транзакция ждёт другую. Что это за механизм?

Подсказка: Подумайте о том, как PostgreSQL управляет блокировками строк.

Ответ:

- **Row-level Lock (блокировка строки)**
- Table-level Lock
- MVCC не использует блокировки
- Транзакции не блокируют данные

Шаг 02:

Миссия 2

Ирина открыла две транзакции, которые обновляют строки в разном порядке, и получила ошибку deadlock. Что произошло?

Подсказка: Подумайте о взаимной блокировке двух транзакций.

Ответ:

- **Возникла взаимная блокировка (deadlock)**
- Ошибка синтаксиса SQL
- Транзакции автоматически откатываются без причины
- База данных зависла навсегда

Шаг 03:

Миссия 3

Сергей хочет предотвратить deadlocks при параллельных обновлениях таблицы заказов. Что стоит использовать?

Подсказка: Подумайте, как упорядочить доступ к ресурсам.

Ответ:

- **Всегда обновлять строки в одном и том же порядке**
- Отключить блокировки
- Использовать Read Uncommitted
- Использовать VACUUM перед UPDATE

Шаг 04:

Миссия 4

 Сопоставьте тип блокировки с её назначением

Подсказка: Подумайте, какая блокировка защищает строку, а какая всю таблицу.

Ответ:

- **Row-level Lock** — Блокирует отдельные строки, позволяя параллельное изменение других
- **Table-level Lock** — Блокирует всю таблицу для операций записи
- **Share Lock** — Разрешает читать, запрещает запись другим транзакциям
- **Exclusive Lock** — Запрещает доступ другим транзакциям

 Шаг 05:

Миссия 5

 Сопоставьте типы deadlock с примерами

Подсказка: Подумайте, какие операции блокируют друг друга.

Ответ:

- **Simple Deadlock** — Две транзакции одновременно ждут друг друга, обновляя одну строку
- **Complex Deadlock** — Несколько транзакций блокируют разные ресурсы в цикле
- **Phantom Deadlock** — Возникает при добавлении новых строк, создавая неожиданные блокировки
- **Resource Deadlock** — Блокировка на уровне таблицы или индекса

 Шаг 06:

Миссия 6

 Сопоставьте способы обнаружения и решения deadlock с их описанием

Подсказка: Подумайте о логах и автоматических механизмах PostgreSQL.

Ответ:

- **Logging Deadlocks** — Использование журналов PostgreSQL для выявления циклов блокировок
- **Automatic Deadlock Resolution** — PostgreSQL откатывает одну из транзакций для разрыва цикла
- **Manual Investigation** — Анализ блокировок с помощью pg_locks и системных представлений
- **Preventive Ordering** — Обновление ресурсов в фиксированном порядке

 Шаг 07:

Миссия 7

 Сопоставьте причины возникновения deadlocks с примерами

Подсказка: Подумайте, какие действия вызывают циклы блокировок.

Ответ:

- **Concurrent Updates** — Две транзакции одновременно обновляют одни и те же строки
- **Inconsistent Lock Order** — Транзакции блокируют ресурсы в разном порядке
- **Long Transactions** — Длительные транзакции удерживают блокировки, увеличивая риск deadlock
- **Missing Indexes** — Приводят к полным таблицам и блокировкам на уровне таблицы

 Шаг 08:

Миссия 8

Расположите шаги работы разработчика для предотвращения deadlocks:

Подсказка: Сначала планирование, потом порядок обновлений, блокировки, проверка.

Ответ:

- Определить ресурсы для обновления
- Обновлять ресурсы в фиксированном порядке
- Использовать SELECT FOR UPDATE при необходимости
- Закоммитить транзакцию

🔮 Шаг 09:

Миссия 9

Расположите действия DBA для обнаружения и устранения deadlocks:

Подсказка: Мониторинг → анализ → решение → проверка.

Ответ:

- Мониторинг блокировок через pg_locks
- Анализ циклов блокировок
- Откат конфликтующих транзакций
- Проверка устранения deadlocks

🔮 Шаг 10:

Миссия 10

📖 Сопоставьте уровни изоляции транзакций с их характеристиками

Подсказка: Подумайте, какие аномалии каждая изоляция предотвращает.

Ответ:

- **Read Uncommitted** — Может видеть незакоммиченные изменения других транзакций
- **Read Committed** — Видит только данные, закоммиченные на момент запроса
- **Repeatable Read** — Гарантирует одинаковые результаты повторного чтения в рамках транзакции
- **Serializable** — Максимальная изоляция, предотвращает все аномалии

🔮 Шаг 11:

Миссия 11

📖 Сопоставьте ситуации с правильной реакцией PostgreSQL

Подсказка: Подумайте, как СУБД реагирует на конфликтующие транзакции.

Ответ:

- **Две транзакции обновляют одну строку одновременно** — Первая блокирует вторую до коммита
- **Deadlock возникает между транзакциями** — СУБД откатывает одну транзакцию
- **Транзакция читает данные без блокировки** — Используется MVCC, видны только коммитнутые данные
- **Длинная транзакция держит блокировки** — Другие транзакции ждут или могут возникнуть deadlocks

🔮 Шаг 12:

Миссия 12

📖 Сопоставьте термины с определениями в контексте concurrency control

Подсказка: Подумайте о механизмах управления параллелизмом в PostgreSQL.

Ответ:

- **MVCC** — Многоверсионная конкуренция, позволяет читать без блокировок
- **Lock** — Механизм, предотвращающий одновременное изменение одних данных
- **Deadlock** — Ситуация, когда транзакции взаимно блокируют друг друга
- **Phantom Read** — Явление, когда повторный запрос видит новые строки

 Шаг 13:

Миссия 13

Расположите шаги устранения deadlocks при параллельных обновлениях:

Подсказка: Сначала анализируем, потом действуем, проверяем результат.

Ответ:

- Выявить транзакции в цикле блокировок
- Откатить одну из транзакций
- Проверить журнал и pg_locks
- Перезапустить откатившуюся транзакцию

 Шаг 14:

Миссия 14

Расположите действия разработчика для безопасной работы с конкурентными транзакциями:

Подсказка: Сначала планируем, потом читаем, затем обновляем, коммитим.

Ответ:

- Определить порядок операций
- Использовать SELECT FOR UPDATE при чтении
- Выполнить обновление данных
- Закоммитить транзакцию

 Шаг 15:

Миссия 15

Расположите шаги DBA для профилактики deadlocks в производственной базе:

Подсказка: Мониторинг → анализ → оптимизация → контроль.

Ответ:

- Мониторинг блокировок с помощью pg_locks
- Анализ проблемных транзакций
- Оптимизация порядка обновлений и индексов
- Контроль и тестирование новых сценариев

 Шаг 16:

Миссия 16

Расположите действия команды разработчиков для минимизации риска deadlocks в релизе:

Подсказка: Сначала анализируем код, потом тестируем, затем выкатываем на прод.

Ответ:

- Анализ блокировок и порядка транзакций в коде
- Добавление тестов на параллельные сценарии
- Внедрение порядка обновлений и использования блокировок

- Выкатка на production и мониторинг

🔧 Шаг 17:

Миссия 17

Расположите шаги расследования deadlock после инцидента на проде:

Подсказка: Сначала фиксируем, потом анализируем, затем устраняем, проверяем.

Ответ:

- Собрать логи и pg_locks информацию
- Определить транзакции, вызвавшие deadlock
- Откатить или исправить проблемные транзакции
- Проверить устранение и профилактику на тестовой среде

📖 Урок 3 - Диагностика проблем с блокировками

🔧 Шаг 01:

Миссия 1

Вы заметили, что транзакция вашего приложения зависла. Что из этого поможет быстро определить блокировки в PostgreSQL?

Подсказка: Сначала стоит узнать, кто держит блокировку и на каком объекте.

Ответ:

- **Выполнить запрос к pg_locks и pg_stat_activity**
- Перезапустить сервер PostgreSQL
- Проверить логи веб-сервера
- Отключить интернет на сервере

🔧 Шаг 02:

Миссия 2

Какой тип блокировки чаще всего вызывает задержки при обновлении нескольких строк в таблице?

Подсказка: Подумайте о том, какая блокировка ставится на уровне строки и таблицы.

Ответ:

- **Row-level lock (FOR UPDATE)**
- Access Exclusive lock на всей таблице
- Share Update Exclusive lock
- Advisory lock на клиенте

🔧 Шаг 03:

Миссия 3

Какой инструмент поможет визуализировать блокировки в вашей базе PostgreSQL?

Подсказка: Иногда проще увидеть блокировки в графическом виде, чем читать таблицы.

Ответ:

- **pgAdmin Dashboard**
- psql alone без запросов
- Логи операционной системы
- EXPLAIN на SELECT без блокировок

Шаг 04:

Миссия 4

 Задача: сопоставьте тип блокировки с его описанием

Типы блокировок:

Ответ:

- Access Share Lock
- Row Exclusive Lock
- Share Lock
- Access Exclusive Lock

Описание:

1. Позволяет только чтение, блокируя модификации
2. Блокирует все операции, включая чтение и запись
3. Разрешает одновременные блокировки на строках, ограничивая только запись
4. Используется для операций модификации отдельных строк

Подсказка: Подумайте, какие блокировки строго запрещают доступ к таблице, а какие позволяют совместную работу.

- **Access Share Lock** — Позволяет только чтение, блокируя модификации
- **Access Exclusive Lock** — Блокирует все операции, включая чтение и запись
- **Row Exclusive Lock** — Используется для операций модификации отдельных строк
- **Share Lock** — Разрешает одновременные блокировки на строках, ограничивая только запись

Шаг 05:

Миссия 5

 Сопоставьте термин с его определением в контексте блокировок

Термины:

Ответ:

- Deadlock
- Lock Timeout
- Blocking
- Advisory Lock

Определения:

1. Ситуация, когда транзакции ждут друг друга бесконечно
2. Транзакция не может получить блокировку в течение заданного времени
3. Ожидание другой транзакции для доступа к ресурсу
4. Произвольная блокировка, установленная пользователем для синхронизации

Подсказка: Обратите внимание на разницу между автоматическими и пользовательскими блокировками.

- **Deadlock** — Ситуация, когда транзакции ждут друг друга бесконечно
- **Lock Timeout** — Транзакция не может получить блокировку в течение заданного времени
- **Blocking** — Ожидание другой транзакции для доступа к ресурсу
- **Advisory Lock** — Произвольная блокировка, установленная пользователем для синхронизации

Шаг 06:

Миссия 6

 Сопоставьте SQL-запрос с его результатом по диагностике блокировок

Запросы:

Ответ:

- `SELECT * FROM pg_locks;`
- `SELECT * FROM pg_stat_activity;`
- `SELECT blocked_locks.pid AS blocked_pid, blocking_locks.pid AS blocking_pid FROM pg_locks blocked_locks JOIN pg_locks blocking_locks ...;`
- `SELECT * FROM pg_stat_user_tables;`

Результаты:

1. Информация о текущих блокировках и ожидающих транзакциях
2. Список всех активных подключений к базе данных
3. Соединение блокирующих и заблокированных процессов
4. Статистика по таблицам пользователя

Подсказка: Каждый запрос возвращает разную информацию, важно знать цель.

- **`SELECT * FROM pg_locks;`** — Информация о текущих блокировках и ожидающих транзакциях
- **`SELECT * FROM pg_stat_activity;`** — Список всех активных подключений к базе данных
- **`SELECT blocked_locks.pid AS blocked_pid, blocking_locks.pid AS blocking_pid FROM pg_locks blocked_locks JOIN pg_locks blocking_locks ...;`** — Соединение блокирующих и заблокированных процессов
- **`SELECT * FROM pg_stat_user_tables;`** — Статистика по таблицам пользователя

Шаг 07:

Миссия 7

 Сопоставьте действие администратора с его эффектом при блокировках

Действия:

Ответ:

- Отмена зависшей транзакции
- Увеличение `lock_timeout`
- Добавление индекса
- Переписывание запроса на меньший объем данных

Эффекты:

1. Снимает блокировку и разблокирует другие транзакции
2. Позволяет дольше ждать освобождения блокировки
3. Сокращает количество строк, блокируемых в таблице
4. Уменьшает нагрузку на блокировки при чтении

Подсказка: Разные действия направлены на разные уровни проблемы: время ожидания, структура данных, объем транзакции.

- **Отмена зависшей транзакции** — Снимает блокировку и разблокирует другие транзакции
- **Увеличение `lock_timeout`** — Позволяет дольше ждать освобождения блокировки

- **Добавление индекса** — Уменьшает нагрузку на блокировки при чтении
- **Переписывание запроса на меньший объем данных** — Сокращает количество строк, блокируемых в таблице

Шаг 08:

Миссия 8

Расположите шаги диагностики блокировок в правильном порядке:

1. Проверка активности транзакций
2. Идентификация заблокированных процессов
3. Определение блокирующих процессов
4. Принятие решения: отмена или оптимизация транзакции

Подсказка: Сначала узнаем, кто активен, затем кто ждёт, потом кто блокирует и наконец действуем.

Ответ:

- Проверка активности транзакций
- Идентификация заблокированных процессов
- Определение блокирующих процессов
- Принятие решения: отмена или оптимизация транзакции

Шаг 09:

Миссия 9

Расположите действия администратора для снятия deadlock в правильной последовательности:

1. Выявление зависших транзакций
2. Выбор транзакции для отмены
3. Отмена транзакции
4. Проверка состояния блокировок после отмены

Подсказка: Важно сначала понять проблему, потом выбрать решение и только после проверки завершить процесс.

Ответ:

- Выявление зависших транзакций
- Выбор транзакции для отмены
- Отмена транзакции
- Проверка состояния блокировок после отмены

Шаг 10:

Миссия 10

Расположите шаги оптимизации запросов для уменьшения блокировок:

1. Анализ текущих запросов на блокировки
2. Идентификация тяжелых операций UPDATE/DELETE
3. Добавление индексов и оптимизация WHERE
4. Переписывание транзакций на меньший объем данных

Подсказка: Сначала собираем данные, затем анализируем и только потом оптимизируем.

Ответ:

- Анализ текущих запросов на блокировки

- Идентификация тяжелых операций UPDATE/DELETE
- Добавление индексов и оптимизация WHERE
- Переписывание транзакций на меньший объем данных

Шаг 11:

Миссия 11

Расположите шаги анализа зависших транзакций в правильном порядке:

1. Определение транзакций с длительной блокировкой
2. Проверка, какие ресурсы они блокируют
3. Решение о прерывании или оптимизации транзакции
4. Применение изменений и повторная проверка состояния

Подсказка: Начните с наблюдения, затем анализ и только потом вмешательство.

Ответ:

- Определение транзакций с длительной блокировкой
- Проверка, какие ресурсы они блокируют
- Решение о прерывании или оптимизации транзакции
- Применение изменений и повторная проверка состояния

Шаг 12:

Миссия 12

Расположите действия при частых блокировках на уровне строк:

1. Выявление проблемных запросов
2. Анализ транзакций на конфликтующие UPDATE/DELETE
3. Добавление индексов и оптимизация WHERE
4. Разделение больших транзакций на меньшие

Подсказка: Сначала выясняем причины, затем оптимизируем и уменьшаем объем работы.

Ответ:

- Выявление проблемных запросов
- Анализ транзакций на конфликтующие UPDATE/DELETE
- Добавление индексов и оптимизация WHERE
- Разделение больших транзакций на меньшие

Шаг 13:

Миссия 13

Расположите этапы работы с deadlock в PostgreSQL:

1. Выявление зависших транзакций
2. Использование pg_locks и pg_stat_activity для диагностики
3. Отмена одной из транзакций
4. Проверка, что блокировка снята и база работает стабильно

Подсказка: Deadlock требует аккуратного анализа и последовательного вмешательства.

Ответ:

- Выявление зависших транзакций
- Использование pg_locks и pg_stat_activity для диагностики

- Отмена одной из транзакций
- Проверка, что блокировка снята и база работает стабильно

Шаг 14:

Миссия 14

Расположите шаги оптимизации параллельного выполнения транзакций:

1. Идентификация транзакций, вызывающих блокировки
2. Переписывание транзакций для меньшего объема блокировок
3. Добавление индексов и оптимизация выборки
4. Мониторинг и проверка эффективности изменений

Подсказка: Сначала находим проблемные места, потом оптимизируем и оцениваем результат.

Ответ:

- Идентификация транзакций, вызывающих блокировки
- Переписывание транзакций для меньшего объема блокировок
- Добавление индексов и оптимизация выборки
- Мониторинг и проверка эффективности изменений

Шаг 15:

Миссия 15

Расположите последовательность действий при использовании advisory locks:

1. Установка advisory lock в начале транзакции
2. Выполнение операций, требующих синхронизации
3. Освобождение advisory lock после завершения
4. Проверка, что другие транзакции получили доступ

Подсказка: Пользовательские блокировки требуют внимательного управления временем удержания.

Ответ:

- Установка advisory lock в начале транзакции
- Выполнение операций, требующих синхронизации
- Освобождение advisory lock после завершения
- Проверка, что другие транзакции получили доступ

Шаг 16:

Миссия 16

Расположите шаги анализа долгих транзакций с точки зрения нагрузки на сервер:

1. Сбор информации о длительных транзакциях через pg_stat_activity
2. Определение затронутых таблиц и строк
3. Проверка блокировок через pg_locks
4. Решение об оптимизации или отмене транзакций

Подсказка: Сначала собираем данные, затем анализируем и только после этого вмешиваемся.

Ответ:

- Сбор информации о длительных транзакциях через pg_stat_activity
- Определение затронутых таблиц и строк

- Проверка блокировок через `pg_locks`
- Решение о оптимизации или отмене транзакций

Шаг 17:

Миссия 17

Расположите шаги комплексного аудита блокировок на сервере PostgreSQL:

1. Сбор информации о всех текущих блокировках
2. Анализ зависших и часто блокируемых таблиц
3. Определение и внедрение оптимизаций (индексы, разбиение транзакций)
4. Мониторинг после внедрения изменений

Подсказка: Аудит требует циклической работы: сбор, анализ, оптимизация, контроль.

Ответ:

- Сбор информации о всех текущих блокировках
- Анализ зависших и часто блокируемых таблиц
- Определение и внедрение оптимизаций (индексы, разбиение транзакций)
- Мониторинг после внедрения изменений

Урок 4 - Advisory locks и pg_locks

Шаг 01:

Миссия 1

Разработчик Артём запускает два скрипта, которые одновременно пытаются захватить advisory lock с одним и тем же ключом. Что произойдёт?

Подсказка: Advisory lock работает похоже на эксклюзивный доступ: второй ждёт, пока первый освободит.

Ответ:

- **Второй процесс будет ждать освобождения lock первым**
- Оба процесса получат lock одновременно
- Второй процесс завершится с ошибкой
- Lock автоматически разделится между ними

Шаг 02:

Миссия 2

Сергей написал функцию, которая берёт advisory lock, но забыл его освободить. Что произойдёт после завершения транзакции?

Подсказка: Подумайте, к чему привязан lock — к сессии или транзакции.

Ответ:

- **Lock освободится автоматически**
- Lock останется навсегда
- Lock заблокирует только SELECT-запросы
- Lock перенесётся на другую транзакцию

Шаг 03:

Миссия 3

Fullstack-разработчик Игорь запускает `pg_try_advisory_lock()` в двух сессиях подряд. Вторая возвращает `false`. Что это означает?

Подсказка: Если lock занят, вторая попытка не сможет его взять.

Ответ:

- **Lock уже удерживается другой сессией**
- Функция вызвана с неправильным параметром
- Postgres не поддерживает advisory locks
- Lock автоматически освобождается через 1 секунду

Шаг 04:

Миссия 4

Программистка Катя читает `pg_locks`. Какой столбец показывает, что блокировка относится к advisory lock?

Подсказка: Внимание на колонку с типом блокировки.

Ответ:

- **locktype = advisory**
- granted = false
- relation = advisory
- mode = share

Шаг 05:

Миссия 5

 Задача: Сопоставление функций advisory lock

Формулировка: Сопоставьте функцию с её назначением.

Функции:

Ответ:

- `pg_advisory_lock()`
- `pg_try_advisory_lock()`
- `pg_advisory_unlock()`
- `pg_locks`

Назначения:

1. Устанавливает эксклюзивный advisory lock и ждёт при занятости.
2. Пробует установить advisory lock, возвращает true/false.
3. Снимает advisory lock.
4. Показывает все текущие блокировки в системе.

Подсказка:

Подумайте: у кого-то из них только чтение, у кого-то — действие с lock.

- **`pg_advisory_lock()`** — Устанавливает эксклюзивный advisory lock и ждёт при занятости.
- **`pg_try_advisory_lock()`** — Пробует установить advisory lock, возвращает true/false.
- **`pg_advisory_unlock()`** — Снимает advisory lock.
- **`pg_locks`** — Показывает все текущие блокировки в системе.

Шаг 06:

Миссия 6

 Задача: Типы advisory lock

Формулировка: Сопоставьте тип advisory lock с его характеристикой.

Ответ:

- Session-level lock
- Transaction-level lock
- Exclusive lock
- Shared lock

1. Снимается автоматически при закрытии соединения.
2. Снимается автоматически при завершении транзакции.
3. Доступ разрешён только одному процессу.
4. Доступ могут получить несколько процессов одновременно.

 **Подсказка:**

Сессия живёт дольше транзакции, а shared/exclusive — это про количество участников.

- **Session-level lock** — Снимается автоматически при закрытии соединения.
- **Transaction-level lock** — Снимается автоматически при завершении транзакции.
- **Exclusive lock** — Доступ разрешён только одному процессу.
- **Shared lock** — Доступ могут получить несколько процессов одновременно.

 **Шаг 07:**

Миссия 7

 Задача: Интерпретация колонок в pg_locks

Формулировка: Сопоставьте колонку с её назначением.

Ответ:

- pid
- locktype
- granted
- relation

1. Идентификатор процесса, который держит lock.
2. Тип блокировки (advisory, relation и др.).
3. Показывает, получен ли lock или процесс ждёт.
4. ID таблицы или объекта, к которому относится lock.

 **Подсказка:**

Подумайте: pid — процесс, relation — объект, granted — статус.

- **pid** — Идентификатор процесса, который держит lock.
- **locktype** — Тип блокировки (advisory, relation и др.).
- **granted** — Показывает, получен ли lock или процесс ждёт.
- **relation** — ID таблицы или объекта, к которому относится lock.

 **Шаг 08:**

Миссия 8

 Задача: Использование advisory lock в реальных сценариях

Формулировка: Сопоставьте сценарий с его описанием.

Ответ:

- Очередь задач
 - Миграции БД
 - Генерация отчёта
 - Крон-задача
1. Предотвращает одновременный запуск двух планировщиков.
 2. Гарантирует, что скрипт не запустится дважды параллельно.
 3. Исключает конфликт при изменении схемы базы.
 4. Не даёт двум воркерам обработать одну задачу.

 Подсказка:

Подумайте: миграции — это про изменения структуры, очередь — про уникальность задачи.

- **Очередь задач** — Не даёт двум воркерам обработать одну задачу.
- **Миграции БД** — Исключает конфликт при изменении схемы базы.
- **Генерация отчёта** — Гарантирует, что скрипт не запустится дважды параллельно.
- **Крон-задача** — Предотвращает одновременный запуск двух планировщиков.

 Шаг 09:

Миссия 9

 Задача: Сопоставьте lock с его особенностью

Ответ:

- Row lock
 - Table lock
 - Advisory lock
 - Deadlock
1. Программно управляемая блокировка, не зависящая от конкретной таблицы.
 2. Блокировка на уровне строки.
 3. Ситуация, когда два процесса ждут друг друга бесконечно.
 4. Блокировка всей таблицы.

 Подсказка:

Advisory lock — ручное управление, deadlock — взаимная блокировка.

- **Row lock** — Блокировка на уровне строки.
- **Table lock** — Блокировка всей таблицы.
- **Advisory lock** — Программно управляемая блокировка, не зависящая от конкретной таблицы.
- **Deadlock** — Ситуация, когда два процесса ждут друг друга бесконечно.

 Урок 5 - Разделение OLTP и OLAP-нагрузки

 Шаг 01:

Миссия 1

Разработчик Андрей заметил, что отчёты тормозят работу транзакций. Какое определение лучше всего описывает OLTP-нагрузку?

Подсказка: Подумайте о коротких транзакциях, работающих с небольшим числом строк.

Ответ:

- **Множество коротких транзакций для обработки повседневных операций**
- Анализ больших массивов исторических данных
- Генерация отчётов за несколько лет
- Предварительная агрегация данных

Шаг 02:

Миссия 2

Аналитик Лена строит дашборд, который обращается к миллионам строк. К какому типу нагрузки это относится?

Подсказка: Здесь важны длинные запросы и агрегированные результаты.

Ответ:

- OLTP
- **OLAP**
- Mixed workload
- In-memory caching

Шаг 03:

Миссия 3

Саша запускает отчёт во время пиковых транзакций интернет-магазина. Почему OLAP-запросы мешают OLTP?

Подсказка: Думайте о долгих сканах таблиц и блокировках ресурсов.

Ответ:

- **Они выполняют долгие операции и конкурируют за ресурсы**
- Они всегда работают быстрее транзакций
- Они автоматически запускаются в другой базе
- Они блокируют только индексы, не мешая транзакциям

Шаг 04:

Миссия 4

Разработчица Даша думает, как разгрузить основную базу. Что чаще всего используют для разделения OLTP и OLAP?

Подсказка: Это связано с копированием данных в отдельное хранилище.

Ответ:

- **Репликацию или выделенный Data Warehouse**
- Автоматическое разделение таблиц
- Отключение всех индексов
- Прямой доступ к файлам диска

Шаг 05:

Миссия 5

Тимлид Аркадий предложил использовать реплики. Какой тип реплики лучше подходит для аналитических запросов?

Подсказка: OLAP хорошо работает там, где читают, но не пишут.

Ответ:

- Primary
- **Standby-реплика (чтение)**
- Logical replication only
- Replica с синхронной записью

 **Шаг 06:**

Миссия 6

Аналитик Петя заметил, что отчёт иногда сильно тормозит. Что поможет уменьшить влияние OLAP на OLTP при работе в одной базе?

Подсказка: Индексы и материализованные представления — хорошие друзья аналитиков.

Ответ:

- **Использование индексов для аналитических запросов**
- **Материализованные представления для агрегаций**
- Отключение транзакций
- Удаление всех связей между таблицами

 **Шаг 07:**

Миссия 7

DevOps-инженер Миша планирует шардировать данные. Зачем это полезно при смешанной нагрузке?

Подсказка: Вспомните про распределение нагрузки и уменьшение конкуренции.

Ответ:

- **Чтобы разделить нагрузку и снизить конкуренцию**
- Чтобы все запросы выполнялись в одной ноде
- Чтобы отключить репликацию
- Чтобы ускорить транзакции за счёт удаления индексов

 **Шаг 08:**

Миссия 8

 **Задача:** Сопоставьте нагрузку с её характеристикой

Ответ:

- OLTP
- OLAP
- Mixed workload
- Data Warehouse

1. Часто используется для хранения исторических данных и агрегаций.
2. Короткие транзакции и высокая конкуренция за ресурсы.
3. Сочетает аналитические и транзакционные операции.
4. Аналитические запросы с большим объёмом данных.

 **Подсказка:**

Подумайте о разнице между повседневными транзакциями и долгими аналитическими запросами.

- **OLTP** — Короткие транзакции и высокая конкуренция за ресурсы.

- **OLAP** — Аналитические запросы с большим объёмом данных.
- **Mixed workload** — Сочетает аналитические и транзакционные операции.
- **Data Warehouse** — Часто используется для хранения исторических данных и агрегаций.

🔧 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте инструмент с его назначением

Ответ:

- Материализованное представление
 - Реплика
 - Индекс
 - Партиционирование
1. Ускоряет поиск и фильтрацию строк.
 2. Хранит результат запроса для быстрой выдачи.
 3. Разделяет данные на сегменты для удобства запросов.
 4. Даёт возможность выполнять аналитические запросы отдельно от транзакционных.

🔧 Подсказка:

Реплика — это про копию, индекс — про поиск, материализованное представление — про сохранённый результат.

- **Материализованное представление** — Хранит результат запроса для быстрой выдачи.
- **Реплика** — Даёт возможность выполнять аналитические запросы отдельно от транзакционных.
- **Индекс** — Ускоряет поиск и фильтрацию строк.
- **Партиционирование** — Разделяет данные на сегменты для удобства запросов.

🔧 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте проблему с её решением

Ответ:

- Тяжёлый отчёт блокирует транзакции
 - Большая таблица сканируется полностью
 - Транзакции ждут из-за аналитики
 - Отчёты работают медленно
1. Вынести запросы на реплику.
 2. Добавить индекс или партиционирование.
 3. Использовать материализованные представления.
 4. Разделить OLTP и OLAP по серверам.

🔧 Подсказка:

Вспомните: индекс помогает искать, реплика разгружает, материализация ускоряет отчёты.

- **Тяжёлый отчёт блокирует транзакции** — Разделить OLTP и OLAP по серверам.
- **Большая таблица сканируется полностью** — Добавить индекс или партиционирование.
- **Транзакции ждут из-за аналитики** — Вынести запросы на реплику.
- **Отчёты работают медленно** — Использовать материализованные представления.

🔮 Шаг 11:

Миссия 11

📖 Задача: Сопоставьте роль с её интересами в OLTP/OLAP

Ответ:

- Разработчик
- Аналитик
- DevOps
- Бизнес-менеджер

1. Интересуется производительностью транзакций.
2. Хочет получать быстрые отчёты.
3. Следит за балансом нагрузки и репликами.
4. Смотрит на бизнес-метрики и тренды.

🔮 **Подсказка:**

Разработчик заботится о коде и транзакциях, аналитик — о данных, DevOps — о системе.

- **Разработчик** — Интересуется производительностью транзакций.
- **Аналитик** — Хочет получать быстрые отчёты.
- **DevOps** — Следит за балансом нагрузки и репликами.
- **Бизнес-менеджер** — Смотрит на бизнес-метрики и тренды.

🔮 Шаг 12:

Миссия 12

📖 Задача: Сопоставьте стратегию с её преимуществом

Ответ:

- Партиционирование
- Репликация
- Шардирование
- Data Warehouse

1. Позволяет выполнять распределённые запросы на разных нодах.
2. Обеспечивает разделение нагрузки между чтением и записью.
3. Ускоряет выборки в больших таблицах.
4. Даёт отдельную систему для аналитики.

🔮 **Подсказка:**

Data Warehouse = аналитика, партиционирование = большие таблицы, репликация = копии для чтения.

- **Партиционирование** — Ускоряет выборки в больших таблицах.
- **Репликация** — Обеспечивает разделение нагрузки между чтением и записью.
- **Шардирование** — Позволяет выполнять распределённые запросы на разных нодах.
- **Data Warehouse** — Даёт отдельную систему для аналитики.

Модуль 10 - Бонус. Восстановление — путь возрождения

Урок 1 - Резервное копирование разными способами

Шаг 01:

Миссия 1

Молодой разработчик Артём впервые услышал про резервное копирование PostgreSQL. Какой самый простой способ сделать резервную копию базы?

Подсказка: Подумайте про инструмент, который создает дамп базы данных в файл.

Ответ:

- **pg_dump**
- VACUUM FULL
- ANALYZE
- REINDEX

Шаг 02:

Миссия 2

Инженер Марина забыла, как восстановить базу из дампа, сделанного с помощью pg_dump . Какая команда поможет ей?

Подсказка: Вспомните утилиту, которая работает с дампом SQL напрямую.

Ответ:

- **psql -f dump.sql mydb**
- pg_restore -f dump.sql mydb
- initdb -D mydb
- createdb -f dump.sql mydb

Шаг 03:

Миссия 3

Сисадмин Дима хочет сделать бинарную резервную копию всей кластера PostgreSQL. Какой инструмент ему нужен?

Подсказка: Этот инструмент копирует не только схему, но и файлы базы в двоичном виде.

Ответ:

- **pg_basebackup**
- pg_restore
- pg_config
- pgbench

Шаг 04:

Миссия 4

DevOps Катя настраивает регулярное резервное копирование. Что правильнее всего использовать для автоматизации?

Подсказка: Вспомните стандартный инструмент планировщика задач в Linux.

Ответ:

- **cron + pg_dump/pg_basebackup**
- REINDEX по расписанию

- ANALYZE каждый час
- initdb ежедневно

🔧 Шаг 05:

Миссия 5

Разработчик Иван случайно удалил таблицу. Он хочет восстановить только её, не затрагивая остальное. Какой подход поможет?

Подсказка: Подумайте про выборочное восстановление из дампа.

Ответ:

- **pg_restore с указанием таблицы**
- pg_basebackup и замена всего кластера
- TRUNCATE таблицы
- VACUUM FULL

🔧 Шаг 06:

Миссия 6

📖 Задача: Сопоставьте инструменты резервного копирования с их назначением.

Подсказка: Вспомните, какие утилиты работают с дампами, а какие с бинарными файлами.

Ответ:

- **pg_dump** — Создаёт SQL-дамп базы данных.
- **pg_restore** — Восстанавливает данные из дампа.
- **pg_basebackup** — Делает бинарную копию всего кластера.
- **cron** — Запускает команды резервного копирования по расписанию.

🔧 Шаг 07:

Миссия 7

📖 Ситуация: Сисадмин хочет правильно распределить типы копий по назначению. Сопоставьте тип резервной копии с её описанием.

Подсказка: Подумайте, чем отличается полный дамп от инкрементального подхода.

Ответ:

- **Полный дамп** — Сохраняет всю базу целиком.
- **Инкрементальная копия** — Сохраняет только изменения с момента последней копии.
- **Физическая копия** — Копирует бинарные файлы кластера.
- **Логическая копия** — Сохраняет структуру и данные в SQL-формате.

🔧 Шаг 08:

Миссия 8

📖 Разработчик Пётр запутался в форматах резервного копирования. Помогите ему сопоставить формат и его преимущество.

Подсказка: Форматы бывают SQL-текст и бинарные архивы.

Ответ:

- **SQL-дамп** — Можно открыть и отредактировать вручную.
- **Custom формат** — Поддерживает выборочное восстановление.
- **Tar-архив** — Подходит для архивации и переноса.
- **Directory формат** — Удобен для параллельного восстановления.

💡 Шаг 09:

Миссия 9

📖 DBA Саша думает, что каждая стратегия резервного копирования имеет типичные сценарии применения. Сопоставьте стратегию и сценарий.

Подсказка: Вспомните: инкрементальные копии экономят место, полные удобны для быстрого восстановления.

Ответ:

- **Полный бэкап раз в неделю** — Подходит для маленькой базы.
- **Инкрементальные копии каждый час** — Экономия места при большом объёме данных.
- **Ежедневный pg_dump** — Простой вариант для разработки.
- **pg_basebackup с WAL** — Для настройки репликации и восстановления после сбоя.

💡 Шаг 10:

Миссия 10

📖 Стажёр Олег путается между инструментами PostgreSQL. Помогите ему сопоставить команду с её основной задачей.

Подсказка: Обратите внимание, что некоторые команды делают копии, а другие восстанавливают.

Ответ:

- **pg_dumpall** — Создает дамп всех баз кластера.
- **createdb** — Создает пустую базу данных.
- **dropdb** — Удаляет базу данных.
- **pg_restore** — Восстанавливает базу из дампа.

💡 Шаг 11:

Миссия 11

📖 DBA Настя учит стажёра. Сопоставьте утилиту с тем, что она НЕ делает.

Подсказка: Подумайте об ограничениях каждой команды.

Ответ:

- **pg_dump** — Не копирует роли и tablespaces.
- **pg_dumpall** — Не делает бинарную копию файлов.
- **pg_basebackup** — Не позволяет выборочно восстановить одну таблицу.
- **pg_restore** — Не может работать без дампа, созданного в поддерживаемом формате.

💡 Шаг 12:

Миссия 12

📖 Разработчик Вова думает про хранение копий. Сопоставьте подход к хранению с его преимуществом.

Подсказка: Подумайте про облако, локальные диски и удалённые сервера.

Ответ:

- **Локальный диск** — Просто и быстро, но риск при сбое сервера.
- **Удалённый сервер** — Защита от поломки основного узла.
- **Облако** — Гибкость и доступ из любой точки.
- **Лента/архив** — Долгосрочное хранение для аудита.

Шаг 13:

Миссия 13

Расположите шаги восстановления базы из дампа SQL в правильном порядке.

Подсказка: Сначала нужно подготовить пустую базу, а потом загрузить туда дамп.

Ответ:

- Создать пустую базу данных
- Запустить psql с дампом
- Проверить структуру и данные

Шаг 14:

Миссия 14

Разработчик хочет настроить ежедневные бэкапы. Расположите шаги настройки cron для pg_dump в правильном порядке.

Подсказка: Вспомните, что сначала надо создать скрипт, затем прописать расписание.

Ответ:

- Написать скрипт с pg_dump
- Проверить выполнение скрипта вручную
- Добавить запись в crontab
- Проверить, что бэкап создаётся по расписанию

Шаг 15:

Миссия 15

DevOps хочет восстановить кластер из физической копии. Расположите действия в правильном порядке.

Подсказка: Важно остановить сервер перед заменой файлов.

Ответ:

- Остановить PostgreSQL
- Заменить директорию данных на копию
- Запустить PostgreSQL
- Проверить целостность базы

Шаг 16:

Миссия 16

DBA Игорь готовит стратегию Disaster Recovery. Расположите шаги действий при сбое.

Подсказка: Сначала нужно локализовать проблему, потом развернуть бэкап.

Ответ:

- Определить характер сбоя
- Выбрать актуальный бэкап
- Развернуть резервную копию
- Проверить работоспособность

Урок 2 - Восстановление до точки во времени (PITR)

Шаг 01:

Миссия 1

Артём случайно удалил важные строки и узнал про возможность восстановления до точки во времени (PITR). Что необходимо включить в настройках PostgreSQL, чтобы PITR стало возможным?

Подсказка: Это связано с сохранением WAL-журналов.

Ответ:

- **archive_mode=on** и настройка **archive_command**
- fsync=off
- shared_buffers=8GB
- synchronous_commit=off

Шаг 02:

Миссия 2

Марина хочет сделать восстановление до определённого момента. Что обязательно нужно сохранить для PITR кроме base backup?

Подсказка: Вспомните, что фиксирует каждую транзакцию после бэкапа.

Ответ:

- **WAL-файлы**
- pg_stat_activity
- pg_hba.conf
- postgresql.conf

Шаг 03:

Миссия 3

Сисадмин Дима делает base backup. Какой инструмент чаще всего используют для создания полной копии кластера для PITR?

Подсказка: Этот инструмент умеет работать с репликацией и WAL.

Ответ:

- **pg_basebackup**
- pg_restore
- initdb
- pgbench

Шаг 04:

Миссия 4

DevOps Катя готовит recovery.conf. Какой параметр позволяет указать момент времени, до которого нужно восстановить базу?

Подсказка: Этот параметр принимает timestamp или LSN.

Ответ:

- **recovery_target_time**
- restore_command
- archive_command

- `checkpoint_timeout`

🔮 Шаг 05:

Миссия 5

Иван случайно удалил таблицу и хочет восстановить базу на момент до удаления. Какое условие должно быть выполнено, чтобы PITR сработало?

Подсказка: Вспомните про актуальность файлов журналов.

Ответ:

- **WAL-журналы, охватывающие нужный момент, должны быть сохранены**
- Нужно остановить сервер до удаления
- Должен быть только дамп SQL
- Достаточно файла `postgresql.conf`

🔮 Шаг 06:

Миссия 6

📖 Сопоставьте параметр PITR с его назначением.

Подсказка: Одни параметры задают цель восстановления, другие — команды для работы с WAL.

Ответ:

- **`restore_command`** — Команда для получения WAL-файлов из архива.
- **`recovery_target_time`** — Момент времени, до которого нужно восстановить базу.
- **`recovery_target_xid`** — Идентификатор транзакции, до которой нужно восстановиться.
- **`archive_command`** — Команда для сохранения WAL-файлов в архив.

🔮 Шаг 07:

Миссия 7

📖 DBA Саша объясняет стажёру типы `recovery_target`. Сопоставьте параметр и его описание.

Подсказка: Обратите внимание: можно восстановиться по времени, LSN или XID.

Ответ:

- **`recovery_target_time`** — Остановка восстановления в заданный момент времени.
- **`recovery_target_lsn`** — Остановка на конкретном номере журнала WAL.
- **`recovery_target_xid`** — Остановка на определённой транзакции.
- **`recovery_target_name`** — Использует именованную точку восстановления.

🔮 Шаг 08:

Миссия 8

📖 Разработчик Пётр сравнивает базовый бэкап и PITR. Сопоставьте понятие и его суть.

Подсказка: PITR всегда требует `base backup` и WAL.

Ответ:

- **Base backup** — Полная физическая копия кластера.
- **WAL-файлы** — Журналы транзакций после `base backup`.
- **PITR** — Восстановление кластера до определённого момента.
- **`pg_basebackup`** — Инструмент для создания полной копии базы.

🔮 Шаг 09:

Миссия 9

📖 DevOps Катя разбирает workflow восстановления. Сопоставьте шаг с его описанием.

Подсказка: Сначала нужен base backup, потом WAL, затем настройка recovery.

Ответ:

- **Получение base backup** — Создание полной копии кластера.
- **Архивация WAL** — Сохранение журналов транзакций.
- **Настройка recovery.conf** — Определение цели восстановления.
- **Применение WAL** — Доведение базы до нужного момента.

🔮 Шаг 10:

Миссия 10

📖 Настя обсуждает с коллегами сценарии PITR. Сопоставьте ситуацию и действие.

Подсказка: PITR полезен при ошибках пользователей и логических сбоях.

Ответ:

- **Удалили таблицу** — Восстановить до момента перед удалением.
- **Сломанная транзакция** — Остановить восстановление на нужном XID.
- **Известное время сбоя** — Указать recovery_target_time.
- **Маркированная точка** — Использовать recovery_target_name.

🔮 Шаг 11:

Миссия 11

📖 Олег перепутал recovery.conf и postgresql.conf. Сопоставьте файл и его назначение.

Подсказка: Один файл для конфигурации сервера, другой — для восстановления.

Ответ:

- **postgresql.conf** — Основные параметры работы сервера.
- **pg_hba.conf** — Настройки доступа пользователей.
- **recovery.conf** — Задаёт цель восстановления и команды.
- **pg_ident.conf** — Сопоставление системных и PostgreSQL-пользователей.

🔮 Шаг 12:

Миссия 12

📖 Стажёр Вова учит разницу между логическим и физическим восстановлением. Сопоставьте метод и его характеристику.

Подсказка: PITR всегда физический процесс.

Ответ:

- **PITR** — Физическое восстановление с использованием WAL.
- **pg_restore** — Логическое восстановление из дампа.
- **pg_dump** — Создание логической копии базы.
- **pg_basebackup** — Физическая копия кластера для PITR.

🔮 Шаг 13:

Миссия 13

Расположите шаги восстановления базы до точки во времени (PITR) в правильном порядке.

Подсказка: Сначала нужен base backup, потом WAL, затем настройки восстановления.

Ответ:

- Скопировать base backup
- Подготовить WAL-файлы
- Создать recovery.conf
- Запустить PostgreSQL

🔮 Шаг 14:

Миссия 14

DevOps хочет восстановить базу по XID. Расположите шаги в правильном порядке.

Подсказка: Recovery target всегда указывают в recovery.conf.

Ответ:

- Создать recovery.conf
- Указать recovery_target_xid
- Запустить PostgreSQL
- Проверить состояние базы

🔮 Шаг 15:

Миссия 15

DBA Игорь тестирует PITR. Расположите действия при настройке archive_command.

Подсказка: Важно сначала выбрать каталог, затем прописать команду.

Ответ:

- Выбрать каталог для хранения WAL
- Прописать archive_command в postgresql.conf
- Перезапустить PostgreSQL
- Проверить, что WAL архивируются

🔮 Шаг 16:

Миссия 16

Настя восстанавливает базу к точке по времени. Расположите шаги в правильном порядке.

Подсказка: Обратите внимание на recovery_target_time.

Ответ:

- Скопировать base backup
- Поместить WAL в pg_wal
- Указать recovery_target_time в recovery.conf
- Запустить PostgreSQL

🔮 Шаг 17:

Миссия 17

Компания тестирует disaster recovery с использованием PITR. Расположите шаги в правильной последовательности.

Подсказка: После восстановления убедитесь, что данные целы.

Ответ:

- Восстановить base backup

- Применить WAL-журналы
- Задать `recovery_target`
- Проверить работоспособность

Урок 3 - Настройка архивации WAL

Шаг 01:

Миссия 1

Разработчик Маша впервые настраивает PostgreSQL для архивации WAL. Она открыла файл `postgresql.conf`. Какой параметр нужно включить в первую очередь?

Подсказка: Этот параметр отвечает за то, чтобы сервер вообще начал думать об архивации.

Ответ:

- **`archive_mode = on`**
- `wal_buffers = 16MB`
- `fsync = on`
- `checkpoint_timeout = 5min`

Шаг 02:

Миссия 2

Андрей хочет указать команду для копирования WAL-файлов в архив. Какой параметр он должен использовать?

Подсказка: Здесь важно указать путь или скрипт, куда будут уходить файлы WAL.

Ответ:

- **`archive_command`**
- `shared_buffers`
- `wal_level`
- `work_mem`

Шаг 03:

Миссия 3

Сисадмин Дима написал в конфиге `archive_command = 'cp %p /backup/%f'`. Что означают `%p` и `%f`?

Подсказка: Эти переменные всегда указывают на исходный путь и имя конечного файла.

Ответ:

- **`%p`** — полный путь к исходному WAL-файлу
- **`%f`** — имя файла для архивации
- **`%p`** — PID процесса
- **`%f`** — дата архивации

Шаг 04:

Миссия 4

Инженер Оля хочет проверить, включена ли архивация. Какой способ проверки правильный?

Подсказка: Для уверенности лучше смотреть системные параметры PostgreSQL.

Ответ:

- **`SELECT name, setting FROM pg_settings WHERE name='archive_mode';`**

- Проверить таблицу `pg_stat_activity`
- Посмотреть список пользователей
- Выполнить `VACUUM FULL`

🔧 Шаг 05:

Миссия 5

Сопоставьте параметры конфигурации архивации WAL с их назначением.

Подсказка: Внимательно вспомни, какой параметр включает режим, а какой задаёт команду.

Ответ:

- **archive_mode** — Включает или отключает режим архивации WAL
- **archive_command** — Определяет команду для сохранения WAL-файлов
- **archive_timeout** — Максимальное время между архивацией даже без активности
- **wal_level** — Задаёт уровень логирования, необходимый для архивации

🔧 Шаг 06:

Миссия 6

Сисадмин Коля проверяет работу архивации. Сопоставьте команду с её результатом.

Подсказка: Думай о том, что выводят системные представления PostgreSQL.

Ответ:

- **pg_stat_archiver** — Содержит статистику выполнения архивации
- **pg_stat_activity** — Показывает активные процессы и запросы
- **pg_ls_waldir()** — Возвращает список текущих WAL-файлов в каталоге
- **pg_switch_wal()** — Принудительно создаёт новый WAL-сегмент

🔧 Шаг 07:

Миссия 7

Аня столкнулась с проблемой: архивация включена, но файлы не появляются в каталоге.

Сопоставьте возможную причину и объяснение.

Подсказка: Здесь важно понять, что не только параметр, но и команда или права доступа могут быть причиной.

Ответ:

- **archive_command неверен** — Команда возвращает ошибку при копировании
- **Недостаточно прав доступа** — Postgres не может записать файлы в указанный каталог
- **wal_level слишком низкий** — Не создаётся достаточно информации для архивации
- **Архивация выключена** — `archive_mode=off`

🔧 Шаг 08:

Миссия 8

Свяжите тип ошибки архивации с её вероятной причиной.

Подсказка: Обрати внимание на сообщения в логах PostgreSQL.

Ответ:

- **Permission denied** — Нет прав на запись в папку архива
- **No such file or directory** — Указанный путь не существует
- **Command exited with non-zero status** — Скрипт архивации завершился с ошибкой

- **Archive command failed too many times** — Многократный сбой привёл к остановке архивации

🔧 Шаг 09:

Миссия 9

Разработчик хочет реализовать кастомный скрипт для архивации. Сопоставьте инструмент и его назначение.

Подсказка: Часто для скриптов используют стандартные средства Linux.

Ответ:

- **scp** — Копирование WAL-файлов на удалённый сервер
- **rsync** — Эффективная передача WAL с проверкой дельт
- **gzip** — Сжатие WAL перед копированием
- **tar** — Архивация нескольких файлов в один архив

🔧 Шаг 10:

Миссия 10

Сопоставьте параметр PostgreSQL с контекстом его применения при архивации.

Подсказка: Вспомни, что влияет на производительность и надёжность.

Ответ:

- **max_wal_senders** — Количество процессов для репликации и архивации
- **wal_compression** — Сжимает данные внутри WAL
- **archive_timeout** — Гарантирует периодическую архивацию
- **checkpoint_timeout** — Регулирует частоту контрольных точек

🔧 Шаг 11:

Миссия 11

Расставьте шаги включения архивации WAL в правильном порядке.

Подсказка: Логично сначала изменить конфиг, затем перезапустить сервер и проверить работу.

Ответ:

- Включить `archive_mode` в `postgresql.conf`
- Указать `archive_command`
- Перезапустить PostgreSQL
- Проверить работу архивации через `pg_stat_archiver`

🔧 Шаг 12:

Миссия 12

Расположите шаги проверки архивации WAL после настройки.

Подсказка: Проверьте статистику, создайте новый WAL и убедитесь в его копировании.

Ответ:

- Посмотреть `pg_stat_archiver`
- Выполнить `SELECT pg_switch_wal()`
- Убедиться в появлении файла в архиве
- Проверить логи PostgreSQL

Шаг 13:

Миссия 13

Упорядочьте действия для восстановления из архива WAL.

Подсказка: Нужно сначала развернуть бэкап, затем подать архивные WAL в нужном порядке.

Ответ:

- Развернуть базовый бэкап
- Указать `restore_command`
- Запустить сервер PostgreSQL
- Применить WAL-файлы до нужного момента

Шаг 14:

Миссия 14

Определи порядок действий при диагностике ошибки архивации.

Подсказка: Начиная с логов, затем переходи к проверке команды и прав.

Ответ:

- Проверить журналы PostgreSQL
- Проверить корректность `archive_command`
- Проверить права на каталог
- Тестировать скрипт вручную

Шаг 15:

Миссия 15

Расположите шаги настройки архивации WAL с использованием `rsync` на удалённый сервер.

Подсказка: Не забудь про `ssh`-ключи и корректный путь назначения.

Ответ:

- Сгенерировать `ssh`-ключ и скопировать на удалённый сервер
- Прописать `archive_command` с `rsync`
- Перезапустить PostgreSQL
- Создать тестовый WAL-файл

Шаг 16:

Миссия 16

Упорядочьте шаги, чтобы настроить автоматическую очистку старых WAL-файлов в архиве.

Подсказка: Это чаще решается внешними скриптами и планировщиками задач.

Ответ:

- Написать `shell`-скрипт для удаления старых файлов
- Протестировать скрипт вручную
- Добавить скрипт в `cron`
- Проверить, что архив корректно очищается

Урок 4 - Горячие бэкапы и offsite-хранилища

Шаг 01:

Миссия 1

Алексей решил настроить горячие бэкапы для базы, не останавливая сервис. Что в PostgreSQL позволяет это сделать?

Подсказка: Подумайте о механизме, который копирует данные во время работы сервера.

Ответ:

- **pg_basebackup**
- VACUUM FULL
- pg_ctl stop
- REINDEX DATABASE

Шаг 02:

Миссия 2

Ольга хранит бэкапы на удалённом сервере. Какой параметр PostgreSQL помогает отправлять WAL-файлы в offsite-хранилище?

Подсказка: Это настройка в postgresql.conf, связанная с копированием.

Ответ:

- **archive_command**
- shared_buffers
- work_mem
- max_connections

Шаг 03:

Миссия 3

Команда DevOps решила проверять доступность offsite-хранилища. Что будет, если archive_command вернёт ненулевой код?

Подсказка: Вспомните, как PostgreSQL реагирует на неудачное копирование WAL.

Ответ:

- **PostgreSQL будет повторять команду до успешного выполнения**
- PostgreSQL удалит WAL-файл
- PostgreSQL остановит сервер
- PostgreSQL проигнорирует ошибку

Шаг 04:

Миссия 4

Иван задумался: можно ли делать горячий бэкап с помощью snapshot на уровне файловой системы?

Подсказка: Подумайте, какие ФС позволяют быстро копировать блоки данных без остановки PostgreSQL.

Ответ:

- **Да, например, с использованием LVM snapshot или ZFS snapshot**
- Нет, PostgreSQL это запрещает
- Да, но только с VACUUM FULL
- Можно только при выключенном сервере

Шаг 05:

Миссия 5

Сергей хочет убедиться, что его архивирование WAL включено. Какой параметр необходимо выставить в `postgresql.conf`?

Подсказка: Архивация не будет работать без включения этого флага.

Ответ:

- **archive_mode = on**
- `wal_level = minimal`
- `hot_standby = off`
- `fsync = off`

Шаг 06:

Миссия 6

 Задача: Сопоставьте команды с их назначением при работе с горячими бэкапами.

Команды:

Ответ:

- `pg_start_backup()`
- `pg_stop_backup()`
- `archive_command`
- `restore_command`

Назначение:

1. Завершает процесс горячего бэкапа
2. Запускает процесс горячего бэкапа
3. Отвечает за копирование WAL-файлов в архив
4. Отвечает за извлечение WAL-файлов из архива при восстановлении

 **Подсказка:**

Вспомните: сначала запускается бэкап, затем завершается. Две другие команды относятся к копированию и восстановлению WAL.

- **`pg_start_backup()`** — Запускает процесс горячего бэкапа
- **`pg_stop_backup()`** — Завершает процесс горячего бэкапа
- **`archive_command`** — Отвечает за копирование WAL-файлов в архив
- **`restore_command`** — Отвечает за извлечение WAL-файлов из архива при восстановлении

Шаг 07:

Миссия 7

 Задача: Сопоставьте типы хранилищ с их характеристиками.

Хранилища:

Ответ:

- Локальный диск
- S3-хранилище
- Удалённый сервер через `rsync`
- Лента (tape)

Характеристики:

1. Высокая скорость, но отсутствие отказоустойчивости
2. Облачное, масштабируемое и географически распределённое

3. Позволяет синхронизировать данные между серверами
4. Долговечное, но медленное устройство

🔔 Подсказка:

Подумайте, что чаще используют в современном DevOps, а что больше подходит для долгосрочного архива.

- **Локальный диск** — Высокая скорость, но отсутствие отказоустойчивости
- **S3-хранилище** — Облачное, масштабируемое и географически распределённое
- **Удалённый сервер через rsync** — Позволяет синхронизировать данные между серверами
- **Лента (tape)** — Долговечное, но медленное устройство

🔔 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте проблемы и их последствия при хранении бэкапов.

Проблемы:

Ответ:

- Отсутствие offsite-копий
- Повреждение WAL-файлов
- Недостаток места в архиве
- Нет проверки целостности бэкапа

Последствия:

1. Невозможность восстановления после потери датацентра
2. Сбой при восстановлении до точки во времени
3. Остановка архивации и рост размеров pg_wal
4. Обнаружение ошибки только в момент аварии

🔔 Подсказка:

Попробуйте связать каждую проблему с самым очевидным риском.

- **Отсутствие offsite-копий** — Невозможность восстановления после потери датацентра
- **Повреждение WAL-файлов** — Сбой при восстановлении до точки во времени
- **Недостаток места в архиве** — Остановка архивации и рост размеров pg_wal
- **Нет проверки целостности бэкапа** — Обнаружение ошибки только в момент аварии

🔔 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте инструменты с их ролью в процессе бэкапа.

Инструменты:

Ответ:

- pg_basebackup
- rsync
- walg
- scp

Роль:

1. Создание полного бэкапа PostgreSQL

2. Синхронизация файлов между серверами
3. Архивирование в облачные хранилища
4. Копирование файлов по SSH

🔗 **Подсказка:**

Вспомните: один инструмент встроен в PostgreSQL, другой часто используют для облаков, ещё два — классические утилиты Linux.

- **pg_basebackup** — Создание полного бэкапа PostgreSQL
- **rsync** — Синхронизация файлов между серверами
- **wal-g** — Архивирование в облачные хранилища
- **scp** — Копирование файлов по SSH

🔗 **Шаг 10:**

Миссия 10

📖 Задача: Сопоставьте команды восстановления с их задачами.

Команды:

Ответ:

- restore_command
- pg_waldump
- pg_receivewal
- pg_restore

Задачи:

1. Загрузка WAL-файлов из архива
2. Анализ содержимого WAL
3. Стриминг WAL в реальном времени
4. Восстановление дампа, созданного pg_dump

🔗 **Подсказка:**

Обратите внимание: pg_restore работает не с WAL, а с дампами.

- **restore_command** — Загрузка WAL-файлов из архива
- **pg_waldump** — Анализ содержимого WAL
- **pg_receivewal** — Стриминг WAL в реальном времени
- **pg_restore** — Восстановление дампа, созданного pg_dump

🔗 **Шаг 11:**

Миссия 11

Расположите шаги настройки горячего бэкапа с pg_basebackup в правильном порядке:

1. Запуск pg_basebackup с указанием параметров
2. Включение archive_mode и archive_command
3. Убедиться, что wal_level = replica
4. Создание каталога для бэкапа

🔗 **Подсказка:**

Сначала проверяют конфигурацию и готовят каталог, потом запускают команду бэкапа.

Ответ:

- Убедиться, что wal_level = replica

- Включение `archive_mode` и `archive_command`
- Создание каталога для бэкапа
- Запуск `pg_basebackup` с указанием параметров

🔧 Шаг 12:

Миссия 12

Расположите шаги восстановления из горячего бэкапа с использованием offsite-хранилища:

1. Скопировать данные из offsite-хранилища
2. Установить `restore_command`
3. Запустить сервер PostgreSQL
4. Подготовить каталог данных (очистка или новый `initdb`)

🔧 Подсказка:

Подумайте: сначала очищается каталог, затем копируются файлы, потом указывают `restore_command` и только в конце стартует сервер.

Ответ:

- Подготовить каталог данных (очистка или новый `initdb`)
- Скопировать данные из offsite-хранилища
- Установить `restore_command`
- Запустить сервер PostgreSQL

📖 Модуль 11 - Бонус. Микросервисы — путь разделения

📖 Урок 1 - PostgreSQL в микросервисной архитектуре

🔧 Шаг 01:

Миссия 1

Программист Аня запускает новый микросервис для логистики. Ей нужно выбрать, как микросервис будет подключаться к PostgreSQL. Какой вариант самый надёжный?

Подсказка: Подумай, что позволит распределить нагрузку и избежать единой точки отказа.

Ответ:

- **Через пул соединений с балансировкой нагрузки**
- Каждый сервис открывает собственное соединение напрямую
- Использовать один глобальный суперпользователь для всех сервисов
- Подключаться через локальный файл CSV

🔧 Шаг 02:

Миссия 2

Сергей пишет сервис оплаты. Он заметил, что транзакции блокируют друг друга. Какое решение уменьшит количество блокировок?

Подсказка: Вспомни про уровни изоляции транзакций.

Ответ:

- **Использовать уровень `READ COMMITTED`**
- Повысить уровень до `SERIALIZABLE` для всех операций
- Отключить транзакции совсем
- Всегда использовать `SELECT FOR UPDATE`

🔗 Шаг 03:

Миссия 3

Команда запускает сервис доставки писем. Они думают, как хранить адреса пользователей. Какое решение лучше для микросервисной архитектуры?

Подсказка: Учитывай принцип разделения данных.

Ответ:

- **Каждый сервис хранит только свои данные в отдельной схеме или базе**
- Все сервисы используют одну большую таблицу users
- Все данные пишутся в один общий JSON-файл
- Адреса сохраняются в кэше без базы данных

🔗 Шаг 04:

Миссия 4

Инженер Дима проектирует сервис корзины покупок. Что стоит использовать, чтобы несколько пользователей не перезаписывали данные друг друга?

Подсказка: Обрати внимание на уникальные ключи и ограничения.

Ответ:

- **Использовать уникальные индексы и внешние ключи**
- Ограничить количество пользователей до 1
- Записывать корзины только в текстовые логи
- Доверять, что пользователи не ошибутся

🔗 Шаг 05:

Миссия 5

Архитектор Катя замечает, что микросервис аналитики слишком медленно выполняет запросы. Что поможет ускорить выборки?

Подсказка: Вспомни про индексирование.

Ответ:

- **Создать индекс по часто фильтруемым полям**
- Удалить все индексы
- Хранить данные в одной строке JSON
- Увеличить длину строковых полей

🔗 Шаг 06:

Миссия 6

Программисты обсуждают, как разделить доступ к PostgreSQL для разных микросервисов. Какой подход правильный?

Подсказка: Разные сервисы = разные пользователи.

Ответ:

- **Создать отдельного пользователя БД для каждого сервиса**
- Давать всем сервисам доступ под postgres
- Использовать общий пароль на всю команду
- Хранить пароль в исходном коде

🔗 Шаг 07:

Миссия 7

Разработчик Паша подключил микросервис к PostgreSQL через ORM. Почему всё равно важно знать SQL?

Подсказка: ORM не всегда оптимален.

Ответ:

- **Чтобы оптимизировать сложные запросы и индексы вручную**
- SQL нужен только администраторам
- ORM полностью заменяет SQL
- **SQL** — это устаревший язык

🔗 Шаг 08:

Миссия 8

Инженер Ира настраивает репликацию PostgreSQL для микросервисов. Какой тип репликации чаще используется для распределения нагрузки на чтение?

Подсказка: Вспомни, что позволяет разгрузить master-сервер.

Ответ:

- **Streaming replication (реплика для чтения)**
- Синхронизация через CSV-файлы
- Каждый сервис пишет в свою базу без синхронизации
- Репликация только вручную администратором

🔗 Шаг 09:

Миссия 9

📄 Сопоставьте понятия PostgreSQL с их описанием.

Подсказка: Вспомни, что отвечает за хранение, а что за оптимизацию.

Ответ:

- **Индекс** — Структура для ускорения поиска строк
- **Схема** — Логическая группировка таблиц и объектов
- **Транзакция** — Набор операций, выполняемых как единое целое
- **Репликация** — Копирование данных на другой сервер

🔗 Шаг 10:

Миссия 10

Сопоставьте сервис и то, как он может использовать PostgreSQL.

Подсказка: Каждый сервис имеет свою специфику данных.

Ответ:

- **Сервис заказов** — Хранение истории покупок
- **Сервис оплаты** — Запись транзакций с высокой надёжностью
- **Сервис доставки** — Учёт адресов и статусов посылок
- **Сервис аналитики** — Сбор агрегированных метрик

🔗 Шаг 11:

Миссия 11

Сопоставьте проблему и возможное решение в PostgreSQL.

Подсказка: Важно связать симптом с инструментом.

Ответ:

- **Медленные запросы** — Создание индексов
- **Частые блокировки** — Использование уровня READ COMMITTED
- **Перегрузка соединениями** — Использование пула соединений
- **Потеря данных при сбое** — Настройка репликации

Шаг 12:

Миссия 12

Сопоставьте команду SQL и её назначение.

Подсказка: Подумай, какие операции изменяют, а какие только читают.

Ответ:

- **SELECT** — Выборка данных
- **INSERT** — Добавление новых строк
- **UPDATE** — Изменение существующих строк
- **DELETE** — Удаление строк

Шаг 13:

Миссия 13

Сопоставьте инструмент PostgreSQL с его описанием.

Подсказка: Обрати внимание на админские утилиты.

Ответ:

- **pg_dump** — Создание дампа базы данных
- **pg_restore** — Восстановление из дампа
- **psql** — Клиент для выполнения SQL-запросов
- **pg_stat_activity** — Мониторинг активных соединений

Шаг 14:

Миссия 14

Расставьте шаги настройки пула соединений в правильном порядке.

Подсказка: Сначала установка, потом конфигурация и тест.

Ответ:

- Установить pgBouncer
- Настроить параметры пула
- Изменить строку подключения сервисов
- Проверить работу под нагрузкой

Шаг 15:

Миссия 15

Расположите шаги настройки репликации PostgreSQL.

Подсказка: Подумай: сначала главный сервер, потом реплика, потом проверка.

Ответ:

- Настроить master-сервер

- Создать слот репликации
- Настроить реплику
- Проверить синхронизацию

🔧 Шаг 16:

Миссия 16

Упорядочьте действия при оптимизации запроса.

Подсказка: Сначала анализируем, потом исправляем.

Ответ:

- Запустить EXPLAIN для анализа
- Найти узкие места
- Создать или изменить индексы
- Повторно протестировать

📖 Урок 2 - Изоляция данных на уровне сервисов

🔧 Шаг 01:

Миссия 1

Аня проектирует сервис заказов и думает: стоит ли ему напрямую использовать таблицу клиентов из сервиса CRM? Как правильнее организовать хранение данных?

Подсказка: Вспомни про принцип изоляции данных: каждый сервис владеет только своими данными.

Ответ:

- **Сервис заказов должен хранить свои данные в отдельной базе или схеме**
- Оба сервиса используют одну общую таблицу клиентов
- Все сервисы пишут в один Excel-файл
- **Изоляция не важна, главное** — скорость разработки

🔧 Шаг 02:

Миссия 2

Сергей пишет сервис оплаты. Коллега предлагает хранить таблицу платежей прямо в базе сервиса заказов. Какой вариант лучше?

Подсказка: Подумай о разграничении ответственности между сервисами.

Ответ:

- **У сервиса оплаты должна быть собственная база или схема**
- Все данные можно хранить в базе заказов
- Лучше хранить все в одном JSON-файле
- Достаточно просто добавить новую таблицу в любую базу

🔧 Шаг 03:

Миссия 3

Катя проектирует сервис доставки. Нужно ли этому сервису иметь доступ на запись в базу сервиса оплаты?

Подсказка: Изоляция данных означает, что прямых изменений чужих данных быть не должно.

Ответ:

- **Нет, сервис доставки должен работать только со своей базой**
- Да, это ускорит процесс
- Только если администратор разрешит
- Можно, если таблица маленькая

Шаг 04:

Миссия 4

Паша заметил, что сервис аналитики напрямую читает таблицы заказов из базы сервиса заказов. Как правильно?

Подсказка: Изоляция достигается через API или репликацию, а не прямое подключение к чужим таблицам.

Ответ:

- **Аналитике лучше работать через реплики или API**
- **Прямое подключение к таблицам** — нормальная практика
- Можно скопировать данные вручную в Excel
- Аналитика вообще не нуждается в изоляции

Шаг 05:

Миссия 5

Ира настраивает микросервис авторизации. Коллега советует хранить пользователей в той же базе, что и сервис заказов. Какой подход правильный?

Подсказка: Каждая область ответственности должна быть изолирована.

Ответ:

- **Пользователи должны храниться отдельно в сервисе авторизации**
- Все сервисы могут использовать одну таблицу users
- Можно вообще не хранить пользователей
- Изоляция не нужна, если мало данных

Шаг 06:

Миссия 6

Команда обсуждает: как правильно настроить доступ к базе PostgreSQL в микросервисной архитектуре?

Подсказка: Безопасность и изоляция важнее удобства.

Ответ:

- **У каждого сервиса свой пользователь базы с ограниченными правами**
- Все сервисы работают от суперпользователя postgres
- Пароль базы хранится в коде репозитория
- Лучше не использовать пользователей вообще

Шаг 07:

Миссия 7

Сервис лояльности нужно интегрировать с сервисом заказов. Как лучше организовать обмен данными?

Подсказка: Сервисы не должны напрямую менять чужие таблицы.

Ответ:

- **Через API или асинхронную очередь событий**
- Через прямой доступ к чужим таблицам
- Через общий Excel-файл
- Через общий Google-документ

 **Шаг 08:**

Миссия 8

Разработчик замечает, что при тестах сервис аналитики блокирует работу сервиса заказов. В чём ошибка?

Подсказка: Вспомни про нарушение принципа изоляции.

Ответ:

- **Аналитика использует живую базу заказов вместо копии/реплики**
- У заказов слишком мало данных
- У аналитики слишком быстрый сервер
- Это нормально, блокировки неизбежны

 **Шаг 09:**

Миссия 9

 Сопоставьте сервис и его правильный способ изоляции данных.

Подсказка: Каждый сервис должен управлять только своими данными.

Ответ:

- **Сервис заказов** — Собственная база заказов
- **Сервис оплаты** — Отдельная база платежей
- **Сервис доставки** — Хранение адресов и статусов в своей базе
- **Сервис аналитики** — Использует реплики для чтения

 **Шаг 10:**

Миссия 10

Сопоставьте проблему и правильное решение для изоляции данных.

Подсказка: Прямая работа с чужими таблицами — частая ошибка.

Ответ:

- **Сервис аналитики напрямую читает таблицы заказов** — Использовать реплики или ETL
- **Сервис доставки пишет в таблицу оплаты** — Разделить базы и использовать API
- **Все сервисы используют одного пользователя БД** — Создать отдельные роли
- **Данные пользователей хранятся в сервисе заказов** — Выделить сервис авторизации

 **Шаг 11:**

Миссия 11

Сопоставьте инструмент PostgreSQL и его роль в изоляции данных.

Подсказка: Вспомни про роли, схемы и права.

Ответ:

- **Роли пользователей** — Ограничение прав доступа

- **Схемы** — Логическая изоляция объектов
- **Отдельные базы** — Физическая изоляция данных
- **GRANT/REVOKE** — Гибкая настройка прав

🔮 Шаг 12:

Миссия 12

Сопоставьте подход к интеграции и его характеристику.

Подсказка: Изоляция требует продуманного обмена данными.

Ответ:

- **Общий доступ к таблицам** — Нарушение изоляции
- **API-вызовы** — Безопасный обмен данными
- **Асинхронные очереди** — Слабая связанность сервисов
- **Репликация** — Чтение без нагрузки на master

🔮 Шаг 13:

Миссия 13

Сопоставьте микросервис и опасность при нарушении изоляции.

Подсказка: Подумай, чем грозит прямой доступ к чужим данным.

Ответ:

- **Сервис заказов пишет в базу доставки** — Риск потери согласованности
- **Сервис оплаты читает напрямую заказы** — Блокировка транзакций
- **Сервис аналитики подключается к прод-базе** — Перегрузка боевой системы
- **Сервис авторизации использует общую таблицу** — Уязвимость безопасности

🔮 Шаг 14:

Миссия 14

Расположите шаги при проектировании изоляции данных сервисов.

Подсказка: Сначала определяем границы, затем базы, потом доступ.

Ответ:

- Определить границы данных каждого сервиса
- Создать отдельные базы или схемы
- Назначить отдельных пользователей
- Настроить API/очереди для обмена

🔮 Шаг 15:

Миссия 15

Расположите шаги при подключении нового микросервиса к кластеру PostgreSQL.

Подсказка: Последовательность: база → пользователь → права → тест.

Ответ:

- Создать новую базу или схему
- Завести пользователя
- Выдать права
- Проверить подключение

🔧 Шаг 16:

Миссия 16

Расставьте шаги при миграции данных из монолита в микросервисы.

Подсказка: Сначала анализируем, потом переносим и проверяем.

Ответ:

- Выделить границы доменов
- Создать базы для сервисов
- Перенести данные
- Настроить взаимодействие через API

📖 Урок 3 - Двухфазный коммит и его альтернатива

🔧 Шаг 01:

Миссия 1

Андрей проектирует систему платежей. Несколько сервисов пишут в разные базы PostgreSQL, и он думает: стоит ли использовать двухфазный коммит?

Подсказка: Подумайте о согласованности данных, когда операции распределены по разным базам.

Ответ:

- **Двухфазный коммит нужен для согласованности транзакций между разными базами**
- Это используется только для ускорения индексов
- Подходит только для репликации чтения
- Используется исключительно для оптимизации кэша

🔧 Шаг 02:

Миссия 2

Ольга реализует заказ товаров в e-commerce. Один сервис пишет в PostgreSQL, другой — в Kafka. Она думает, как обеспечить атомарность.

Подсказка: Вспомните, что двухфазный коммит не всегда лучший выбор при интеграции с небазовыми системами.

Ответ:

- Использовать только двухфазный коммит в PostgreSQL
- **Рассмотреть паттерн outbox для согласованности с внешними системами**
- Отправлять данные в Kafka напрямую без гарантии транзакции
- Просто надеяться, что сбоя не будет

🔧 Шаг 03:

Миссия 3

Светлана слышала про альтернативы двухфазному коммиту и задумывается: какой из них подходит для асинхронных систем?

Подсказка: Иногда вместо блокировки транзакций выбирают компенсацию ошибок.

Ответ:

- Репликация логов
- **Паттерн Saga**

- Горизонтальное шардирование
- Только кэширование

🔗 Шаг 04:

Миссия 4

Команда разрабатывает систему бронирования билетов. Нужно гарантировать согласованность в нескольких сервисах, но не хочется держать долгие блокировки.

Подсказка: Подумайте о подходе, где ошибки обрабатываются компенсирующими действиями.

Ответ:

- **Использовать Saga с компенсирующими транзакциями**
- Настроить двухфазный коммит везде
- Отказаться от логирования ошибок
- Хранить все операции в одном сервисе без разделения

🔗 Шаг 05:

Миссия 5

📖 Сопоставьте термины с их определением в контексте двухфазного коммита.

Подсказка: Вспомните этапы prepare и commit.

Ответ:

- **Prepare phase** — Фаза, когда участники резервируют изменения
- **Commit phase** — Фаза, когда участники фиксируют изменения окончательно
- **Coordinator** — Процесс, управляющий распределенной транзакцией
- **Participant** — Узел, участвующий в транзакции

🔗 Шаг 06:

Миссия 6

📖 Сопоставьте альтернативные подходы и их особенности.

Подсказка: Saga и Outbox решают разные задачи.

Ответ:

- **Saga** — Управление последовательностью локальных транзакций с компенсациями
- **Outbox** — Шаблон для надежной публикации событий во внешние системы
- **2PC** — Механизм глобальной атомарности между участниками
- **Event sourcing** — Сохранение состояния как последовательности событий

🔗 Шаг 07:

Миссия 7

📖 Сопоставьте ситуации и подходящие решения.

Подсказка: Обратите внимание: онлайн-платежи требуют атомарности, а склад — компенсируемости.

Ответ:

- **Платеж через банк** — Лучше подходит двухфазный коммит
- **Бронирование билета с возможностью отмены** — Лучше подходит Saga
- **Публикация сообщения в Kafka** — Лучше подходит Outbox
- **Сохранение бизнес-событий для аналитики** — Лучше подходит Event sourcing

🔔 Шаг 08:

Миссия 8

📖 Сопоставьте понятие с риском.

Подсказка: Вспомните, в чем слабые стороны 2PC.

Ответ:

- **Coordinator failure** — Транзакция может зависнуть
- **Network partition** — Участники остаются в неопределенном состоянии
- **Long lock** — Задержка и блокировка ресурсов
- **Compensating transaction** — Сложность корректной реализации

🔔 Шаг 09:

Миссия 9

📖 Сопоставьте преимущества разных подходов.

Подсказка: Сравните гибкость Saga и строгость 2PC.

Ответ:

- **2PC** — Гарантия атомарности, но риски блокировок
- **Saga** — Гибкость и масштабируемость, но сложность компенсаций
- **Outbox** — Надежная интеграция с брокерами сообщений
- **Event sourcing** — Историчность данных и возможность пересборки

🔔 Шаг 10:

Миссия 10

Расположите шаги двухфазного коммита в правильном порядке:

Подсказка: Сначала запрос на подготовку, потом подтверждение фиксации.

Ответ:

- Координатор отправляет запрос Prepare
- Участники подтверждают готовность
- Координатор отправляет Commit
- Участники фиксируют изменения

🔔 Шаг 11:

Миссия 11

Расположите шаги паттерна Saga в правильном порядке:

Подсказка: Сначала выполняется локальная транзакция, затем при ошибке — компенсация.

Ответ:

- Запуск первой локальной транзакции
- Выполнение следующей локальной транзакции
- Обнаружение ошибки
- Вызов компенсирующих операций

🔔 Шаг 12:

Миссия 12

Расположите действия в шаблоне Outbox:

Подсказка: Сначала запись события в таблицу, потом публикация.

Ответ:

- Сервис пишет бизнес-событие в таблицу Outbox
- Транзакция коммитится вместе с бизнес-данными
- Процессор считывает событие из Outbox
- Событие публикуется во внешнюю систему

 **Шаг 13:**

Миссия 13

Расположите шаги обработки сбоя в двухфазном коммите:

Подсказка: Подумайте, как координируется восстановление.

Ответ:

- Выявление сбоя координатора
- Повторная проверка состояния участников
- Принятие решения commit/rollback
- Согласование финального состояния

 **Шаг 14:**

Миссия 14

Расположите шаги использования Event Sourcing:

Подсказка: События сначала записываются, потом применяются к состоянию.

Ответ:

- Фиксация события в журнал
- Публикация события для слушателей
- Применение события к агрегату
- Сборка состояния из истории

 **Шаг 15:**

Миссия 15

Расположите подходы по степени синхронности (от самого синхронного к асинхронному):

Подсказка: Вспомните: 2PC требует блокировок, а Saga работает асинхронно.

Ответ:

- 2PC
- Outbox
- Saga
- Event sourcing

Урок 4 - CQRS и event sourcing с PostgreSQL

 **Шаг 01:**

Миссия 1

Программист Артём проектирует систему заказов. Он впервые слышит про CQRS. Что является главным принципом CQRS?

Подсказка: Подумайте о разделении обязанностей между операциями чтения и записи.

Ответ:

- **Разделение моделей чтения и записи**
- Хранение истории изменений в одной таблице
- Отказ от транзакций в пользу асинхронных событий
- Использование только денормализованных данных

🔮 **Шаг 02:**

Миссия 2

Инженер Оля читает логи событий. Ей нужно объяснить коллеге, что такое Event Sourcing. Какое определение правильное?

Подсказка: Вспомните — мы сохраняем не текущее состояние, а всю историю изменений.

Ответ:

- **Подход, при котором система хранит события изменений, а состояние восстанавливается из них**
- Метод оптимизации запросов для ускорения SELECT
- Техника шардинга данных по серверам
- Механизм автоматической генерации индексов

🔮 **Шаг 03:**

Миссия 3

Разработчики спорят, зачем в CQRS часто используют отдельную базу для чтения. В чём главный плюс такого подхода?

Подсказка: Вспомните о распределении нагрузки и оптимизации разных типов запросов.

Ответ:

- **Чтение можно масштабировать независимо от записи**
- Чтение становится полностью транзакционным
- Снижается количество данных в системе
- События больше не нужны

🔮 **Шаг 04:**

Миссия 4

Программист Даша тестирует прототип на PostgreSQL. Она хочет хранить события заказов. Как лучше хранить события в базе?

Подсказка: События — это не состояние, а факты, которые произошли.

Ответ:

- **Хранить каждое событие как отдельную запись в журнале событий**
- Сохранять только текущее состояние агрегата
- Перезаписывать строку при каждом изменении
- Хранить только агрегированные данные

🔮 **Шаг 05:**

Миссия 5

Старший разработчик Пётр спрашивает: почему Event Sourcing удобно совмещать с CQRS?

Подсказка: Вспомните про независимость моделей и удобство репликации истории.

Ответ:

- **События легко реплицируются и обновляют модели чтения**
- CQRS требует полного отказа от транзакций
- Event Sourcing работает только в NoSQL
- CQRS делает события ненужными

Шаг 06:

Миссия 6

Разработчик Иван сталкивается с проблемой: при реконструкции состояния из событий это занимает много времени. Какой приём помогает ускорить процесс?

Подсказка: Подумайте о сохранении контрольных точек, чтобы не проигрывать всю историю.

Ответ:

- **Использовать снимки для сохранения промежуточного состояния**
- Хранить события в нескольких базах
- Удалять старые события
- Дублировать все события в таблице чтения

Шаг 07:

Миссия 7

Команда хочет использовать PostgreSQL для CQRS. Какие инструменты помогут организовать асинхронную репликацию событий?

Подсказка: Подумайте о механизмах уведомлений и очередей внутри PostgreSQL.

Ответ:

- **LISTEN/NOTIFY**
- **Logical Replication**
- VACUUM FULL
- pg_dump

Шаг 08:

Миссия 8

 Задача: сопоставьте термины CQRS с их объяснениями.

Подсказка: Вспомните, что одна модель отвечает за записи, а другая за чтения.

Ответ:

- **Команда** — Операция, изменяющая состояние системы.
- **Запрос** — Операция, возвращающая данные без изменения состояния.
- **Модель записи** — Часть системы, обрабатывающая команды.
- **Модель чтения** — Часть системы, оптимизированная для запросов.

Шаг 09:

Миссия 9

 Задача: сопоставьте особенности Event Sourcing с их описанием.

Подсказка: Подумайте о хранении истории и её использовании.

Ответ:

- **Событие** — Факт, который произошёл в системе.

- **Журнал событий** — Основное хранилище фактов изменений.
- **Снапшот** — Сохранённое состояние для ускорения восстановления.
- **Проекция** — Специализированное представление данных для чтения.

🔗 Шаг 10:

Миссия 10

📖 Задача: сопоставьте инструменты PostgreSQL с их ролью в CQRS и Event Sourcing.

Подсказка: Вспомните, какие фичи PostgreSQL помогают строить асинхронные системы.

Ответ:

- **LISTEN/NOTIFY** — Механизм публикации и подписки внутри базы.
- **Logical Replication** — Средство передачи изменений в другие базы.
- **JSONB** — Формат хранения событий в гибкой структуре.
- **Materialized View** — Инструмент для построения проекций данных.

🔗 Шаг 11:

Миссия 11

📖 Задача: сопоставьте проблему и решение при работе с Event Sourcing.

Подсказка: Каждое решение отвечает на конкретный вызов.

Ответ:

- **Медленное восстановление состояния** — Использование снапшотов.
- **Рост объёма событий** — Архивация старых данных.
- **Сложность запросов для аналитики** — Построение проекций.
- **Согласованность между моделями** — Асинхронное обновление проекций.

🔗 Шаг 12:

Миссия 12

📖 Задача: сопоставьте тип команды с её примером в e-commerce.

Подсказка: Подумайте, какие действия меняют состояние заказа.

Ответ:

- **Создание заказа** — Команда: CreateOrder
- **Добавление товара** — Команда: AddItem
- **Оплата заказа** — Команда: PayOrder
- **Отмена заказа** — Команда: CancelOrder

🔗 Шаг 13:

Миссия 13

Расположите шаги обработки команды в CQRS в правильном порядке.

Подсказка: Сначала поступает команда, потом события, затем обновления моделей чтения.

Ответ:

- Команда поступает в модель записи
- Генерация событий
- Обновление проекций для чтения
- Запрос к модели чтения возвращает данные

🔗 Шаг 14:

Миссия 14

Расставьте шаги восстановления состояния агрегата при Event Sourcing.

Подсказка: Подумайте о последовательности: загрузка событий → применение → получение состояния.

Ответ:

- Загрузка всех событий агрегата
- Применение событий к объекту
- Формирование актуального состояния
- Возврат готового агрегата

🔗 Шаг 15:

Миссия 15

Расставьте шаги создания проекции в CQRS.

Подсказка: Сначала события генерируются, потом применяются к проекции, и лишь затем запросы читают данные.

Ответ:

- Генерация событий
- Применение событий к таблице проекции
- Обновление данных в проекции
- Чтение проекции через запросы

🔗 Шаг 16:

Миссия 16

Расставьте шаги при работе со снимком в Event Sourcing.

Подсказка: Восстановление начинается не с нуля, а с сохранённого состояния.

Ответ:

- Загрузка последнего снимка
- Загрузка событий после снимка
- Применение событий к состоянию
- Получение актуального состояния

📖 Урок 5 - Использование LISTENNOTIFY и очередей

🔗 Шаг 01:

Миссия 1

Аня разрабатывает сервис уведомлений для своего микросервиса. Какой подход PostgreSQL позволит её сервису получать события в реальном времени?

Подсказка: Подумайте о механизмах, которые поддерживают асинхронные уведомления между сервисами.

Ответ:

- **Использовать LISTEN/NOTIFY**
- Использовать pg_dump для выгрузки данных
- Создать индекс на каждой таблице
- Выполнять SELECT каждые 5 секунд

Шаг 02:

Миссия 2

Иван строит очередь задач на базе PostgreSQL для микросервисов. Какое преимущество LISTEN/NOTIFY перед постоянными опросами таблицы?

Подсказка: Подумайте о нагрузке на базу и задержках в обработке событий.

Ответ:

- **Снижение нагрузки и мгновенное оповещение**
- Увеличение объема таблиц
- Повышение частоты резервного копирования
- Блокировка всей таблицы на время обработки

Шаг 03:

Миссия 3

Мария настроила несколько микросервисов, подписанных на одно уведомление. Что произойдет, если несколько подписчиков одновременно получат событие?

Подсказка: Подумайте о том, как PostgreSQL рассылает уведомления.

Ответ:

- **Каждый подписчик получит уведомление независимо**
- Уведомление будет доставлено только одному подписчику
- Уведомление потеряется при конкуренции
- Все подписчики блокируются до обработки уведомления

Шаг 04:

Миссия 4

 Задача: Сопоставьте события и реакции микросервисов

Формулировка: Соотнесите событие с правильной реакцией подписчика.

События:

Ответ:

- Новый заказ создан
- Статус платежа изменен
- Товар закончился на складе
- Ошибка при обработке задачи

Реакции:

1. Отправить уведомление пользователю
2. Перезапустить задачу в очереди
3. Обновить интерфейс админки
4. Отправить email клиенту

Подсказка:

Подумайте, какие события требуют действий со стороны пользователя, а какие — внутренних процессов.

- **Новый заказ создан** — Отправить email клиенту
- **Статус платежа изменен** — Обновить интерфейс админки
- **Товар закончился на складе** — Отправить уведомление пользователю

- **Ошибка при обработке задачи** — Перезапустить задачу в очереди

🔮 Шаг 05:

Миссия 5

📖 Задача: Сопоставьте типы очередей с их особенностями

Типы очередей:

Ответ:

- FIFO
- LIFO
- Приоритетная
- Отложенная

Особенности:

1. Обработка задач по мере поступления
2. Последние задачи обрабатываются первыми
3. Задачи с высоким приоритетом обрабатываются раньше
4. Задачи выполняются через заданное время

💡 **Подсказка:**

Подумайте, как выстроена очередь по времени и приоритету.

- **FIFO** — Обработка задач по мере поступления
- **LIFO** — Последние задачи обрабатываются первыми
- **Приоритетная** — Задачи с высоким приоритетом обрабатываются раньше
- **Отложенная** — Задачи выполняются через заданное время

🔮 Шаг 06:

Миссия 6

📖 Задача: Сопоставьте команды PostgreSQL с их назначением для очередей

Команды:

Ответ:

- LISTEN
- NOTIFY
- UNLISTEN
- pg_notify

Назначение:

1. Подписка на канал уведомлений
2. Отправка уведомления на канал
3. Отмена подписки на канал
4. Отправка уведомления с произвольным payload

💡 **Подсказка:**

Подумайте о роли команд: кто слушает, кто сообщает, кто отписывается.

- **LISTEN** — Подписка на канал уведомлений
- **NOTIFY** — Отправка уведомления на канал
- **UNLISTEN** — Отмена подписки на канал
- **pg_notify** — Отправка уведомления с произвольным payload

Шаг 07:

Миссия 7

 Задача: Сопоставьте типичные ошибки работы с очередями и их последствия

Ошибки:

Ответ:

- Игнорирование подтверждений задач
- Бесконечное ожидание уведомлений
- Отправка одинакового уведомления многократно
- Блокировка таблицы очереди

Последствия:

1. Задачи теряются и не выполняются
2. Сервис зависает, не обрабатывая новые события
3. Клиенты получают повторяющиеся уведомления
4. Другие процессы не могут вставлять новые задачи

 Подсказка:

Смотрите на каждую ошибку с точки зрения производительности и надежности.

- **Игнорирование подтверждений задач** — Задачи теряются и не выполняются
- **Бесконечное ожидание уведомлений** — Сервис зависает, не обрабатывая новые события
- **Отправка одинакового уведомления многократно** — Клиенты получают повторяющиеся уведомления
- **Блокировка таблицы очереди** — Другие процессы не могут вставлять новые задачи

Шаг 08:

Миссия 8

Расположите шаги настройки очереди с LISTEN/NOTIFY в правильном порядке:

1. Создать канал уведомлений
2. Подписаться на канал с помощью LISTEN
3. Отправить событие с помощью NOTIFY
4. Обработать уведомление в подписчике

 Подсказка:

Сначала создаём канал, затем подписываемся, отправляем уведомление и обрабатываем его.

Ответ:

- Создать канал уведомлений
- Подписаться на канал с помощью LISTEN
- Отправить событие с помощью NOTIFY
- Обработать уведомление в подписчике

Шаг 09:

Миссия 9

Расположите шаги обработки отложенной задачи:

1. Поместить задачу в очередь с отложенным временем
2. Дождаться наступления времени выполнения

3. Извлечь задачу из очереди
4. Выполнить задачу и подтвердить её обработку

 Подсказка:

Подумайте, когда задача должна появиться в обработчике, и что нужно сделать после выполнения.

Ответ:

- Поместить задачу в очередь с отложенным временем
- Дождаться наступления времени выполнения
- Извлечь задачу из очереди
- Выполнить задачу и подтвердить её обработку

 Шаг 10:

Миссия 10

Расположите действия для безопасной работы с приоритетной очередью:

1. Добавить задачи с приоритетом
2. Извлечь задачу с наивысшим приоритетом
3. Обработать задачу
4. Подтвердить выполнение

 Подсказка:

Сначала вставляем задачи, затем извлекаем наиболее важные и подтверждаем выполнение.

Ответ:

- Добавить задачи с приоритетом
- Извлечь задачу с наивысшим приоритетом
- Обработать задачу
- Подтвердить выполнение

 Шаг 11:

Миссия 11

Сергей интегрирует микросервисы через PostgreSQL и хочет, чтобы каждый сервис реагировал на события независимо. Какой режим работы LISTEN/NOTIFY лучше выбрать?

Подсказка: Подумайте о независимости обработки для каждого подписчика.

Ответ:

- **Каждый сервис подписан на один канал и обрабатывает события самостоятельно**
- Все сервисы объединены в один процесс обработки
- Сервис опрашивает базу каждые 10 секунд
- Отправлять уведомления только одному сервису

 Шаг 12:

Миссия 12

Лена хочет минимизировать потерю уведомлений при сбое подписчика. Что поможет в PostgreSQL?

Подсказка: Подумайте о механизмах подтверждения или резервирования задач.

Ответ:

- **Использовать отдельную таблицу очереди и подтверждение обработки**

- Отправлять уведомления повторно без хранения
- Не использовать LISTEN/NOTIFY вообще
- Выключить подписчиков при сбое

Модуль 12 - Бонус. Мониторинг — путь наблюдения

Урок 1 - Метрики PostgreSQL под высокой нагрузкой

Шаг 01:

Миссия 1

Андрей следит за базой PostgreSQL под высокой нагрузкой. Какой метрики стоит первым делом обратить внимание для оценки текущей активности базы?

Подсказка: Подумайте о том, что отражает количество операций с базой.

Ответ:

- **Количество активных подключений (connections)**
- Размер таблиц на диске
- Скорость сети между серверами
- Версия PostgreSQL

Шаг 02:

Миссия 2

Ирина заметила, что база часто использует диск под нагрузкой. Какой показатель напрямую показывает интенсивность записи данных?

Подсказка: Подумайте о том, какие операции создают нагрузку на I/O.

Ответ:

- **Количество операций записи на диск (writes)**
- Количество SELECT-запросов
- Количество соединений с базой
- Размер кэша PostgreSQL

Шаг 03:

Миссия 3

Сергей наблюдает рост задержки запросов. Какая метрика поможет понять, перегружена ли база из-за блокировок?

Подсказка: Подумайте о том, что замедляет выполнение транзакций.

Ответ:

- **Количество блокировок (locks)**
- Объем памяти в кэше
- Количество индексов в таблицах
- Скорость CPU на сервере

Шаг 04:

Миссия 4

 **Задача:** Сопоставьте метрику PostgreSQL с её описанием

Метрики:

Ответ:

- `pg_stat_activity`
- `pg_stat_io`
- `pg_locks`
- `pg_stat_database`

Описание:

1. Отслеживание текущих подключений и выполняющихся запросов
2. Сбор информации о блокировках
3. Статистика ввода-вывода по таблицам и индексам
4. Сводная статистика по всем базам

 Подсказка:

Подумайте, какая метрика показывает активность пользователей, а какая — нагрузку на диски.

- **`pg_stat_activity`** — Отслеживание текущих подключений и выполняющихся запросов
- **`pg_stat_io`** — Статистика ввода-вывода по таблицам и индексам
- **`pg_locks`** — Сбор информации о блокировках
- **`pg_stat_database`** — Сводная статистика по всем базам

 Шаг 05:

Миссия 5

 **Задача:** Сопоставьте показатели нагрузки с их последствиями

Показатели:

Ответ:

- Высокий уровень блокировок
- Рост времени выполнения запросов
- Увеличение числа подключений
- Высокая нагрузка на диск

Последствия:

1. Замедление отклика приложения
2. Очереди транзакций и возможные deadlock
3. Неэффективное использование ресурсов сервера
4. Замедление операций INSERT/UPDATE

 Подсказка:

Свяжите показатели с реальными проблемами, которые они могут вызвать.

- **Высокий уровень блокировок** — Очереди транзакций и возможные deadlock
- **Рост времени выполнения запросов** — Замедление отклика приложения
- **Увеличение числа подключений** — Неэффективное использование ресурсов сервера
- **Высокая нагрузка на диск** — Замедление операций INSERT/UPDATE

 Шаг 06:

Миссия 6

 **Задача:** Сопоставьте метрики с типом ресурса, за который они отвечают

Метрики:

Ответ:

- CPU usage
- Buffer cache hit ratio
- Disk I/O
- Connections count

Ресурс:

1. Процессор
2. Память
3. Диск
4. Сетевое подключение/пользователи

 Подсказка:

Подумайте, какая метрика указывает на эффективность использования конкретного ресурса.

- **CPU usage** — Процессор
- **Buffer cache hit ratio** — Память
- **Disk I/O** — Диск
- **Connections count** — Сетевое подключение/пользователи

 Шаг 07:

Миссия 7

 **Задача:** Сопоставьте инструмент мониторинга PostgreSQL с его функцией

Инструменты:

Ответ:

- pg_stat_statements
- EXPLAIN
- pg_top
- auto_explain

Функции:

1. Анализ частоты и времени выполнения запросов
2. Показ текущей нагрузки на сервер PostgreSQL
3. Показывает план выполнения конкретного запроса
4. Автоматическая запись планов долгих запросов

 Подсказка:

Обратите внимание, какой инструмент нужен для анализа исторических данных, а какой — для текущей нагрузки.

- **pg_stat_statements** — Анализ частоты и времени выполнения запросов
- **EXPLAIN** — Показывает план выполнения конкретного запроса
- **pg_top** — Показ текущей нагрузки на сервер PostgreSQL
- **auto_explain** — Автоматическая запись планов долгих запросов

 Шаг 08:

Миссия 8

Расположите шаги мониторинга нагрузки PostgreSQL под высокой нагрузкой:

1. Сбор метрик CPU, памяти, диска и подключений

2. Сбор статистики по запросам и блокировкам
3. Анализ данных и выявление узких мест
4. Оптимизация запросов и конфигурации сервера

 Подсказка:

Сначала измеряем, потом анализируем и только после этого вносим улучшения.

Ответ:

- Сбор метрик CPU, памяти, диска и подключений
- Сбор статистики по запросам и блокировкам
- Анализ данных и выявление узких мест
- Оптимизация запросов и конфигурации сервера

 Шаг 09:

Миссия 9

Расположите действия DBA при обнаружении роста времени выполнения запросов:

1. Проверить текущие блокировки и активные транзакции
2. Использовать `pg_stat_statements` для выявления медленных запросов
3. Проанализировать индексы и планы выполнения
4. Внести оптимизации и протестировать изменения

 Подсказка:

Сначала выявляем проблему, потом анализируем её причины и только потом исправляем.

Ответ:

- Проверить текущие блокировки и активные транзакции
- Использовать `pg_stat_statements` для выявления медленных запросов
- Проанализировать индексы и планы выполнения
- Внести оптимизации и протестировать изменения

 Шаг 10:

Миссия 10

Расположите шаги оценки влияния нагрузки на дисковую подсистему:

1. Собрать статистику по Disk I/O
2. Определить таблицы и индексы с наибольшей активностью
3. Выявить узкие места и задержки операций
4. Оптимизировать индексы и конфигурацию хранения

 Подсказка:

Важно сначала собрать данные, затем проанализировать и только после этого оптимизировать.

Ответ:

- Собрать статистику по Disk I/O
- Определить таблицы и индексы с наибольшей активностью
- Выявить узкие места и задержки операций
- Оптимизировать индексы и конфигурацию хранения

 Шаг 11:

Миссия 11

Олег хочет отслеживать метрики для долгоживущих транзакций. Какой подход лучше всего подойдет?

Подсказка: Подумайте о метриках, которые показывают длительность работы транзакций.

Ответ:

- **Использовать `pg_stat_activity` и смотреть поле `state_duration`**
- Считать количество SELECT-запросов
- Собрать метрики CPU только
- Перезапустить базу каждые 10 минут

🔧 **Шаг 12:**

Миссия 12

Наталья хочет визуализировать метрики PostgreSQL в реальном времени. Какой подход наиболее эффективен?

Подсказка: Подумайте о внешних инструментах для сбора и отображения метрик.

Ответ:

- **Использовать Prometheus + Grafana для мониторинга метрик**
- Сохранять метрики в текстовый файл и открывать блокнотом
- Запускать EXPLAIN для каждого запроса
- Отключить все индексы для ускорения базы

📖 Урок 2 - Настройка мониторинга с Prometheus

🔧 **Шаг 01:**

Миссия 1

Вы подключаете PostgreSQL к Prometheus. Какой компонент отвечает за сбор метрик с базы?

Подсказка: Подумайте, что нужно для передачи данных в Prometheus.

Ответ:

- **Exporter (например, `postgres_exporter`)**
- Alertmanager
- Grafana
- Node.js

🔧 **Шаг 02:**

Миссия 2

Какой формат данных обычно использует Prometheus для хранения метрик?

Подсказка: Обратите внимание на временные ряды.

Ответ:

- **Time series**
- JSON
- CSV
- XML

🔧 **Шаг 03:**

Миссия 3

Ваша команда заметила высокую нагрузку на PostgreSQL. Что из перечисленного полезно включить для наблюдения?

Подсказка: Подумайте о метриках, которые помогут оценить состояние сервера.

Ответ:

- **Количество активных соединений**
- **Задержка выполнения запросов**
- Количество пользователей в GitHub
- Время отклика веб-браузера

Шаг 04:

Миссия 4

Вы настраиваете PostgreSQL exporter. Какая настройка критична для корректного сбора метрик?

Подсказка: Сосредоточьтесь на подключении к базе.

Ответ:

- **Указание правильных учетных данных PostgreSQL**
- Настройка Grafana Dashboard
- Запуск Node Exporter
- Включение SSL в веб-браузере

Шаг 05:

Миссия 5

 Сопоставьте понятия с их объяснениями.

Понятия:

Ответ:

- Prometheus
- Exporter
- Alertmanager
- Grafana

Определения:

1. Система мониторинга и хранения временных рядов
2. Компонент, собирающий метрики с источника
3. Инструмент для настройки и отправки уведомлений о событиях
4. Визуализация метрик и построение дашбордов

Подсказка: Подумайте, какая роль каждого компонента в процессе наблюдения.

- **Prometheus** — Система мониторинга и хранения временных рядов
- **Exporter** — Компонент, собирающий метрики с источника
- **Alertmanager** — Инструмент для настройки и отправки уведомлений о событиях
- **Grafana** — Визуализация метрик и построение дашбордов

Шаг 06:

Миссия 6

Сопоставьте метрики PostgreSQL с их описанием:

Ответ:

- pg_stat_activity
 - pg_stat_database
 - pg_locks
 - pg_stat_bgwriter
1. Содержит активные соединения и текущие запросы
 2. Статистика по всей базе данных
 3. Информация о блокировках
 4. Статистика фоновой записи и буферов

Подсказка: Подумайте, какие метрики относятся к соединениям, блокировкам и записи.

- **pg_stat_activity** — Содержит активные соединения и текущие запросы
- **pg_stat_database** — Статистика по всей базе данных
- **pg_locks** — Информация о блокировках
- **pg_stat_bgwriter** — Статистика фоновой записи и буферов

 Шаг 07:

Миссия 7

Сопоставьте типы метрик с примером:

Ответ:

- Counter
 - Gauge
 - Histogram
 - Summary
1. Общее число подключений к базе
 2. Текущее использование CPU
 3. Распределение времени выполнения запросов по бинам
 4. Перцентиль времени отклика SQL-запроса

Подсказка: Обратите внимание, что одни метрики накапливаются, другие показывают текущее состояние.

- **Counter** — Общее число подключений к базе
- **Gauge** — Текущее использование CPU
- **Histogram** — Распределение времени выполнения запросов по бинам
- **Summary** — Перцентиль времени отклика SQL-запроса

 Шаг 08:

Миссия 8

Сопоставьте сценарий с подходящей метрикой для мониторинга:

Ответ:

- Высокая задержка выполнения запросов
 - Частые блокировки таблиц
 - Резкое увеличение числа подключений
 - Проблемы с записью на диск
1. pg_stat_activity

2. pg_locks
3. pg_stat_database
4. pg_stat_bgwriter

Подсказка: Внимательно подумайте, какие метрики отражают проблемы с соединениями, блокировками и I/O.

- **Высокая задержка выполнения запросов** — pg_stat_activity
- **Частые блокировки таблиц** — pg_locks
- **Резкое увеличение числа подключений** — pg_stat_database
- **Проблемы с записью на диск** — pg_stat_bgwriter

Шаг 09:

Миссия 9

Расположите шаги настройки Prometheus для PostgreSQL в правильном порядке:

1. Установка PostgreSQL Exporter
2. Настройка подключения к базе
3. Добавление target в Prometheus
4. Проверка метрик и визуализация в Grafana

Подсказка: Подумайте о логике установки и проверки работы системы.

Ответ:

- Установка PostgreSQL Exporter
- Настройка подключения к базе
- Добавление target в Prometheus
- Проверка метрик и визуализация в Grafana

Шаг 10:

Миссия 10

Расположите действия при обнаружении высокой нагрузки на PostgreSQL:

1. Проверка активных запросов
2. Сбор метрик через exporter
3. Анализ блокировок и долгих транзакций
4. Настройка алертов на повторение проблемы

Подсказка: Начните с диагностики, затем переходите к мониторингу и автоматизации.

Ответ:

- Проверка активных запросов
- Сбор метрик через exporter
- Анализ блокировок и долгих транзакций
- Настройка алертов на повторение проблемы

Шаг 11:

Миссия 11

Расположите этапы создания дашборда в Grafana:

1. Добавление источника данных Prometheus
2. Создание панели для отображения метрик

3. Настройка визуализации и легенд
4. Сохранение и публикация дашборда

Подсказка: Сначала подключаем данные, затем строим визуализацию и сохраняем.

Ответ:

- Добавление источника данных Prometheus
- Создание панели для отображения метрик
- Настройка визуализации и легенд
- Сохранение и публикация дашборда

Шаг 12:

Миссия 12

Вы хотите получать уведомления о проблемах с базой. Какой инструмент Prometheus позволяет это делать?

Подсказка: Он занимается алертами, а не хранением метрик.

Ответ:

- **Alertmanager**
- Exporter
- Grafana
- Node Exporter

Шаг 13:

Миссия 13

Какой способ безопасно подключить PostgreSQL exporter к базе?

Подсказка: Думайте о безопасности и минимальных правах.

Ответ:

- **Создать отдельного пользователя с правами только для чтения метрик**
- **Использовать защищенное соединение SSL**
- Использовать root пользователя базы
- Делать метрики общедоступными в интернете

Урок 3 - Использование Grafana для визуализации

Шаг 01:

Миссия 1

Вы только что установили Grafana для мониторинга PostgreSQL под нагрузкой. Что первым делом нужно сделать?

Подсказка: Без данных в Grafana будет пусто.

Ответ:

- **Подключить источник данных (Data Source) для PostgreSQL / Prometheus**
- Сразу создать графики (Dashboards)
- Настроить алерты
- Проверить пользователи и роли

Шаг 02:

Миссия 2

Ваш коллега предлагает собрать Dashboard «Показатели соединений». Что минимум стоит отобразить?

Подсказка: Мониторьте активные соединения и лимиты.

Ответ:

- **Количество активных соединений**
- **Максимальное число соединений (max_connections)**
- Количество таблиц в базе
- Число функций в PostgreSQL

🔧 Шаг 03:

Миссия 3

Вы хотите построить график задержки запросов (query latency). Что на оси Y логично отложить?

Подсказка: Время измеряется в миллисекундах или секундах.

Ответ:

- **Среднее время ответа (ms или s)**
- Количество запросов
- Общий объём данных
- Число ошибок

🔧 Шаг 04:

Миссия 4

Dashboard уже начал наполняться метриками. Что стоит сделать, чтобы в будущем была история использования?

Подсказка: Grafana сама не хранит долго метрики — требуется external store.

Ответ:

- **Настроить retention и long-term storage (Thanos, Cortex и т.д.)**
- Ставить snapshot дашбордов
- Периодически вручную экспортировать данные
- Удалять старые панели

🔧 Шаг 05:

Миссия 5

Сопоставьте элементы Grafana с их ролями:

Элементы:

Ответ:

- Data Source
- Dashboard
- Panel
- Alerting Rule

Назначения:

1. Источник метрик, к которому Grafana обращается
2. Набор панелей для визуализации разных метрик
3. Единичный график / гистограмма / таблица в Dashboard
4. Условие, которое генерирует сигнал при превышении порога

Подсказка: Подумайте, что служит “фундаментом”, что — визуализацией, что — оповещением.

- **Data Source** — Источник метрик, к которому Grafana обращается
- **Dashboard** — Набор панелей для визуализации разных метрик
- **Panel** — Единичный график / гистограмма / таблица в Dashboard
- **Alerting Rule** — Условие, которое генерирует сигнал при превышении порога

🔧 Шаг 06:

Миссия 6

Сопоставьте метрику с её смыслом (для PostgreSQL):

Метрика:

Ответ:

- pg_stat_activity_count
- pg_stat_database_xact_commit
- pg_stat_database_deadlocks
- pg_stat_database_temp_bytes

Описание:

1. Количество активных соединений
2. Число коммитов транзакций
3. Число взаимных блокировок (deadlocks)
4. Объём временных файлов на диск

Подсказка: Используйте префикс и контекст названия метрики.

- **pg_stat_activity_count** — Количество активных соединений
- **pg_stat_database_xact_commit** — Число коммитов транзакций
- **pg_stat_database_deadlocks** — Число взаимных блокировок (deadlocks)
- **pg_stat_database_temp_bytes** — Объём временных файлов на диск

🔧 Шаг 07:

Миссия 7

Сопоставьте тип визуализации Grafana с задачей:

Тип визуализации:

Ответ:

- Graph (time-series)
- Bar Gauge / Stat
- Table
- Heatmap

Задача:

1. Показать изменение метрики во времени
2. Показать текущее значение и статус порога
3. Показать список баз с их показателями
4. Показать распределение латенций по диапазонам

Подсказка: Подумайте, что удобнее для временных рядов, что — для списка, что — для распределения.

- **Graph (time-series)** — Показать изменение метрики во времени

- **Bar Gauge / Stat** — Показать текущее значение и статус порога
- **Table** — Показать список баз с их показателями
- **Heatmap** — Показать распределение латенций по диапазонам

🔮 Шаг 08:

Миссия 8

Расположите шаги от простого к наиболее комплексному при настройке дашбордов для PostgreSQL:

1. Создать отдельную панель для каждой метрики
2. Объединить панели в дашборд по смыслу (connections, latency, locks)
3. Добавить фильтры (templating) по базе или схеме
4. Настроить alert'ы и пороговые значения

Подсказка:

Начинаем с базового, затем группируем, потом делаем фильтрацию и завершаем настройками уведомлений.

Ответ:

- Создать отдельную панель для каждой метрики
- Объединить панели в дашборд по смыслу (connections, latency, locks)
- Добавить фильтры (templating) по базе или схеме
- Настроить alert'ы и пороговые значения

🔮 Шаг 09:

Миссия 9

Расположите шаги от менее ресурсоёмкого до более при построении запросов к метрикам:

1. `SELECT count(*) FROM metrics`
2. `SELECT quantile_over_time(...)` (гистограмма)
3. `SELECT rate(...) / deriv()`
4. `SELECT histogram_quantile(percentile, bucket)`

Подсказка:

Начинайте с простого агрегата, затем используйте функции над гистограммами, потом сложные трансформации.

Ответ:

- `SELECT count(*) FROM metrics`
- `SELECT quantile_over_time(...)` (гистограмма)
- `SELECT rate(...) / deriv()`
- `SELECT histogram_quantile(percentile, bucket)`

🔮 Шаг 10:

Миссия 10

Расположите шаги по приоритизации метрик на дашборде:

1. Добавить далеко неважные метрики
2. Добавить основные (connections, latency, errors)
3. Добавить вторичные (locks, temp files, IO)
4. Оптимизировать дашборд, удалив редко используемые панели

Подсказка:

Собираем сначала ключевые, потом дополняем, затем чистим.

Ответ:

- Добавить далеко неважные метрики
- Добавить основные (connections, latency, errors)
- Добавить вторичные (locks, temp files, IO)
- Оптимизировать дашборд, удалив редко используемые панели

 Шаг 11:**Миссия 11**

Расположите шаги при отладке некорректного отображения метрики:

1. Проверить, что данные доходят до Prometheus
2. Проверить, что Data Source в Grafana правильно сконфигурирован
3. Проверить запрос в панели (query) на ошибки
4. Проверить временные диапазоны и фильтры панели

Подсказка:

Сначала проверяем источник данных, затем шаги внутренние.

Ответ:

- Проверить, что данные доходят до Prometheus
- Проверить, что Data Source в Grafana правильно сконфигурирован
- Проверить запрос в панели (query) на ошибки
- Проверить временные диапазоны и фильтры панели

 Шаг 12:**Миссия 12**

Расположите шаги миграции дашбордов между инстансами Grafana:

1. Экспортировать JSON-дэфиницию дашборда
2. Подготовить подходящие Data Source на целевом инстансе
3. Импортировать JSON на целевой инстанс
4. Проверить, что все панели корректно отображаются

Подсказка:

Нельзя просто импортировать — сначала нужно обеспечить совместимость источников.

Ответ:

- Экспортировать JSON-дэфиницию дашборда
- Подготовить подходящие Data Source на целевом инстансе
- Импортировать JSON на целевой инстанс
- Проверить, что все панели корректно отображаются

 Шаг 13:**Миссия 13**

После недели работы вы видите, что Dashboard «latency» иногда падает в ноль. В чём может быть причина?

Подсказка: Пустые значения часто связаны с отсутствием данных в диапазоне.

Ответ:

- Период выборки слишком велик, и нет точек для этого интервала
- Prometheus не собирал метрики в этот момент
- Grafana сбойнула визуализацию
- PostgreSQL отключился навсегда

 **Шаг 14:**

Миссия 14

Расположите шаги для построения полного рабочего цикла мониторинга PostgreSQL через Grafana:

1. Собирать метрики (экспортёр / agent)
2. Хранить метрики (Prometheus / long-term store)
3. Подключить Grafana к хранилищу как Data Source
4. Создать Dashboard и панели
5. Настроить алерты и уведомления
6. Проверять, что весь цикл работает, и реагировать

Подсказка:

Мониторинг — это цикл от сбора до реакции.

Ответ:

- Собирать метрики (экспортёр / agent)
- Хранить метрики (Prometheus / long-term store)
- Подключить Grafana к хранилищу как Data Source
- Создать Dashboard и панели
- Настроить алерты и уведомления
- Проверять, что весь цикл работает, и реагировать

Урок 4 - Алерты при сбоях и задержках

 **Шаг 01:**

Миссия 1

Ты мониторишь базу данных PostgreSQL, и заметил, что она начала тормозить. Что первым делом ты должен сделать?

Подсказка: Подумай, что будет первым индикатором проблемы.

Ответ:

- Проверить метрики производительности сервера и базы данных
- Перезапустить сервер базы данных
- Очистить кэш приложения
- Попробовать выполнить запросы вручную в консоли

 **Шаг 02:**

Миссия 2

 **Задача:** Сопоставьте возможные проблемы с их возможными решениями

Проблемы:

Ответ:

- Высокая нагрузка на базу данных

- Задержки в отклике запросов
- Проблемы с индексацией
- Низкая скорость записи данных

Решения:

1. Использование более мощного оборудования для сервера
2. Оптимизация запросов и проверка планов выполнения
3. Пересмотр стратегии индексации и добавление новых индексов
4. Использование оптимизированных методов записи данных, например, через bulk insert

🔔 Подсказка:

Иногда решение проблемы заключается в поиске её корня. Попробуй подумать, какая проблема может вызвать другую.

- **Высокая нагрузка на базу данных** — Использование более мощного оборудования для сервера
- **Задержки в отклике запросов** — Оптимизация запросов и проверка планов выполнения
- **Проблемы с индексацией** — Пересмотр стратегии индексации и добавление новых индексов
- **Низкая скорость записи данных** — Использование оптимизированных методов записи данных, например, через bulk insert

🔔 Шаг 03:

Миссия 3

Ты настроил мониторинг для PostgreSQL, но вдруг один из алертов сработал. Что делать?

Подсказка: Первый шаг — проверить, что именно вызвало срабатывание алерта.

Ответ:

- **Проверить журнал ошибок и логи базы данных**
- Отправить запрос в техническую поддержку
- Незамедлительно перезапустить базу данных
- Игнорировать ошибку, если это не критично

🔔 Шаг 04:

Миссия 4

Расположите действия для устранения сбоя при высоких задержках запросов в PostgreSQL в правильном порядке:

1. Провести анализ запросов и их выполнения
2. Определить, какие ресурсы являются узким местом
3. Оптимизировать индексацию и переписать проблемные запросы
4. Перепроверить настройки сервера PostgreSQL

🔔 Подсказка:

Если ты заметил задержки, важно сначала понять, что именно их вызывает. Это поможет определить, что нужно улучшать.

Ответ:

- Провести анализ запросов и их выполнения
- Определить, какие ресурсы являются узким местом
- Оптимизировать индексацию и переписать проблемные запросы
- Перепроверить настройки сервера PostgreSQL

Шаг 05:

Миссия 5

Ты заметил, что база данных начинает работать медленно в пиковые часы. Какой параметр в первую очередь стоит мониторить?

Подсказка: Важно отслеживать, как ведет себя база именно в периоды нагрузки.

Ответ:

- **Загруженность процессора (CPU)**
- Число активных подключений
- Размер базы данных
- Частота выполнения запросов

Шаг 06:

Миссия 6

Ты заметил, что база данных начала работать медленно после установки нового индекса. Что стоит проверить в первую очередь?

Подсказка: Иногда новые индексы могут влиять на производительность. Важно проверить, не стал ли новый индекс причиной проблемы.

Ответ:

- **Проверить, не увеличилась ли стоимость выполнения запросов из-за индекса**
- Проверить, не используется ли индексы в запросах вообще
- Переписать все запросы на более сложные конструкции
- Удалить индекс и запустить сервер базы данных заново

Шаг 07:

Миссия 7

 Задача: Сопоставьте виды мониторинга с их описаниями

Типы мониторинга:

Ответ:

- Мониторинг производительности
- Мониторинг ресурсов
- Мониторинг сетевых проблем
- Мониторинг безопасности

Описание:

1. Отслеживание состояния сервера, нагрузки на процессор, памяти и другие ресурсы
2. Отслеживание сетевых соединений и времени отклика
3. Анализ атак на базу данных или несанкционированных доступов
4. Анализ скорости выполнения запросов и общего времени отклика базы данных

Подсказка:

Подумай, какие проблемы могут касаться сети, какие — процессора, а какие — безопасности.

- **Мониторинг производительности** — Анализ скорости выполнения запросов и общего времени отклика базы данных
- **Мониторинг ресурсов** — Отслеживание состояния сервера, нагрузки на процессор, памяти и другие ресурсы
- **Мониторинг сетевых проблем** — Отслеживание сетевых соединений и времени отклика

- **Мониторинг безопасности** — Анализ атак на базу данных или несанкционированных доступов

Шаг 08:

Миссия 8

Ты решил настроить алерты для критичных запросов в PostgreSQL. Что первым делом нужно определить?

Подсказка: Алерты должны быть настроены на основе того, что является критичным для твоей базы данных.

Ответ:

- **Какие запросы могут затруднить работу базы данных при пиковых нагрузках**
- Какие запросы вызывают наибольшее количество ошибок в журнале
- Какие запросы вызывают наибольшее количество соединений с сервером
- Какие запросы используют наибольшее количество памяти

Шаг 09:

Миссия 9

Расположите шаги для анализа сбоев по нагрузке на сервер PostgreSQL в правильном порядке:

1. Определить, какие запросы занимают больше всего времени
2. Проверить статистику нагрузки на сервер и работу его процессоров
3. Рассмотреть планы выполнения запросов для выявления возможных проблем
4. Провести тесты и нагрузочные тесты для выявления слабых мест

 **Подсказка:**

Сначала стоит определить, какие именно запросы вызывают нагрузку, а затем провести более детальный анализ работы сервера.

Ответ:

- Определить, какие запросы занимают больше всего времени
- Проверить статистику нагрузки на сервер и работу его процессоров
- Рассмотреть планы выполнения запросов для выявления возможных проблем
- Провести тесты и нагрузочные тесты для выявления слабых мест

Шаг 10:

Миссия 10

Во время наблюдения за базой данных ты заметил, что время выполнения запросов начинает увеличиваться. Что стоит сделать в первую очередь?

Подсказка: Важно понять, в чем именно заключается причина увеличения времени выполнения запросов.

Ответ:

- **Проверить план выполнения запросов и статистику индексов**
- Перезапустить базу данных и проверить, поможет ли это
- Очистить кэш и пересоздать индексы
- Добавить больше памяти на сервер

Шаг 11:

Миссия 11

Ты заметил, что нагрузка на сервер PostgreSQL значительно возросла. Что первым делом следует проверить?

Подсказка: Прежде чем делать масштабирование, важно выявить, что именно вызывает нагрузку.

Ответ:

- Проверить запросы, которые занимают больше всего времени и ресурсов
- Увеличить количество оперативной памяти на сервере
- Перезапустить сервер базы данных
- Удалить старые индексы и запросы из базы данных

Шаг 12:

Миссия 12

Расположите шаги для настройки алертов в PostgreSQL по приоритету:

1. Определить, какие метрики важны для отслеживания
2. Настроить уведомления при достижении пороговых значений метрик
3. Настроить сбор статистики с помощью `pg_stat_statements`
4. Проверить корректность работы алертов и их настройку

Подсказка:

Важно сначала настроить сбор нужных данных, а потом настроить уведомления и проверить работу системы.

Ответ:

- Определить, какие метрики важны для отслеживания
- Настроить уведомления при достижении пороговых значений метрик
- Настроить сбор статистики с помощью `pg_stat_statements`
- Проверить корректность работы алертов и их настройку

Шаг 13:

Миссия 13

 Задача: Сопоставьте типы алертов с их описаниями

Типы алертов:

Ответ:

- Нагрузка на процессор
- Количество соединений с базой данных
- Время выполнения запросов
- Использование памяти

Описание:

1. Отслеживание процессора для выявления перегрузок
2. Определение количества подключений к базе данных для предотвращения блокировок
3. Отслеживание времени выполнения запросов для поиска проблем с производительностью
4. Мониторинг использования памяти для предотвращения ее исчерпания

Подсказка:

Подумай, какие метрики могут быть связаны с производительностью, а какие — с состоянием ресурсов сервера.

- **Нагрузка на процессор** — Отслеживание процессора для выявления перегрузок

- **Количество соединений с базой данных** — Определение количества подключений к базе данных для предотвращения блокировок
- **Время выполнения запросов** — Отслеживание времени выполнения запросов для поиска проблем с производительностью
- **Использование памяти** — Мониторинг использования памяти для предотвращения ее исчерпания

🔧 Шаг 14:

Миссия 14

Ты настроил мониторинг для PostgreSQL и получаешь уведомления о высоком времени выполнения запросов. Как ты будешь действовать?

Подсказка: Важно не только получать уведомления, но и правильно анализировать проблему.

Ответ:

- **Проверить планы выполнения запросов и их оптимизацию**
- Остановить базу данных и перезапустить её
- Удалить старые запросы из базы данных
- Увеличить память и процессорные ресурсы на сервере

🔧 Шаг 15:

Миссия 15

Расположите шаги для оптимизации работы базы данных PostgreSQL по приоритету:

1. Изучить планы выполнения запросов и индексы
2. Обновить статистику базы данных и индексов
3. Настроить параметры конфигурации для работы с большими объемами данных
4. Провести нагрузочное тестирование

🔧 **Подсказка:**

Сначала стоит проверить текущие индексы и запросы, а потом уже переходить к настройке и тестированию.

Ответ:

- Изучить планы выполнения запросов и индексы
- Обновить статистику базы данных и индексов
- Настроить параметры конфигурации для работы с большими объемами данных
- Провести нагрузочное тестирование

📖 Урок 5 - Логирование долгих и медленных запросов

🔧 Шаг 01:

Миссия 1

Вы — DBA в стартапе, приложение иногда замедляется. Какой параметр PostgreSQL нужно включить, чтобы логировать запросы, дольше заданного порога?

Подсказка: Нужно активировать сам механизм логирования медленных запросов.

Ответ:

- **log_min_duration_statement**
- shared_buffers
- max_connections
- work_mem

🔧 Шаг 02:

Миссия 2

Вы включили `log_min_duration_statement = 500`. Что именно он будет логировать?

Подсказка: «500» — это миллисекунды.

Ответ:

- **Все запросы, которые выполнялись дольше 500 мс**
- Все запросы, быстрее 500 мс
- Все запросы независимо от длительности
- **Никакие запросы** — параметр неправильно работает

🔧 Шаг 03:

Миссия 3

Программист жалуется: «я вижу в логах много запросов 0 ms — это медленные запросы?» Что вы посоветуете?

Подсказка: 0 ms может означать, что запись длительности отключена или округление.

Ответ:

- **Проверь** — включено ли `log_statement_stats` или `track_activity_query_size`
- Отключи `log_min_duration_statement`
- Увеличь `max_wal_size`
- Уменьши `shared_buffers`

🔧 Шаг 04:

Миссия 4

Вы настроили `auto_explain.log_min_duration = 1000`. Когда будет логироваться план?

Подсказка: Это план только при условии, что запрос выполнялся дольше порога.

Ответ:

- **Для запросов, медленнее 1000 мс**
- Для всех запросов
- Ни для каких запросов
- Для тех, что быстрее 1000 мс

🔧 Шаг 05:

Миссия 5

📄 Сопоставьте параметр PostgreSQL и его роль в логировании медленных запросов:

Параметры:

Ответ:

- `log_duration`
- `log_min_duration_statement`
- `auto_explain.log_min_duration`
- `auto_explain.log_analyze`

Роли:

1. включает логирование длительности всех запросов
2. устанавливает порог (в мс) для логирования запросов как медленных

3. определяет порог для автоматического EXPLAIN
4. указывает, следует ли автоматически запускать EXPLAIN ANALYZE

Подсказка: Обратите внимание: два параметра начинаются с `auto_explain`, они связаны друг с другом.

- **log_duration** — включает логирование длительности всех запросов
- **log_min_duration_statement** — устанавливает порог (в мс) для логирования запросов как медленных
- **auto_explain.log_min_duration** — определяет порог для автоматического EXPLAIN
- **auto_explain.log_analyze** — указывает, следует ли автоматически запускать EXPLAIN ANALYZE

Шаг 06:

Миссия 6

 Сопоставьте этапы анализа медленного запроса и его цель:

Этапы:

Ответ:

- Идентификация в логах
- EXPLAIN / EXPLAIN ANALYZE
- Проверка индексного использования
- Рефакторинг запроса или добавление индексов

Цели:

1. выявить запрос, превышающий порог
2. оценить план и время исполнения с детализацией
3. понять, используются ли нужные индексы
4. оптимизировать, чтобы снизить нагрузку

Подсказка: Последовательность действий логична: сначала найти, потом проанализировать, потом понять индексы, потом улучшить.

- **Идентификация в логах** — выявить запрос, превышающий порог
- **EXPLAIN / EXPLAIN ANALYZE** — оценить план и время исполнения с детализацией
- **Проверка индексного использования** — понять, используются ли нужные индексы
- **Рефакторинг запроса или добавление индексов** — оптимизировать, чтобы снизить нагрузку

Шаг 07:

Миссия 7

 Сопоставьте состояние логирования и его следствие для анализа:

Состояния:

Ответ:

- `log_min_duration_statement = -1`
- `log_min_duration_statement = 0`
- `auto_explain.enabled = off`
- `auto_explain.log_analyze = off`

Следствия:

1. никакие медленные запросы не логируются

2. логируются все запросы независимо от длительности
3. не запускается автоматический EXPLAIN
4. EXPLAIN без ANALYZE (без фактической трассировки времени)

Подсказка: Значение -1 часто означает «отключено», 0 — «всегда логировать».

- **log_min_duration_statement = -1** — никакие медленные запросы не логируются
- **log_min_duration_statement = 0** — логируются все запросы независимо от длительности
- **auto_explain.enabled = off** — не запускается автоматический EXPLAIN
- **auto_explain.log_analyze = off** — EXPLAIN без ANALYZE (без фактической трассировки времени)

Шаг 08:

Миссия 8

Расположите шаги реакции на всплеск медленных запросов:

1. включение логирования медленных запросов
2. сбор логов и фильтрация по длительности
3. анализ планов с EXPLAIN
4. внесение оптимизаций (индексы, переписывание)
5. мониторинг после изменений

Подсказка: Сначала нужно начать логирование, потом собрать данные, проанализировать, изменить и оценить результат.

Ответ:

- включение логирования медленных запросов
- сбор логов и фильтрация по длительности
- анализ планов с EXPLAIN
- внесение оптимизаций (индексы, переписывание)
- мониторинг после изменений

Шаг 09:

Миссия 9

Упорядочите слои, через которые проходит информация о медленном запросе:

1. сетевая задержка / клиент
2. приложение (ORM или SQL)
3. PostgreSQL серверный слой (логирующий механизм)
4. файловая система логов / внешний агрегатор
5. система мониторинга / аналитика

Подсказка: Данные проходят от приложения до логов и далее до анализа.

Ответ:

- сетевая задержка / клиент
- приложение (ORM или SQL)
- PostgreSQL серверный слой (логирующий механизм)
- файловая система логов / внешний агрегатор
- система мониторинга / аналитика

Шаг 10:

Миссия 10

Упорядочите приоритеты при настройке логирования медленных запросов:

1. минимальное влияние на производительность
2. достаточная детализация (время, план)
3. безопасность логов
4. возможность агрегации и аналитики
5. хранение и ротация логов

Подсказка: Приоритеты начинаются с «не повредить систему».

Ответ:

- минимальное влияние на производительность
- достаточная детализация (время, план)
- безопасность логов
- возможность агрегации и аналитики
- хранение и ротация логов

Шаг 11:

Миссия 11

Упорядочите уровни агрегации после логирования медленных запросов:

1. сырые записи запросов с длительностью
2. группировка по шаблонам (normalized queries)
3. сводные метрики (percentiles, среднее время)
4. визуализация в дашборде
5. уведомления при аномалиях

Подсказка: Начните с сырых данных, затем группируйте, анализируйте и визуализируйте.

Ответ:

- сырые записи запросов с длительностью
- группировка по шаблонам (normalized queries)
- сводные метрики (percentiles, среднее время)
- визуализация в дашборде
- уведомления при аномалиях

Шаг 12:

Миссия 12

Упорядочите стадии при обработке EXPLAIN-лога:

1. разбор формата журнала
2. извлечение плана и метрик
3. нормализация запроса (удаление конкретных значений)
4. группировка с похожими шаблонами
5. сравнение планов между запусками

Подсказка: Лог превращается в план, затем в шаблон, затем агрегируется.

Ответ:

- разбор формата журнала

- извлечение плана и метрик
- нормализация запроса (удаление конкретных значений)
- группировка с похожими шаблонами
- сравнение планов между запусками

Шаг 13:

Миссия 13

Упорядочите события при неожиданном росте «медленных» запросов:

1. уведомление из мониторинга
2. поиск корреляции с нагрузкой
3. фильтрация логов по времени и длительности
4. анализ планов EXPLAIN
5. оптимизация / перераспределение ресурсов

Подсказка: Сначала вы узнаете, что что-то не так, затем локализуете во времени, потом анализируете, затем реагируете.

Ответ:

- уведомление из мониторинга
- поиск корреляции с нагрузкой
- фильтрация логов по времени и длительности
- анализ планов EXPLAIN
- оптимизация / перераспределение ресурсов

Шаг 14:

Миссия 14

Упорядочите подходы к снижению числа длительных запросов:

1. замена медленных JOIN на более простые запросы
2. введение лимитов / пагинации
3. регулярное персонифицированное профилирование
4. пересмотр блокировок / параллелизма
5. увеличение ресурсов (CPU, память, I/O)

Подсказка: Часто начинают с изменения логики, прежде чем масштабировать железо.

Ответ:

- замена медленных JOIN на более простые запросы
- введение лимитов / пагинации
- регулярное персонифицированное профилирование
- пересмотр блокировок / параллелизма
- увеличение ресурсов (CPU, память, I/O)

Шаг 15:

Миссия 15

Упорядочите уровни детализации логов, от менее подробного до наиболее подробного:

1. только время выполнения запроса
2. время + текст запроса
3. время + текст + план выполнения

4. время + текст + план + аналитика (ANALYZE)
5. дополнительные статистики (буферы, I/O, время ожидания)

Подсказка: Чем дальше — тем больше слоёв информации добавляется.

Ответ:

- только время выполнения запроса
- время + текст запроса
- время + текст + план выполнения
- время + текст + план + аналитика (ANALYZE)
- дополнительные статистики (буферы, I/O, время ожидания)

Шаг 16:

Миссия 16

Упорядочите стратегию реагирования на медленные запросы по степени зрелости:

1. вручную исключать проблемные запросы
2. скрипты агрегации логов / парсеры
3. интеграция с системой мониторинга (Prometheus, ELK и др.)
4. автоматическая выдача алертов и трейсинг запросов
5. самообучающаяся система рекомендаций оптимизации

Подсказка: Начинайте с простого подхода, затем автоматизируйте, и затем идите к умной системе.

Ответ:

- вручную исключать проблемные запросы
- скрипты агрегации логов / парсеры
- интеграция с системой мониторинга (Prometheus, ELK и др.)
- автоматическая выдача алертов и трейсинг запросов
- самообучающаяся система рекомендаций оптимизации

Модуль 13 - Бонус. Антипаттерны — путь ошибок

Урок 1 - Типичные ошибки масштабирования

Шаг 01:

Миссия 1

Вы — разработчик, и на одном из серверов база PostgreSQL стала медленно отвечать при росте нагрузки. Какое из решений первоочередно проверить, чтобы исключить типичную ошибку масштабирования?

Подсказка: Часто беда — не масштабирование, а конфигурация, которую забыли отрегулировать.

Ответ:

- Проверить настройки ``shared_buffers`` и ``work_mem``
- Добавить ещё один физический диск
- Мгновенно вводить шардирование
- Переходить на другой СУБД

🔧 Шаг 02:

Миссия 2

После увеличения `work_mem` приложение начало аварийно падать с ошибками памяти. В чём, скорее всего, ошибка?`

Подсказка: `work_mem` применяется на каждый запрос и каждый сортируемый фрагмент.`

Ответ:

- Слишком высокий `work_mem` на большое число параллельных запросов`
- Неправильный порядок колонок в индексах
- Недостаточный `shared_buffers``
- Отсутствие SSD для журнала WAL

🔧 Шаг 03:

Миссия 3

Команда поставила увеличение `max_connections` до тысячи без планирования. Какой риск вы бы описали коллегам?`

Подсказка: Больше число соединений — не всегда лучше, особенно если каждое соединение забирает память.

Ответ:

- Перегрузка памяти из-за множества одновременных подключений
- Нарушение целостности данных
- Увеличение количества WAL-файлов
- Снижение скорости записи на диск

🔧 Шаг 04:

Миссия 4

Вы решили масштабировать — добавили реплику, но забыли на приложении перераспределить чтения. Что наиболее типично произойдет?

Подсказка: Если приложение всё ещё шлёт всё на мастер, реплика простаивает.

Ответ:

- Реплика будет простаивать, а нагрузка останется на мастере
- Реплика выполнит транзакции вместо мастера
- Данные начнут рассинхронизироваться автоматически
- Запросы будут выполняться быстрее на мастере

🔧 Шаг 05:

Миссия 5

В логах вы видите частые блокировки (deadlock) при высокой нагрузке — что из этого чаще всего является причиной?

Подсказка: Блокировки часто возникают из-за пересечения обновлений в разном порядке.

Ответ:

- Несогласованный порядок обращения к таблицам в транзакциях
- Перегрузка сети
- Недостаточный WAL
- Слишком маленький `shared_buffers``

🔗 Шаг 06:

Миссия 6

Приложение начало делать слишком много мелких запросов вместо одного крупного, при этом нагрузка выросла. Это — какая ошибка масштабирования?

Подсказка: Часто это называют «N+1 problem».

Ответ:

- **Проблема N+1 запросов**
- Избыточные JOINы
- Переполнение пулла соединений
- Неправильно настроенный планировщик

🔗 Шаг 07:

Миссия 7

📄 Сопоставьте антипаттерн с его описанием:

Антипаттерны:

Ответ:

- Избыточные соединения (too many connections)
- Проблема N+1
- Отсутствие индексов
- Безрассудное вертикальное масштабирование

Описание:

1. каждый запрос делает дополнительный запрос к БД для связанных данных
2. множество неиспользуемых соединений губит память и перебирает пулы
3. все таблицы сканируются, потому что нет нужных индексов
4. просто прокачивать CPU/память, игнорируя архитектуру

Подсказка: Наиболее популярные ошибки — буква N, соединения, отсутствие индексов, надпись «vertical scaling».

- **Избыточные соединения (too many connections)** — множество неиспользуемых соединений губит память и перебирает пулы
- **Проблема N+1** — каждый запрос делает дополнительный запрос к БД для связанных данных
- **Отсутствие индексов** — все таблицы сканируются, потому что нет нужных индексов
- **Безрассудное вертикальное масштабирование** — просто прокачивать CPU/память, игнорируя архитектуру

🔗 Шаг 08:

Миссия 8

📄 Сопоставьте признаки, которые вы увидите, с возможным антипаттерном:

Признаки:

Ответ:

- много процессов ждёт свободного соединения
- жёсткие блокировки при одновременном обновлении
- медленный полный скан таблицы
- сверхбыстрый рост нагрузки без улучшения времени отклика

Антипаттерны:

1. too many connections
2. deadlock / блокировки
3. отсутствие индексов / seq scan
4. неустойчивое вертикальное масштабирование

Подсказка: Сопоставьте прямой симптом с наиболее вероятной причиной.

- **много процессов ждёт свободного соединения** — too many connections
- **жёсткие блокировки при одновременном обновлении** — deadlock / блокировки
- **медленный полный скан таблицы** — отсутствие индексов / seq scan
- **сверхбыстрый рост нагрузки без улучшения времени отклика** — неустойчивое вертикальное масштабирование

Шаг 09:

Миссия 9

 Сопоставьте контекст и ошибку масштабирования:

Контексты:

Ответ:

- каждый веб-запрос строит 5 отдельных запросов к БД
- на пике 500 клиентов все просто перестают подключаться
- даже на новом железе время ответа растёт
- отчёты работают медленно, сканируя огромные таблицы

Антипаттерны:

1. N+1 problem
2. too many connections
3. вертикальное масштабирование без архитектуры
4. отсутствие индексов / sequential scan

Подсказка: Ищите шаблоны: много мелких запросов — N+1, перегрузка соединениями и др.

- **каждый веб-запрос строит 5 отдельных запросов к БД** — N+1 problem
- **на пике 500 клиентов все просто перестают подключаться** — too many connections
- **даже на новом железе время ответа растёт** — вертикальное масштабирование без архитектуры
- **отчёты работают медленно, сканируя огромные таблицы** — отсутствие индексов / sequential scan

Шаг 10:

Миссия 10

 Сопоставьте антипаттерн и его потенциальное решение:

Антипаттерны:

Ответ:

- too many connections
- N+1 problem
- отсутствие индексов
- вертикальное масштабирование без архитектуры

Решения:

1. использовать пул соединений (PgBouncer и др.)
2. заменить множество запросов на JOIN или batch
3. добавить или пересмотреть индексы, покрывающие запросы
4. внедрить шардирование / распределённую архитектуру

Подсказка: Решения часто прямо отвечают корню проблемы.

- **too many connections** — использовать пул соединений (PgBouncer и др.)
- **N+1 problem** — заменить множество запросов на JOIN или batch
- **отсутствие индексов** — добавить или пересмотреть индексы, покрывающие запросы
- **вертикальное масштабирование без архитектуры** — внедрить шардирование / распределённую архитектуру

Шаг 11:

Миссия 11

 Сопоставьте различные слои, где может проявляться ошибка масштабирования, с её признаком:

Слои:

Ответ:

- приложение / ORM
- сеть / соединения
- СУБД / планировщик
- хранилище / диск I/O

Признаки:

1. ORM делает много лишних запросов (N+1)
2. соединения не устанавливаются или таймаутятся
3. неэффективные планы (полные сканы вместо индексов)
4. время ожидания I/O, дисковый лаг

Подсказка: Ошибки могут быть не только на СУБД — приложение, сеть и диск тоже влияют.

- **приложение / ORM** — ORM делает много лишних запросов (N+1)
- **сеть / соединения** — соединения не устанавливаются или таймаутятся
- **СУБД / планировщик** — неэффективные планы (полные сканы вместо индексов)
- **хранилище / диск I/O** — время ожидания I/O, дисковый лаг

Шаг 12:

Миссия 12

Расположите шаги расследования производительности при росте нагрузки:

1. сбор метрик CPU, RAM, диск, сеть
2. анализ логов ошибок и таймаутов
3. просмотр запросов с высокой длительностью
4. определение узких мест (I/O, ЦП, блокировки)
5. внедрение корректирующих мер (индексы, пул соединений и др.)

Подсказка: Начинайте с внешнего мониторинга, затем углубляйтесь до запросов и корректируйте.

Ответ:

- сбор метрик CPU, RAM, диск, сеть
- анализ логов ошибок и таймаутов
- просмотр запросов с высокой длительностью
- определение узких мест (I/O, ЦП, блокировки)
- внедрение корректирующих мер (индексы, пул соединений и др.)

 Шаг 13:**Миссия 13**

Упорядочите уровни реакции на рост количества соединений:

1. исследовать причины (утечки, долгие транзакции)
2. внедрить пул соединений
3. ограничить `max\_connections` и queueing
4. шардирование или разделение по базе
5. архивация старых данных и очистка

Подсказка: Сначала исследование, потом оптимизация, затем архитектура.

Ответ:

- исследовать причины (утечки, долгие транзакции)
- внедрить пул соединений
- ограничить `max\_connections` и queueing
- шардирование или разделение по базе
- архивация старых данных и очистка

 Шаг 14:**Миссия 14**

Упорядочите слои, на которых можно предпринять оптимизацию, от ближнего к пользователю до глубин БД:

1. кэширование в приложении / CDN
2. ограничения количества запросов (rate limiting)
3. конвейер или batch запрос
4. оптимизация SQL и индексов
5. масштабирование архитектуры (реплики, шарды)

Подсказка: Сначала ближе к клиенту, потом глубже внутрь системы.

Ответ:

- кэширование в приложении / CDN
- ограничения количества запросов (rate limiting)
- конвейер или batch запрос
- оптимизация SQL и индексов
- масштабирование архитектуры (реплики, шарды)

 Шаг 15:**Миссия 15**

Расположите этапы эволюции архитектуры приложения под нагрузкой:

1. монолит + одна база

2. введение пула соединений
3. отделение чтения/записи (реплики)
4. горизонтальное масштабирование / шарды
5. микросервисы и распределённые БД

Подсказка: Это как взросление: от простого монолита к распределённой системе.

Ответ:

- монолит + одна база
- введение пула соединений
- отделение чтения/записи (реплики)
- горизонтальное масштабирование / шарды
- микросервисы и распределённые БД

Шаг 16:

Миссия 16

Расположите стратегии по степени автоматизации и интеллекта:

1. ручное вмешательство и отладка
2. скрипты агрегации логов и оповещений
3. интеграция с системой мониторинга (Prometheus, ELK и т.д.)
4. автоматическая адаптация параметров под нагрузку
5. машинно-обучающийся анализ поведения запросов

Подсказка: От простого к автоматическому и далее к «умному» подходу.

Ответ:

- ручное вмешательство и отладка
- скрипты агрегации логов и оповещений
- интеграция с системой мониторинга (Prometheus, ELK и т.д.)
- автоматическая адаптация параметров под нагрузку
- машинно-обучающийся анализ поведения запросов

Урок 2 - Неверное использование транзакций

Шаг 01:

Миссия 1

Во время разработки микросервиса программист обёрнул каждую отдельную операцию INSERT/UPDATE в свою транзакцию. Какой из недостатков такого подхода наиболее вероятен?

Подсказка: Частые транзакции создают лишнюю нагрузку на журнал WAL и блокировки.

Ответ:

- **Увеличенная нагрузка на WAL и частые fsync**
- Нулевая изоляция данных
- Автоматическая репликация перестает работать
- Запросы становятся атомарными

Шаг 02:

Миссия 2

Собралась команда и решила держать одну глобальную транзакцию на всё приложение, чтобы «гарантировать целостность данных». Какой эффект это вызовет?

Подсказка: Долгие транзакции — враг конкурентности и VACUUM.

Ответ:

- **Блокировки и рост числа «защищённых старых» строк (bloat)**
- Ускорение всех запросов
- Полное отключение индексов
- Снижение потребления памяти

🔮 Шаг 03:

Миссия 3

Инженер написал код, в котором внутри одной транзакции делает множество операций чтения, после чего выполняет модификацию. В чём потенциальная опасность?

Подсказка: Длительные транзакции держат MVCC версии живыми дольше.

Ответ:

- **Невозможность чистого удаления старых версий (vacuum задерживается)**
- Снижение пропускной способности сети
- Автоматическое закрытие сессии
- Потеря данных

🔮 Шаг 04:

Миссия 4

Разработчик забыл делать COMMIT и у него транзакция остаётся открытой очень долго. Что станет ключевой проблемой?

Подсказка: Так могут «замораживаться» обновления и мешать очистке.

Ответ:

- **Рост bloat и невозможность удалить старые строки**
- Снижение размера базы
- Уменьшение числа индексов
- Автоматическое закрытие реплики

🔮 Шаг 05:

Миссия 5

В одном из сервисов транзакции покрывают весь цикл обработки HTTP-запроса (включая внешние API-вызовы). Почему это плохо?

Подсказка: Внешние вызовы могут работать медленно или ждать, и транзакция «висит».

Ответ:

- **Высокий риск блокировок и дедлоков при задержках внешних вызовов**
- Невозможность открыть соединение к БД
- Ускорение запросов
- Устранение всех гонок

🔮 Шаг 06:

Миссия 6

Команда пытается обойти ограничения базы и открывает подчинённые транзакции с SAVEPOINT-ами повсеместно. Какая ошибка здесь может появиться?

Подсказка: SAVEPOINT не делает отдельные независимые транзакции.

Ответ:

- **Ложное ощущение независимости и сложности отката**
- SAVEPOINT создаёт отдельную базу
- Полная изоляция от других транзакций
- Автоматическое коммитирование

Шаг 07:

Миссия 7

 Сопоставьте антипаттерн транзакций с его символическим признаком:

Антипаттерны:

Ответ:

- Мелкие транзакции (каждый шаг в транзакции)
- Огромная глобальная транзакция
- Транзакция через внешние вызовы (API, HTTP)
- Забытый COMMIT / слишком долгий open транзакция

Признаки:

1. частые fsync и нагрузка WAL
2. рост bloat, блокировки и задержки VACUUM
3. задержки из-за внешнего API, зависание внутри транзакции
4. старые версии строк не удаляются, долгие блокировки

Подсказка: Сопоставьте проявление с антипаттерном: множество мелких транзакций — нагрузка WAL, глобальная — bloat и блокировки и т.д.

- **Мелкие транзакции (каждый шаг в транзакции)** — частые fsync и нагрузка WAL
- **Огромная глобальная транзакция** — рост bloat, блокировки и задержки VACUUM
- **Транзакция через внешние вызовы (API, HTTP)** — задержки из-за внешнего API, зависание внутри транзакции
- **Забытый COMMIT / слишком долгий open транзакция** — старые версии строк не удаляются, долгие блокировки

Шаг 08:

Миссия 8

 Сопоставьте симптом и вероятность неправильной транзакции:

Симптомы:

Ответ:

- журнал WAL растёт стремительно
- VACUUM не успевает убрать мёртвые строки
- запрос внутри транзакции зависает из-за ожидания HTTP
- иногда транзакция длится минуты без COMMIT

Вероятные ошибки:

1. мелкие транзакции слишком часты
2. глобальная транзакция покрывает всё приложение
3. транзакция обёрнута вокруг внешнего вызова
4. забыли коммит / приложение не закрывает транзакции

Подсказка: WAL — это частые коммиты, bloat — долгий open, HTTP — внешние вызовы.

- **журнал WAL растёт стремительно** — мелкие транзакции слишком часты
- **VACUUM не успевает убрать мёртвые строки** — глобальная транзакция покрывает всё приложение
- **запрос внутри транзакции зависает из-за ожидания HTTP** — транзакция обёрнута вокруг внешнего вызова
- **иногда транзакция длится минуты без COMMIT** — забыли коммит / приложение не закрывает транзакции

Шаг 09:

Миссия 9

 Сопоставьте стратегию исправления и антипаттерн транзакций:

Антипаттерны:

Ответ:

- Мелкие транзакции
- Глобальная транзакция
- Транзакция через внешний вызов
- Забытый COMMIT / слишком долгий open

Коррекции:

1. объединить несколько операций в одну транзакцию, но не дробить слишком мелко
2. разделить область ответственности, использовать несколько мелких транзакций
3. вынести вызовы API за пределы транзакции или сделать компенсации

Подсказка: Коррекции часто прямо обращаются к корню проблемы — уменьшать, ограничивать, вынести.

- **Мелкие транзакции** — объединить несколько операций в одну транзакцию, но не дробить слишком мелко
- **Глобальная транзакция** — разделить область ответственности, использовать несколько мелких транзакций
- **Транзакция через внешний вызов** — вынести вызовы API за пределы транзакции или сделать компенсации
- **Забытый COMMIT / слишком долгий open** — следить за временем транзакции и явно коммитить/откатывать

Шаг 10:

Миссия 10

 Сопоставьте слой приложения / архитектуры и риск, связанный с неправильной транзакцией:

Слои:

Ответ:

- ORM / бизнес-логика
- Слой интеграции с внешними API
- Слой доступа к БД (DAO / репозитории)
- Инфраструктура БД

Риски:

1. ORM генерирует множество мелких транзакций автоматически
2. внешние задержки втягивают транзакцию внутрь API-вызовов

3. неявные коммиты или отсутствие управления временем транзакции
4. bloat, блокировки, длинные open транзакции

Подсказка: ORM часто управляет транзакциями автоматически, внешние слои — задержками, DAO — commit logic и БД — последствия.

- **ORM / бизнес-логика** — ORM генерирует множество мелких транзакций автоматически
- **Слой интеграции с внешними API** — внешние задержки втягивают транзакцию внутрь API-вызовов
- **Слой доступа к БД (DAO / репозитории)** — неявные коммиты или отсутствие управления временем транзакции
- **Инфраструктура БД** — bloat, блокировки, длинные open транзакции

Шаг 11:

Миссия 11

Упорядочите шаги диагностики проблем с транзакциями:

1. сбор метрик bloat, время транзакций, количество открытых транзакций
2. анализ логов блокировок и задержек
3. поиск транзакций с длительным execution time
4. сопоставление с внешними вызовами, HTTP, API
5. корректировка: объединение транзакций, вынос API, контроль времени

Подсказка: Диагностика начинается с измерений, затем логов, затем корреляций и исправлений.

Ответ:

- сбор метрик bloat, время транзакций, количество открытых транзакций
- анализ логов блокировок и задержек
- поиск транзакций с длительным execution time
- сопоставление с внешними вызовами, HTTP, API
- корректировка: объединение транзакций, вынос API, контроль времени

Шаг 12:

Миссия 12

Упорядочите приоритеты исправления ошибок транзакций (от лёгкого к тяжёлому):

1. явно закрывать транзакции (COMMIT / ROLLBACK)
2. ограничивать продолжительность транзакции
3. вынести внешние вызовы из транзакций
4. объединить мелкие транзакции с логикой
5. перераспределить логику на уровни, возможно разбить на микрооперации

Подсказка: Сначала нужно базовое исправление (закрывать и ограничивать), затем архитектурные меры.

Ответ:

- явно закрывать транзакции (COMMIT / ROLLBACK)
- ограничивать продолжительность транзакции
- вынести внешние вызовы из транзакций
- объединить мелкие транзакции с логикой
- перераспределить логику на уровни, возможно разбить на микрооперации

🔗 Шаг 13:

Миссия 13

Упорядочите виды транзакционных ошибок по возрастающему ущербу для системы:

1. отсутствие COMMIT (транзакции остаются открытыми)
2. мелкие транзакции слишком часто
3. транзакции охватывают внешние вызовы
4. глобальная транзакция покрытия всего приложения
5. комбинация всех трёх ошибок одновременно

Подсказка: Сначала одна типовая ошибка, затем более комплексные и опасные сценарии.

Ответ:

- отсутствие COMMIT (транзакции остаются открытыми)
- мелкие транзакции слишком часто
- транзакции охватывают внешние вызовы
- глобальная транзакция покрытия всего приложения
- комбинация всех трёх ошибок одновременно

🔗 Шаг 14:

Миссия 14

Упорядочите этапы эволюции транзакционной модели от плохой к оптимальной:

1. **каждая операция** — своя транзакция
2. одна глобальная транзакция на всю логику
3. транзакции захватывают внешние вызовы
4. транзакции контролируются по времени и границам
5. микротранзакции и компенсации за пределами БД

Подсказка: Сначала типичные ошибки, затем постепенное улучшение к сложной, устойчивой модели.

Ответ:

- **каждая операция** — своя транзакция
- одна глобальная транзакция на всю логику
- транзакции захватывают внешние вызовы
- транзакции контролируются по времени и границам
- микротранзакции и компенсации за пределами БД

🔗 Шаг 15:

Миссия 15

Упорядочите уровни, на которых можно применять контроль транзакций (от внешнего к внутреннему):

1. архитектурные решения (сервисная композиция, CQRS)
2. границы бизнес-логики (не оборачивать API)
3. слой доступа к данным (DAO)
4. ORM / фреймворк (контроль транзакций)
5. ниже БД (настройки таймаутов, параметров)

Подсказка: Контроль надо сначала на более высоком уровне, затем спускаться вниз к БД.

Ответ:

- архитектурные решения (сервисная композиция, CQRS)
- границы бизнес-логики (не оборачивать API)
- слой доступа к данным (DAO)
- ORM / фреймворк (контроль транзакций)
- ниже БД (настройки таймаутов, параметров)

 Шаг 16:

Миссия 16

Упорядочите шаги внедрения безопасной транзакционной модели:

1. инвентаризация текущих транзакций и сценариев
2. определение допустимой продолжительности транзакций
3. внесение границ транзакций и удаление внешних вызовов внутри них
4. тестирование под нагрузкой границ транзакций
5. мониторинг и контроль bloat, блокировок, длительности

Подсказка: Сначала нужно понять, где проблема, затем ввести ограничения, тестировать и контролировать.

Ответ:

- инвентаризация текущих транзакций и сценариев
- определение допустимой продолжительности транзакций
- внесение границ транзакций и удаление внешних вызовов внутри них
- тестирование под нагрузкой границ транзакций
- мониторинг и контроль bloat, блокировок, длительности

 Шаг 17:

Миссия 17

Упорядочите стратегии контроля транзакции по степени автоматизации и интеллекта:

1. ручной контроль и ревью транзакций
2. скрипты анализа длительных транзакций
3. интеграция с системой мониторинга (Prometheus, APM)
4. автоматическое аварийное прерывание долгих транзакций
5. адаптивное ограничение транзакции на основе ML / исторических данных

Подсказка: От ручного контроля к автоматике, затем к «умному» решению.

Ответ:

- ручной контроль и ревью транзакций
- скрипты анализа длительных транзакций
- интеграция с системой мониторинга (Prometheus, APM)
- автоматическое аварийное прерывание долгих транзакций
- адаптивное ограничение транзакции на основе ML / исторических данных

 Урок 3 - Ошибки индексации и запросов

 Шаг 01:

Миссия 1

Интерн Маша пишет запросы через ORM, и она добавила индекс на колонку, но запрос всё равно сканирует всю таблицу. Что могло пойти не так?

Подсказка: Индекс не будет использован, если типы не совпадают.

Ответ:

- **ORM передаёт значение с несоответствующим типом (например, текст вместо integer)**
- Индекс был создан на другую схему
- **Таблица слишком маленькая** — индекс не имеет смысла
- Запрос использует ORDER BY, который индекс не поддерживает

🔮 Шаг 02:

Миссия 2

Стартап-команда быстро реализовала текстовый поиск через `LIKE '%keyword% '` без индексов. Что из этого наиболее вероятно при росте базы?

Подсказка: Полнотекстовый поиск требует специальных механизмов.

Ответ:

- **Запросы станут медленными, полные сканирования таблицы**
- Индекс B-tree будет автоматически применён
- LIKE '%keyword%' начнёт использовать hash индекс
- База сама преобразует LIKE в full text search

🔮 Шаг 03:

Миссия 3

Разработчик Юра создал составной (composite) индекс `(user\_id, status)` и делает запрос `WHERE status = 'active'`. Почему индекс может быть неэффективен?

Подсказка: Порядок колонок в составном индексе важен.

Ответ:

- **Индекс не может быть использован, поскольку первый столбец `user\_id` не ограничен**
- Composite индекс автоматически работает наоборот
- PostgreSQL всегда игнорирует составные индексы
- **Статус слишком часто встречается** — планировщик игнорирует индекс

🔮 Шаг 04:

Миссия 4

Виктория настроила GIN-индекс на поле JSONB, но запрос с `jsonb\_extract\_path\_text(...) = 'value'` всё равно медленно. Что могло пойти не так?

Подсказка: Используемые функции должны быть поддерживаемыми для индекса.

Ответ:

- **Выражение в WHERE не совпадает с выражением, на котором построен индекс**
- GIN индекс не подходит для JSONB
- JSONB-индекс нельзя комбинировать с фильтром
- Индекс GIN не используется в выборках

🔗 Шаг 05:

Миссия 5

Команда аналитиков решила добавить индекс на колонку `created\_at`, но потом отметить, что вставки стали медленнее. Почему?

Подсказка: Индексы влияют на вставки и обновления.

Ответ:

- **Каждая вставка должна поддерживать структуру индекса** — накладная нагрузка
- Индекс автоматически запрещает вставки
- Индекс заставляет PostgreSQL делать двойные записи
- Вставки блокируются намеренно при наличии индекса

🔗 Шаг 06:

Миссия 6

Инженер Алекс внедрил новый индекс, но запросы на выборку стали медленнее. Что могло пойти не так?

Подсказка: Индекс помогает на операциях поиска, но может мешать при других операциях.

Ответ:

- **Избыточный индекс создаёт накладные расходы при чтении**
- Индекс автоматически перестраивается напряжённо
- PostgreSQL пропускает индекс без причин
- Индекс влияет только на INSERT/UPDATE, не на SELECT

🔗 Шаг 07:

Миссия 7

Сергей добавил несколько индексов на разные сочетания колонок, но аналитические запросы всё ещё медленные. Что стоит попробовать первым?

Подсказка: Составные индексы обрабатывают несколько колонок в нужном порядке.

Ответ:

- **Создать один составной индекс в правильном порядке колонок**
- Добавить индекс на каждую колонку отдельно
- Удалить все индексы
- Сразу использовать кластеризацию

🔗 Шаг 08:

Миссия 8

Ольга увидела, что план запроса использует последовательное сканирование вместо индекса — хотя индекс есть. Как это объяснить?

Подсказка: Планировщик выбирает то, что дешевле по оценке.

Ответ:

- **Оценка планировщика показала Seq Scan дешевле**
- Индекс испорчен
- PostgreSQL игнорирует новые индексы
- Индекс удалился сам

🔮 Шаг 09:

Миссия 9

При вставке большого объёма данных Иван отключил индексы, а после включил снова. Но обновление индексов сильно тормозит. Почему?

Подсказка: Построение индексов после загрузки данных может быть тяжёлым.

Ответ:

- **Индекс строится по всему объёму, что дорого**
- Индексы никогда не пересоздаются
- PostgreSQL пропускает rebuild
- Индекс уже был оптимальным

🔮 Шаг 10:

Миссия 10

Лилия видит, что при большом количестве параллельных запросов конкуренция за индекс блокировки растёт. Что может помочь?

Подсказка: Разделение нагрузки и частичная индексация помогают.

Ответ:

- **Использовать partial или покрывающие индексы**
- Отказаться от индексов полностью
- Увеличить lock_timeout
- Использовать только sequential scan

🔮 Шаг 11:

Миссия 11

Сопоставьте причины игнорирования индексов и их объяснение.

Причина:

Ответ:

- **Функция на столбце**
- **Низкий селективный столбец**
- **Тип данных не совпадает**
- **Запрос без фильтра по проиндексированной колонке**

Объяснение:

1. **Селективность низкая** — индекс неэффективен
2. Запрос не может использовать индекс, если фильтр не по колонке
3. Типы не совпадают, требуется приведение
4. Функция разрушает возможность использовать индекс

Подсказка: Обратите внимание на то, что мешает прямому сравнению колонок.

- **Функция на столбце** — Функция разрушает возможность использовать индекс
- **Низкий селективный столбец** — Селективность низкая — индекс неэффективен
- **Тип данных не совпадает** — Типы не совпадают, требуется приведение
- **Запрос без фильтра по проиндексированной колонке** — Запрос не может использовать индекс, если фильтр не по колонке

🔗 Шаг 12:

Миссия 12

Сопоставьте тип индексации с её особенностью:

Тип:

Ответ:

- B-tree
- Hash
- GIN / GiST
- BRIN

Особенность:

1. Хороша для full-text и массивов
2. Простая структурированная сортировка
3. Малый объём и блочные диапазоны
4. Только равенство, не поддерживает диапазоны

Подсказка: Подумайте, какие типы индексов хорошо работают для полнотекстового поиска или больших таблиц.

- **B-tree** — Простая структурированная сортировка
- **Hash** — Только равенство, не поддерживает диапазоны
- **GIN / GiST** — Хороша для full-text и массивов
- **BRIN** — Малый объём и блочные диапазоны

🔗 Шаг 13:

Миссия 13

Сопоставьте вид плана выполнения запроса и фактор, который его вызывает:

Вид плана:

Ответ:

- Seq Scan
- Index Scan
- Bitmap Index Scan
- Index Only Scan

Причина:

1. Все нужные столбцы уже содержатся в индексе
2. Большая доля строк выбирается
3. Используется несколько индексов и объединение битмап
4. Индекс применим, но нужен доступ к таблице для других столбцов

Подсказка: Подумайте, когда полезен bitmap scan, и когда можно обойтись без доступа к таблице.

- **Seq Scan** — Большая доля строк выбирается
- **Index Scan** — Индекс применим, но нужен доступ к таблице для других столбцов
- **Bitmap Index Scan** — Используется несколько индексов и объединение битмап
- **Index Only Scan** — Все нужные столбцы уже содержатся в индексе

Шаг 14:

Миссия 14

Сопоставьте проблему и потенциальное решение:

Проблема:

Ответ:

- Индекс слишком большой
- Фрагментация индекса
- Перекрывающиеся индексы
- Неиспользуемые индексы

Решение:

1. Удалить лишние индексы
2. Реорганизовать или пересоздать индекс (REINDEX / CLUSTER)
3. Проверить покрытие и объединить индексы в один составной
4. Использовать частичную индексацию или уменьшить размер

Подсказка: Иногда стоит объединить или сократить. Иногда — реорганизация.

- **Индекс слишком большой** — Использовать частичную индексацию или уменьшить размер
- **Фрагментация индекса** — Реорганизовать или пересоздать индекс (REINDEX / CLUSTER)
- **Перекрывающиеся индексы** — Проверить покрытие и объединить индексы в один составной
- **Неиспользуемые индексы** — Удалить лишние индексы

Шаг 15:

Миссия 15

Сопоставьте этап процесса анализа индекса с его задачей:

Этап:

Ответ:

- Сбор статистики
- Обновление статистики (ANALYZE)
- Проверка использования индекса
- Переоценка плана

Задача:

1. Узнать распределение значений
2. Обновить метаданные и статистику
3. Посмотреть, что индекс реально задействован в планах
4. Принять новый план на основании статистики

Подсказка: Статистика — основа выбора плана.

- **Сбор статистики** — Узнать распределение значений
- **Обновление статистики (ANALYZE)** — Обновить метаданные и статистику
- **Проверка использования индекса** — Посмотреть, что индекс реально задействован в планах
- **Переоценка плана** — Принять новый план на основании статистики

Шаг 16:

Миссия 16

Расположите шаги оптимизации индексации перед нагрузочным тестом:

1. Проверка текущего плана и статистики
2. Удаление перекрывающихся и неиспользуемых индексов
3. Создание необходимых составных или частичных индексов
4. Запуск EXPLAIN и сравнение планов
5. Проведение нагрузочного теста

Подсказка: Сначала – анализ, затем чистка, потом создание, проверка и наконец тесты.

Ответ:

- Проверка текущего плана и статистики
- Удаление перекрывающихся и неиспользуемых индексов
- Создание необходимых составных или частичных индексов
- Запуск EXPLAIN и сравнение планов
- Проведение нагрузочного теста

Шаг 17:

Миссия 17

Расположите шаги при решении фрагментации индекса:

1. Оценка степени фрагментации
2. Выбрать подход между REINDEX и CLUSTER
3. Временная блокировка или использование CONCURRENT
4. Выполнение выбранной операции
5. Проверка, что индекс работает быстрее

Подсказка: Фрагментация — сначала замеры, затем выбор метода, затем исполнение и проверка.

Ответ:

- Оценка степени фрагментации
- Выбрать подход между REINDEX и CLUSTER
- Временная блокировка или использование CONCURRENT
- Выполнение выбранной операции
- Проверка, что индекс работает быстрее

Шаг 18:

Миссия 18

Расположите этапы при тестировании альтернативных планов запросов в PostgreSQL:

1. Включить настройку планировщика (enable_xxx off/on)
2. Запустить EXPLAIN для варианта А
3. Запустить EXPLAIN для варианта В
4. Сравнить затраты и выбрать лучший план
5. Вернуться к обычным настройкам

Подсказка: Временное вмешательство в планировщик — эксперимент, не навсегда.

Ответ:

- Включить настройку планировщика (enable_xxx off/on)
- Запустить EXPLAIN для варианта A
- Запустить EXPLAIN для варианта B
- Сравнить затраты и выбрать лучший план
- Вернуться к обычным настройкам

 Шаг 19:

Миссия 19

Расположите шаги отладки медленного запроса с индексами:

1. Использовать EXPLAIN ANALYZE
2. Проверить использование индекса
3. Сравнить с планом без индекса
4. Попробовать альтернативные индексы
5. Провести нагрузочное тестирование

Подсказка: Анализ, проверка, эксперименты, тест.

Ответ:

- Использовать EXPLAIN ANALYZE
- Проверить использование индекса
- Сравнить с планом без индекса
- Попробовать альтернативные индексы
- Провести нагрузочное тестирование

 Урок 4 - Неэффективное использование ORM

 Шаг 01:

Миссия 1

Вы пишете веб-приложение на Node.js + ORM (например, Sequelize). Вы замечаете, что для каждого запроса к списку пользователей ORM генерирует 3 дополнительных запроса к таблицам профилей, ролей и статистики. Какое действие будет наиболее правильным на первом шаге?

Подсказка: Ищите, где ORM скрывает лишние запросы.

Ответ:

- **Проверить, какие связи (associations) объявлены как lazy/eager загрузка**
- Немедленно переписать весь доступ к БД на чистый SQL
- Добавить кеширование на уровне API перед ORM
- **Игнорировать** — ORM сам оптимизирует позже

 Шаг 02:

Миссия 2

Вы обнаружили, что ORM в вашем проекте при обновлении множества записей делает UPDATE по одной строке за раз в цикле. Что наиболее корректно предпринять?

Подсказка: ORM часто скрывает bulk-операции.

Ответ:

- **Посмотреть, поддерживает ли ORM bulk update / batch update**

- **Оставить как есть** — циклы незначительны
- Делать параллельные независимые запросы
- Переписать ORM-логику на транзакции вручную

🔗 Шаг 03:

Миссия 3

При сложном фильтре ORM генерирует подзапросы, которые выполняются медленно. Какой подход следует попробовать первым?

Подсказка: Можно вручную написать часть запроса для узкого участка.

Ответ:

- **Вынести сложную часть фильтра в raw SQL-фрагмент в ORM**
- Разбить запрос на множество ORM-запросов
- **Удалить фильтр** — пусть клиент сам отфильтрует
- Отключить все индексы и посмотреть скорость

🔗 Шаг 04:

Миссия 4

Вы видите, что ORM постоянно запрашивает одни и те же данные (например, справочники) в разных местах. Что разумно сделать?

Подсказка: Часто стоит кешировать или мемоизировать.

Ответ:

- **Добавить кеш на уровне приложения или внедрить identity map**
- Сделать запросы чаще, чтобы уменьшить одни и те же данные
- Отключить использование ORM для справочников
- Всегда загружать справочники отдельно вне ORM

🔗 Шаг 05:

Миссия 5

Проект растёт, нагрузка увеличивается. Вы замечаете, что ORM-модели становятся узким местом. Какое архитектурное решение стоит рассмотреть?

Подсказка: Иногда ORM лучше ограничить или изолировать.

Ответ:

- **Выделить критичные запросы в слой «сервис → DAO с чистым SQL»**
- Увеличить слой abstraction и связывать через ORM всё
- Перенести всю логику в frontend, чтобы сервер меньше работал
- Сменить ORM на более тяжелый фреймворк

🔗 Шаг 06:

Миссия 6

📖 Сопоставьте проблему и её характерное проявление при неэффективном использовании ORM:

Проблемы:

Ответ:

- «N+1 запросов»
- Отсутствие batch-обновлений
- Скрытые подзапросы

- Дублирование кэша на уровне приложения и БД

Проявления:

1. ORM делает «SELECT» внутри цикла
2. ORM не использует UPDATE ... WHERE id IN (...) для множества записей
ORM сам генерирует вложенные SELECT в FROM или WHERE
3. Кеш в приложении и в БД дают разный результат, сложно согласовать

Подсказка: Подумайте: как проявляется каждая проблема на уровне SQL?

- **«N+1 запросов»** — ORM делает «SELECT» внутри цикла
- **Отсутствие batch-обновлений** — ORM не использует UPDATE ... WHERE id IN (...) для множества записей
- **Скрытые подзапросы** — ORM сам генерирует вложенные SELECT в FROM или WHERE
- **Дублирование кэша на уровне приложения и БД** — Кеш в приложении и в БД дают разный результат, сложно согласовать

Шаг 07:

Миссия 7

 Сопоставьте тип ORM-решения и тип оптимизации, которую стоит применить:

Тип ORM-решения:

Ответ:

- Lazy loading associations
- Eager loading с join
- Raw SQL фрагменты
- DataLoader / batching

Оптимизации:

1. Подгружать связи через JOIN, а не отдельными запросами
2. Использовать batch запросы через IN или CASE
3. Встраивать более сложную логику в SQL руками
4. Группировать множественные запросы в один запрос

Подсказка: Оцените, как ORM транслирует связи и как можно сгруппировать запросы.

- **Lazy loading associations** — Группировать множественные запросы в один запрос
- **Eager loading с join** — Подгружать связи через JOIN, а не отдельными запросами
- **Raw SQL фрагменты** — Встраивать более сложную логику в SQL руками
- **DataLoader / batching** — Использовать batch запросы через IN или CASE

Шаг 08:

Миссия 8

 Сопоставьте персонажа-разработчика и неверный ORM-подход, который он использует:

Персонажи:

Ответ:

- Аня, backend-разработчик
- Борис, fullstack-инженер
- Вика, команда аналитики
- Глеб, тимлид

Подходы (неэффективные):

1. Строит один гигантский ORM-запрос со множеством join'ов
2. Запрашивает справочники через ORM в каждой функции
3. Обновляет записи по одной через модель.save() внутри цикла
4. Использует ORM для аналитических агрегаций вместо SQL

Подсказка: Подумайте, какие задачи чаще всего решают эти роли.

- **Аня, backend-разработчик** — Запрашивает справочники через ORM в каждой функции
- **Борис, fullstack-инженер** — Строит один гигантский ORM-запрос со множеством join'ов
- **Вика, команда аналитики** — Использует ORM для аналитических агрегаций вместо SQL
- **Глеб, тимлид** — Обновляет записи по одной через модель.save() внутри цикла

Шаг 09:

Миссия 9

 Сопоставьте стратегию кеширования и её ограничение при использовании ORM под нагрузкой:

Стратегии кеширования:

Ответ:

- In-memory cache в приложении
- Redis cache поверх запросов ORM
- Кеширование на уровне представлений БД (materialized view)
- Кеширование HTTP-ответов на уровне CDN / прокси

Ограничения/проблемы:

1. Не делится между инстансами приложения
2. Нужно инвалидация при изменении данных через ORM
3. Материализованный вид требует периодического обновления
4. Кеширование ответа не подходит для динамичного API под разными параметрами

Подсказка: Подумайте, какая часть кеша ближе к базе, а какая ближе к клиенту.

- **In-memory cache в приложении** — Не делится между инстансами приложения
- **Redis cache поверх запросов ORM** — Нужно инвалидация при изменении данных через ORM
- **Кеширование на уровне представлений БД (materialized view)** — Материализованный вид требует периодического обновления
- **Кеширование HTTP-ответов на уровне CDN / прокси** — Кеширование ответа не подходит для динамичного API под разными параметрами

Шаг 10:

Миссия 10

 Сопоставьте уровень ORM-наблюдаемости и тип метрики, которую стоит измерять:

Уровни наблюдаемости:

Ответ:

- Трассировки SQL-запросов
- Логи ORM-операций (create, update, delete)
- Профилирование медленных методов в коде
- Метрики счётчиков ошибок/таймаутов

Метрики:

1. Запросы длительностью > 1 с
2. Количество вызовов ORM.create / ORM.update
3. Таймауты и исключения в работе ORM
4. Время выполнения метода вокруг вызовов ORM

Подсказка: Соотнесите каждый инструмент с тем, какую информацию он может раскрыть.

- **Трассировки SQL-запросов** — Запросы длительностью > 1 с
- **Логи ORM-операций (create, update, delete)** — Количество вызовов ORM.create / ORM.update
- **Профилирование медленных методов в коде** — Время выполнения метода вокруг вызовов ORM
- **Метрики счётчиков ошибок/таймаутов** — Таймауты и исключения в работе ORM

Шаг 11:

Миссия 11

 Сопоставьте тип данных/сценарий и ошибку, которую ORM часто реализует неэффективно:

Типы:

Ответ:

- JSON / JSONB поля
- Географические (PostGIS) типы
- Полнотекстовый поиск (tsvector)
- Массивы и вложенные коллекции

Ошибки:

1. Принудительная десериализация/сериализация на каждую строку
2. Использование generic операторов вместо специализированных функций Postgres
3. Неэффективный фильтр LIKE вместо ts_query
4. ORM распаковывает массивы в множество JOIN-ов

Подсказка: Подумайте, как ORM может слабо оптимизировать специфичные типы PostgreSQL.

- **JSON / JSONB поля** — Принудительная десериализация/сериализация на каждую строку
- **Географические (PostGIS) типы** — Использование generic операторов вместо специализированных функций Postgres
- **Полнотекстовый поиск (tsvector)** — Неэффективный фильтр LIKE вместо ts_query
- **Массивы и вложенные коллекции** — ORM распаковывает массивы в множество JOIN-ов

Шаг 12:

Миссия 12

Расположите ниже этапы оптимизации ORM-слоя под нагрузкой в логическом порядке:

1. Профилирование запросов и логирование медленных SQL
2. Определение узких мест в ORM (N+1, batch, join-проблемы)
3. Внедрение исправлений (batch, raw SQL, кеширование)
4. Мониторинг после изменений, проверка эффекта
5. Рефакторинг нестратегических частей кода с ORM

Подсказка: Не стоит внедрять правки до того, как вы поймёте, что именно тормозит.

Ответ:

- Профилирование запросов и логирование медленных SQL
- Определение узких мест в ORM (N+1, batch, join-проблемы)
- Внедрение исправлений (batch, raw SQL, кеширование)
- Мониторинг после изменений, проверка эффекта
- Рефакторинг нестратегических частей кода с ORM

 Шаг 13:

Миссия 13

Расположите слои, через которые проходит запрос от API до БД, включая точки оптимизации ORM:

1. Контроллер / API-роутинг
2. ORM / слой моделей
3. SQL / запрос к PostgreSQL
4. Кеширование / слой Redis или in-memory
5. Слой бизнес-логики / сервисы

Подсказка: Запрос идёт сверху вниз, но оптимизации могут встраиваться между слоями.

Ответ:

- Контроллер / API-роутинг
- Слой бизнес-логики / сервисы
- ORM / слой моделей
- Кеширование / слой Redis или in-memory
- SQL / запрос к PostgreSQL

 Шаг 14:

Миссия 14

Расставьте приоритеты по степени сложности внедрения оптимизации (от лёгкой к самой трудной):

1. Добавить кеширование на уровне приложения
2. Использовать batch операции ORM
3. Внедрить raw SQL для критичных запросов
4. Перейти к CQRS / разделению чтения и записи
5. Рефакторинг ORM-моделей и убрать лишние связи

Подсказка: Простые изменения часто дают быстрый выигрыш, но архитектурные переходы дороже.

Ответ:

- Добавить кеширование на уровне приложения
- Использовать batch операции ORM
- Внедрить raw SQL для критичных запросов
- Рефакторинг ORM-моделей и убрать лишние связи
- Перейти к CQRS / разделению чтения и записи

 Шаг 15:

Миссия 15

Распределите шаги при масштабировании приложения, когда ORM уже оптимизирован, но нагрузка растёт:

1. Шардинг / разделение данных
2. Вертикальное масштабирование базы (увеличение ресурсов)
3. Репликация чтения (read replica)
4. Горизонтальное масштабирование приложения
5. Внедрение CQRS / разделение путей чтения и записи

Подсказка: Какие меры самые лёгкие и какие наиболее фундаментальные?

Ответ:

- Вертикальное масштабирование базы (увеличение ресурсов)
- Репликация чтения (read replica)
- Горизонтальное масштабирование приложения
- Шардинг / разделение данных
- Внедрение CQRS / разделение путей чтения и записи

Шаг 16:

Миссия 16

Расположите техники, которые стоит применять для автоматизации мониторинга ORM-эффекта:

1. Логирование цепочек SQL-запросов
2. Настройка алёртов на медленные запросы > порога
3. Сбор метрик по количеству ORM-вызовов
4. Дашборды времени отклика API и зависимости от ORM
5. Регулярный аудит запросов и профилей

Подсказка: Автоматизация начинается с простых сигналов и идёт к регулярным аудитам.

Ответ:

- Логирование цепочек SQL-запросов
- Сбор метрик по количеству ORM-вызовов
- Настройка алёртов на медленные запросы > порога
- Дашборды времени отклика API и зависимости от ORM
- Регулярный аудит запросов и профилей

Шаг 17:

Миссия 17

Расположите фазы эволюции системы в сторону отказа от ORM-зависимости:

1. Оптимизация ORM-функций и исправление «узких мест»
2. Гибридный подход: ORM + ручные DAO для критичных частей
3. Вынос бизнес-логики в отдельные сервисы с DAO
4. Полный отказ от ORM в критичных подсистемах
5. Переход на микросервисы с собственными моделями доступа к БД

Подсказка: Изменения идут от менее радикального к наиболее радикальному.

Ответ:

- Оптимизация ORM-функций и исправление «узких мест»
- Гибридный подход: ORM + ручные DAO для критичных частей
- Вынос бизнес-логики в отдельные сервисы с DAO

- Полный отказ от ORM в критичных подсистемах
- Переход на микросервисы с собственными моделями доступа к БД

Урок 5 - Грабли архитектуры и микросервисов

Шаг 01:

Миссия 1

Вы работаете над системой обмена сообщениями между микросервисами, и один сервис делает прямые вызовы к БД другого микросервиса. Чем это грозит?

Подсказка: Нарушается граница микросервиса.

Ответ:

- **Сильно снизится изоляция и возрастёт связность между сервисами**
- Увеличится латентность всех запросов
- Это сразу приведёт к полной остановке системы
- Это безопасно, если использовать пул соединений

Шаг 02:

Миссия 2

Ваша команда решила, что каждый микросервис будет иметь свою собственную базу данных. Какую из следующих проблем вы можете получить?

Подсказка: Данные могут «расходиться».

Ответ:

- **Сложность обеспечения согласованности между БД**
- Невозможность горизонтального шардинга
- Потеря связи с внешними сервисами
- Невозможность масштабировать сервисы

Шаг 03:

Миссия 3

Один микросервис запрашивает большую агрегированную отчётность, и отправляет SQL-запрос напрямую к общей базе другого сервиса. Выбор архитектуры уязвим. Как лучше было организовать доступ?

Подсказка: Используйте границы сервиса и API.

Ответ:

- **Сделать API-слой / endpoint в том микросервисе, который владеет данными**
- Разрешить всем напрямую писать SQL к чужой БД
- Копировать всю базу локально в каждый сервис
- Использовать единый монолит как централизованную БД

Шаг 04:

Миссия 4

Вы реализуете распределённую транзакцию между четырьмя микросервисами, каждый работает на своей БД. Что является самым опасным риском?

Подсказка: Что сложно обеспечить в распределённой среде?

Ответ:

- **Блокировки и дедлоки в разных базах и невозможность отката во всех случаях**
- Дублирование данных между сервисами
- Снижение читаемости кода
- Невозможность масштабировать базы

 **Шаг 05:**

Миссия 5

Вы решили внести API-gateway, через который все запросы к микросервисам проходят. Что может стать неожиданной проблемой?

Подсказка: Один узел может стать бутылочным горлышком.

Ответ:

- **Gateway может стать single point of failure и узким местом**
- Он автоматически кэширует всё без настройки
- Он полностью скрывает ошибки downstream
- Он увеличивает число баз данных

 **Шаг 06:**

Миссия 6

 Сопоставьте архитектурную проблему и её следствие в микросервисной системе:

Проблемы:

Ответ:

- Сильная связность между сервисами
- Единый shared database
- Дублирование данных между сервисами
- Отсутствие API-шлюза / маршрутизации

Следствия:

1. Изменение одного сервиса рушит другие
2. Контуры ответственности размыты, микросервисы конкурируют за таблицы
3. Сложность синхронизации и рассинхронизация данных
4. Непонятно, куда направлять запросы, дублирование логики маршрута

Подсказка: Смотрите, какая проблема даёт «эффект домино».

- **Сильная связность между сервисами** — Изменение одного сервиса рушит другие
- **Единый shared database** — Контуры ответственности размыты, микросервисы конкурируют за таблицы
- **Дублирование данных между сервисами** — Сложность синхронизации и рассинхронизация данных
- **Отсутствие API-шлюза / маршрутизации** — Непонятно, куда направлять запросы, дублирование логики маршрута

 **Шаг 07:**

Миссия 7

 Сопоставьте способ межсервисной коммуникации и потенциальную проблему архитектуры:

Подходы:

Ответ:

- Synchronous HTTP / REST вызовы
- gRPC вызовы с таймаутами
- Сообщения через очередь / брокер (event-driven)
- Saga / orchestration

Проблемы:

1. Формирование цепочек зависимостей и блокировки при сбоях
2. Таймауты и ухудшение отклика при высоких нагрузках
3. Сложность согласованности и компенсационных операций
4. Высокая латентность из-за сетевых накладных расходов

Подсказка: Подумайте, как каждый способ коммуникации проявляет слабость.

- **Synchronous HTTP / REST вызовы** — Высокая латентность из-за сетевых накладных расходов
- **gRPC вызовы с таймаутами** — Таймауты и ухудшение отклика при высоких нагрузках
- **Сообщения через очередь / брокер (event-driven)** — Сложность согласованности и компенсационных операций
- **Saga / orchestration** — Формирование цепочек зависимостей и блокировки при сбоях

Шаг 08:

Миссия 8

 Сопоставьте трансформацию модели данных и её слабость при микросервисной архитектуре:

Модели данных:

Ответ:

- Согласованная модель во всех сервисах
- Свободные модели с копиями атрибутов
- Антикоррупционный слой
- Событийная репликация / CQRS

Слабости:

1. Усложнение согласования изменений схемы между командами
2. Дублирование логики преобразования и миграций
3. Зависимость от трансляции при запросах
4. Поздняя согласованность и временные рассинхронизации

Подсказка: Рассмотрите, где возникает рассинхронизация и где схема должна трансформироваться.

- **Согласованная модель во всех сервисах** — Усложнение согласования изменений схемы между командами
- **Свободные модели с копиями атрибутов** — Дублирование логики преобразования и миграций
- **Антикоррупционный слой** — Зависимость от трансляции при запросах
- **Событийная репликация / CQRS** — Поздняя согласованность и временные рассинхронизации

Шаг 09:

Миссия 9

 Сопоставьте микросервисную стратегию и её основное препятствие:

Стратегии:

Ответ:

- API composition (композиция запросов через API)
- CQRS / разделение чтения и записи
- Запрос через BFF / агрегатор сервисов
- Событийное взаимодействие через шину событий

Препятствия:

1. Высокая нагрузка на агрегатор и узкие места в связке сервисов
2. Сложность обеспечения согласованности данных между Read и Write
3. Избыточные вызовы между сервисами вместо единого запроса
4. Неопределённость порядка обработки и рассинхронизация событий

Подсказка: Подумайте, где возникает задержка или повторные вызовы.

- **API composition (композиция запросов через API)** — Избыточные вызовы между сервисами вместо единого запроса
- **CQRS / разделение чтения и записи** — Сложность обеспечения согласованности данных между Read и Write
- **Запрос через BFF / агрегатор сервисов** — Высокая нагрузка на агрегатор и узкие места в связке сервисов
- **Событийное взаимодействие через шину событий** — Неопределённость порядка обработки и рассинхронизация событий

Шаг 10:

Миссия 10

Расположите шаги, когда вы решаете уйти от shared database к независимым БД в микросервисах:

1. Идентификация границ данных и владельцев объектов
2. Создание API-интерфейсов для доступа к данным другого сервиса
3. Миграция функциональности и перенаправление вызовов
4. Деплой независимых БД и миграций
5. Удаление прямых обращений к старой общей базе

Подсказка: Переход должен быть по шагам: сначала проектирование, потом реализация, затем переключение.

Ответ:

- Идентификация границ данных и владельцев объектов
- Создание API-интерфейсов для доступа к данным другого сервиса
- Миграция функциональности и перенаправление вызовов
- Деплой независимых БД и миграций
- Удаление прямых обращений к старой общей базе

Шаг 11:

Миссия 11

Расставьте уровни сложности и риска при введении микросервисов над почти монолитной системой:

1. Добавить экземпляры API-шлюза перед монолитом

2. Выделить минимальный сервис (например, только один модуль)
3. Перенести коммуникацию через очередь между сервисами
4. Выделить ещё больше сервисов, разделив логику
5. Перейти на полностью распределённую архитектуру с микросервисами для всего

Подсказка: Чем более глубокая декомпозиция — тем выше риск.

Ответ:

- Добавить экземпляры API-шлюза перед монолитом
- Выделить минимальный сервис (например, только один модуль)
- Перенести коммуникацию через очередь между сервисами
- Выделить ещё больше сервисов, разделив логику
- Перейти на полностью распределённую архитектуру с микросервисами для всего

 Шаг 12:

Миссия 12

Поставьте в правильный порядок слои взаимодействия между микросервисами и БД:

1. Клиент → API Gateway / BFF
2. Микросервис (логика)
3. Сервис-API внутренняя маршрутизация / агрегация
4. Доступ к хранилищу данных (собственная БД сервиса)
5. Межсервисное общение через очередь / вызовы

Подсказка: Запрос идёт сверху вниз, затем может взаимодействовать с другими сервисами.

Ответ:

- Клиент → API Gateway / BFF
- Микросервис (логика)
- Сервис-API внутренняя маршрутизация / агрегация
- Межсервисное общение через очередь / вызовы
- Доступ к хранилищу данных (собственная БД сервиса)

 Шаг 13:

Миссия 13

Расставьте уровни принятия отказа от распределённых транзакций в микросервисах:

1. Введение compensating actions / компенсирующих операций
2. Использование eventual consistency и разрешение конфликтов
3. Переключение на асинхронные процессы для согласованности
4. Полный отказ от ACID в кросс-сервисных сценариях
5. Добавление мониторинга и алёртов для рассинхронизаций

Подсказка: Начинайте с мягких мер, затем постепенно усиливайте.

Ответ:

- Введение compensating actions / компенсирующих операций
- Использование eventual consistency и разрешение конфликтов
- Переключение на асинхронные процессы для согласованности
- Полный отказ от ACID в кросс-сервисных сценариях
- Добавление мониторинга и алёртов для рассинхронизаций

🔮 Шаг 14:

Миссия 14

Расположите этапы в стратегии введения Event sourcing / CQRS в микросервис-контексте:

1. Определение агрегатов и событий
2. Проектирование схемы записи и проекций
3. Поддержка совместимости старых сущностей
4. Миграция части операций к событийной модели
5. Полный переход и удаление старых путей записи

Подсказка: Не можно переходить сразу — важна плавность.

Ответ:

- Определение агрегатов и событий
- Проектирование схемы записи и проекций
- Поддержка совместимости старых сущностей
- Миграция части операций к событийной модели
- Полный переход и удаление старых путей записи

🔮 Шаг 15:

Миссия 15

Расположите шаги обеспечения согласованности данных через событийную шину между сервисами:

1. Производство события после успешной локальной транзакции в сервисе
2. Доставка события в очередь / брокер
3. Потребление события другими сервисами и запись проекций
4. Обработка дубликатов и idempotency
5. Мониторинг и дедлайн доставки / компенсации

Подсказка: Событие должно быть надёжно произведено и надёжно потреблено.

Ответ:

- Производство события после успешной локальной транзакции в сервисе
- Доставка события в очередь / брокер
- Потребление события другими сервисами и запись проекций
- Обработка дубликатов и idempotency
- Мониторинг и дедлайн доставки / компенсации

🔮 Шаг 16:

Миссия 16

Расположите стадии зрелости микросервисной архитектуры, когда вы постепенно избавляетесь от антипаттернов:

1. Монолит с модулями и монолитным БД
2. Микросервисы с shared database / сильной связностью
3. Микросервисы с отдельными БД, но синхронными вызовами
4. Асинхронные взаимодействия, eventual consistency
5. Событийная архитектура, CQRS / Event sourcing, с минимальной связностью

Подсказка: Архитектурный путь обычно идёт через постепенное ослабление связей.

Ответ:

- Монолит с модулями и монолитным БД
- Микросервисы с shared database / сильной связностью
- Микросервисы с отдельными БД, но синхронными вызовами
- Асинхронные взаимодействия, eventual consistency
- Событийная архитектура, CQRS / Event sourcing, с минимальной связностью